

Automated Analysis and Mutation of Software Artifacts against EDR

Xavier Ogay - 301681 - xavier.ogay@epfl.ch - [\[GitHub Repository\]](#)

Abstract

Modern Endpoint Detection and Response (EDR) systems layer kernel telemetry, behavioural heuristics, and cloud-assisted machine learning into detection pipelines whose internal triggering logic remains largely opaque. Despite their centrality to enterprise security, no principled methodology exists to systematically explain *why* a given artifact is detected, evaluation still depends on manual red-team craft that is slow, subjective, and difficult to reproduce.

We present AutoMutate++, a closed-loop framework that transforms EDR evaluation from ad-hoc experimentation into a repeatable, feedback-driven engineering process. Starting from a universally detected payload, the system automatically mutates, executes, observes, and avoids detection in successive rounds. A *multi-layer mutation engine* applies deterministic transformations across three compilation stages, source-level AST rewrites, LLVM IR control-flow perturbations, and post-link PE binary edits, over a modular, gene-based template whose seven independent behavioural slots span a combinatorial search space far exceeding what manual campaigns can cover. A *three-run differential protocol* executes each variant under baseline, instrumented, and EDR-free conditions, filtering instrumentation artefacts and carrier bugs to produce a trustworthy detection oracle. Finally, a *token-based attribution* pipeline extracts normalised triage tokens from ETW telemetry and ranks them by Bayesian lift, yielding avoid/seek hypotheses.

Evaluated against Microsoft Defender for Endpoint with four selector strategies as an ablation study, AutoMutate++ demonstrates that feedback-driven selectors outperform random search, confirming that the infrastructure provides a meaningful fitness signal. The genetic-algorithm selector successfully transforms universally flagged artifacts into evasive variants. The infrastructure orchestrates runs asynchronously, scaling to sustained campaigns while accumulating a structured database of mutations, telemetry, and verdicts that serves as a reusable corpus for EDR research. Because mutations, selectors, and the attribution oracle are fully modular, the framework generalises to longer campaigns, alternative payloads, and other EDR targets without architectural change. Equally, the framework serves defensive teams: blue-team op-

erators can leverage AutoMutate++ to stress-test and benchmark their EDR of choice, systematically uncovering weak spots before real adversaries do.

To our knowledge, AutoMutate++ is the first system to unify multi-layer deterministic mutation, differential outcome filtering, and statistical token-level attribution in a single closed loop, shifting EDR evaluation from opaque craft to transparent, reproducible engineering.

Keywords: EDR, AV, Bypass, Mutation, Automatic, Framework

Typographic conventions. This document uses a consistent set of typographic markers to distinguish technical entities such as API names, syscall names, file paths, registry keys, and hexadecimal values from regular text. The legend below summarizes these conventions.

Formatting Legend

Inline code	<code>size_t len = 0;</code>
API name	<code>VirtualProtect</code>
DLL	<code>ntdll.dll</code>
Registry	<code>HKCU\Software\Microsoft\Windows</code>
Hex / opcode	<code>0x90</code> <code>E8 ?? ?? ?? ??</code> <code>0x401000</code>
Syscall	<code>NtCreateThreadEx</code>
Path	<code>C:\Windows\System32\winhttp.dll</code>
Process	<code>explorer.exe</code>
CLI flag	<code>-headless -vv</code>
Hash	<code>SHA256: 2f1c...9a</code>
Network	<code>10.0.0.5 c2.redteam.local</code>
Callout	

Note

Prefer RW→RX over RWX in staging.

Contents

Introduction	1
<i>Overview and Contributions</i>	1
Related Work	2
<i>EDR Detection Landscape</i>	2
<i>Machine Learning in Malware Detection</i>	2
<i>EDR Evaluation: Empirical Gaps</i>	3
<i>Related Evasion Frameworks</i>	4
<i>Fuzzing as Methodological Foundation</i>	5
Background	8
<i>Operating System and the User/Kernel Separation</i>	8
<i>Windows Technical Substrate</i>	9
<i>The Portable Executable (PE) Format</i>	9
<i>Windows Primitives for Code Execution</i>	10
<i>Antivirus and Classical Detection Approaches</i>	10
<i>Signature-Based Detection</i>	10
<i>Heuristic Analysis</i>	11
<i>Modern Extensions</i>	11
<i>Endpoint Detection and Response (EDR)</i>	11
<i>User-Mode Components</i>	12
<i>Kernel-Mode Components</i>	12
<i>Event Tracing for Windows (ETW)</i>	12
<i>Detection Pipeline and Implications</i>	13
<i>Detection trade-offs: the “Bubbles of Bane”</i>	13
<i>Malware Obfuscation Fundamentals</i>	14
<i>Theoretical Limits of Malware Detection</i>	15
<i>Fuzzers and their role in security testing</i>	16
<i>Programmatic Code Transformation</i>	16
<i>Levels of Transformation</i>	17
<i>Evolutionary Search and Genetic Algorithms</i>	17
<i>Exploration vs. Exploitation</i>	18
<i>Differential Analysis and Causality</i>	18
<i>Delta Debugging</i>	18
<i>Trace Refinement</i>	18
<i>Telemetry Abstraction and Triage Tokens</i>	18
<i>gRPC and Protocol Buffers</i>	19
Design	20
<i>Design Goals and Constraints</i>	21
<i>System Decomposition</i>	21
<i>Controller Design: Jobs, Rounds, and Fair Scheduling</i>	21
<i>RunPool and capability-based routing.</i>	21
<i>Three-Run Differential Protocol</i>	21
<i>Differential Categories and Trustworthiness</i>	23
<i>Build System Design: Modular Templates as a Controlled Search Space</i>	23
<i>Modular Gene Model</i>	24
<i>Three-Layer Mutation Architecture</i>	24
<i>Deconditioner Design</i>	24
<i>Deterministic Compilation Pipeline</i>	25
<i>Instrumentation Design: Evidence That Survives Process Death</i>	26
<i>Conditional instrumentation via #ifdef macros.</i>	27
<i>Worker Agent Design: Robust Execution as a Ten-Phase Pipeline</i>	27
<i>Execution as phases.</i>	27

Per-run detection classification.	27
Execution Guarantees and Transport	28
Lab Infrastructure	29
Data Model and Storage: Making Runs Queryable	29
Evasion Loop Architecture: Fuzzing for Detection Bypass	29
Composite Evasion Scoring	30
Token-Driven Feedback Loop	30
Token abstraction.	30
Extract–Score–Guide pipeline.	31
Design choices that enable feedback.	32
Selector Strategies as Ablation Framework	32
Safety and Experiment Hygiene	32
Summary	33
Implementation	34
Controller: Dispatch Reliability Mechanisms	34
Dryrun Dispatch and Grace Period	34
Asynchronous Coverage Correction	35
Builder: Artifact Construction and Mutation Pipeline	35
Template Assembly	35
Payload Encoding Schemes	35
AST Mutations via Tree-Sitter	36
IR Mutations via Text-Based LLVM IR Manipulation	37
Binary Mutations at the PE Byte Level	38
Benign Catalog: Dependency-Aware API Insertion	38
Builder: Instrumentation and Runtime Infrastructure	38
Shellcode Stub Prepend	38
Shellcode Checkpoint Patching	39
Instrumentation Runtime	40
Deferred Loop Tracing	41
Named Pipe Trace Transport	41
Worker Agent Implementation	42
Triage and Feedback Loop	43
Token Extraction and Scoring Pipeline	43
Token Set Differencing	44
Mutation Parameter Distance	45
Evolutionary Mutation Selection	46
Evaluation	47
Benchmark Structure and Methodology	47
Setup	47
Question	48
Observation	48
Deduction	49
Summary	51
Future Work	52
EDR-Side Token Extraction	52
Alternative telemetry collectors.	52
API Indirection via Callback Mechanisms	52
Advanced Carrier Strategies	52
Additional tokens	53
Temporal tokens.	53
Localization tokens.	53
MSVC Toolchain Adoption	53
Mutation Token Profiling	54

<i>Machine Learning on Accumulated Data</i>	54
<i>Completing the Token Feedback Loop</i>	54
Conclusion	55
Appendices	60
<i>Build and Execution Instructions</i>	60
<i>Loader Template</i>	62
<i>Pattern Signature Internals</i>	64
<i>Wildcard Patterns in Signatures</i>	64
<i>PE File Format Details</i>	65
<i>Vectored Exception Handling and INT3 Checkpoints</i>	67
<i>Kernel Patch Protection and the End of Third-Party Kernel Hooking</i>	68
<i>Elasticsearch Index Schema</i>	69
<i>Token Comparison Pipeline</i>	70
<i>Controller Communication and Synchronization</i>	71
<i>Worker Exit Code Classification</i>	72
<i>Controller Verdict Resolution</i>	72
<i>Instrumentation: Evolution from Weak Symbols to Conditional Compilation</i>	73
<i>Named Pipe Trace Protocol</i>	75
<i>Mutation Catalog</i>	76
<i>List of techniques</i>	82
<i>Component-Level Evaluation Diagnostics</i>	86
<i>Coverage Selector (n=39)</i>	86
<i>Genetic Selector (n=51)</i>	91
<i>Random Selector (n=50)</i>	100
<i>Token Selector (n=40)</i>	104
<i>Infrastructure Evaluation Diagnostics</i>	108
<i>I1: Encoding Validation</i>	108
<i>I10: Oracle Stability</i>	108
<i>I11: Cross-Selector Comparison</i>	109
<i>I12: Guidance Utilisation</i>	111
<i>I13: Convergence Analysis</i>	111
<i>I14: Line Tracing</i>	112
<i>Condensed Metrics Tables</i>	112

Introduction

A red-team operator spends a week hand-crafting a shell-code loader, tweaking API-call sequences, obfuscating strings, and testing each iteration against a production endpoint detection and response (EDR) agent. The payload survives one signature update before it is flagged, and the cycle restarts. Scale that single operator's effort to the broader threat landscape where an estimated 560,000 new malware samples appear every day [29] and a fundamental mismatch emerges: the arms race between offensive tooling and endpoint defence has accelerated far beyond the reach of any manual evaluation methodology.

EDR products have become the last line of host-level defence deployed on virtually every corporate endpoint [6]. They fuse kernel-mode callbacks, user-mode hooks, behavioural heuristics, and cloud-assisted machine-learning classifiers into a multi-layered detection pipeline [43, 41]. Yet the organisations that stake their security posture on these products possess no systematic, reproducible means of verifying whether they actually work. Manual red-team engagements are expensive, cover a handful of scenarios, and produce results that cannot be generalised [39]. Nominal ATT&CK coverage numbers overstate actual detection capability [74]. Even the ground-truth malware labels on which evaluation depends are unstable, with more than half of all samples receiving a different verdict after only twelve months [81]. Taken together, these limitations constitute an *evaluation gap*: defenders cannot objectively measure what their EDR detects, what it misses, and why.

On the evasion side of this gap, attackers and researchers have no shortage of automated tools, but each addresses only a fragment of the problem. Binary-level mutators such as ARMED [46] perturb static features like headers and section entropy yet leave behavioural signals untouched. GAN-based generators like MalGAN [34] operate entirely in feature space and never produce an executable artefact that can be detonated against a real EDR. Single-layer AST frameworks such as Leucism [48] iteratively mutate and trace executions to localize detections, but remain largely manual and lack differential isolation, continuous scoring, and multi-layer coverage. Meanwhile, the software-testing community has shown that large search spaces can be explored systematically through token-level mutations [65], coverage-guided scheduling [13], and statistical input-state correlation [7], principles that have never been applied to EDR evaluation. Bridging these two worlds is the central motivation of this thesis.

The following subsection details the architecture and contributions of AutoMutate++, the framework developed to close this evaluation gap.

Overview and Contributions

We present AutoMutate++, a closed-loop framework for automated analysis of Windows EDR detection

mechanisms. Unlike manual red-team workflows, which require an operator to hypothesise, mutate, execute, and interpret results one artifact at a time, AutoMutate++ automates the entire cycle: it generates multi-layer artifact mutations, executes them under production EDR monitoring, collects telemetry via ETW (§ [Event Tracing for Windows \(ETW\)](#)), extracts normalised *triage tokens*, scores each token by its statistical correlation with detection, and feeds those scores back into the next round of mutation selection.

Packer-based evasion tools operate at a single layer (binary) and treat detection as a black box (evaded or not), without explaining *which* behavioural signal triggered the verdict. AutoMutate++ bridges this gap by combining multi-layer mutation with token-level attribution with the goal, at term, to try to produce ranked, explainable hypotheses rather than opaque pass/fail outcomes.

Our main contributions are:

1. **A closed-loop mutation-triage architecture.** An automated pipeline that iterates through build, execution, telemetry collection, token extraction, scoring, and mutation selection without manual intervention. Each round's triage guidance is fed asynchronously to the next round's selector, closing the feedback loop.
2. **A multi-layer mutation engine.** A minimal set of 22 mutations spanning three compilation stages, 10 AST-level transforms, 3 LLVM IR transforms, and 9 post-link binary transforms, applied in a deterministic, reproducible pipeline with seeded randomness and pinned toolchains.
3. **A three-run differential protocol.** Each mutation round executes the same artifact under three conditions: baseline (no instrumentation), instrumented (line-level tracing), and dryrun (no AV/EDR) to isolate real detections from instrumentation artifacts and carrier bugs, ensuring only trustworthy outcomes inform the learning loop.
4. **Token-based detection attribution.** A 9-category triage token system (API calls, API arguments, call-sequence n -grams, ETW events, loaded images, execution checkpoints, module choices, and mutation identifiers) with statistical scoring: $\text{lift}(T) = P(\text{detected} \mid T) / P(\text{detected})$, weighted by observation confidence. High-lift tokens become *avoid* targets; low-lift tokens become *seek* targets for future rounds.
5. **Four interchangeable selection strategies.** Random (zero-intelligence baseline), coverage-driven (ϵ -greedy), genetic-algorithm (tournament selection with crossover), and token-guided selectors, all sharing a common trait interface and assignable per job for controlled ablation studies.

6. **A modular template architecture.** A gene-based artifact template (inspired by SuperMega [26]) with seven independently swappable module slots (carrier, decoder, anti-emulation, guardrail, virtualprotect, decoy, deconditioner), enabling combinatorial exploration of evasion-relevant design choices.

Related Work

This section surveys the current landscape of endpoint detection, evasion research, machine-learning-based malware classification, and automated testing methodologies that inform the design of AutoMutate++. We organize the discussion into five areas: the EDR detection landscape, machine learning in malware detection, empirical gaps in EDR evaluation, related evasion frameworks, and fuzzing as a methodological foundation.

EDR Detection Landscape

EDR evolution. Arfeen et al. [6] define EDR as the convergence of real-time monitoring, behavioural analysis, and automated response on the endpoint itself. Kaur et al. [43] trace this convergence across four distinct phases, summarized in table 1. AutoMutate++ operates directly in the fourth phase, where detection is multi-layered, cloud-assisted, and increasingly opaque, making systematic, automated evaluation both more necessary and more difficult.

EDR self-protection mechanisms. Cailleret [17] catalogues the layered self-protection architecture that modern EDRs employ to resist tampering. table 2 summarizes the principal mechanisms. These protections ensure that an attacker cannot simply disable the sensor; consequently, evasion research must focus on altering the *artifact* rather than the *observer*.

Memory scanning and cloud-assisted analysis. Beyond static file inspection, modern EDRs perform periodic memory sweeps and trigger on-demand scans when suspicious events occur, for example, a call to `VirtualProtect` that transitions a page to RWX, or the creation of a remote thread in an unrelated process. Cloud backends further extend the detection surface by routing suspicious samples or behavioural summaries to heavyweight ML classifiers that cannot run locally. On the evasion side, techniques such as sleep encryption (SWAPPALA, Ekko) [64], module stomping, and phantom DLL hollowing represent advanced countermeasures that keep malicious content out of scannable memory at rest. While AutoMutate++ does not implement these techniques directly, it models the observable behavioral primitives they rely on: memory protection transitions, API call sequences, and execution timing as AST-level mutations. The feedback loop can then measure whether these lower-level behavioral changes correlate with detection, providing indirect evidence about EDR sensitivity

to the patterns that advanced techniques like module stomping exploit.

Modern evasion taxonomy. Geng et al. [31] propose a three-category taxonomy of evasion techniques: *transformation-based*, *concealment-based*, and *attack-based*. table 3 maps each category to the corresponding AutoMutate++ mutation layer, illustrating how the three-layer architecture provides coverage across the full taxonomy.

Modern evasion landscape. Rutishauser [64] provides an extensive compendium of anti-EDR techniques deployed in contemporary offensive tooling: memory encryption, call-stack spoofing, ETW patching, direct and indirect syscalls, process-injection variants, and API unhooking [78]. Van de Wouw et al. [78] further systematize API unhooking by cataloguing the runtime restoration of hooked `ntdll.dll` stubs as a prerequisite for many of the techniques listed above. HarfangLab [72] complements this offensive view by enumerating the telemetry sources (kernel callbacks, ETW providers, minifilters) that EDRs consume. Together, these works define the attack-defence surface that AutoMutate++ must navigate.

MalDev myths. Rutishauser [62] debunks several widely held assumptions in the malware-development community. table 4 summarizes the principal myths and their empirical status. These findings directly inform which mutations are worth exploring: for instance, entropy reduction and IAT manipulation are deprioritized relative to API-call-sequence mutations, which correlate more strongly with behavioural detections.

Machine Learning in Malware Detection

PE header features. Kim et al. [45] demonstrate that 87 features extracted solely from the PE header achieve over 80% classification accuracy on a corpus of more than 500,000 samples. This result motivates the binary mutation layer of AutoMutate++: if header features alone carry significant discriminative power, then normalizing those features toward benign distributions is a necessary evasion step.

Production ML pipelines. Kaspersky [41] describes a two-stage production architecture comprising a lightweight pre-filter (locality-sensitive hashing, exemplar networks) that runs on the endpoint and a precision classifier (behaviour-log neural networks, deep stacked ensembles) that runs in the cloud. Ferdous et al. [29] report that the malware ecosystem generates approximately 560,000 new unique samples daily, underscoring the reliance of modern detection on ML rather than manual signature authoring. This two-tier architecture means that evasion must address both the local pre-filter, which AutoMutate++ targets via binary mutations that shift lightweight features, and the cloud classifier, which

Table 1: Four phases of EDR evolution (adapted from Kaur et al. [43]).

Period	Phase	Representative Techniques
2005–2010	Signature era	Signature-based, heuristic, log analysis, HIDS
2011–2015	Behavioural era	Behavioural analysis, memory forensics, sandboxing, anomaly detection
2016–2020	ML era	ML + behavioural analytics, endpoint isolation, cloud EDR, fileless detection
2021–2023	AI/Automation	XDR, zero trust, AI-powered detection, automated incident response

Table 2: EDR self-protection mechanisms (Cailleret [17]).

Mechanism	Purpose
Protected Process Light (PPL)	Prevents user-mode tampering with the sensor process
Early Launch Anti-Malware (ELAM)	Loads the driver before third-party code executes
Kernel callbacks	Monitors process, thread, image-load, and registry events
Isolated user mode (VBS)	Hardware-backed isolation of credential material
Restricted DLL loading	Blocks unsigned DLLs from the sensor address space
Exploit protection (ACG, CIG)	Prevents dynamic code generation and unsigned code injection

it targets via behavioural mutations at the AST and IR layers.

Obfuscation-resilient detection. Madamidola et al. [50] construct a 340 KB Random Forest model that achieves 99.8% accuracy at 5.7 μ s per file, with zero-shot generalization to previously unseen obfuscation families, by relying on memory-forensic features rather than static file features. This work highlights that obfuscation alone is insufficient if the detection model observes runtime memory layouts. AutoMutate++ accounts for this threat by including behavioural mutations (e.g., sleep encryption, stack spoofing) that alter the runtime memory footprint, not only the on-disk binary.

Label instability. Zhu et al. [81] (USENIX Security 2020) present the most comprehensive study of ground-truth reliability in malware classification. They survey 115 papers, analyse over 300 million data points, and track 14,000+ files over one year on VirusTotal. Their principal findings are alarming: more than 50% of labels exhibit hazard flips (benign $\leftrightarrow$ malicious transitions) within the observation window; anti-virus engines cluster into correlated groups that propagate false reputations; and obfuscation-induced false positives are common. The absence of a consensus threshold demonstrates that external verdicts are unreliable as ground truth. This finding motivates the differential protocol in AutoMutate++: rather than relying on unstable aggregated verdicts, the framework observes the real-time detection response of a specific EDR under controlled conditions, producing a reproducible binary signal per variant. The framework uses direct observation of detection events across controlled execution runs to construct its own feedback signal.

EDR Evaluation: Empirical Gaps

Karantzias & Patsakis. Karantzias and Patsakis [39] evaluate eleven commercial EDR products against four APT-style attack scenarios built with Cobalt Strike. Their central finding is that EDRs fail to prevent, or even log, the bulk of attack steps. They note that, at the time of writing, “no empirical assessment of EDR efficacy exists in the scientific literature,” positioning their work as a first attempt to fill this void. However, their methodology is entirely manual: attack variants are hand-crafted, the number of variations is small (four scenarios), and no systematic mutation or feedback is applied.

Virkud et al. Virkud et al. [74] (USENIX Security 2024) conduct a large-scale analysis of ATT&CK coverage across major EDR vendors. Nominal technique coverage ranges from 48% to 55%, but drops to 25–26% after filtering out rules that the vendor itself labels as low-risk or informational. Fully 27.7% of ATT&CK techniques have no implemented detection rule across any tested product. Furthermore, labelling is inconsistent: the same telemetry event may be classified as “critical” by one vendor and “informational” by another. These findings demonstrate that nominal ATT&CK coverage is an unreliable proxy for actual detection capability. AutoMutate++ addresses this gap by measuring detection outcomes empirically against a target EDR, rather than inferring coverage from vendor-reported technique mappings.

EvilEDR. Alachkar et al. [2] (USENIX Security 2025) invert the conventional threat model by demonstrating that EDR agents can be repurposed as offensive tools. Using legitimate EDR capabilities, live response, file transfer, telemetry collection, and endpoint isolation, they show that a compromised management console can weaponize the very sensors meant to protect endpoints. They validate the attack against Microsoft Defender for

Table 3: Mapping Geng et al. [31] evasion categories to AutoMutate++ layers.

Geng et al. Category	AutoMutate++ Layer	Concrete Implementation
Transformation-based	AST, IR, Binary	String XOR encoding, constant obfuscation, opaque predicates (IR), NOP insertion (IR)...
Concealment-based	AST, Behavioural modules	anti-emulation (heap stress, CPU burn, FS enumeration), benign preambles, timing jitter insertion, deconditioning
Attack-based	Feedback loop	Closed-loop Modification–Query–Attack cycle: mutation engine generates variants, RedEDR ETW collection observes telemetry, token triage guides subsequent mutation selection

Table 4: MalDev myths vs. empirical evidence (Rutishauser [62]).

Myth	Reality
PE section choice matters for evasion	EDRs do not observe which section data originates from at runtime
High entropy triggers automated alerts	Entropy is rarely a reliable automated indicator; mostly useful for manual RE
IAT content reveals malicious intent	Legitimate software imports the same “suspicious” functions; IAT heuristics are unreliable
Code signing prevents scanning	Reasonable security software mostly ignores signing status
Patching ETW blinds EDRs	Only user-mode ETW is patchable; kernel-level providers and callbacks remain active

Endpoint, Elastic Defend, Sophos Intercept X, and Trend Micro Vision One. Their analysis of EDR internals also confirms that kernel-level ETW is a primary telemetry source for modern sensors. AutoMutate++ leverages this same mechanism as its instrumentation layer, consuming ETW events to observe what the EDR itself observes during controlled execution runs.

Dynamic analysis infrastructure. A recurring question is whether evasion research can be conducted in sandboxed environments rather than against live EDR deployments. table 5 compares three widely used dynamic analysis platforms. AutoMutate++ uses real EDR installations inside managed virtual machines because, sandboxes lack kernel-level telemetry sources (ETW, minifilters, kernel callbacks), do not reproduce the full detection pipeline (cloud look-up, reputation scoring, behavioural correlation over time), and are increasingly detected by malware through anti-sandbox checks. Bulazel and Yener [15] further show that even Microsoft Defender’s own emulator diverges from native execution semantics, producing false negatives that would not occur on a real endpoint. Chen et al. [20] present Kollect, an ETW-based kernel log collector for Windows that efficiently captures and parses kernel-level events with minimal overhead,

enabling lossless, real-time log collection for security analysis.

Table 5: Comparison of dynamic analysis platforms vs. live EDR evaluation.

Aspect	Cuckoo	PANDA	Unicorn	Live EDR (AutoMutate++)
Kernel telemetry	Partial (hooks)	Full (record/replay)	None (CPU only)	ETWs
Automated pipeline	Plugin-based	Scripted replay	Manual	gRPC-orchestrated rounds
Behavioural correlation	Limited	Post-hoc	None	Per-round differential
Anti-evasion resistance	Low	Medium	Low	Native execution
Detection fidelity	Heuristic-based	Offline analysis	None	Production EDR verdicts

Related Evasion Frameworks

This subsection reviews the five most closely related automated evasion frameworks and positions AutoMutate++ relative to each.

Leucism. Lepori [48] developed *Leucism*, a proof-of-concept framework comprising three components: an AST transformer that applies obfuscation passes to C-based shellcode loaders; a code instrumenter that inserts trace points to collect execution traces; and a behaviour mutator that adjusts program parameters between rounds. Leucism successfully bypassed all four tested EDR products (Microsoft Defender, Bitdefender, Elastic Defend, Palo Alto Cortex XDR) for initial access. Its feedback mechanism compares truncated execution traces between detected and undetected variants to identify the source line responsible for detection.

However, Leucism operates at a single transformation layer (AST), relies on manual parameter selection between rounds, and does not formalize the feedback loop: the operator inspects execution graphs and decides which transformations to apply next. Its evaluation signal is binary (detected or not), providing no continuous fitness gradient.

Despite these architectural differences, AutoMutate++ directly adopts and systematizes several of Lepori’s empirically validated techniques: volatile constant decomposition (Section [AST Mutations via Tree-Sitter](#)), staged memory protection transitions (virtualprotect gene), sub-byte payload encoding (deconditioner design, Section [Deconditioner Design](#)), and behavioral mutation via benign API call insertion (Section [Benign Catalog: Dependency-Aware API Insertion](#)). Each technique is integrated as a parameterizable gene that benefits from automated selection, dependency-aware scheduling, and differential evaluation capabilities absent from the original manual workflow.

table 6 provides a detailed comparison across eight dimensions.

Table 6: Detailed comparison: Leucism [48] vs. AutoMutate++.

Aspect	Leucism	AutoMutate++
Mutation layers	1 (AST)	3 (AST + IR + Binary)
Token scoring	None	Lift × confidence
Mutation strategy	Generational mutation	4 selectors (Random, Token, Coverage, Greedy)
Differential protocol	Trace comparison	7-category classification
Telemetry	Instrumentation execution traces	ETWs and instrumentation
Feedback signal	Execution progress before detection	Continuous evasion score
Search guidance	Execution-progress guided mutation	Avoid/seek lists from token analysis
Build determinism	Not discussed	Seeded RNG, reproducible builds

ARMED. Labaca-Castro et al. [46] present ARMED, a framework that applies nine LIEF-based (Library to Instrument Executable Formats) perturbations to PE binaries § [The Portable Executable \(PE\) Format](#) (e.g., section appending, header manipulation, and import-table modifications). Their evaluation shows that a small number of perturbations can reduce detection rates by up to 80% across VirusTotal scanners while preserving functionality. ARMED operates exclusively at the binary level and focuses on static detection engines. Although mutations are validated in a sandbox to ensure functionality, the framework does not leverage behavioral telemetry for mutation guidance. The results demonstrate that PE-level mutations have strong evasion potential, motivating the binary mutation layer in AutoMutate++, but ARMED does not address behavioral detection mechanisms.

Chameleon. Jana and Shmatikov [36] develop *Chameleon*, which exploits 45 parser-gap vulnerabilities across 36 antivirus scanners. By crafting files that are parsed differently by the scanner and the operating system, Chameleon bypasses detection without alter-

ing the payload. This approach is complementary to mutation-based evasion: it targets structural discrepancies in file parsing rather than the content of the executable itself. AutoMutate++ does not currently exploit parser gaps but could incorporate such mutations in its binary layer as a future extension.

MalGAN. Hu and Tan [34] propose *MalGAN*, a generative adversarial network that learns to perturb feature vectors so that a substitute detector classifies them as benign. MalGAN operates in *feature space*: the generator outputs a modified feature vector, not an executable binary. While MalGAN demonstrates the theoretical vulnerability of ML classifiers to adversarial examples, the gap between a perturbed feature vector and a functional executable remains unbridged. AutoMutate++ closes this gap by operating on real source code, IR, and binaries, producing artefacts that compile, link, and execute.

Meng et al. [53] present *MYSTIQUE*, a framework for generating Android malware variants using a feature-based model and a multi-objective evolutionary algorithm. MYSTIQUE modularises attack features and evasion techniques extracted from real malware and evolves new APKs by selecting and assembling feature combinations. The evolutionary search optimises three objectives, aggressiveness, evasiveness, and detectability, where detectability is measured using multiple anti-malware tools. While MYSTIQUE demonstrates the effectiveness of evolutionary search for probing detection capabilities, it targets Android applications and static or hybrid mobile malware detectors. AutoMutate++ adopts a similar search philosophy but focuses on Windows PE artifacts and introduces multi-layer mutations, token-based feedback, and a differential protocol tailored to behavioural EDR systems.

Comparative summary. table 7 synthesizes the comparison across all six systems. AutoMutate++ is the only framework that combines multi-layer mutations, token-level feedback, a differential protocol, and deterministic builds.

Fuzzing as Methodological Foundation

AutoMutate++ adapts principles from coverage-guided fuzzing to the domain of EDR evaluation. In traditional fuzzing, the objective is to maximize code coverage by mutating inputs; in AutoMutate++, the objective is to maximize detection-correlation coverage by mutating artefacts. This subsection traces the conceptual parallels.

Token-level fuzzing. Salls et al. [65] introduce mutations that operate between the byte level and the grammar level, targeting individual tokens. Applied to JavaScript engines, the approach discovers 29 bugs that neither byte-level nor grammar-level fuzzers find. The parallel in AutoMutate++ is the triage token system, which decomposes EDR telemetry into typed tokens (API calls,

Table 7: Comparative summary of automated evasion and malware-generation frameworks.

Dimension	Leucism	ARMED	Chameleon	MaIGAN	MYSTIQUE	AutoMutate++
Year	2025	2019	2012	2017	2016	2026
Mutation layer	AST	Binary (PE)	Binary/APK	Feature space	Feature model → APK	AST + IR + Binary
Target detection	Behavioral EDR	Static AV	Static AV	ML classifiers	AMTs (AV/ML/static/dynamic)	Static + behavioral
Feedback	Execution trace comparison	Detection rate	None	GAN loss	Multi-objective fitness	Token scoring
Executables produced	Yes	Yes	Yes	No	Yes	Yes
Selectors / search	Generational mutation	Random perturbations	None	GAN generator	Evolutionary algorithm	4 selectors
Telemetry	Execution traces	None	None	None	None	9 token categories
Differential protocol	Trace comparison	No	No	No	No	Yes (7 categories)
Deterministic builds	No	No	N/A	N/A	No	Yes

memory operations, registry accesses, etc.) and selects mutations that target specific token categories.

Learning input tokens. IFuzzer [51] automates the extraction of valid tokens from a target program via dynamic tainting, achieving 17% more coverage than manual token specification. AutoMutate++ draws on the same principle: rather than manually defining which telemetry events matter, the framework automatically extracts tokens from ETW traces and scores them by detection correlation.

Grammar-aware AST mutations. Superior [76] extends AFL with grammar-aware trimming and tree-based mutation of AST nodes, discovering 31 bugs (16 CVEs) in JavaScript and XML parsers. AutoMutate++ applies an analogous strategy at the source-code level: tree-sitter parses the C source into an AST, and mutations operate on well-typed subtrees to preserve compilability.

Feasibility-aware guidance. AFLGopher [9] estimates branch feasibility to avoid wasting cycles on unreachable code, achieving $3.76\text{--}5.60\times$ speedup over AFL++. AutoMutate++ applies a similar principle through lift $\times$ confidence scoring § **Degenerate lift guard**: mutations that target tokens with high detection lift and high statistical confidence are prioritized, while mutations targeting low-lift tokens are deprioritized.

Stateful API fuzzing. RESTler [8] models producer–consumer dependencies across API calls to fuzz stateful REST services, finding 28 bugs in GitLab and other services. AutoMutate++ manages analogous statefulness: each mutation round depends on the telemetry and detection outcomes of previous rounds, and the TriageGuidance structure carries forward avoid/seek lists across the feedback loop.

Windows harness synthesis. Jung et al. [38] present WINNIE, a system that automatically synthesizes fuzzing harnesses for closed-source Windows applications and introduces an efficient fork-server implementation for Windows. By extracting API call sequences from execution traces, WINNIE generates harnesses that bypass GUI code and enable efficient fuzzing of program logic. Using this approach, the authors fuzzed 59 Windows binaries and discovered 61 bugs. While WINNIE

focuses on enabling scalable fuzzing of closed-source Windows software, AutoMutate++ addresses a different systems challenge: orchestrating controlled execution experiments on Windows VMs with automated artefact deployment and telemetry collection for behavioural analysis.

Table 8: Progression toward automated, multi-layer EDR evaluation.

Approach	Example	Mutations	Feedback	Automation
Manual pentesting	Red team	Human-chosen	Human analysis	None
Manual research	Karantzas [4]	4 fixed	Manual log review	Partial
Binary mutation	ARMED [4]	Random (binary)	Detection rate	Full (static)
AST feedback loop	Leucism [48]	Population (AST)	Trace truncation	Semi manual (single-layer)
Multi-layer scoring	This work	22 mutations, 3 layers	Token scoring, 4 selectors	Full (multi-layer)

Advanced input-to-state techniques. RedQueen [7] exploits input-to-state correspondence to solve magic-byte checks without symbolic execution. AutoMutate++ draws on an analogous intuition: just as RedQueen observes that input bytes correlate with comparison operands, the token scoring mechanism observes that specific telemetry tokens *correlate* with detection outcomes, without requiring access to EDR rule internals. VUzzer [60] introduces application-aware mutation by prioritising inputs that reach deeper code paths; AutoMutate++ adapts this principle as *outcome-aware* mutation, where the selector prioritises artefact changes that alter the detection verdict rather than targeting EDR logic directly. GREYONE [30] infers lightweight taint relationships without full symbolic execution; AutoMutate++ similarly infers token–detection relationships from statistical co-occurrence (lift $\times$ confidence) rather than from EDR internals, treating the defender as a black box.

Mutation scheduling. The fuzzing literature offers mutation-scheduling strategies that inform AutoMu-

tate++'s selector design. MOpt [49] formulates operator selection as a particle-swarm-optimised multi-armed bandit; AutoMutate++ provides four interchangeable selectors (Random, Coverage, Token, Fuzzer) that can be assigned per job, though it does not yet dynamically switch between them within a single campaign. AFLFast [13] introduces power schedules allocating more energy to seeds exercising rare paths; the CoverageSelector applies a similar principle by favouring mutations whose prior variants achieved higher code-path coverage. FairFuzz [47] targets rare branches specifically; the TokenSelector focuses analogously on high-lift tokens, telemetry signals most correlated with detection. NEUZZ [66] uses neural networks to model input-coverage relationships, pointing toward a potential ML-guided extension of AutoMutate++'s token scoring.

Structure-aware fuzzing without grammars. Grimoire [12] synthesises structural mutations without an explicit grammar by learning recurring patterns from the input corpus. While AutoMutate++'s binary mutation layer does not infer PE structures from observation (it relies on explicit PE parsing via `goblin`), Grimoire motivates the broader principle that structural mutations, such as Rich header injection, import reshaping, and section renaming, can influence detection independently of payload content. Extending AutoMutate++ with corpus-driven structural learning remains future work.

Mapping summary. table 9 maps each fuzzing concept to its AutoMutate++ equivalent, making explicit the methodological lineage.

Synthesis and gap analysis. table 8 traces the progression from manual evaluation to fully automated, multi-layer evasion testing. Manual red-team engagements produce high-fidelity results but do not scale. Early academic evaluations (Karantzas and Patsakis [39]) introduce reproducibility but test only a handful of fixed scenarios. Binary-only mutation frameworks (ARMED [46]) automate the static dimension but ignore behavioural detection. Single-layer feedback frameworks (Leucism [48]) close the behavioural loop for one compilation layer but lack the multi-layer coverage, continuous scoring, and differential isolation needed for systematic exploration.

The undecidability of virus detection (Cohen [21]) and the NP-completeness of obfuscation recognition (Spinelis [69]; Borello and Mé [14]; see Section **Theoretical Limits of Malware Detection**) guarantee that no detection system can be both complete and sound. AutoMutate++ exploits this theoretical gap operationally: by combining three mutation layers, four selection strategies, a token-based feedback mechanism, and a differential protocol that constructs its own ground truth, it provides the first fully automated framework for systematic, multi-dimensional EDR evaluation. The following subsection details the specific contributions of this work.

Table 9: Mapping of fuzzing concepts to AutoMutate++ equivalents.

Fuzzing Concept	Fuzzing Tool	AutoMutate++ Equivalent
Token-level mutations	Token-Level AFL [65]	9-category triage token system
Automated token extraction	IFuzzer [51]	token extraction
Feasibility-aware guidance	AFLGopher [9]	Lift $\times$ confidence scoring
AST-level source mutations	Superion [76]	C source mutations (AST transforms)
Stateful sequencing	RESTler [8]	Round-based state with TriageGuidance
Execution orchestration	WINNIE [38]	gRPC-orchestrated VM execution pipeline (slower)
Coverage metric	Branch/edge coverage	Detection verdict as fitness signal
Feedback loop	Coverage-guided (AFL [80])	Token-scored selection/ Coverage based (4 strategies)
Token-detection correlation	RedQueen [7]	Statistical token-outcome correlation
Taint inference	GREYONE [30]	Behavioural token extraction
Mutation scheduling	MOpt [49]	Per-job selector assignment (not runtime)
Power schedules	AFLFast [13]	CoverageSelector ϵ -greedy exploration
Rare-branch targeting	FairFuzz [47]	TokenSelector high-lift focus
Neural-guided mutation	NEUZZ [66]	ML-guided mutation (potential extension)
Structure-aware mutation	Grimoire [12]	PE-aware binary transforms (explicit parsing, need to be explored)

Background

This chapter establishes the technical foundations for the system described in subsequent chapters. It is organized in three thematic blocks. The first covers the defensive landscape: the Windows operating system, its binary format and runtime primitives, classical antivirus approaches, and modern Endpoint Detection and Response. The second block addresses the evasion landscape: malware obfuscation techniques and the theoretical limits of detection. The third block introduces the algorithmic and methodological foundations, fuzzers, code transformation, evolutionary search, differential analysis, triage tokens, and the gRPC communication substrate, that underpin the automated mutation strategy.

Operating System and the User/Kernel Separation

Modern operating systems such as **Windows** are designed with a strict separation between *user space* and *kernel space*, a principle that ensures both security and stability. User processes, applications and services launched by end users or the system, execute in **user mode**, a restricted execution environment. This restriction prevents them from directly accessing hardware or critical resources, thereby protecting the operating system from accidental corruption or malicious tampering. When user-mode processes need privileged functionality, they must invoke *the Windows API* (e.g., via `kernel32.dll`, `user32.dll`), which funnels requests down through `ntdll.dll` into the kernel.

At the heart of the system lies the Windows kernel (`ntoskrnl.exe`/`ntoskrnl.dll`), which executes in **kernel mode**. The kernel has unrestricted access to memory, hardware, and CPU instructions. It pro-

vides fundamental services such as process and thread scheduling, interrupt handling, and memory management. Above the kernel, *executive services* and *system service routines* expose controlled entry points to user space, such as `NtAllocateVirtualMemory` or `NtCreateThreadEx`. These calls pass through the *system call dispatcher*, which safely transitions execution from user mode to kernel mode.

Between these two domains, Windows enforces a well-defined boundary: user-mode code cannot directly manipulate hardware or kernel data structures. This boundary is fundamental for isolating applications from one another and from the OS itself. However, it also creates an asymmetry: **kernel-mode components, such as drivers, have far more visibility and control than user processes.**

This asymmetry is directly relevant to the domain of *Endpoint Detection and Response (EDR)*. EDR products often include **kernel drivers** (e.g., mini-filters, process and thread callbacks, image load notifications) that integrate deeply with Windows internals. Running in kernel mode, these drivers can intercept system events that user-mode monitoring alone cannot reliably capture, such as low-level process creation, driver loading, or direct memory access attempts. For instance, Windows provides callback mechanisms (e.g., `PoSetCreateProcessNotifyRoutine`, `ObRegisterCallbacks`) that EDRs leverage to detect suspicious activity in real time.

Figure 1 illustrates this layered structure. User applications interact through the Win32 subsystem and `ntdll.dll`, while kernel-mode services and drivers extend the kernel to provide monitoring, logging, and enforcement. Below the kernel, the *Hardware Abstraction*

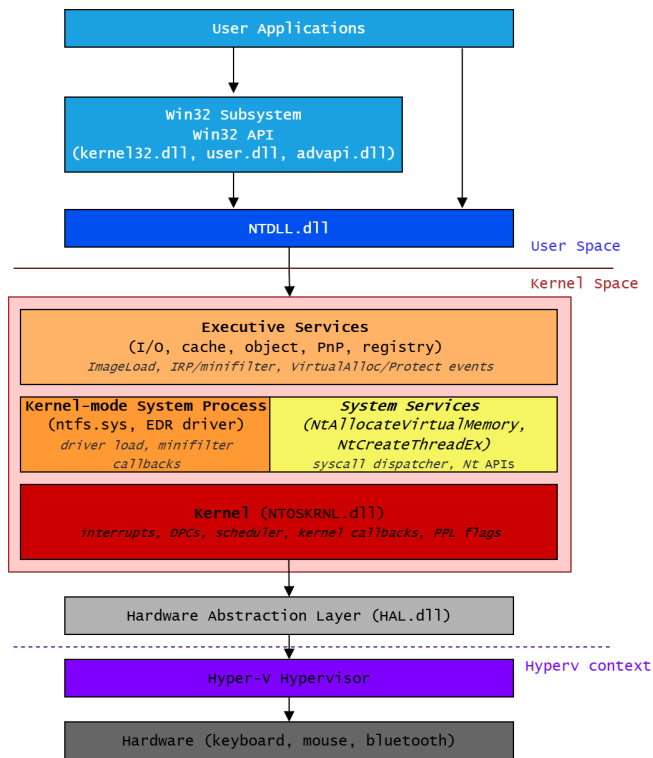


Figure 1: Windows internals - adapted and extended from [19] and [55].

Layer (HAL) and optional Hyper-V hypervisor mediate access to hardware resources.

Understanding this architecture is crucial for this thesis. Since our work explores automated binary mutation and feedback-driven evasion against EDRs, it must account for the fact that the **detection surface spans both user-mode API usage and kernel-level monitoring hooks**. While our artifacts operate in user space, their behavior is scrutinized in kernel space, where EDRs reside. Thus, the user/kernel separation is not merely a theoretical construct but the foundation of how detection and evasion mechanisms interact.

Windows Technical Substrate

Having established the user/kernel boundary, we now describe two Windows-specific foundations that subsequent chapters rely on: the Portable Executable binary format and the runtime primitives used for code injection. Both are needed to understand how loaders work and how EDRs monitor them.

The Portable Executable (PE) Format

The **Portable Executable (PE)** format is the native binary format for executables, DLLs, and drivers on Windows. Because the binary mutation layer of this thesis directly manipulates PE metadata, import tables, and section contents, features shown to carry significant discriminative power for malware classification [45], a working understanding of PE structure is a prerequisite for the design and implementation chapters that follow.

A PE file is organized as a sequence of nested headers followed by a section table and the sections themselves [56, 59]. The **DOS header** (beginning with the magic bytes MZ) provides backward compatibility and contains `e_lfanew`, a pointer to the PE signature. After the four-byte PE signature (`PE\0\0`), the **COFF header** records the target machine type, the number of sections, a timestamp, and the size of the optional header. The **Optional header** (mandatory for executables despite its name) specifies the entry point address, the preferred image base, section and file alignment values, and, critically, the *Data Directory* array, an indexed table of RVA/size pairs pointing to structures such as the Import Directory, Export Directory, Resource Directory, and Debug Directory. Finally, the **section table** describes each section's name, virtual size and address, raw data offset, and permission flags (`.text` for code, `.rdata` for read-only data, `.data` for mutable data, `.rsrc` for resources). A detailed field-level description of each header is provided in Appendix [PE File Format Details](#).

Import Address Table (IAT). At load time, the Windows loader walks the *Import Directory Table* to resolve the DLLs and functions a binary depends on [56]. For each imported DLL, the loader reads the *Import Lookup Table* (ILT), resolves each function name (or ordinal) via the corresponding Hint/Name entry, and writes the resulting virtual address into the *Import Address Table* (IAT). Code generated by the compiler calls imported functions through IAT pointers. EDRs may intercept these calls through several mechanisms, most commonly inline hooks in `ntdll.dll`, but also user-mode API shims or, occasionally, IAT patching [33]. From a static-analysis perspective, minimal import tables combined with dynamic API resolution (e.g., importing only `LoadLibraryA` and `GetProcAddress`) are commonly observed in malware and may raise heuristic suspicion [5].

Rich header. Between the DOS stub and the PE signature lies an undocumented structure known as the **Rich header**. Produced by the Microsoft Visual C++ linker, it records a set of *product ID / build count* pairs, each identifying a specific MSVC toolchain component and how many object files it contributed. The entire block is XOR-encoded with a checksum key derived from the DOS header content and the compiler records themselves. Rich header metadata is frequently used in malware clustering and toolchain fingerprinting; anomalies, such as a binary that claims to be MSVC-built but lacks a Rich header, or one whose Rich header records are inconsistent with its section layout, serve as heuristic indicators of tampering or non-standard build processes [77]. Lepori [48] confirmed that compiler provenance is a practical detection signal: executables compiled with MinGW are flagged by multiple EDR products based solely on metadata differences from MSVC-compiled binaries. The encoding algorithm and

field-level interpretation are detailed in Appendix [PE File Format Details](#).

Resources and version information. The `.rsrc` section contains structured resource entries, most notably `VS_VERSIONINFO` (product name, company name, file version), application manifests (which may declare UAC elevation requirements), and icons. Metadata consistency is used as a heuristic signal in static malware analysis: legitimate software typically includes coherent version information and a manifest, so their absence or inconsistency may raise flags in automated scanning pipelines. Because resources are easily modified without affecting code execution, they represent a low-cost mutation surface for shifting static detection scores.

The binary mutation layer of this thesis directly manipulates Rich headers, import tables, section contents, and PE metadata, understanding these structures is therefore necessary to appreciate why such mutations shift detection outcomes.

Windows Primitives for Code Execution

The user/kernel separation introduced in [fig. 1](#) governs *what* a process may do; the primitives described here govern *how* injected code obtains executable memory at runtime. Understanding these primitives is essential because the experimental pipeline rehearses each step with benign data to condition the EDR baseline before committing the real payload.

Virtual memory management. Two Win32 functions form the foundation of runtime memory control. `VirtualAlloc` reserves or commits a region of pages in the calling process's address space, returning a pointer to the new region. `VirtualProtect` changes the protection attributes of committed pages. The key permission classes are **RW** (read-write, suitable for data buffers), **RX** (read-execute, suitable for code), and **RWX** (read-write-execute, which allows simultaneous writing and execution). RWX allocations and RW→RX transitions are commonly monitored indicators of code injection, alongside thread start-address validation, cross-process memory writes, and section mapping anomalies [33, 57]. Lepori [48] showed that direct RW→RX transitions are specifically flagged by Elastic Defend, and that inserting an intermediate read-only step (RW→R→RX) reduces detection, a finding that motivated the `virtualprotect` gene variants in Section [Modular Gene Model](#). Building on the kernel-level API routing described earlier, these functions ultimately pass through `ntdll.dll` into the kernel via `NtAllocateVirtualMemory` and `NtProtectVirtualMemory`, respectively.

Shellcode injection pattern. The canonical injection sequence proceeds in four stages: (1) allocate a region with RW permissions, (2) write the payload into the region, (3) change the region's protection to RX, and (4) transfer control to the region (e.g., via a function pointer cast or

`CreateThread`). Each stage produces distinct telemetry, allocation size, write target, protection-flag change, and thread creation, that EDRs correlate into a behavioral signature. This is the sequence that the *deconditioner* concept (Section [Build System Design: Modular Templates as a Controlled Search Space](#)) rehearses with benign data: the hypothesis, first articulated by Cailleret [16], is that by executing the same alloc→write→protect→execute pattern with innocuous content, the pipeline may influence the EDR's behavioral baseline so that the final, real execution is less distinguishable from previously observed benign activity.

PEB walking. The **Process Environment Block** (PEB) is a per-process structure maintained by the Windows loader. Among other fields, it contains a pointer to the `PEB_LDR_DATA` structure, which in turn holds a doubly-linked list of `LDR_DATA_TABLE_ENTRY` records, one for every module (DLL) mapped into the process. By traversing this in-memory list, code can resolve the base address of any loaded DLL and, from there, walk its export table to obtain function pointers, all without appearing in the binary's Import Address Table (Section [The Portable Executable \(PE\) Format](#)). The result is a binary whose static imports appear minimal and benign, while full API access is obtained dynamically at runtime. This technique is relevant because PEB-walk-based carriers produce a fundamentally different static profile than IAT-based ones, a distinction the mutation engine must account for.

Direct syscalls. As noted in the discussion of kernel-mode EDR components, user-mode API hooks in `ntdll.dll` are a primary interception point for EDRs. *Direct syscalls* bypass this layer entirely: the program reads the syscall number from the `ntdll.dll` stub at runtime and issues the `syscall` instruction directly, transitioning to kernel mode without executing any hooked code in user space. While this defeats user-mode hooks, kernel-level telemetry (ETW providers, kernel callbacks) still observes the resulting system call. Direct syscalls are relevant both as an evasion technique against user-mode monitoring and as an implementation detail of the experimental runtime described in Section [Worker Agent Design: Robust Execution as a Ten-Phase Pipeline](#).

Antivirus and Classical Detection Approaches

Before the emergence of modern Endpoint Detection and Response platforms, **antivirus (AV)** software was the primary line of defense on Windows endpoints. Traditional AV solutions focused mainly on **static analysis** of binaries, supplemented by simple heuristics. The underlying principle was to identify malicious code before execution by inspecting the binary on disk.

Signature-Based Detection

The most classical method relies on **signatures**. Each known malware sample is reduced to a compact fingerprint (e.g., a hash value or a byte-pattern signature), which

is stored in the AV database. When a new binary is encountered, the AV computes its fingerprint and compares it against the database:

- **Hash signatures:** Fast lookups of exact file hashes (MD5, SHA-1, SHA-256) for known malware.
- **Pattern signatures:** Matching specific byte sequences or opcode patterns characteristic of malware families.

This approach is highly effective against known malware but fails against new or obfuscated variants [58], leading to the so-called “zero-day” problem.

Beyond simple hash lookups, modern pattern signatures decompose each detection rule into weighted *sub-rules*: a binary is flagged when the cumulative weight of matched sub-rules meets or exceeds a score threshold, allowing detection to tolerate partial obfuscation [23]. Signatures further employ *wildcards* that accommodate minor byte-level variations across samples of the same family [61]. A detailed treatment of sub-rule weighting, wildcard encoding, and a representative YARA-notation [75] example is provided in Appendix [Pattern Signature Internals](#).

Heuristic Analysis

To overcome strict dependence on signatures, AV solutions also employ **heuristic analysis**. Here, simple rules are applied to identify suspicious characteristics, such as:

- **File metadata:** Is the binary signed with a trusted certificate? Does its filename or version information resemble known system components?
- **Embedded strings and resources:** Are there suspicious hardcoded URLs, commands, or function names present?
- **Unusual headers or sections:** Does the binary have malformed PE headers, compressed/encrypted sections, or abnormal entry points [45]?

These heuristics allow AVs to flag potentially malicious binaries even if they have no exact signature match. Additionally, some antivirus engines employ lightweight **emulation**: the suspect binary is executed inside an instrumented virtual CPU within the AV engine (e.g., Microsoft Defender’s `mpengine.dll` emulator) to observe its unpacking or decryption routines before allowing real execution [15]. This technique is particularly relevant because the anti-emulation gene in our framework (Section [Build System Design: Modular Templates as a Controlled Search Space](#)) specifically targets this detection layer.

Modern Extensions

In recent years, classical AVs have incorporated more advanced techniques to remain effective against modern threats:

- **Behavioral monitoring:** Instead of relying solely on static file inspection, AVs now observe runtime behavior (e.g., process injection, registry persistence, suspicious API call sequences) like Avast’s Behavior Shield, Bitdefender’s Advanced Threat Control, or Kaspersky’s System Watcher [67, 11, 40].
- **Cloud-assisted scanning:** Suspect binaries are hashed or uploaded to a vendor backend, where large-scale databases and machine learning models analyze them with richer context than what is possible locally. Vendors such as Avast CyberCapture or ESET LiveGrid [68, 28] use those techniques.
- **Machine learning classifiers:** Features such as opcode frequency, API call graphs, and entropy of file sections are extracted and fed into ML models that generalize detection beyond simple heuristics as described by Bitdefender, Avast, and Kaspersky [10, 68, 41].
- **N-gram behavioral models:** An *N-gram* is a contiguous sequence of *N* API calls (or system events) extracted from an execution trace. By building frequency profiles of these short call sequences, defenders obtain compact *behavioral fingerprints* that capture not just which APIs a process invokes but in what order. Deviations from known-benign N-gram distributions flag suspicious processes even when individual calls appear harmless [44, 4].

Despite these improvements, classical AV solutions remain primarily **file-centric**. Their strength lies in preventing execution of known malware families, but they lack the deep, continuous monitoring capabilities that characterize modern EDR systems.

While antivirus solutions remain effective at preventing the execution of known threats through signatures, heuristics, and basic behavioral checks, their visibility is largely limited to the binary itself and its immediate execution context. They excel at identifying what a file *is*, but struggle to monitor what a process *does* once running. This limitation has driven the evolution toward **Endpoint Detection and Response (EDR)**, which extends protection beyond static file analysis to continuous runtime monitoring, deeper operating system integration, and richer behavioral correlation across endpoints.

Endpoint Detection and Response (EDR)

Endpoint Detection and Response systems are modern security solutions that extend beyond classical antivirus approaches [6, 43]. Their design goal is not only to block known malware but also to provide continuous monitoring, behavioral analysis, and rapid response to novel or obfuscated threats, as confirmed by Lepori [48], whose experiments showed that the same obfuscation technique may evade one EDR product while being

detected by another, reflecting vendor-specific behavioral rule sets. Cailleret [17] demonstrated this opacity from the defender's perspective: by introspecting MDE's own ETW events, the EDRi framework revealed that the same executable triggers different analysis stages (static lookup, emulation, memory scan, behavioural correlation) depending on whether it has been seen before, making detection outcomes path-dependent. To achieve this, EDRs embed themselves across multiple layers of the operating system (Windows), combining both **user-mode agents** and **kernel-mode drivers**.

EDR integration leverages the separation of user and kernel space (see fig. 1). While user processes are limited to interacting through high-level APIs, kernel components can observe and mediate low-level system activity. This asymmetry is key to how EDRs detect behaviors such as code injection, driver loading, and suspicious handle creation. As shown conceptually in fig. 2, the architecture of an EDR agent typically follows a modular design composed of several coordinated components: a *kernel-mode driver* acting as the privileged sensor, a *user-mode agent service* aggregating and correlating data, a *hooking DLL* for API interception in processes, and supporting elements such as *static scanners*, *network filters*, and *filesystem minifilters*.

User-Mode Components

In user space, EDRs typically include service processes and lightweight agents that perform:

- **Telemetry collection** via high-level APIs (process creation chains, registry writes, network connections).
- **Content inspection** through mechanisms like the Antimalware Scan Interface (AMSI), which allows dynamic scanning of scripts and memory buffers before execution.
- **Event tracing (ETW)**: EDRs subscribe as consumers to ETW sessions for continuous telemetry (Section [Event Tracing for Windows \(ETW\)](#)).
- **Communication with the cloud backend**, uploading suspicious samples and behavioral traces for correlation and classification across endpoints.

Although user-mode collection mechanisms such as API hooking or AMSI inspection are valuable, sophisticated malware can bypass them (e.g., by directly invoking system calls). In contrast, Event Tracing for Windows bridges both worlds: its kernel providers emit high-fidelity events that are asynchronously delivered to user-mode consumers, making ETW a more robust telemetry source than purely user-mode hooks. Nevertheless, because ETW is primarily observational, EDR vendors still rely on kernel drivers when they require real-time interception, blocking, or enforcement. The next section describes these kernel-mode components.

Kernel-Mode Components

Kernel integration is fundamental for modern EDR solutions. By loading signed drivers into kernel space, EDRs gain privileged access to core OS structures:

- **System call monitoring**, by analyzing arguments and return values at the boundary between user space and kernel space (see fig. 3).
- **Process, thread, and image load notifications** using callbacks such as `PsSetCreateProcessNotifyRoutineEx` and `PsSetLoadImageNotifyRoutine`.
- **Object access callbacks**, via `ObRegisterCallbacks`, to detect unauthorized handle creation or sensitive memory access.
- **Mini-filter drivers** for monitoring file and registry operations at the I/O layer. These drivers register with the Filter Manager (`FltMgr.sys`) to intercept I/O Request Packets (**IRPs**) before they reach the filesystem or registry, allowing the EDR to inspect, log, or block operations such as file creation, modification, and persistence attempts in critical paths like AppData or `Run` keys.
- **Network filter driver**, built atop the Windows Filtering Platform (WFP), capable of monitoring or blocking suspicious outbound connections (for instance, beaconing C2 traffic).
- Optionally, an **Early Launch Anti-Malware (ELAM)** driver and service pair, ensuring the agent's defensive components initialize before untrusted drivers at boot. While ELAM is relevant to comprehensive EDR deployments, it does not directly affect the mutation surface explored in this thesis. ELAM components can also receive *Secure ETW* events from protected providers, expanding the agent's visibility during early system startup.

While most of these events (*process creation*, *image loading*, *registry access*, *file I/O*) are also exposed through kernel ETW providers, ETW primarily serves as an observational telemetry channel. In contrast, direct callbacks and mini-filter drivers provide real-time interception and enforcement capabilities. Modern EDRs typically combine both approaches: ETW for scalable event collection and correlation, and kernel drivers for immediate prevention and enforcement of integrity.

Kernel drivers, therefore, provide the visibility required to correlate user-level requests with system-level activity, bridging the blind spots of user-mode monitoring.

Event Tracing for Windows (ETW)

ETW is a cornerstone of modern EDR monitoring. ETW provides a high-performance, low-overhead logging

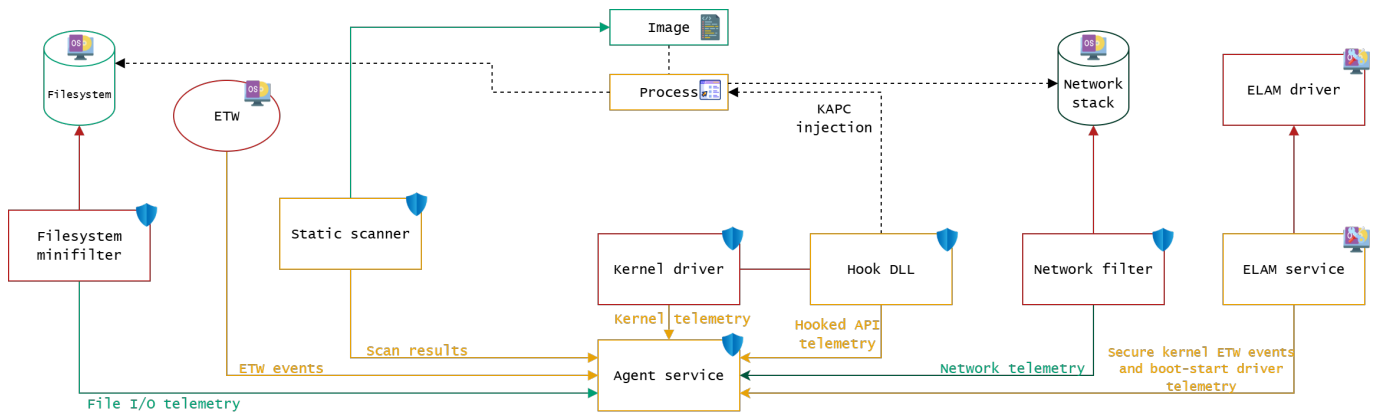


Figure 2: Conceptual EDR agent architecture - adapted and extended from [33].

infrastructure embedded in the Windows kernel. Through ETW, thousands of events can be recorded per second, spanning from kernel-level activity to application-defined logging in user space (see fig. 4).

EDRs rely on ETW to:

- Collect fine-grained telemetry about system behavior without requiring invasive hooks for every event.
- Subscribe to providers such as Microsoft-Windows-Kernel-Process, Microsoft-Windows-Kernel-Audit-API-Calls, or the security-focused Microsoft-Windows-Threat-Intelligence provider to track sensitive actions like process injections, module loads, and memory protection changes.
- Correlate ETW events with other telemetry sources, such as AMSI scans or mini-filter callbacks, to construct a full behavioral trace of an endpoint.

Within the EDR architecture (fig. 2), the **Agent service** operates as an ETW consumer, subscribing to kernel and user providers through user-mode ETW APIs. Although ETW itself is kernel-implemented, event delivery occurs through shared buffers read by this user-mode consumer, integrating kernel and user telemetry streams into the same analytical pipeline.

Importantly, not all ETW events offer the same tamper resistance. Events logged directly by kernel providers (e.g., process creation, memory protection changes) are recorded on the kernel side of the user-to-kernel security boundary, making them significantly harder to suppress or forge [73]. In contrast, user-mode ETW providers, where the logging call originates inside the traced process itself, remain vulnerable to tampering, since the adversary controls the process's address space. Leading EDRs such as Elastic Endpoint and Microsoft Defender for Endpoint therefore prefer kernel ETW over user-mode `ntdll` hooks [73]. This reliance became especially pronounced after Kernel Patch Protection (KPP) removed the ability for third-party drivers to inline-hook kernel functions

(Appendix [Kernel Patch Protection and the End of Third-Party Kernel Hooking](#)), making asynchronous kernel ETW the primary mechanism for after-the-fact visibility into kernel actions.

However, malware can still attempt to tamper with ETW consumer processes or spoof client-supplied data fields, making ETW both a detection surface and an evasion target. In this thesis, ETW is particularly relevant as it provides the observable ground truth, collected via RedEDR [25], against which binary mutations can be evaluated.

Detection Pipeline and Implications

The collected telemetry from user and kernel components is processed through a multi-stage detection pipeline:

1. Local heuristics and rule-based detections for speed-sensitive responses.
2. Correlation of kernel callbacks, ETW streams, and AMSI scans into process trees and behavior graphs.
3. Cloud-based enrichment, where suspicious artifacts are analyzed with machine learning and cross-endpoint correlation.
4. Response actions, such as process termination, memory scanning, or endpoint isolation.

For this thesis, ETWs and kernel callbacks provide the most reliable telemetry for modern EDRs and therefore ETWs will be the primary data sources driving the experimental feedback loop.

Detection trade-offs: the “Bubbles of Bane”

The detection surface of modern endpoint security can be cleanly described as three overlapping domains: file (static) scanning, memory scanning, and telemetry / behaviour analysis. Each domain provides a different strength and produces different signals for defenders. Figure 5 visualises these domains as the “Bubbles of Bane”: signature-based file scanning (classic AV),

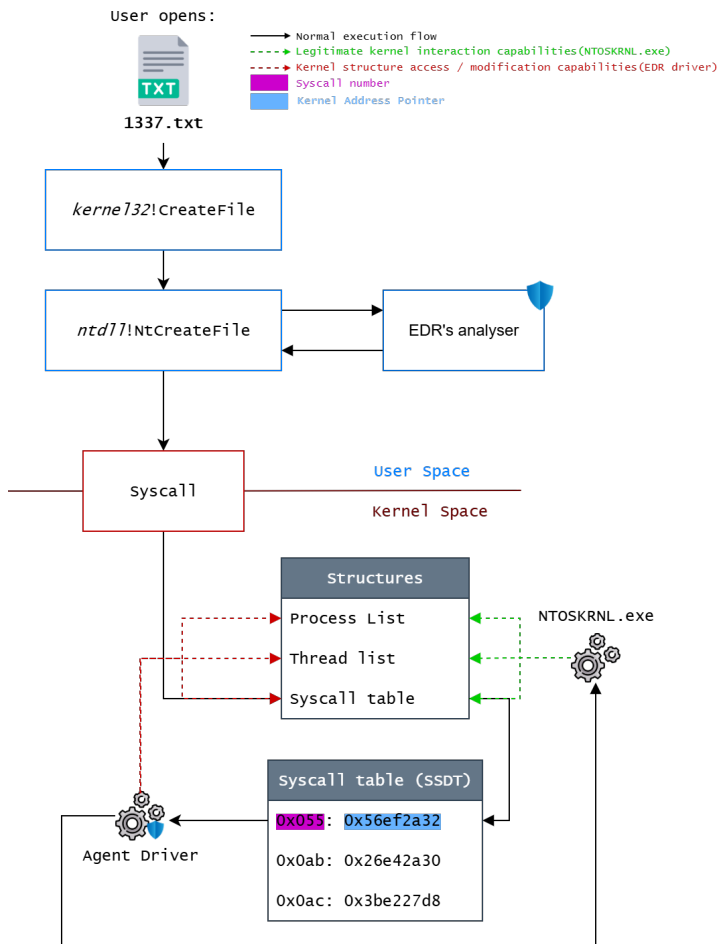


Figure 3: Opening a File under Conceptual EDR - adapted and extended from [19].

memory scanners that inspect process address spaces, and telemetry-driven behavioural analysis performed by EDRs.

The practical consequence of this decomposition is a set of trade-offs for payload and loader design. Strategies that reduce visibility in one bubble typically increase exposure in another. For example, encrypting a payload on disk (to evade file scanning) forces the payload to be decrypted in memory before execution (increasing the risk of memory-scanner detection). Conversely, techniques that try to eliminate in-memory artefacts (for instance by encrypting memory while sleeping) tend to produce behavioural telemetry (timers, protection flips, repeated allocations) that EDRs can correlate and act upon. The net effect is a balancing problem: avoiding detection requires carefully trading off static footprint, memory artefacts, and runtime telemetry.

This work takes the “Bubbles of Bane” model as a guiding design principle. Concretely, the experimental pipeline described in this thesis will aim to:

- prioritize reducing high-signal runtime artefacts (minimizing write-then-execute and RWX transitions where possible) while
- limiting static strings and exports visible in file-backed

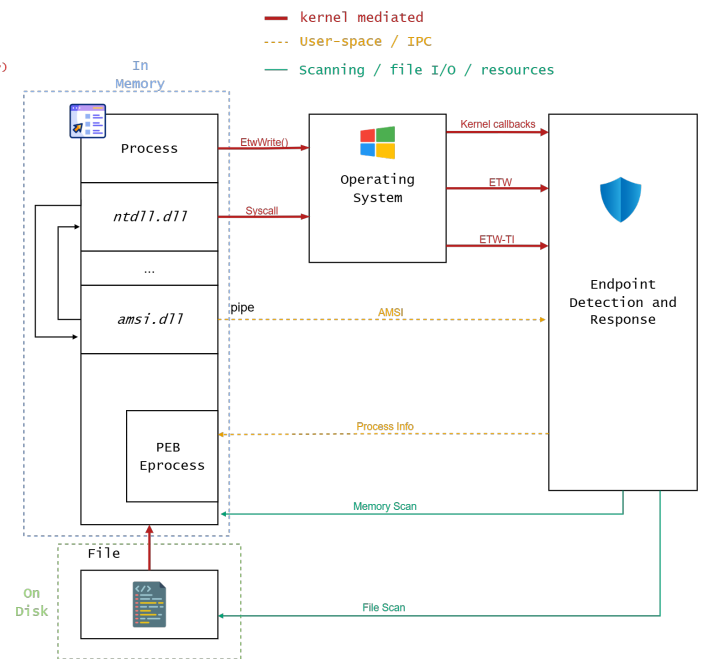


Figure 4: Conceptual EDR Query Process Information - adapted and extended from [24].

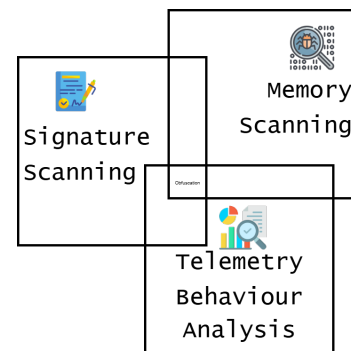


Figure 5: Bubbles of Banes - from [24].

sections, and

- introducing targeted obfuscation of the runtime payload to make straightforward memory-scanning signatures less effective.

To that end, the remainder of the thesis will evaluate specific obfuscation and mutation strategies against all three detection domains (file, memory, telemetry). The concrete loader architecture realizing these trade-offs is presented in Section **Build System Design: Modular Templates as a Controlled Search Space**.

Malware Obfuscation Fundamentals

Malware obfuscation is the deliberate transformation of malicious code to resist detection while preserving its runtime semantics. The techniques span data-flow and control-flow manipulation and have evolved from simple encryption to fully metamorphic engines [14, 58]. Understanding these fundamentals is necessary because

AutoMutate++ applies mutations at multiple compilation layers that directly parallel these historical obfuscation strategies.

Data-flow obfuscation. Borello and Mé [14] identify four canonical data-flow transformations: (i) *instruction substitution*, which replaces instruction blocks with semantically equivalent alternatives (e.g., `ADD Reg, 0` $\equiv$ `NOP`); (ii) *instruction permutation*, which reorders independent instructions while preserving data dependencies; (iii) *variable/register substitution*, which exchanges registers or switches variable scoping to alter syntactic signatures; and (iv) *dead-code insertion*, which injects semantically inert instructions or unreachable blocks to inflate and diversify the binary.

Control-flow obfuscation. Three principal control-flow techniques appear throughout the literature [14, 42]. *Unconditional jumps* split sequential code into blocks connected by `jmp` instructions, fragmenting the original control-flow graph. *Opaque predicates* introduce conditional branches whose outcome is known to the obfuscator but computationally hard for any deobfuscator to resolve. *Control-flow flattening* (CFF) replaces structured control flow with a dispatch loop governed by a state variable, collapsing the entire function into a single switch-like construct. CFF is particularly effective against pattern-based detectors because it destroys the hierarchical nesting that signatures typically rely on.

Historical progression. The evolution of evasive malware follows four stages of increasing sophistication [48, 58]. First, *encoding and packing* encrypts the malicious payload so that static analysis reveals only the encrypted blob; O’Kane et al. report that 48.92 % of malware samples observed in the wild are packed [58]. Second, *polymorphic* malware changes its encryption key and scheme with every replication, yet the decryption routine itself remains a detectable constant. Third, *metamorphic* malware mutates the entire executable on every generation, defeating all static signatures. Fourth, *stealth* techniques actively modify the detection software or its data structures to suppress alerts.

Polymorphic engines. A polymorphic engine follows a seven-step cycle [58]: (1) decrypt the virus body, (2) execute the payload, (3) generate a new encryption key, (4) re-encrypt the body with the new key, (5) produce a new decryption routine, (6) combine the new routine with the encrypted body, and (7) replicate. The critical weakness is that the decryption routine, despite syntactic variation, maintains structural patterns detectable by emulation-based scanners that can execute the stub and compare the decrypted body against known signatures.

Metamorphic architecture. Borello and Mé [14] describe a five-component metamorphic architecture: (1) a *disassembler* that extracts the virus’s own code semantics, (2) an *IR builder* that creates an intermediate model,

(3) an *obfuscation transformer* that applies one or more data-flow and control-flow transforms, (4) a *code generator* that produces new machine code, and (5) an *assembler/linker* that creates the final executable. Each generation is syntactically different but semantically equivalent. Table 10 summarises the techniques employed by well-known metamorphic viruses.

Table 10: Obfuscation techniques used by notable metamorphic viruses (adapted from [14]).

Virus	Instr. Sub.	Instr. Perm.	Var. Sub.	Dead Code	Ctrl. Flow
Evol (2000)	–	–	–	–	✓
Regswap (2000)	–	–	✓	–	–
ZPerm (2000)	–	✓	–	✓	✓
ZMist (2001)	–	✓	✓	✓	✓
MetaPHOR (2001)	✓	✓	✓	✓	–

LLVM IR obfuscation. Kastler et al. [42] demonstrate that LLVM Intermediate Representation enables compiler-level obfuscation: CFF, instruction substitution, dead-code insertion, and polymorphic code generation can all be implemented as optimization passes operating on IR before back-end code emission. Because these transformations are applied before register allocation and instruction scheduling, they produce binaries that are structurally diverse yet compiled through a legitimate toolchain. This observation directly justifies the IR mutation layer described in subsequent chapters.

Shellcode loader anatomy. Rutishauser [63] identifies a three-phase structure common to shellcode loaders: allocation of executable memory, copying the shellcode into the allocated region, and transfer of control. The runtime injection primitives underlying these loaders, `VirtualAlloc`, `VirtualProtect`, and related APIs, were introduced in Section [Windows Primitives for Code Execution](#). Antivirus engines primarily detect the file on disk (static analysis), whereas EDRs target the behavioral telemetry generated at runtime.

Theoretical Limits of Malware Detection

Before evaluating detection systems empirically, it is instructive to examine the fundamental computational limits that govern malware identification. A series of results from theoretical computer science establishes that no detection method can be simultaneously efficient, complete, and sound, motivating practical evaluation frameworks that probe the gap between theory and deployed capability.

Undecidability of general virus detection. Cohen [21] formally defined computer viruses and proved that determining whether an arbitrary program is viral is undecidable, by analogy with the Halting Problem. The proof constructs a program D that, given any program P , decides whether P

is a virus; a self-referential argument shows that D must simultaneously classify P as viral and non-viral, yielding a contradiction. The result implies that no single algorithm can correctly classify all programs. However, practical detection systems do not attempt to solve the general case; they target specific, bounded-length virus families.

NP-completeness of bounded-length detection. Spinellis [69] proves that the reliable identification of bounded-length mutating viruses is NP-complete. The proof proceeds by reduction from SAT: a virus is constructed as a triple (f, s, c) where f is the replication function, s is a Boolean satisfiability flag, and c is a candidate-assignment counter. The virus iterates through all 2^n possible assignments; a virus detector that answers correctly in polynomial time would therefore solve SAT. The practical implication is that “good-enough” detectors that tolerate over-reporting can operate in linear time at the cost of false positives, a trade-off Spinellis considers acceptable because “it is quite rare for a non-viral program to match the detection pattern of a known virus” [69].

Extension to metamorphic viruses. Borello and Mé [14] extend the NP-completeness result to metamorphic viruses that employ control-flow obfuscation. The proof uses a reduction from 3-SAT: a program P is partitioned into k basic blocks, a permutation σ is applied, and opaque predicates are inserted to enforce the original execution order. A transformation T is a k -obfuscator if and only if the correct execution order is preserved under σ . The construction is polynomial in the size of the Boolean formula. The authors build a practical obfuscator using program scissoring, dead-code blocks that contain fragments of real program code, and a tuneable obfuscation parameter K that controls the degree of block interleaving.

Semantics-aware detection. Dalla Preda et al. [22] propose the A_{MD} algorithm, which matches instruction semantics, templates with polymorphic variables and symbolic constants, rather than syntactic byte patterns. Templates preserve def-use chains and permit arbitrary register bindings, making the detector resilient to register renaming, code transposition, and dead-code insertion. The *Template Matching Problem* itself is undecidable in the general case; consequently, A_{MD} is *sound* (every match is a true positive) but *incomplete* (some variants may be missed). Experimental evaluation shows 100 % detection of the Netsky and Beagle families using a single template per family, compared with 13 separate signatures required by McAfee, and 0 % false positives on a corpus of 2,000 benign programs. The principal limitation is *equivalent-functionality substitution*: replacing an instruction sequence with a semantically identical but structurally unrelated one defeats template matching.

The hierarchy presented in Table 11 confirms that no detection methodology can be simultaneously efficient and perfectly reliable. Practical detectors must accept

Table 11: Hierarchy of impossibility results for malware detection.

Problem	Result	Reference
General virus detection	Undecidable	Cohen [21]
Bounded polymorphic identification	NP-complete	Spinellis [69]
Metamorphic detection (CF obf.)	NP-complete	Borello [14]
Template Matching Problem	Undecidable	Dalla Preda [22]
Practical semantic detection	Sound, incomplete	Dalla Preda [22]

either false positives, false negatives, or exponential computational cost. This theoretical ceiling motivates AutoMutate++ as an empirical evaluation framework: rather than attempting perfect detection, the framework systematically probes which mutations real EDR products actually detect, measuring the practical gap between theoretical limits and deployed capability.

Fuzzers and their role in security testing

A **fuzzer** is an automated tool that repeatedly generates candidate inputs (or program variants here) and observes the target system’s reactions to find interesting behaviours, crashes, or policy violations. In security research fuzzers are typically used to explore large, complex search spaces by applying random or guided mutations and using an oracle (crash, coverage, or other feedback) to steer future generations. Fuzzing excels where the input space is combinatorial and where small changes can provoke different runtime behaviour, precisely the setting encountered when exploring how endpoint protections react to binary mutations, loader variants, and runtime/source code transformations.

This thesis adapts the fuzzer paradigm to a domain-specific “evasion loop,” treating the loader and payload as the input to be mutated, the compilation and mutation process as the generator, and the EDR’s telemetry and enforcement outcome as the oracle. The concrete architecture of this evasion loop, including how telemetry-derived triage tokens guide subsequent mutations, is presented in Section [Evasion Loop Architecture: Fuzzing for Detection Bypass](#).

Programmatic Code Transformation

To automate the generation of evasive artifacts, one must define the “genotype” of the software, the representation upon which mutations are applied. The choice of representation dictates the granularity, safety, and semantic depth of possible transformations.

Levels of Transformation

Code transformation can occur at three primary levels: source code (AST), intermediate representation (IR), and machine code (binary). To situate these levels, consider the full compilation pipeline for a C program targeting Windows: source code is first *parsed* into an Abstract Syntax Tree, then *lowered* into LLVM Intermediate Representation (.ll/.bc), *optimized* by a sequence of LLVM passes, *code-generated* into a native object file (.o/.obj), and finally *linked* into a Portable Executable. The three mutation layers of this thesis each intervene at a different stage of this pipeline: AST mutations operate before lowering, IR mutations operate after lowering but before code generation, and binary mutations operate on the linked PE. This pipeline perspective clarifies both the expressiveness and the limitations of each layer.

Binary Rewriting Early metamorphic engines operated directly on assembly code. While this allows for operation on compiled binaries without source access, it is inherently fragile. Displacing a single instruction can invalidate jump offsets and break execution flow. Furthermore, binary-level transformations are often limited to local peephole optimizations or simple instruction substitution, which modern heuristic scanners can easily normalize [27].

Intermediate Representation (LLVM IR) The LLVM Intermediate Representation (IR) is a low-level, RISC-like language used by the compiler backend. It provides a middle ground, allowing for control flow flattening and basic block manipulation independent of the source language. However, IR lowers high-level constructs; for instance, `for` and `while` loops are converted into generic conditional branches and labels. Reconstructing high-level logic from IR to perform complex semantic mutations (such as loop re-rolling or variable type masquerading) is computationally expensive and prone to error. Additionally, obfuscations inserted at the IR level are often viewed as "inefficiencies" by the subsequent compiler optimization passes (e.g., `-O2`), which may strip them out before binary emission [42]. LLVM also provides **SanitizerCoverage**, a compiler pass that inserts lightweight callbacks (e.g., `__sanitizer_cov_trace_pc_guard`) at basic-block or function-entry boundaries, yielding code-coverage data without manual instrumentation. This mechanism is leveraged by the instrumentation design (Section [Instrumentation Design: Evidence That Survives Process Death](#)) for runtime coverage tracking of mutated artifacts.

Abstract Syntax Tree (AST) The Abstract Syntax Tree (AST) represents the hierarchical structure of the source code (see fig. 6). Unlike IR, the AST preserves the programmer's intent, including type information, loop structures, and variable scopes. Frameworks such as Clang LibTooling allow for "Source-to-Source" transformation. An alternative is **Tree-sitter**, an incremental, error-tolerant parser that produces a *concrete syntax tree*

(CST), a tree that preserves all source tokens including whitespace and punctuation. Unlike Clang's AST, which requires a full compilation environment (headers, build flags, standard-library paths), Tree-sitter operates on raw source text with byte-offset-based editing: each node records its start and end byte positions, enabling precise surgical modifications without invoking a compiler front-end. The trade-off is that transformations must account for position shifts, inserting text at one location invalidates byte offsets of all subsequent nodes, requiring either bottom-up editing or an offset-adjustment pass. This thesis uses Tree-sitter for its AST-level mutations due to its language-agnostic design goal for future ability and its tolerance of incomplete or non-compiling source fragments, which is particularly valuable in a fuzzing context where intermediate mutations may temporarily break compilation.

For evolutionary malware, the AST offers a superior genotype for several reasons:

1. **Semantic Richness:** Transformations can leverage type information (e.g., changing a `struct` layout) which is lost in lower representations.
2. **Correctness:** Modifications applied at the AST level are re-compiled by the standard toolchain, ensuring that register allocation and stack management are handled correctly by the compiler backend.
3. **Resilience:** Structural changes at the source level (such as control flow restructuring) are integral to the program logic and survive compiler optimization.

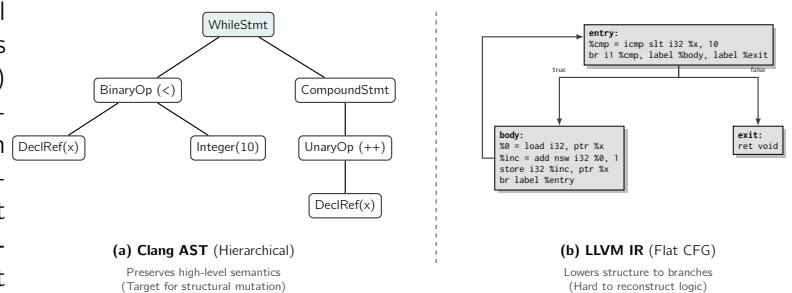


Figure 6: Structural Comparison of Code Representations.

The AST (a) retains the explicit notion of a loop and variable types, facilitating complex logic swapping. The IR (b) flattens this into a generic Control Flow Graph (CFG) of basic blocks and jumps, obscuring the original intent.

Table 16 in the design chapter compares these three substrates along the dimensions of abstraction, persistence through optimization, and targeted detection surface.

Evolutionary Search and Genetic Algorithms

Generating a variant that is simply "different" is insufficient; the variant must be *evasive*. This transforms the malware generation process into an optimization problem.

Table 12: Comparison of Mutation Substrates

Feature	AST (Clang)	LLVM IR	Binary
Abstraction	High (Source)	Medium (SSA)	Low (Machine)
Context	Retains types, loops, names	Lost high-level structure	Raw instructions
Safety	High (Compiler handles backend)	Moderate (Invalid IR possible)	Low (Fragile offsets)
Obfuscation	Deep structural changes	Control flow flattening	Instruction substitution

By treating the source code as the *genotype* (the representation manipulated by mutations) and the runtime telemetry plus detection verdict as the *phenotype* (the observable outcome), Genetic Algorithms (GA) can be employed to evolve the artifact. The specific genotype–phenotype mapping, multi-objective fitness formulation, and selector strategies used in this thesis are design decisions presented in Section [Token-Driven Feedback Loop](#).

Exploration vs. Exploitation

Evolutionary search addresses *how* to evolve candidate solutions, but a prior question is *when* to explore untested regions of the search space versus exploit the best-known variants. In a system with a limited experimental budget, each trial requires building, deploying, and observing a binary, repeating the current best variant (pure exploitation) risks missing superior alternatives, while always trying random variants (pure exploration) wastes budget on uninformative experiments. The *epsilon-greedy* strategy, one of the simplest action-selection strategies from multi-armed bandit theory [70, Chapter 2], offers a straightforward resolution: with probability ϵ the system selects a random action (explore), and with probability $1 - \epsilon$ it selects the action with the highest estimated reward (exploit). Despite its simplicity, epsilon-greedy provides a tunable and well-understood baseline that is effective when the action space is moderate and, when paired with recency-weighted value estimates, well-suited to nonstationary settings, both properties of the mutation-testing loop studied here. Section [Token-Driven Feedback Loop](#) describes how the controller applies this strategy in its selector components.

Differential Analysis and Causality

A critical challenge in automated evasion is the "Assignment of Blame." When an EDR blocks an artifact, the verdict is often opaque (e.g., "Malicious Activity Detected"). Without granular feedback, a fuzzer operates blindly. Differential Analysis aims to convert this binary verdict into causal features.

Delta Debugging

Originally proposed by Zeller for software debugging [37], Delta Debugging (*ddmin*) acts as a scientific method to isolate failure-inducing inputs. In the context of evasion, we apply this to identifying "detection-inducing" features.

By submitting closely related variants, one detected

and one not, and comparing their execution traces, we can attempt to isolate the specific API call, basic block, or argument that triggered the alert. This is formally described as:

$$\Delta = \text{Trace}_{\text{detected}} - \text{Trace}_{\text{benign}}$$

Trace Refinement

Modern EDRs often trigger on sequences of events rather than single atomic actions (see Section ["Detection trade-offs: the 'Bubbles of Bane'"](#)). Therefore, differential analysis must compare multiple layers of telemetry:

- **Set Difference:** Identifying API calls present in the detected trace but absent in the benign one.
- **Sequence Divergence:** Identifying where the execution path diverges (e.g., using execution traces to find the specific `if` branch taken before termination).
- **Argument Analysis:** Detecting changes in arguments (e.g., `RWX` vs `RW` memory protection flags) that correlate with detection.

This feedback loop closes the circuit: the Differential Analysis engine informs the Mutation engine *what* to change (the "hot" feature), and the Evolutionary logic decides *how* to change it (the specific AST transformation). These refinement strategies are formalized through the triage token taxonomy (Section [Token-Driven Feedback Loop](#)): event tokens capture set differences, sequence tokens encode ordering divergence, and argument-class tokens represent coarse parameter classes.

Telemetry Abstraction and Triage Tokens

While ETW provides high-fidelity observations of endpoint activity, raw telemetry streams are typically not directly actionable for systematic experimentation: they are high-volume, heterogeneous across providers, and sensitive to contextual noise (e.g., background system activity). Moreover, EDR detections are often triggered by *combinations* of signals across time, rather than by a single isolated event. This creates an attribution challenge: a blocked artifact yields an observable outcome (alert or termination), but not an immediate explanation of which feature caused it.

To bridge this gap, this thesis introduces **triage tokens** as a compact, normalized representation of detection-relevant telemetry. A triage token is a feature extracted

from one or more observations that is: (i) comparable across runs, (ii) rankable by correlation with detection outcomes, and (iii) mappable back to specific mutation targets or source locations. Concretely, triage tokens serve as the *unit of explanation* and the *unit of guidance* for the feedback loop. The complete token taxonomy, the lift-based ranking metric, and the reproducibility protocol are presented as design decisions in Section [Token-Driven Feedback Loop](#).

gRPC and Protocol Buffers

gRPC is a high-performance Remote Procedure Call framework built on HTTP/2 and **Protocol Buffers** (Protobuf) for interface definition and serialization [32]. A service is defined in a `.proto` file that specifies the RPCs it exposes and the message types they exchange; from this definition, language-specific client and server stubs are generated at compile time, providing type-safe communication with no manual serialization code.

gRPC supports four communication patterns: *unary* (single request, single response), *server streaming* (single request, stream of responses), *client streaming* (stream of requests, single response), and *bidirectional streaming* (both sides send independent streams over a single connection). The choice of pattern has architectural consequences: unary RPCs are stateless and simple to reason about, but cannot deliver data incrementally during a long-running operation; bidirectional streams enable real-time data exchange at the cost of connection lifecycle management. Both patterns are used in this thesis: unary RPCs for discrete operations such as capability queries, and a bidirectional stream for real-time telemetry delivery during artifact execution (Section [Worker Agent Design: Robust Execution as a Ten-Phase Pipeline](#)).

The concepts introduced in this chapter, the Windows platform and its detection infrastructure, malware obfuscation and its theoretical limits, and the algorithmic toolkit of fuzzers, code transformation, evolutionary search, differential analysis, and triage tokens, provide the theoretical foundation for the system described next. Section [Design](#) presents the concrete architecture and design decisions of **AutoMutate++**, showing how these ideas are integrated into an automated, feedback-driven experimentation pipeline.

Design

Evaluating EDR detection capabilities requires systematically varying the structural and behavioral properties of executable artifacts, executing them under realistic monitoring, and analyzing which observable features correlate with detection outcomes. Manual approaches to this process are generally slow, hardly reproducible, and unable to explore the mutation space at scale. AutoMutate++ automates this workflow as a *closed experimental loop*: an integrated system that mutates, builds, executes, observes, and learns in successive rounds without manual intervention. The implementation spans multiple Rust crates communicating over gRPC, with Elasticsearch as the canonical store for runs, telemetry, and aggregated results.

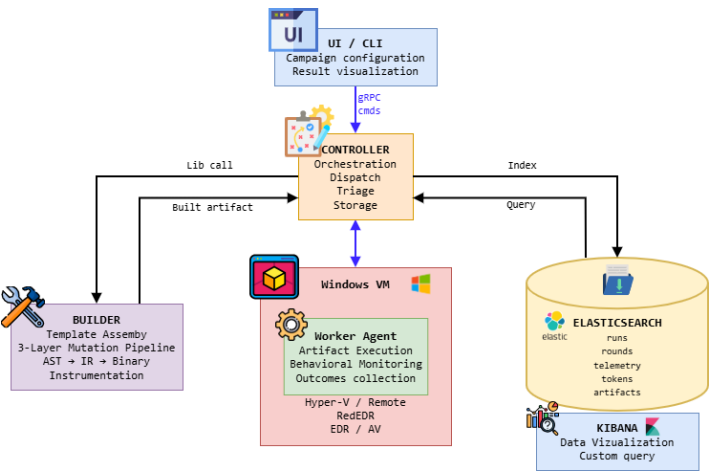


Figure 7: Simplified architecture of AutoMutate++, showing the five major components: Controller, Build System, Worker Agent, Storage Layer, UI, and the data flows that connect them.

The central design idea is a feedback loop (Figure 8): in each round, the system applies a set of mutations to a loader template, compiles the resulting artifact, and dispatches it for execution inside instrumented Windows VMs. Multi-source telemetry, ETW events from RedEDR, line-level execution traces, and structured checkpoints, is collected and stored alongside the EDR detection verdict. From this evidence the system extracts *behavioral tokens*: fine-grained indicators of what the artifact did at runtime. Token-level scoring then quantifies the statistical association between each token and detection, producing a guidance signal that steers mutation selection in subsequent rounds. Figure 7 provides a component-level view of this architecture.

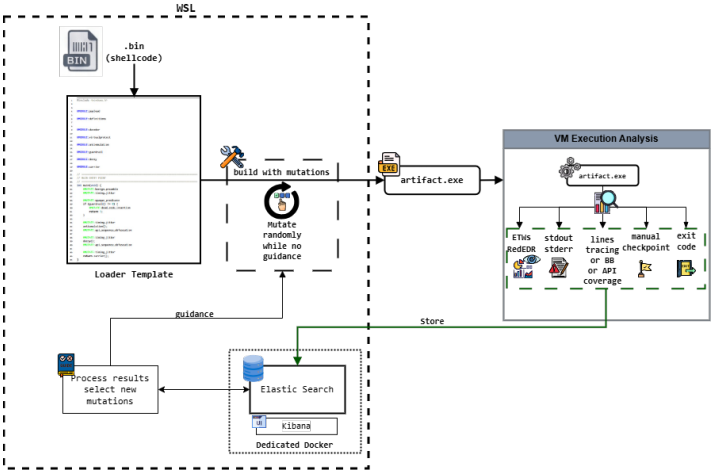


Figure 8: Overview of the AutoMutate++ closed experimental loop. Mutated artifacts are built from a loader template, executed in instrumented VMs, and analyzed using multi-source telemetry. Token-level behavioral signals extracted from execution results guide the selection of mutations in subsequent rounds.

Job	– A complete experiment specification: target payload, capability requirements, mutation strategy, and number of rounds.
Round	– Atomic experimental unit: one mutation configuration evaluated via up to three correlated runs.
Run	– A single execution of a built artifact inside an instrumented VM (baseline, instrumented, or dryrun).
Baseline	– Run without instrumentation (<code>trace=off</code> , <code>dryrun=false</code>); provides the ground-truth EDR verdict.
Instrumented	– Run with line-level tracing (<code>trace=lines</code> , <code>dryrun=false</code>); maximizes observability.
DryRun	– Run without active EDR (<code>dryrun=true</code>); sanity-checks that the artifact itself is functional.
Artifact	– The compiled PE binary under test, composed of a loader and an encrypted payload.
Loader	– Carrier code that decodes and executes the payload at runtime via a configurable API sequence.
Payload	– The encrypted shellcode delivered and executed by the loader.
Mutation	– A variant choice for one behavioral gene of the loader (AST, IR, or binary layer).
Token	– A fine-grained behavioral indicator extracted from execution telemetry, scored for detection correlation.

Three foundational principles shape every design decision that follows. First, *differential validity*: a multi-run protocol (Section **Three-Run Differential Protocol**) separates genuine detection from instrumentation-induced artifacts and broken-build noise. Second, *token-driven guidance*: instead of blind or random mutation, the system uses statistically grounded

token scores (Section [Token-Driven Feedback Loop](#)) to prioritize mutations that target the behavioral features most correlated with detection. To isolate the contribution of token guidance, three simpler selectors: random, coverage-driven, and genetic serve as ablation baselines (Section [Selector Strategies as Ablation Framework](#)).

Third, *deterministic reproducibility*: pinned seeds, hermetic builds, and an immutable evidence store ensure that any experimental round can be replayed and audited.

The remainder of this chapter is organized as follows. Section [Design Goals and Constraints](#) states the non-negotiable design constraints. Section [System Decomposition](#) decomposes the system into its five major components. Subsequent sections then detail each component: controller scheduling, the three-run differential protocol, build-system design, worker-agent execution, lab infrastructure, storage, and the token-driven feedback loop, before closing with the automation engine that ties them together.

Design Goals and Constraints

AutoMutate++ is engineered around three non-negotiable constraints:

- **Reproducibility** The system must be able to replay a round and obtain the same build output and comparable telemetry, otherwise any “learning” degenerates into noise.
- **Observability under adversarial termination.** The artifact under test may be terminated at *any* point. The design must preserve partial progress evidence even if the process is killed mid-execution.
- **Differential validity.** Instrumentation is useful, but it can perturb behavior. The design must separate *ground-truth* outcomes from instrumentation-induced artifacts.

A secondary (but important) objective is scalability: multiple jobs should run concurrently across a pool of heterogeneous VMs without starving each other or contaminating per-run telemetry.

System Decomposition

The system decomposes cleanly into five layers (Figure 9):

- **Controller (Scheduling + Orchestration).** Owns job lifecycle, run scheduling, worker routing, and result aggregation.
- **Build System (Artifact Emitter).** Deterministically produces PE artifacts from a modular template and optional mutation specifications.
- **Worker Agent (Execution Runtime).** Executes artifacts inside Windows VMs, enforces timeouts, collects telemetry, and streams results back.

- **Telemetry + Storage (Evidence Plane).** Normalizes and indexes run outputs, ETW events, traces, and summaries.
- **Infrastructure Automation.** Provisions and configures local Hyper-V and/or remote VMs, networking, and prerequisites (toolchains, RedEDR).

The concrete typed channels and synchronization primitives that connect these layers are documented in Appendix [Controller Communication and Synchronization](#).

Controller Design: Jobs, Rounds, and Fair Scheduling

The controller is an event-driven scheduler that turns a job specification into a stream of **rounds**. A *round* is the atomic experimental unit: it represents a single configuration (mutations + argument choices) evaluated via multiple correlated executions. Figure 11 illustrates the internal layering: an Orchestrator spawns per-job workers whose runs are dispatched through a shared pool to VM agents.

RunPool and capability-based routing.

Workers differ by OS version, installed tooling, and enabled sensors. To avoid accidental invalid comparisons (or repeated failures), the controller routes runs through a shared **RunPool** filtered by *capability sets*. A VM executor pulls runs only if it satisfies the required OS and capability constraints. Internally, the RunPool partitions its queues by `required_os`, so VMs targeting different Windows versions never contend for the same queue, a contention-reduction design that prevents OS-heterogeneous pools from becoming a bottleneck. Additionally, a bidirectional dryrun guard ensures that VMs designated for dryrun execution only consume dryrun runs, and vice versa; this capability-matching invariant prevents misrouted experiments. Idle executors are notified via a signal-driven wake mechanism when work arrives, avoiding polling overhead (Appendix [Controller Communication and Synchronization](#)).

RoundAgg: join state for correlated executions. Each round produces up to three correlated runs (baseline, instrumented, and optionally dryrun) that must be aggregated before committing an experimental verdict. The controller maintains a join structure (RoundAgg) that joins all expected run results, with a grace period for the optional dryrun, before computing the round summary.

Three-Run Differential Protocol

A core design decision is to avoid treating instrumented telemetry as ground truth. Instrumentation can perturb behavior (the observer effect), and broken artifacts can masquerade as detections. Each round therefore executes three correlated runs on the *same* artifact:

- **Baseline** (`trace=off`, `dryrun=false`). No instrumentation overhead; provides the ground-truth EDR verdict.

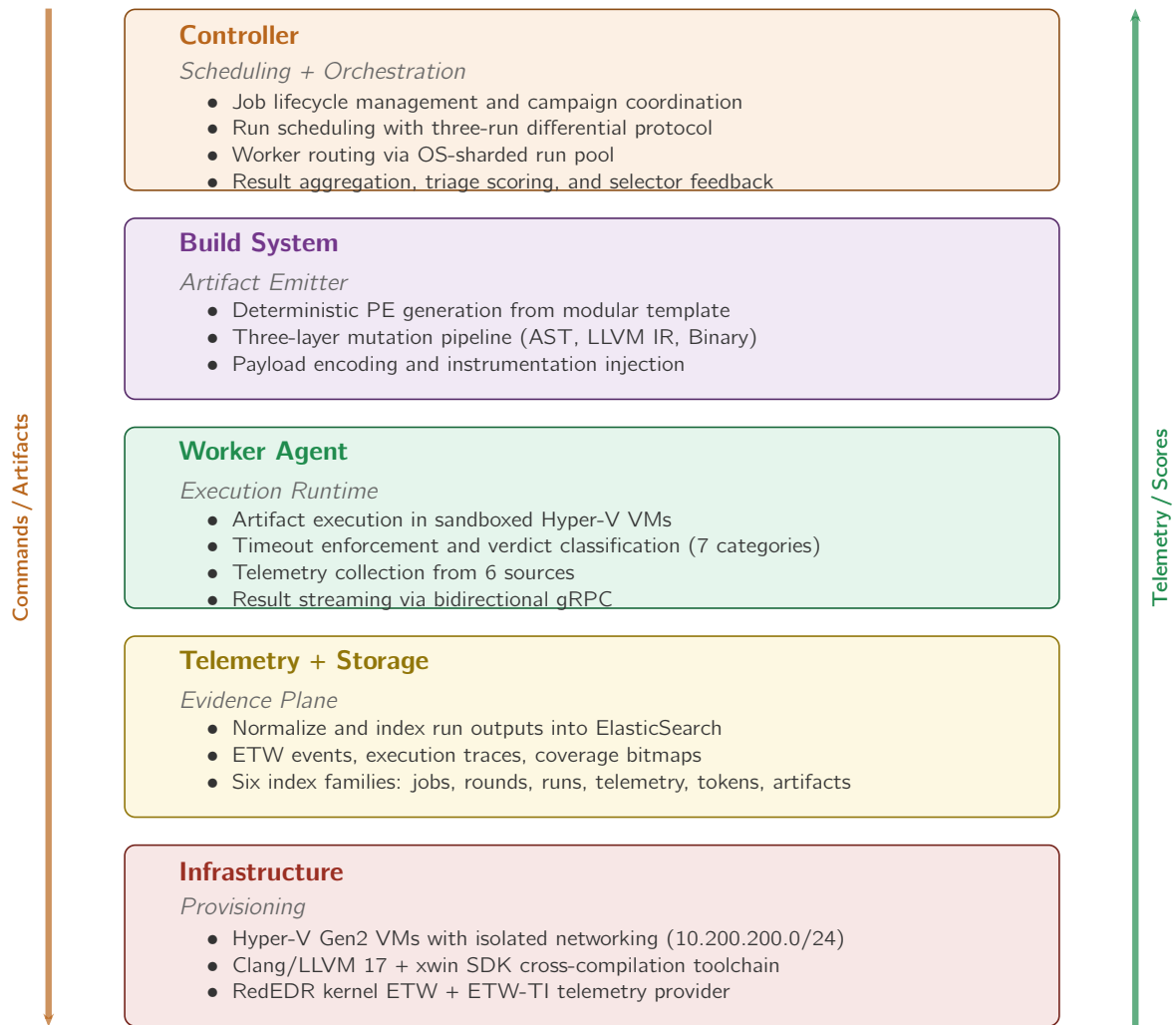


Figure 9: AutoMutate++ decomposition: Controller, Build, Worker Agent, Storage, and Automation.

- **Instrumented** (trace=lines, dryrun=false). Maximizes execution-path observability for localization and causal analysis.
- **DryRun** (trace=off, dryrun=true). Executes the loader and payload *without* an active EDR, serving as a sanity check that the artifact itself is functional (that mutations have not introduced functional bugs/crash).

The baseline and instrumented outcomes form a 2×2 decision matrix (Table 13).

Table 13: Baseline $\times$ Instrumented decision matrix.

	Instr. Detected	Instr. Clean
Base Detected	RealDetection	Flaky
Base Clean	InstrumentationArtifact	Evasion

RealDetection: both runs agree the artifact was caught, high-confidence positive. **Flaky**: the baseline was caught but the instrumented run survived, indicating a non-

reproducible detection. **InstrumentationArtifact**: only the instrumented run was caught, meaning the observer (tracing) caused the detection, discarded for learning. **Evasion**: neither run was detected, the artifact bypassed the EDR.

DryRun override. If the artifact crashes during the dryrun (i.e., it fails without any EDR intervention), the 2×2 result is overridden: the round is classified as **MutationFailed** when mutations were applied, or **PayloadFailed** when the vanilla template itself is broken. This isolates broken-build noise from genuine detection analysis.

Static scan short-circuit. Before any VM dispatch, a static Defender file scan is run on the compiled artifact. If the scan detects the artifact, the round is immediately finalized as **StaticDetection** with an evasion score of 0.0, saving three VM dispatches. This cleanly separates static from dynamic detection in the experimental record: a mutation that defeats static scanning but fails dynamically tells a different story than one that fails both.

This protocol enforces a guiding principle of the thesis:

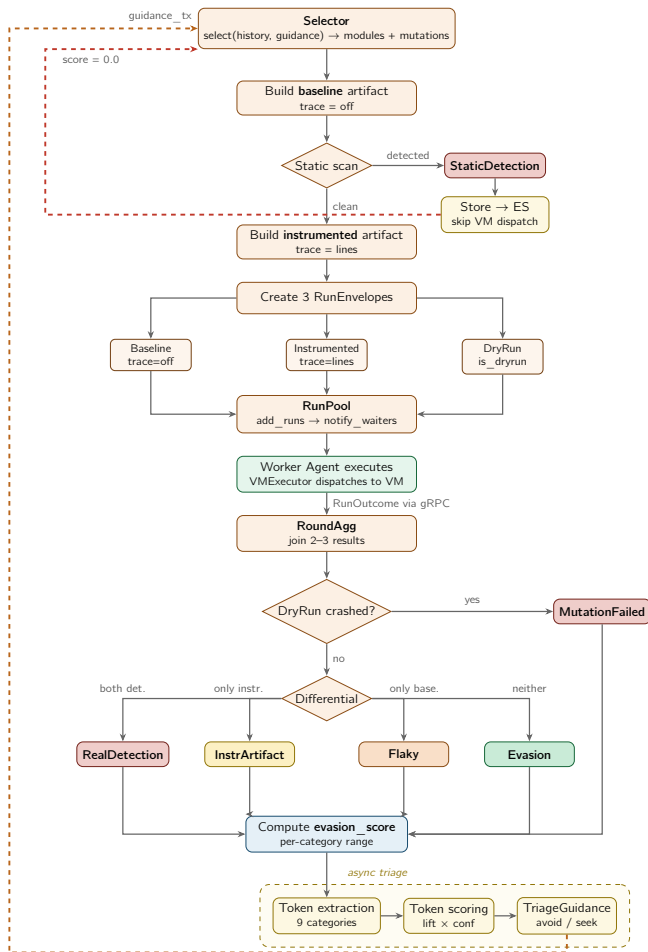


Figure 10: Lifecycle of a single experimental round, showing the feedback loop from selector through build, dispatch, aggregation, and triage back to the next selector invocation. The static-scan short-circuit avoids wasting VM time on artifacts detected at rest. The dashed region indicates asynchronous triage processing.

instrumentation both observes behavior and determines execution outcomes, so broken artifacts must be caught before their results pollute the feedback signal.

Differential Categories and Trustworthiness

The three-run protocol and the static scan produce seven mutually exclusive outcome categories (Table 14).

Only *trustworthy* categories, RealDetection, Evasion, and StaticDetection, feed the learning loop. The remaining four are discarded: InstrumentationArtifact (the observer caused the detection), Flaky (not reproducible), MutationFailed and PayloadFailed (broken build). This trustworthiness predicate acts as a data-quality filter applied before statistical learning, preventing noisy rounds from corrupting lift and confidence calculations.

Build System Design: Modular Templates as a Controlled Search Space

The build system is designed to provide a large mutation/search space without sacrificing determinism. The

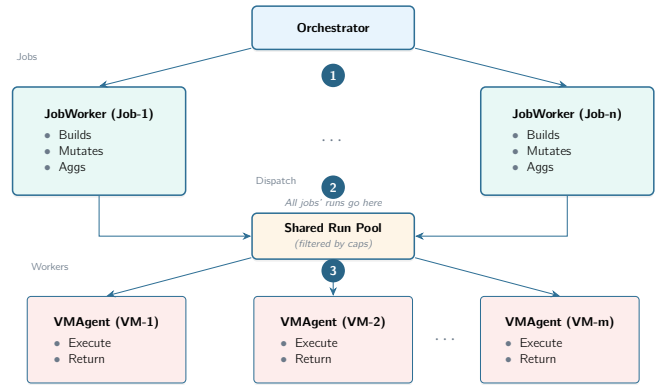


Figure 11: High-level architecture of the orchestration system. The system schedules n jobs whose runs are executed by m VMs.

Table 14: Seven differential categories and their trustworthiness.

Category	Source	Trustworthy?
RealDetection	2×2 : both baseline and instrumented detected	Yes
InstrumentationArtifact	2×2 : only instrumented detected	No
Flaky	2×2 : only baseline detected	No
Evasion	2×2 : neither detected	Yes
MutationFailed	Dryrun nonzero or stalled; artifact broken on clean VM	No
PayloadFailed	Dryrun nonzero, payload execution reached	No
StaticDetection	Defender static file scan before VM dispatch	Yes

core abstraction is a **modular template** whose interchangeable sections are stitched together into a single compilable source file. Figure 12 illustrates the high-level loader architecture: an encrypted payload preceded by anti-emulation guards, behavioral conditioning (deconditioner), and a decode/execute phase, each realized as an independently mutable gene.

Why a canonical loader template. The loader template deliberately uses a canonical shellcode-injection pattern: allocate writable memory, decode payload, transition to executable, transfer control, rather than advanced techniques such as reflective DLL loading, module stomping, or process hollowing (Table 42). This is a deliberate experimental-design choice: the loader skeleton is the *controlled variable*, not the object of study. The research question asks which *observable behavioral features* correlate with EDR detection; answering it requires that each mutation map cleanly to an identifiable change in the telemetry. A simple car-

rier produces a short, decomposable API sequence (e.g. `VirtualAlloc`→`memcpy`→`VirtualProtect`→`CreateThread`) where every call is intentional and every mutation yields a distinct token delta. Complex injection techniques, reflective loading involves manual PE parsing, section mapping, relocation fixups, and IAT resolution, produce dozens of structurally *coupled* API calls. When detection occurs, the feedback loop cannot isolate which part of the reflective loader triggered it, collapsing the differential signal that the token scorer depends on.

The three carrier variants already span the key behavioral primitives that advanced techniques rely on: `alloc_rw_rx` and `change_rw_rx` exercise different memory-protection transition sequences (the same transitions module stomping uses), while `peb_walk` performs runtime API resolution via PEB traversal, the same mechanism reflective loaders use to locate `LoadLibrary` and `GetProcAddress`. The gene model can therefore test *whether* PEB-based resolution or staged protection transitions affect detection, which is the measurable hypothesis, without conflating it with the structural complexity of a full reflective loader. Additionally, the line-level instrumentation system (Section [Instrumentation Design: Evidence That Survives Process Death](#)) assumes a linear `main()`→`guardrail`→`antiemulation`→`deconditioner`→`carrier` flow; techniques involving cross-process injection or suspended-thread manipulation could break the single-process telemetry collection model and require a fundamentally different observation architecture.

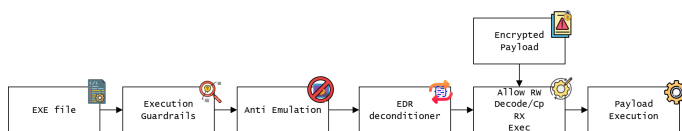


Figure 12: Proposed loader architecture (adapted from [24]). The design separates anti-emulation guards, behavioral conditioning (deconditioner), and decode/execute phases. Each phase is realized as an independently mutable gene described below.

Module assembly via @MODULE. A template file contains markers such as `@MODULE:decoder` or `@MODULE:carrier`. At build time, a template assembler selects one implementation per category and inlines it into a single merged source file. The primary benefit is that each module combination produces a structurally distinct loader while remaining analyzable and reproducible.

Modular Gene Model

The artifact is decomposed into independent behavioral **genes**. Each gene encapsulates a single functional responsibility, and multiple interchangeable *variants* exist per gene. Table 15 lists the gene categories, their approximate variant counts, and the detection surface each primarily targets.

Because genes are selected independently, the combinatorial space is large: $3 \times 4 \times 7 \times 7 \times 2 \times 2 \times 2 = 4704$ distinct configurations before any fine-grained mutation is applied. Gene boundaries are delimited by structured comments (`// --- BEGIN/END MODULE ---`), which enable *scoped mutation*: when triage identifies a specific gene as the detection cause, subsequent mutation rounds can be restricted to that gene alone, leaving all other modules frozen.

The `@MODULE` / `@MUTATE` dual-marker system is a deliberate separation of concerns, structural composition (`@MODULE`) and fine-grained transformation (`@MUTATE`) operate independently. This allows the assembler and the mutator to be developed and tested in isolation. Section [Template Assembly](#) describes how the assembler resolves these markers at build time.

In-module mutability via @MUTATE. Fine-grained transformation points are declared with `@MUTATE` markers, which act as explicit “edit sites.” If a mutation is not applied, markers are stripped to keep the compilation clean. This avoids “accidental mutation” where a tool rewrites unrelated code.

Three-Layer Mutation Architecture

Most prior work on artifact transformation mutates at a single compilation level, either the source code or the final binary [48]. AutoMutate++ applies transformations at *three* distinct stages of the compilation pipeline (introduced in Section [Programmatic Code Transformation](#)), each targeting a different class of detection surface. Table 16 summarizes the layers.

Composability. The three layers are composable: an artifact can receive mutations at all three in a single build. The ordering is fixed (AST → IR → Binary) because each layer operates on the output of the previous compilation stage. This constraint is inherent, IR mutations cannot precede AST mutations because the `.ll` file is produced *from* the mutated C source.

Optimization resistance. A critical design requirement is that mutations must *survive* compiler optimization. IR-level opaque predicates use inline assembly (`asm sideeffect “xor $0, $0”`) specifically because LLVM treats inline assembly as an opaque side-effecting operation and cannot constant-fold it. A “trivial” mode (`icmp eq i32 0, 0`) is provided as a control variable: it folds away at `-O2`, confirming that the robust mode’s persistence is due to assembly opacity rather than optimizer limitations. Sections [AST Mutations via Tree-Sitter](#), [IR Mutations via Text-Based LLVM IR Manipulation](#), and [Binary Mutations at the PE Byte Level](#) detail the realization of each layer.

Deconditioner Design

The deconditioner module introduces a novel *behavioral-level mutation* that does not modify the payload or the

Table 15: Gene categories and their targeted detection surfaces.

Gene	Variants	Primary Detection Surface
carrier	3	Behavioral analysis: API call sequence (alloc→write→protect→execute), incl. staged RW→R→RX [48]
decoder	4	Static analysis: entropy profile, string signatures
antiemulation	7	Emulator/sandbox evasion heuristics
deconditioner	7	Behavioral baseline dilution (Section Deconditioner Design)
guardrail	2	Pre-execution environment validation
virtualprotect	2	Memory protection mechanism: standard API wrapper or undersized chunked calls
decoy	2	Import table and resource metadata

Table 16: Three-layer mutation architecture: each layer targets a distinct detection surface.

Layer	Representation	Survives Opt.?	Target Detection Surface
AST (C source)	Structural	Partially	Behavioral signatures, API sequences
LLVM IR	SSA, typed	Yes*	Control-flow heuristics, basic-block analysis
Binary (PE)	Raw bytes	N/A	Static ML features, PE metadata

*When using inline assembly; see below.

carrier code itself but instead alters the *statistical context* in which the carrier executes. Concretely, the deconditioner rehearses the carrier’s alloc→write→protect→free pattern (the same sequence described in Section [Windows Primitives for Code Execution](#)) using **benign data** before real payload execution.

Hypothesis. EDR behavioral engines maintain internal counters or baselines for suspicious API patterns. By flooding these counters with benign instances of the same pattern, the real execution may blend into the statistical noise, reducing its distinguishability. This is explicitly a hypothesis to be tested experimentally, not an established evasion guarantee. The hypothesis originates from Cailleret [16], who posits that EDRs employ per-process behavioral cutoffs: once a process performs a monitored action below a detection threshold repeatedly without triggering malicious conditions, the EDR deprioritizes further tracking of that pattern. This hypothesis is partly supported by Lepori [48], who showed that inserting benign system calls between suspicious ones (behavior mutation) extended execution further before detection on Elastic Defend. Cailleret [17] independently confirmed a generalised form of this effect: targeted deconditioning, repeated benign dumping of non-critical processes, bypassed MDE’s behavioural freeze on **lsass.exe** memory dumps, suggesting that deconditioning is a generic bypass for behavioural tracking once the relevant trigger is identified. The benign repetitions may also satisfy internal rules that would otherwise trigger runtime memory scanning; after sufficiently many clean evaluations, the EDR temporarily deactivates or deprioritizes those rules for the process. N-gram extraction techniques [44, 4] remain relevant in a complementary role: by extracting the API-call N-grams from the loader and payload execution traces,

the system could identify the specific short patterns that correlate with detection and construct targeted benign deconditioning sequences from those patterns.

Variant strategies. Multiple deconditioner variants explore different flooding approaches: simple sequential loops, threaded allocation (concurrent **VirtualAlloc**/**VirtualFree** pairs), mixed API sets (interleaving memory and file operations), and entropy-varied data buffers. Each variant produces a different behavioral fingerprint, enabling the system to test which flooding strategy, if any, reduces detection rates.

This mechanism is distinct from traditional code transformation: it operates at the behavioral telemetry level rather than the syntactic or structural level. Section [Benign Catalog: Dependency-Aware API Insertion](#) describes the dependency-aware API insertion algorithm that populates the deconditioner’s benign call sequences.

Payload encoding as a first-class dimension. The payload is embedded through an auto-generated header (payload.h). Multiple encodings exist (e.g., XOR-like and dictionary-based mapping), not as a trick, but as a controlled knob to study static scanning sensitivity vs. runtime footprint trade-offs. Lepori [48] demonstrated that simple XOR encoding is insufficient against ML-based static engines (Elastic) and proposed a dictionary-based sub-byte mapping as an alternative; AutoMutate++ adopts and extends this encoding scheme as an option.

Deterministic Compilation Pipeline

The build pipeline is structured as a deterministic chain:

1. **Encode payload** → generate payload.h.
2. **Assemble modules** → produce assembled.c.

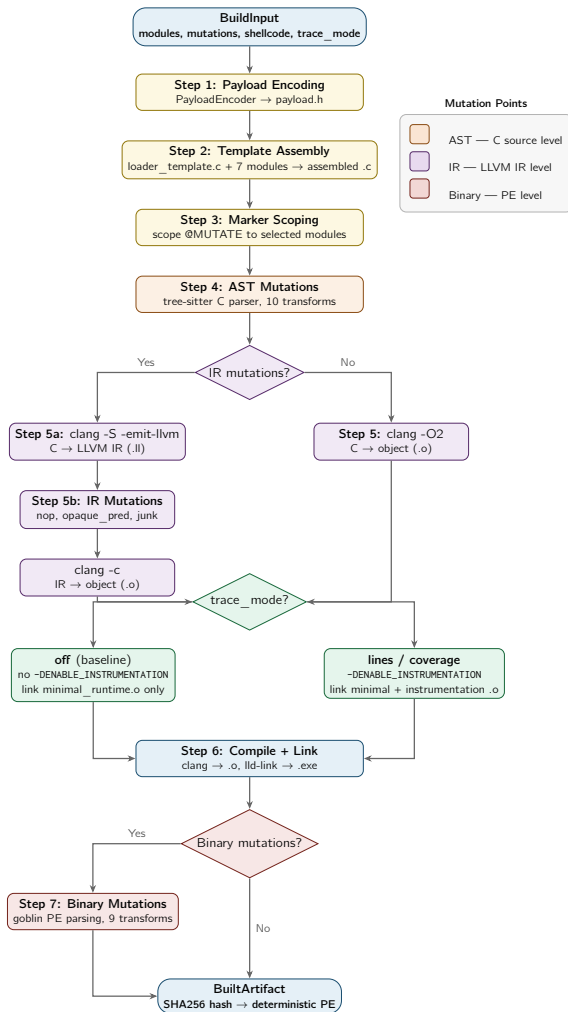


Figure 13: The 7-step build pipeline showing the conditional branch between direct compilation and the LLVM IR path (Step 5), the `trace_mode` decision controlling both the `-DENABLE_INSTRUMENTATION` compile flag and runtime object selection (Step 6), and post-link binary mutations (Step 7). Coloured annotations mark the three mutation insertion points: AST transforms operate on C source, IR transforms on LLVM intermediate representation, and binary transforms on the linked PE executable.

3. **Apply mutations** (optional, at declared sites).
4. **Cross-compile to PE** using Clang/LLVM with a Windows SDK sysroot.
5. **Optionally instrument** (AST line traces, IR-level checkpoints/coverage).
6. **Hash + rename** to a SHA256-named artifact for provenance stability.

The design intent is that the same input spec yields the same artifact bytes, enabling strict round-to-round comparisons.

RNG seed table. Every source of randomness in the pipeline is seeded and deterministic. Table 17 enumerates

the five randomness sources, their generators, and default seeds.

Table 17: Randomness sources and their deterministic seeds.

Component	RNG	Seed
IR mutations	glibc LCG	User-provided or fixed 1234
Benign catalog scheduling	xorshift64	User-provided or fixed 0xBE41
Binary timestamp jitter	FNV-1a hash	Derived from artifact content
Payload encoding	Fixed keys	[0xAA, 0x55] or user-provided
Clang linker	/Brepo flag	Deterministic PE timestamps

The rationale is methodological: both the differential protocol (Section [Three-Run Differential Protocol](#)) and the feedback loop (Section [Token-Driven Feedback Loop](#)) require that *the same mutation recipe always produces the same artifact*. Without reproducibility, detection changes cannot be attributed to specific mutations and causality cannot be established.

Having established how artifacts are built and varied, we now address the runtime challenge: capturing execution evidence that survives abrupt process termination.

Instrumentation Design: Evidence That Survives Process Death

The artifact is executed in an environment where termination may be abrupt. Therefore, observability cannot rely solely on “flush-at-exit” mechanisms.

Three complementary telemetry channels. AutoMutate++ uses three distinct channels for runtime evidence:

- **ETW stream (RedEDR).** Kernel-level events that approximate what defenders see.
- **Line traces (named pipe).** Fine-grained progress markers emitted before statements.
- **Checkpoints (named pipe).** Structured, semantic “stage reached” signals (including optional manual checkpoints).

The key property is that line traces and checkpoints are emitted and flushed *inline*, so that if a process is killed at statement n , events up to n have already been transmitted to the collector task. Coverage bitmaps are treated as best-effort because they are more sensitive to kill timing.

Trace modes. Instrumentation is intentionally expressed as explicit modes: `off` (baseline), `lines` (localization), and extensible modes (API checkpoints, basic-block coverage) whose granularity can be traded against runtime overhead in future experiments to identify the coverage

layer that maximises scoring accuracy. When instrumentation is disabled, macros expand to no-ops, minimizing behavioral drift.

Conditional instrumentation via #ifdef macros.

The build system defines `-DENABLE_INSTRUMENTATION` only when `trace_mode` $\neq$ "off". The header `instrumentation.h` uses this flag to toggle between real function calls and no-ops:

- When defined, macros such as `ARTIFACT_CHECKPOINT(name)` expand to real instrumentation calls, and both `minimal_runtime.o` and `instrumentation_runtime.o` are compiled and linked.
- When not defined, all instrumentation macros expand to `((void)0)` and are eliminated by the optimizer. Neither runtime object is linked, and the baseline binary contains no trace of the instrumentation system.

The baseline and instrumented variants are therefore *intentionally different* binaries: the baseline uses the standard CRT exit path and carries no instrumentation imports, while the instrumented variant includes direct-syscall exit logic, telemetry flush routines, and additional API imports (`Sleep`, `OutputDebugStringA`). Linking `minimal_runtime.o` into the baseline was initially considered (using weak symbols to provide no-op stubs), but testing revealed that it introduced EDR-visible artifacts, inline syscall instructions, `ntdll.dll` string references, and dynamic API resolution patterns, that contaminated the baseline's detection profile. The differential protocol handles the resulting structural divergence through outcome categorization rather than binary identity: if the baseline evades but the instrumented variant is detected, the round is classified as `InstrumentationArtifact` and excluded from token scoring (Table 14). Appendix [Instrumentation: Evolution from Weak Symbols to Conditional Compilation](#) details the original weak-symbol design and the artifacts that motivated its replacement.

Worker Agent Design: Robust Execution as a Ten-Phase Pipeline

Each Windows VM runs a worker agent responsible for executing artifacts and collecting evidence. The agent is built around a multi-phase pipeline to make failure handling explicit and auditable. The agent is deliberately stateless with respect to mutation strategies and scheduling policy, it executes what it receives and reports what it observes, allowing the experimental protocol to evolve independently of the observation infrastructure.

Execution as phases.

A run is executed as:

1. Validate artifact + metadata.
2. Initialize RedEDR collection and sanity-check contamination. In strict mode, if RedEDR reports residual events from a prior run, the engine aborts with a `FailedPrecondition` error rather than risk collecting mixed telemetry, a direct enforcement of the differential-validity design goal (Section [Design Goals and Constraints](#)).
3. Prepare per-run environment (fresh telemetry directory).
4. Spawn artifact process.
5. Start monitoring (polling + timeouts).
6. Wait via a three-way race (`tokio::select!`): process completion, timeout expiration, and idle-detection signal. Whichever fires first determines the termination cause and subsequent handling. On timeout or idle, `kill_process_tree()` terminates the artifact and all its descendants (Section [Worker Agent Implementation](#)).
7. Collect ETW and trace streams concurrently.
8. Classify outcome (exit code vs. external kill vs. timeout).
9. Package telemetry: trace logs are deduplicated by (file, line, function) keeping the record with the highest sequence number (closest to termination), and oversized payloads are progressively truncated from the head, preserving the tail where detection-relevant behavior clusters (Section [Worker Agent Implementation](#)).
10. Cleanup (always, even on partial failure).

Per-run detection classification.

After process termination, the worker classifies each run into one of five provisional verdicts (Table 18). These per-run verdicts are distinct from the round-level differential categories in Table 14: a per-run verdict captures what happened to a *single* execution, whereas the round-level category emerges from comparing *multiple* correlated runs.

The classifier follows a strict decision tree: infrastructure and guardrail failures take highest priority (`InfraError`), followed by clean exit (`Evasion`), timeout, split by whether the Launching checkpoint was reached (`Evasion` vs `Stalled`), then externally terminated or AV-specific NTSTATUS values (`Detected`), and finally crash codes, carrier failure codes, and unrecognised nonzero exits (`Ambiguous`).

The cascade progresses from definitive to ambiguous: *cannot run* $\rightarrow$ *ran successfully* $\rightarrow$ *ran but stuck* $\rightarrow$ *killed by AV* $\rightarrow$ *unclear*, each level eliminates cases before the

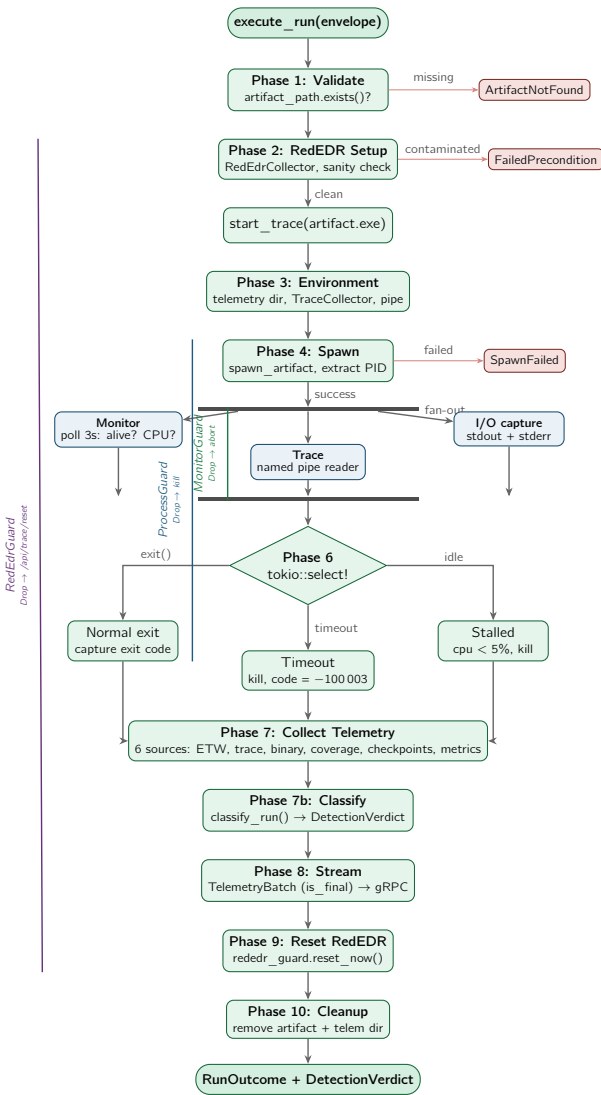


Figure 14: The 10-phase execution pipeline with concurrent task fan-out during Phases 5–6. Three RAIL guards (shown as coloured side-braces) guarantee resource cleanup on all exit paths including panics. The `tokio::select!` race during Phase 6 resolves between normal exit, timeout, and idle detection. Six telemetry sources are collected in Phase 7.

next, so no verdict requires information the worker does not yet have.

The *Ambiguous* verdict is a deliberate design choice: in a system where classification drives automated learning, a wrong classification is worse than no classification. A crash caused by a mutation bug is indistinguishable from an AV-induced crash at the exit-code level; routing both to *Detected* would cause the feedback loop to avoid mutations that merely had implementation bugs. By excluding ambiguous signals, the system preserves the integrity of token statistics at the cost of reduced data volume.

The worker produces a *provisional* verdict based on local signals only; the controller may override it after dryrun comparison (Section [Three-Run Differential Protocol](#),

Table 18: Five provisional per-run verdicts produced by the worker-agent classifier.

Verdict	Meaning	Feedback Use
Evasion	Exit code 0, or timeout after reaching the Launching checkpoint	Positive signal
Detected	AV NTSTATUS or externally terminated	Negative signal
Ambiguous	Crash NTSTATUS, carrier codes 30–39, or unknown nonzero exit code	Pending dryrun
Stalled	Timeout without reaching the Launching checkpoint	Excluded
InfraError	Spawn failure, wait failure, or guardrail rejection (exit codes 10–19)	Excluded

Table 14), promoting *Ambiguous* to *MutationFailed*, *PayloadFailed*, or *Detected* once clean-VM context is available. The dryrun refines classification but is not required: without it, ambiguous runs are simply excluded from the feedback loop rather than misclassified.

Execution Guarantees and Transport

RAIL guards as the cleanup contract. The worker uses RAIL guards for resources that must be released reliably (RedEDR sessions, processes, temporary directories). This choice is not stylistic: it is required because termination and failures can occur at any phase.

One run at a time. The worker enforces a single execution lock. This prevents ETW contamination and makes telemetry attribution unambiguous.

Dual communication model. The gRPC patterns introduced in Section [gRPC and Protocol Buffers](#) are applied through two coexisting communication models. In the *unary* model, the controller invokes `RunSample()` and blocks until the worker returns a classified outcome; telemetry is then retrieved separately via the server-streaming `GetTelemetry()` RPC. This mode is straightforward to debug and sufficient for batch operation, but provides no in-flight visibility. In the *bidirectional stream* model (`EstablishStream()`), both sides exchange typed messages concurrently over a single HTTP/2 connection, enabling real-time telemetry delivery during execution and early-abort decisions.

To prevent the execution engine from coupling to either model, the agent defines a `ControlPlaneSink` trait with two methods: `send_telemetry()` and `send_status()`. Two implementations exist: `StreamSink` (writes to the bidirectional stream) and `NullSink` (discards data silently for unary mode). A factory inspects whether a stream session is active and returns the appropriate implementation as a trait object; the execution engine calls it uniformly, unaware of the underlying transport. This is mostly for legacy controller compatibility.

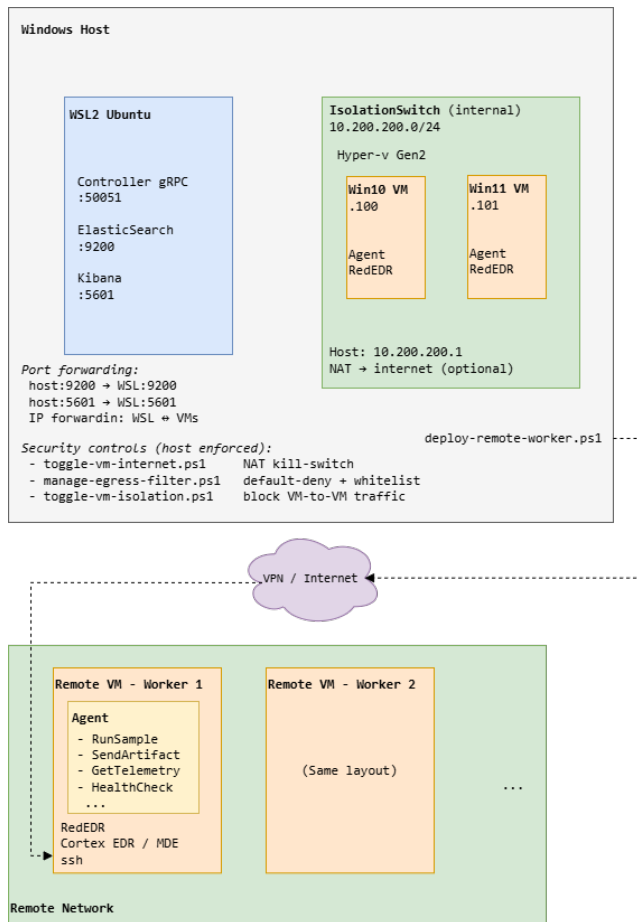


Figure 15: Deployment topology of AutoMutate++. A controller and telemetry stack run in WSL2 on the host system, while artifacts execute in isolated Hyper-V Windows VMs instrumented with RedEDR. Remote worker VMs can connect over a VPN to scale execution and return telemetry.

Lab Infrastructure

The system supports two deployment modes: *local* Hyper-V virtual machines on the controller host, and *remote* worker VMs reachable over a VPN link. In both cases the controller initiates gRPC connections to the workers (controller-dials-worker), so remote workers behind NAT require only that the controller can reach their advertised address. The controller itself runs inside WSL2 on the host machine, which provides a Linux-native toolchain environment while retaining direct access to Windows host facilities, notably, the build pipeline invokes **MpCmdRun.exe** via Windows interop to perform a Defender static file scan on each compiled artifact before VM dispatch, short-circuiting rounds that are detected on disk without consuming VM execution time. All telemetry is indexed in Elasticsearch for cross-run differential analysis. Figure 15 shows the resulting topology.

Hyper-V checkpoints provide clean-state revert between mutation rounds, guaranteeing that each run begins with the same Defender signature database version, no leftover files, registry keys, or cached detections

from previous executions, and identical ETW provider registrations and audit configuration. This revert is especially critical given Cailleret’s [17] finding that MDE dynamically generates and caches new static identifiers after a behavioural detection: once a process injection is detected at runtime, the originating executable is analysed and its identifiers are cached, causing future variants to be blocked statically rather than behaviourally. Without checkpoint revert, this temporal feedback would silently shift the detection baseline between rounds. VMs can be fully air-gapped (NAT adapter disabled) to prevent artifacts from contacting command-and-control infrastructure or triggering Defender cloud-based detection; when cloud detection is itself part of the experiment, egress is selectively re-enabled. The per-worker single-execution lock (Section **Worker Agent Design: Robust Execution as a Ten-Phase Pipeline**) prevents cross-contamination between parallel runs on the same VM.

The three-run differential protocol (Section **Three-Run Differential Protocol**) relies directly on this infrastructure: all runs of a differential round execute on the *similar* VM, with a checkpoint revert between them. When activated (not available for remote vms), this ensures identical OS state, Defender signature versions, and EDR baselines for all runs of a differential round. Because environmental drift is eliminated, any behavioral difference between runs can be attributed to instrumentation rather than to state divergence, which is what makes the “instrumentation artifact” classification trustworthy.

Data Model and Storage: Making Runs Queryable

Every run produces a structured record:

- **Inputs:** module selection, mutation recipe, trace mode, build metadata (flags, toolchain versions).
- **Outputs:** exit code, stdout/stderr snippets, detection classification.
- **Telemetry:** ETW batches, trace events, checkpoints, (optional) coverage artifacts.
- **Aggregation:** round-level comparison between baseline and instrumented outcomes.

Elasticsearch is the canonical store because the design expects heavy cross-run slicing: by module choices, by OS capabilities, by tokens, and by time windows.

Evasion Loop Architecture: Fuzzing for Detection Bypass

This thesis adapts the fuzzer paradigm (Section **Fuzzers and their role in security testing**) to a domain-specific “evasion loop.” Unlike a traditional fuzzer which aims to crash a target process, an evasion fuzzer aims to “crash” the detection logic, specifically, to induce a false negative classification.

The architecture treats the malware loader and payload as the *input*, the compilation and mutation process as the *generator*, and the EDR’s telemetry and enforcement outcome as the *oracle*. In our context, this oracle is not a crash signal but a **policy decision** (blocked vs. allowed), augmented with *telemetry-derived triage tokens* that explain and guide subsequent mutations. To function effectively, such a system requires three distinct capabilities: a method to programmatically modify code structure while preserving semantics, a strategy to guide these modifications toward evasive variants, and a feedback mechanism to determine which specific features triggered detection.

The three-run differential protocol (Section [Three-Run Differential Protocol](#)) ensures that the oracle’s verdict is trustworthy, and the modular build system (Section [Build System Design: Modular Templates as a Controlled Search Space](#)) provides the controlled search space over which the generator operates. The following subsection describes the token-driven selection mechanism that tries to close this loop.

Composite Evasion Scoring

A binary detected/not-detected verdict discards information: an artifact that runs for 80% of the timeout before detection is more promising than one killed instantly. The system therefore computes a *continuous* evasion score within bounded, non-overlapping ranges per category (Table 19). Although evasion scores are defined for all seven categories (for observability and potential future selector usage), only scores from *trustworthy* rounds pass the filter: every consumer, the token scorer and all four selectors, calls `is_trustworthy()` before accumulating a round’s score, so non-trustworthy scores are computed but never enter the feedback loop.

Table 19: Evasion score ranges by differential category. The *base factor* column shows the initial (time-only) formula; after asynchronous coverage computation it is replaced by the blended factor described at [Asynchronous coverage blending](#).

Category	Range	Formula	Base factor
Evasion	0.6–1.0	$0.6 + 0.4 \times f$	$f = 0.5p + 0.5b$
InstrumentationArtifact	0.5–0.7	$0.5 + 0.2 \times f$	$f = s$
RealDetection	0.0–0.4	$0.4 \times f$	$f = s$
Flaky	0.0–0.3	$0.3 \times f$	$f = s$
Static / MutFailed / PayFailed	0.0	0.0	—

s = elapsed/timeout (survival ratio), p = (payload reached),
 b = [exits match $\wedge$ verdicts match] (behavioral match).

The non-overlapping ranges preserve category ordering as an invariant: any Evasion (≥ 0.6) always outscores any RealDetection (≤ 0.4). Within each range, the base factor f provides a gradient that rewards longer execution or deeper payload delivery.

Asynchronous coverage blending. Because coverage is computed asynchronously, the Orchestrator parses coverage after the round completes, the evasion score is initially computed from time-based signals alone (the base factors in Table 19). When coverage data arrives, the base factor is replaced by a blended value:

$$f_{\text{blend}} = 0.7 \times \frac{\text{coverage}}{100} + 0.3 \times f_{\text{time}}$$

where f_{time} is the original base factor. The per-category formula is then reapplied to f_{blend} (e.g. Evasion becomes $0.6 + 0.4 \times f_{\text{blend}}$, RealDetection becomes $0.4 \times f_{\text{blend}}$), preserving the same non-overlapping ranges. Coverage is weighted arbitrarily at 70% because execution depth, how much of the artifact’s code was actually reached, is a better evasion-quality proxy than wall-clock survival alone. The correction is delivered via a `CoverageCorrection` message from the Orchestrator to the JobWorker, which patches the round’s score in-place so that subsequent selector queries see the updated value.

Token-Driven Feedback Loop

The differential protocol and evasion scoring provide a per-round verdict and fitness signal. The feedback loop converts these into mutation guidance for subsequent rounds, closing the evidence-to-action cycle. The loop is organized as a four-stage pipeline: *Extract*, *Score*, *Guide*, and *Select*.

Evolutionary framing. Following the biological analogy introduced in Section [Evolutionary Search and Genetic Algorithms](#), the build system’s source-level representation (AST, modules, mutation parameters) constitutes the *genotype*, the structure manipulated by subtree swapping, node replacement, and code insertion. The *phenotype* is the observable outcome: runtime behavior (API call sequences, memory protection transitions, timing patterns) together with the EDR’s detection verdict. A mutation is not successful solely if it evades detection; it must also function correctly and resemble a legitimate program to avoid heuristic flagging [18]. The evasion score therefore combines detection avoidance with a payload-functionality bonus: an evasive artifact scores $0.6 + 0.2 \times \text{payload_reached} + 0.2 \times \text{behavior_match}$. Broken artifacts, those that crash on a clean VM during the dryrun phase, are classified as `MutationFailed` and assigned a score of 0, removing them from the learning loop entirely rather than rewarding silent failure. Exploration diversity, ensuring the system does not converge on near-duplicate configurations, is maintained separately by the mutation selectors (Section [Selector Strategies as Ablation Framework](#)).

Token abstraction.

Raw telemetry is normalized into deterministic, versioned strings (“tokens”) such as:

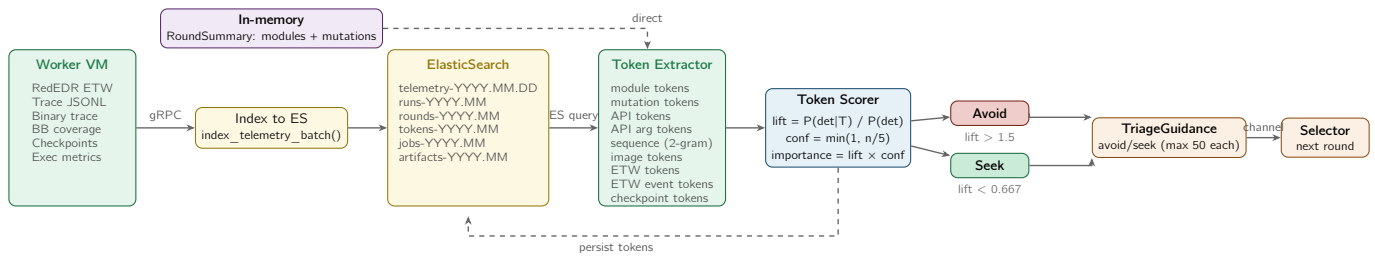


Figure 16: Data lineage from raw telemetry to selector guidance. Six telemetry sources are indexed into ElasticSearch; the token extractor queries three source types (in-memory build inputs, ES API telemetry, ES checkpoint events) to produce nine normalised token categories. The scorer computes lift and confidence per token, classifying tokens as *avoid* (correlated with detection) or *seek* (correlated with evasion), which feed back as TriageGuidance to the next round's selector.

- **Event tokens:** normalized identifiers derived from ETW providers and event types (e.g., process, thread, image-load, memory-management events).
- **Sequence tokens:** short n -grams that represent local ordering patterns in event streams (capturing that some detections depend on ordering, not just presence).
- **Argument-class tokens:** coarse classes of sensitive parameters (e.g., memory protection classes, access rights categories), represented without storing full raw arguments when not needed.
- **Temporal tokens:** relations between events such as short time windows between two classes of actions (capturing timing-sensitive detections).
- **Localization tokens:** execution markers that approximate *where* a detection manifested (e.g., last observed progress point under line tracing).

Implementation status

Event tokens are fully implemented across all ETW provider categories. Sequence tokens currently extract only bigrams ($n=2$). Argument-class tokens cover a subset of security-relevant parameters (memory protection flags, access rights). Temporal and localization tokens are defined in the taxonomy but not yet extracted; they are discussed as future work in § [Additional tokens](#).

These tokens are extracted from three sources: in-memory module and mutation specifications (which modules and parameters were active), Elasticsearch telemetry (ETW events from the run), and Elasticsearch checkpoints (structured stage-reached signals). Additional module-level and mutation-level tokens capture which structural choices were active in each round, enabling attribution of detection outcomes to specific genes.

Tokens are not signatures. Triage tokens are an analytical abstraction used to explain and compare experimental outcomes; they are not vendor detection rules, nor do they imply knowledge of an EDR's internal signatures.

Their purpose is to form testable hypotheses about which observable features correlate with detection, enabling systematic differential analysis and guided mutation.

The resulting pipeline can be summarized as:

Telemetry (ETW/logs/traces) → Normalization
→ Triage Tokens → Differential Ranking
→ Mutation Guidance.

This abstraction makes the feedback mechanism explicit: instead of mutating artifacts blindly, subsequent generations prioritize edits in regions and parameters associated with high-ranked tokens.

Extract–Score–Guide pipeline.

After each trustworthy round, the four-stage pipeline executes asynchronously in the background:

1. **Extract:** token extraction runs from the three sources described above, producing the round's token set.
2. **Score:** each token receives a lift and confidence score (described below).
3. **Guide:** tokens with Lift > 1.5 and Confidence > 0.3 form an **avoid list** (detection-correlated); tokens with Lift < 0.667 and Confidence > 0.3 form a **seek list** (evasion-correlated). The thresholds are reciprocals ($0.667 = 1/1.5$), requiring the same strength of association in both directions: a token must be at least 50% more (or less) likely than the base detection rate to qualify.
4. **Select:** the resulting guidance is delivered as a TriageGuidance message to the job worker before the next round's mutation selection.

Because extraction is asynchronous, the selector initially operates without guidance (pure exploration) and progressively benefits from accumulated evidence as rounds complete.

Scoring and selection. Across many runs, each token T receives an association score with detection outcomes. We adopt *lift*, a standard interestingness measure from association rule mining [1, 71], which measures the ratio of a feature’s conditional detection rate to the overall base detection rate:

$$\text{Lift}(T) = \frac{P(\text{detected} \mid T)}{P(\text{detected})}$$

$$\text{Importance}(T) = \text{Lift}(T) \times \text{Confidence}(T)$$

A lift greater than 1 indicates that the token co-occurs with detection more often than chance (positive association); a lift less than 1 indicates a negative association (correlated with evasion); and a lift of exactly 1 implies statistical independence. Confidence ramps linearly with sample size: $\text{Confidence}(T) = \min(1.0, n_{\text{observations}}/5)$. This downweights lift for small sample sizes, since the variance of conditional probability estimates decreases as the number of observations grows, preventing the system from acting on statistically unreliable associations. High-importance tokens form an **avoid list** (signals correlated with detection). Tokens correlated with non-detection or coverage expansion form a **seek list**. A selector can then score candidate mutation sets as a multi-objective trade-off:

$$f(x) = w_1 \cdot \text{Evasion}(x) + w_2 \cdot \text{Func}(x) + w_3 \cdot \text{Div}(x)$$

A selector can then score candidate mutation sets by balancing evasion performance, payload functionality, and diversity, though each selector strategy weights these objectives differently. The trustworthiness filter separately ensures that broken artifacts and unreliable outcomes never enter the scoring loop.

Design choices that enable feedback.

The build pipeline embeds several structural properties that make closed-loop selection feasible:

- **MutationSpec** is layer-agnostic and serializable (an identifier plus a parameter `HashMap`), so mutation recipes can be stored, replayed, and compared across rounds.
- **BuiltArtifact** captures full provenance: the list of applied mutations, compiler flags, and a SHA-256 content hash.
- `strip_markers_outside_targets()` enables *mutation scoping*: when triage identifies a particular gene (e.g., the carrier) as the detection cause, only that gene’s markers are exposed for subsequent mutation, leaving all other modules frozen.

- Module boundary comments (`// --- BEGIN/END MODULE ---`) enable attribution, mapping a detection outcome back to the gene that contributed the suspicious behavior.

Together, these choices ensure that the build system does not merely *produce* artifacts but also *records* enough metadata to support causal reasoning about why a variant was detected or evaded.

Selector Strategies as Ablation Framework

The feedback loop’s output is consumed by a pluggable *selector*, the component that chooses the next round’s mutation recipe. Four strategies are provided, forming a natural ablation study (Table 20).

Table 20: Four selector strategies and their information sources.

Selector	Algorithm	Uses History	Uses Guidance
Random	Uniform sampling	No	No
Coverage	Epsilon-greedy ($\epsilon=0.3$)	Yes	No
Token	Token-biased ϵ -greedy	Yes	Yes
Fuzzer	Genetic algorithm (tournament $k=3$, crossover)	Yes	No

The four selectors form a layered ablation: **Random** serves as the null hypothesis (no intelligence). **Coverage** adds exploitation by preferring recipes whose prior rounds achieved higher evasion scores, with $\epsilon=0.3$ ensuring 30% of rounds remain exploratory. **Token** augments coverage scoring with the avoid/seek lists from the triage pipeline, plus a novelty bonus for untested configurations. **Fuzzer** replaces the bandit model entirely with an evolutionary search: mutation recipes are treated as chromosomes subject to tournament selection, crossover of parent recipes, and parameter perturbation. The genetic algorithm’s chromosome encoding and evolution phases are detailed in Section [Evolutionary Mutation Selection](#). Comparing evasion rates across these strategies quantifies the marginal value of each intelligence layer.

All four selectors are *stateless*: state arrives via function arguments (round history, triage guidance, search space definition). This ensures fair experimental comparison, no hidden state accumulates differently across strategies.

Safety and Experiment Hygiene

The per-worker execution lock is enforced by the system itself, preventing telemetry contamination from concurrent runs regardless of deployment. The remaining isolation mechanisms checkpoint revert, network air-gap, and VM-level snapshotting are available when workers run on local Hyper-V infrastructure (§. [Lab Infrastructure](#)) but

are not enforced for remote workers. When all mechanisms are active, uncontrolled state drift and uncontrolled egress are eliminated, maximising experimental interpretability.

Summary

AutoMutate++ is designed as an *evidence-first* system: modular, deterministic builds produce controlled search spaces; a three-run differential protocol with seven trustworthiness-classified categories preserves differential validity; a composite evasion score provides a continuous fitness signal; a hardened worker runtime captures partial progress under adversarial termination; a token-driven feedback loop with four pluggable selector strategies closes the evidence-to-action cycle; and token set differencing with parameter distance metrics provides post-hoc causal analysis of detection outcomes.

Implementation

This section describes how each design decision from Section [Design](#) was realized in AutoMutate++. The focus is on non-obvious implementation challenges, key algorithms, and the trade-offs that shaped the final system. The subsections mirror the design chapter's module progression: protocol enforcement, the build pipeline, instrumentation, the worker agent, evasion scoring, and the token-feedback loop.

Controller: Dispatch Reliability Mechanisms

Dryrun Dispatch and Grace Period

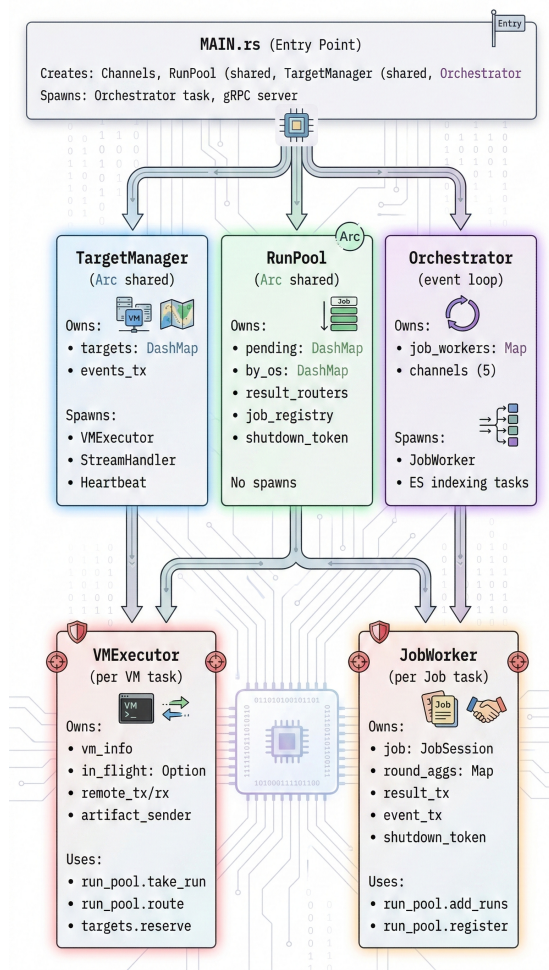


Figure 17: Component Hierarchy & Ownership - Controller

The three-run differential protocol (Section [Three-Run Differential Protocol](#)) requires that each run type reaches the correct VM. Two complementary mechanisms enforce this invariant.

Bidirectional dispatch guard. The RunPool's dispatch loop enforces a strict symmetry between run type and VM capability: a VM whose capability set contains "dryrun" only consumes runs where `is_dryrun=true`, and VMs without this tag never consume dryrun runs. When a VM encounters a non-matching run at the head

of its OS queue, the run is pushed back to the tail, preserving FIFO ordering for other consumers. By design, dryrun VMs are clean machines provisioned without any active EDR or AV, their purpose is to confirm that the artifact executes correctly in the absence of security software, disambiguating functional crashes from EDR kills. The operator declares this by adding "dryrun" to the worker's `extra_capabilities` configuration; conversely, non-dryrun VMs are the EDR-monitored machines where `detect_capabilities()` probes for installed security tools (RedEDR, MDE, Cortex XDR) at startup. On the worker side, `execute_dryrun` reinforces the clean-VM contract by skipping RedEDR activation, telemetry collection, and process monitoring entirely, it spawns the artifact and collects only an exit code. Without this bidirectional guard, a non-dryrun VM (running under active EDR) could consume a dryrun run and misattribute an EDR kill as a functional crash, or conversely a clean dryrun VM could consume a baseline run and produce an unmonitored clean outcome misinterpreted as evasion. The capability tags used for this routing are established at connection time via the worker's stream registration message.

Grace period for the optional dryrun. When the baseline and instrumented results have arrived but the dryrun result is still pending, the JobWorker starts a five-second deadline. If the dryrun result arrives within this window, the round is finalized with all three results; a dryrun crash overrides the differential category to `MutationFailed` or `PayloadFailed` (Section [Differential Categories and Trustworthiness](#)). If the deadline expires, the unclaimed dryrun run is removed from the RunPool and the round is finalized with two results. A 100 ms production tick checks for expired deadlines. The five-second grace period was chosen empirically: shorter values routinely discard valid dryrun results, while longer values reduce throughput by delaying round finalization.

Content-addressed artifact naming. Build artifacts are named by their SHA-256 content hash (Section [Deterministic Compilation Pipeline](#)), providing natural deduplication: when the evolutionary selector (Section [Evolutionary Mutation Selection](#)) produces the same recipe in different rounds, common during the elitism-preserved seeding phase, the identical artifact is not rebuilt or re-uploaded. Artifact files are not deleted after each round because cross-round references may exist; instead, paths are collected in a per-job cleanup list and deduplicated deletion occurs at job completion.

Event-loop priority. The orchestrator's main loop uses Tokio's `biased select` macro with four arms in strict priority order: (1) job control commands (stop/cancel), (2) new job submissions, (3) JobWorker events (round completions, job completions), and (4) target events (VM connect/disconnect, telemetry batches). The `biased` modifier ensures that when multiple channels have pend-

ing messages, the highest-priority arm is always polled first. This eliminates a race condition where a cancelled job's round-completion events, already in the channel buffer, would be processed and indexed to ElasticSearch after the cancellation was acknowledged. With biased polling, the stop command fires the `CancellationToken` before any queued results are consumed, ensuring that the `JobWorker` observes cancellation and ceases emitting new events. The fourth-priority placement of target events (which include high-volume telemetry batches) prevents telemetry indexing from starving job control or submission processing during periods of heavy I/O.

Asynchronous Coverage Correction

The composite evasion score (Section [Composite Evasion Scoring](#)) initially relies on survival time alone. After a round is finalized and indexed, an asynchronous task refines the score with source-level coverage, an eventual-consistency pattern within the experimental loop.

Async spawn and Elasticsearch query. Once the Orchestrator indexes a finalized round, it spawns a detached asynchronous task that queries the trace content from Elasticsearch for the instrumented run. A single retry after one second serves as a safety net for Elasticsearch refresh-latency edge cases.

SourceMap construction. The task parses the assembled C source into a `SourceMap` that identifies function boundaries via heuristic detection of definition patterns. A *cutoff line* is computed as the last carrier line; runtime stubs injected after this point are excluded from coverage accounting. Coverage is defined as the fraction of carrier-region source lines that appear in the trace output. Per-function breakdowns are stored for diagnostic purposes, but only the aggregate coverage value feeds the blended score.

Score correction and in-memory patching. The blended evasion score is computed according to the formula in Section [Composite Evasion Scoring](#), weighting coverage at 70% and the time-based survival factor at 30%. The task sends a `CoverageCorrection` message, carrying the round number, coverage percentage, and blended evasion score, to the corresponding `JobWorker` via a dedicated per-job channel (Appendix [Controller Communication and Synchronization](#)). Upon receipt, the `JobWorker` patches its in-memory round history so that subsequent selector invocations see the improved score. The trade-off is that the selector may operate on a stale time-based score for one to two rounds; this is acceptable because epsilon-greedy exploration (Section [Selector Strategies as Ablation Framework](#)) naturally tolerates transient inaccuracy. This robustness to non-stationary reward signals is a well-known property of ϵ -greedy policies [70].

Builder: Artifact Construction and Mutation Pipeline

Template Assembly

The assembler transforms a template file and a set of selected gene modules (Section [Modular Gene Model](#)) into a single C translation unit ready for compilation. Its implementation is deliberately simple: every `@MODULE:xxx` marker is a unique string that is replaced exactly once via `String::replace`, no regular expressions, no parser.

Module cache. A `HashMap<String, String>` caches the contents of module source files so that repeated builds within the same session avoid redundant file I/O. The `definitions.h` include directive is stripped from each module before insertion, because the template already contains a single top-level include; without stripping, double-inclusion would cause redefinition errors.

Fixed replacement order. Modules are inserted in a fixed sequence: payload header, definitions, decoder, antiemulation, deconditioner, guardrail, virtualprotect, decoy, and finally the carrier. This ordering is significant because the carrier code may reference types or constants defined by earlier modules (e.g., the decoder's output buffer type). Inserting the carrier last guarantees that all dependencies are already present in the translation unit.

Trade-off: no syntactic validation. The assembler performs no validation that the inserted module code is syntactically correct C. Errors surface only at the subsequent Clang compilation step. This is acceptable because modules are hand-written and individually tested; adding a C parser to the assembler would introduce substantial complexity for minimal practical benefit.

Payload Encoding Schemes

Role of encoding in the build pipeline. Before the assembler combines modules into a final C source file, the raw shellcode payload must be transformed into an embeddable C header (`payload.h`). The `PayloadEncoder` (Rust) accepts raw bytes and a chosen encoding type, producing both the encoded byte sequence and the corresponding C decoder constants. This step is deterministic: given the same shellcode and encoding parameters, `prepare_payload` always produces identical output, satisfying the reproducible-build constraint established in Section [Deterministic Compilation Pipeline](#). Four encoding types are supported: None, XOR, English, and SubByte, each targeting a different detection surface.

Rolling XOR encoding. The default scheme applies a 2-byte rolling XOR key (e.g. `[0xAA, 0x55]`) to each payload byte: `encoded[i] = payload[i] ^ key[i % 2]`. The key is emitted as a separate `XOR_KEY[2]` array in the generated header, and the corresponding decoder module (`xor.c`) broadcasts this 2-byte key into an 8-byte mask for bulk processing, XOR-ing 8 bytes per iteration and handling the 0–7 byte tail separately. This widened loop reduces per-byte overhead for large payloads while maintaining identical output to the naive byte-at-a-time loop. The encoder API accepts a custom

key via `with_xor_key()`; this key is then held constant across all rounds of the same job via payload caching (Section [Payload Caching Between Rounds](#)). The fixed key ensures that the same payload always produces the same ciphertext.

```
1  /* xor.c decoder: 8-byte bulk XOR */
2  unsigned __int64 k16 = (unsigned
   ↪ __int64)XOR_KEY[0]
3      | ((unsigned
   ↪ __int64)XOR_KEY[1] <<
   ↪ 8);
4  unsigned __int64 mask = k16 | (k16 << 16);
5  mask = mask | (mask << 32);
6  for (; i < bulk; i += 8) {
7      unsigned __int64 block;
8      __builtin_memcpy(&block, src + i, 8);
9      block ^= mask;
10     __builtin_memcpy(dest + i, &block, 8);
11 }
```

Listing 1: Widened XOR decoding loop from `xor.c`: the 2-byte key is broadcast into a 64-bit mask, XOR-ing 8 bytes per iteration.

English dictionary encoding. English encoding maps each payload byte to one of 256 common English words (the first 60 drawn from a frequency-ranked vocabulary: “the”, “be”, “to”, ...; the remaining 196 are generated tokens `w60-w255`). The encoded output is a space-separated string of words stored as `supermega_payload_str` in the header. The decoder (`english.c`) tokenises this string with `strtok`, performs a linear scan of the 256-entry dictionary for each word, and reconstructs the original byte index. While this linear scan is $O(256n)$ for an n -byte payload, the overhead is negligible for typical shellcode sizes (<10 KB). The key property of English encoding is entropy elimination: the encoded representation consists entirely of printable ASCII words drawn from a small, fixed vocabulary. This makes English encoding a controlled variable for testing whether entropy-based static heuristics contribute to the observed detection rate, though whether any given EDR actually employs such heuristics remains an empirical question that the framework is designed to answer.

SubByte nibble mapping. SubByte encoding [48] splits each payload byte into its high and low 4-bit nibbles and maps each through a 16-entry lookup table of empirically chosen “safe” byte values:

The result is a $2\times$ size expansion: each original byte becomes two encoded bytes, both drawn from a restricted alphabet of 16 values. The decoder (`subbyte.c`) builds a 256-entry reverse lookup at runtime, then reconstructs each original byte from consecutive encoded pairs. The restricted output alphabet was selected to avoid byte values that ML-based classifiers commonly associate

```
1  const unsigned char SUBBYTE_MAPPING[16] =
2      {0, 2, 5, 6, 7, 8, 9, 10, 11, 13, 14, 15,
   ↪ 16, 17, 18, 20};
```

Listing 2: Default sub-byte nibble mapping from `payload.rs`: 16 byte values empirically selected to avoid ML-flagged entropy patterns.

with shellcode (e.g. `0xCC` INT3, `0x90` NOP, `0xFF` indirect call prefixes). Because the mapping is injective (each nibble value maps to a unique output byte), the reverse lookup is unambiguous. SubByte encoding thus provides fine-grained control over the statistical byte distribution of the embedded payload, enabling the system to measure whether specific byte-frequency features drive ML detection scores.

AST Mutations via Tree-Sitter

The `AstMutator` uses Tree-sitter (introduced in Section [Programmatic Code Transformation](#)) to parse C source into a concrete syntax tree, then applies transformations by byte-offset manipulation on the source string. The design rationale for operating at the AST layer is discussed in Section [Three-Layer Mutation Architecture](#). Five implementation challenges merit discussion.

1. Bottom-up application order. Marker-based mutations are applied in *reverse line order* (highest line number first). Because Tree-sitter uses byte offsets into the source buffer, an insertion at line N shifts the byte positions of all content after line N . Processing markers bottom-up ensures that each transformation operates on still-valid offsets. This is a direct consequence of the byte-offset editing model described in Section [Programmatic Code Transformation](#).

2. Two-pass constant obfuscation. The `const_obfuscation` mutation replaces integer literals with references to `volatile` declarations, defeating constant-propagation analysis, following the volatile-decomposition pattern demonstrated by Lepori [48], who showed that integer constants hash values, command tables, and memory protection flags serve as static signature anchors. It requires two passes:

- Pass 1 (inline replacement):** Each integer literal is replaced in-place with a cast expression `(int)__obf_cN`, processing in reverse byte order.
- Pass 2 (declaration insertion):** A `volatile` declaration for each obfuscated constant is inserted before the containing statement, processing in forward line order.

Separating the passes prevents a subtle bug: if declarations were inserted during Pass 1, the byte-offset

shifts would invalidate subsequent inline replacements within the same statement.

3. String XOR runs last. The `string_xor` mutation XOR-encodes all string literals in the source. It must execute *after* all other AST mutations because earlier transformations, `benign_preamble` and `benign_syscall_insert`, introduce new string literals that must also be encoded.

4. Protection macro inlining. Before `const_obfuscation` processes integer literals, it first expands the protection macros `p_RW`, `p_RX`, and `p_RWX` to their numeric values (`04`, `20`, `40`). Tree-sitter parses unexpanded macros as identifiers, not number literals, so without this pre-expansion step, memory protection constants would be missed entirely.

5. Filter chain. The `const_obfuscation` pass skips integer literals that appear in contexts where replacement would produce invalid C: floating-point literals, preprocessor directives, array size expressions, case labels, initializer lists, global-scope constants, and declarations already carrying the `__obf_c` prefix. Each filter was added to prevent a specific class of compilation error discovered during development.

IR Mutations via Text-Based LLVM IR Manipulation

The `IrMutator` operates on `.ll` text files with line-by-line processing, implementing the IR mutation layer described in Section [Three-Layer Mutation Architecture](#). This approach deliberately avoids linking against the LLVM C-API: the `.ll` text format is stable across LLVM versions and human-readable, whereas the C-API changes between releases and requires a matching `libLLVM` at build time.

Three mutations are implemented: NOP insertion, opaque predicates, and junk basic blocks.

Opaque predicates. The core transformation replaces unconditional branches with conditionals that are semantically equivalent but opaque to the optimizer:

```
1 ; Before: unconditional branch
2 br label %target
3
4 ; After (robust mode): optimizer-resistant
  ↳ conditional
5 %__op_N = call i32 @asm_sideeffect "xor $0,
  ↳ $0", "=r"()
6 %__op_cmp_N = icmp eq i32 %__op_N, 0
7 br i1 %__op_cmp_N, label %target, label %target
```

Listing 3: IR opaque predicate: both branch targets are identical, but LLVM cannot prove equivalence through inline assembly.

Both branch targets are identical, the predicate always evaluates to true (`xor reg, reg` yields zero). However, LLVM cannot prove this because inline assembly is treated

as an opaque, side-effecting operation. The “trivial” mode (`icmp eq i32 0, 0`) serves as a control variable: it folds away at `-O2`, confirming that the robust mode’s persistence is due to assembly opacity.

Deterministic insertion. A linear congruential generator with glibc constants (`state × 1103515245 + 12345`) controls which branch instructions are selected for mutation. The same seed produces the same mutation locations and therefore the same artifact, a requirement of the reproducible pipeline (Section [Deterministic Compilation Pipeline](#)).

MSVC-Compatible Build Mode

The default build pipeline cross-compiles from WSL2 using `clang` (targeting `x86_64-pc-windows-msvc`) and the LLVM linker `lld-link`, with Windows headers and libraries provided by the `xwin` SDK. While functionally correct, the resulting PE binaries carry LLD-specific signatures: a synthetic Rich header, an LLD linker version stamp, and characteristic section layouts. These metadata artefacts are known static detection signals for several EDR engines.

When the job-level flag `msvc_compat` is set to `true`, the `JobWorker` constructs an `MsvcCompat` configuration that switches the build pipeline to a two-phase process:

- 1. Compilation.** `clang` is invoked with `--driver-mode=cl`, which makes it behave identically to `clang-cl`: it defines `_MSC_VER`, enables `-fms-compatibility` by default, and accepts MSVC-style flags (`/O2`, `/Fo`, `/imsvc`). The `xwin` SDK headers are passed via `/imsvc` instead of `-isystem`. This step produces a `.obj` file.
- 2. Linking.** The object file is linked by the native MSVC `link.exe`, invoked from WSL2 via `cmd.exe` with a temporary batch script that first calls `vcvarsall.bat x64` to set up the MSVC environment. All WSL paths are converted to Windows paths via `wslpath -wa` before being passed to the linker. The linker flags include `/Brepro` for deterministic PE timestamps, matching the standard pipeline’s reproducibility guarantee.

The MSVC linker embeds genuine Microsoft metadata, an authentic Rich header reflecting the actual MSVC toolchain version, a native linker version stamp, and standard MSVC section layout, making the resulting PE closer from a binary built with Visual Studio. This applies uniformly to all link steps: the single-step `invoke_clang_internal`, baseline linking, and instrumented linking. IR emission remains unaffected: it always uses the standard `clang` driver because `--driver-mode=cl` is incompatible with `-S -emit-llvm`.

The `vcvarsall.bat` path is auto-detected from common Visual Studio 2022 install locations or overridden via the `msvc_vcvarsall` job field. No `/LIBPATH` entries are

passed to **link.exe**; the `vcvarsall.bat` environment sets the native LIB variable, which **link.exe** consults automatically.

Binary Mutations at the PE Byte Level

The `BinaryMutator` takes ownership of the PE byte buffer and manipulates it with manual offset arithmetic, implementing the binary mutation layer from Section [Three-Layer Mutation Architecture](#). After all transforms, the result is validated with `goblin::pe::PE::parse()` to ensure structural correctness. The PE format structures involved are detailed in Section [The Portable Executable \(PE\) Format](#) and Appendix [PE File Format Details](#).

1. Rich header injection. The Rich header occupies the gap between the DOS stub and the PE signature (at offset `e_lfanew`). Injecting a donor Rich header requires shifting all subsequent content and updating the `e_lfanew` pointer. The header is XOR-encoded using a checksum derived from both the DOS header bytes and the embedded compiler records.

2. Consolidated section pattern. Four logically distinct mutations, `string_inject`, `entropy_normalize`, `size_pad`, and `debug_dir`, are merged into a single `.rdata` section rather than creating separate named sections. The rationale is that multiple non-standard section names (e.g., `.strings`, `.padding`) are themselves a detection signal: legitimate MSVC-compiled binaries use a small, predictable set of section names.

3. Import padding. Existing imports are parsed with the `goblin` crate, then a new `.idata` section is built containing both the original Import Directory Table and additional benign entries. The `DataDirectory[1]` pointer is updated to reference the new section. Hint values in the import name table are approximated, the PE loader falls back to binary search when hints are inaccurate, which works correctly but incurs a marginal load-time cost.

4. PE checksum. After all mutations, a PE checksum is computed using the standard 16-bit carry-fold algorithm (matching the behavior of MSVC **link.exe**). The checksum is not a security feature but a compliance signal: Windows validates it for kernel drivers, and some security tools check it for all executables.

5. The `add_section()` primitive. All section-creating mutations funnel through a shared `add_section()` function that handles file alignment (rounding up to `FileAlignment`), section header insertion, `NumberOfSections` increment, and `SizeOfImage` update. Phase-2 mutations (consolidated) collect their data into a single buffer before issuing one `add_section()` call, minimizing structural perturbation.

Benign Catalog: Dependency-Aware API Insertion

The benign catalog [48] implements behavioral obfuscation by inserting sequences of benign Windows API calls into the artifact source. Its goal is to dilute behavioral

fingerprints with innocuous patterns that do not affect payload functionality. Lepori [48] demonstrated that randomly inserting benign system calls between suspicious ones extends execution before detection on Elastic Defend; the benign catalog systematizes this observation with dependency-aware scheduling and topological ordering.

Dependency graph and topological scheduling. The catalog is organized as a `BehaviorGraph`: a directed acyclic graph where nodes are individual API calls and edges represent ordering constraints. For example, the `FileIo` group enforces:

```
1 HANDLE h =
  → CreateFileA("C:\\Windows\\System32\\ntdll.dll",
  → ...); // ID 10
2 ReadFile(h, buf, sizeof(buf), &bytesRead,
  → NULL); // ID 11, depends on 10
3 CloseHandle(h);
  → // ID 12, depends on 11
```

Listing 4: `FileIo` dependency chain: `ReadFile` must follow `CreateFileA`, and `CloseHandle` must follow `ReadFile`.

Inserting `ReadFile` before `CreateFileA` would crash the process. The topological scheduler maintains a *frontier* of nodes whose dependencies are satisfied and selects from it with a seeded xorshift64 PRNG (seed from Table 17). The frontier is kept sorted before random selection to ensure deterministic iteration order.

Declaration deduplication. Variable declarations emitted by the catalog are deduplicated by content using a `HashSet<String>`. All generated variables carry the `__be_` prefix to prevent naming collisions with artifact code.

PEB-walk detection. If the assembled source contains calls to `get_func_by_name` or `get_module_by_name` (indicators of PEB-walk-based API resolution in our template), the catalog restricts itself to the `SystemQuery` group. Groups such as `FileIo` and `RegistryIo` require imported functions that PEB-walk carriers do not have access to; inserting them would produce linker errors.

Builder: Instrumentation and Runtime Infrastructure

Shellcode Stub Prepend

The attribution problem. The loader template allocates memory, decodes the payload, transitions the allocation to executable, and jumps to the decoded shellcode. The last loader-emitted checkpoint before this jump is "Launching". If the EDR terminates the process after this point, the triage system must distinguish two cases: the kill occurred during the final carrier instructions (after the checkpoint but before shellcode entry) versus during shellcode execution itself. Without a signal from the

shellcode, these cases are indistinguishable in telemetry, both produce a `Detected` verdict with "Launching" as the last observed checkpoint. This ambiguity undermines the differential protocol's ability to attribute detection to specific loader behaviours versus payload behaviours

Stub design. To solve this, the builder prepends a 41-byte position-independent stub to the raw shellcode before any further processing. The stub consists of 24 bytes of x86-64 machine code followed by a 17-byte null-terminated string ("payload_executed\0"). The carrier module passes the address of `__artifact_checkpoint` in the RCX register (the first argument in the Windows x64 calling convention). The stub saves RBX, moves the function pointer to RBX, allocates 32 bytes of shadow space, loads the address of the embedded string via a RIP-relative `lea` instruction, calls the checkpoint function, restores the stack, and falls through to the original shellcode via a short `jmp` over the string data:

```

1 0x00 push rbx
2 0x01 mov rbx, rcx           ; save
   ↳ checkpoint fn ptr
3 0x04 sub rsp, 0x20          ; shadow space
   ↳ (Win64 ABI)
4 0x08 lea rcx, [rip+0x09]    ; ->
   ↳ "payload_executed\0"
5 0x0F call rbx              ;
   ↳ checkpoint("payload_executed")
6 0x11 add rsp, 0x20
7 0x15 pop rbx
8 0x16 jmp short +17         ; skip string
   ↳ data
9 0x18 db "payload_executed", 0 ; 17 bytes
10 0x29 ... original shellcode ...

```

Listing 5: Checkpoint stub prepended to shellcode: emits "payload_executed" via the callback pointer passed in rcx (Win64 ABI), then falls through to the original shellcode at offset 0x29.

The `lea rcx, [rip+0x09]` instruction computes the string address at runtime using PC-relative addressing, ensuring the stub remains position-independent regardless of where the shellcode is loaded in memory. The `jmp short +17` displacement equals exactly the length of the embedded string, which is verified by a compile-time unit test in `shellcode_stub.rs`.

Pipeline ordering. The stub is prepended as the first transformation in `prepare_payload`, before both INT3 checkpoint patching and encoding:

raw shellcode $\xrightarrow{\text{prepend stub}}$ $\xrightarrow{\text{INT3 patch}}$ $\xrightarrow{\text{encode}}$ payload.h

This ordering is mandatory for two reasons. First, INT3 patching inserts breakpoints at evenly-spaced offsets within the shellcode body and needs to know that the

first STUB_SIZE (41) bytes are the checkpoint stub, these bytes are excluded from candidate breakpoint positions. Second, the encoder must operate on the final byte sequence (stub + patched shellcode) so that the decoder in the artifact recovers the exact bytes that include the stub. Reversing the order would either corrupt the stub's machine code (if encoding preceded stub prepend) or place breakpoints inside the stub (if INT3 patching did not account for the stub offset).

```

1 // build/src/builder.rs - prepare_payload
   ↳ pipeline
2 let payload_to_encode = if instrumented {
3     prepend_checkpoint_stub(payload) // Step
   ↳ 1: stub
4 } else { payload };
5 let (final_buf, sc_header) = if sc_checkpoints
   ↳ > 0 && instrumented {
6     patch_shellcode(&mut buf, count,
   ↳ STUB_SIZE) // Step 2: INT3
7 } else { (payload_to_encode, None) };
8 let encoded = encoder.encode(&final_buf,
   ↳ encoding); // Step 3: encode

```

Listing 6: Payload preparation pipeline (simplified from `builder.rs:126-175`): stub prepend, optional INT3 checkpoint patching, then encoding. Each step feeds into the next; the output is cached as a `PreparedPayload`.

Baseline exclusion. The stub is only prepended when `trace_mode != "off"`. Baseline artifacts execute raw shellcode without any checkpoint infrastructure: the `EXECUTE_SHELLCODE` macro invokes the shellcode with no arguments, so the rcx register does not contain a valid callback pointer. If the stub were present in baseline builds, its `call rbx` instruction would invoke an uninitialised register value, crashing the process before any payload logic executes.

Beyond this correctness requirement, excluding the stub preserves the differential protocol's invariant that baseline and instrumented builds differ only in observability, not in behaviour: the stub's additional code, stack manipulation, and string data could introduce noise, both static (byte patterns, import references, section entropy) and dynamic (ETW events, call-stack heuristics), that alters the EDR's detection decision. This concern applies equally to every instrumentation component excluded from baseline builds.

Shellcode Checkpoint Patching

The `patch_shellcode()` function inserts INT3 breakpoints into raw shellcode to serve as execution-progress checkpoints. The VEH-based checkpoint mechanism that handles these breakpoints at runtime is described in Appendix [Vectored Exception Handling and INT3 Checkpoints](#).

Recursive descent disassembly. The implementation uses the `iced-x86` crate for *recursive descent* disassembly rather than linear sweep. Linear sweep decodes instructions sequentially from the entry point, which misidentifies inline data (API name hashes, configuration blobs, encrypted strings) after unconditional jumps as code. Shellcode commonly embeds such data inline. Recursive descent follows control flow, branches, calls, and conditional jumps, and only visits reachable code regions, ensuring that data regions are never corrupted by INT3 insertion.

Even-spacing formula. Checkpoints are distributed uniformly across the shellcode body using:

$$\text{interval} = \max(1, \lfloor (\text{usable} + 1) / (\text{count} + 1) \rfloor)$$

where *usable* is the number of candidate instruction offsets and *count* is the requested checkpoint count. The entry point (first instruction) is never patched, it must execute to reach the VEH checkpoint stub. Each checkpoint carries a `progress_pct` field that enables triage to report *how far* shellcode executed before EDR terminated the process.

Payload Caching Between Rounds

The encoding pipeline, stub prepend, INT3 patching, encoding, C header generation, is deterministic: identical inputs produce identical output. Since the four inputs to `prepare_payload()` (raw shellcode bytes, encoding type, trace mode, and checkpoint count) are job-level constants that do not vary across rounds, the `JobWorker` caches the result and reuses it for every subsequent build within the same job.

Three cache slots are maintained on the `JobWorker` struct:

1. **cached_payload** Raw shellcode bytes, read from disk once on the first build call and cloned into every subsequent `BuildInput`.
2. **baseline_payload** A `PreparedPayload` (the generated payload.h header string and optional checkpoint table header) for builds where `trace_mode = "off"`. Lazily initialised on the first baseline or dry-run build.
3. **instrumented_payload** The same, but for builds where `trace_mode ≠ "off"`. These differ because `prepare_payload()` prepends the checkpoint stub to the shellcode and the optional patches INT3 breakpoints before encoding.

Two `PreparedPayload` slots suffice because only two trace-mode variants exist within a job: baseline ("off") and instrumented ("lines"). Each is computed at most once via lazy initialisation: on the first build of that variant, the slot is populated; all subsequent rounds of the same variant clone the cached value and pass it as

`precomputed_payload` to the builder, which skips the encoding pipeline entirely.

Caching is gated on the `cache_payload` flag (default: true). When set to false, `precomputed_payload` is passed as `None` and the builder re-runs the full encoding pipeline on every round. This exists as a debugging escape hatch; under normal operation the determinism guarantee makes caching safe:

$$\text{prepare_payload}(p, e, t, c) = \text{prepare_payload}(p, e, t, c) \\ \forall \text{ invocations with identical } (p, e, t, c) \quad (1)$$

Instrumentation Runtime

AFL-style edge coverage bitmap. The instrumentation runtime (`instrumentation_runtime.c`) maintains a 64 KB bitmap (`__coverage_map`) that records edge transitions between basic blocks, following the AFL coverage model. Each basic block is assigned a unique ID, either statically by the LLVM SanitizerCoverage pass (`__sanitizer_cov_trace_pc_guard_init` assigns IDs 1 through *N* to the guard array) or dynamically from the return address (`__sanitizer_cov_trace_pc` truncates the caller's return address to 32 bits). On each BB entry, the runtime computes an edge hash:

```
1 uint32_t edge = (__coverage_prev_bb << 1) ^
   ↳ bb_id;
2 uint32_t idx = edge % COVERAGE_MAP_SIZE;
3 if (__coverage_map[idx] < 255)
4     __coverage_map[idx]++;
5 __coverage_prev_bb = bb_id;
```

The left-shift of `prev_bb` before XOR ensures that the edge $A \rightarrow B$ hashes differently from $B \rightarrow A$, capturing directional control flow. Hit counts saturate at 255 (single byte) to prevent overflow. Separately, a linear array (`__bb_ids / __bb_hit_counts`, up to 1024 entries) tracks per-BB hit counts for the coverage metadata report (`coverage_bbs.txt`).

Incremental BB flush. In a conventional coverage-guided fuzzer, the bitmap is flushed once at process exit via `atexit`. This is insufficient for EDR evaluation: if the EDR kills the process before `atexit` runs, the entire coverage bitmap is lost. The runtime therefore performs an incremental flush every `BB_FLUSH_INTERVAL` (configurable, default is 50) basic block executions. Each incremental flush overwrites `coverage.bin` with the latest bitmap snapshot. The 50-BB interval represents a trade-off: lower values increase file I/O overhead (each flush writes 64 KB plus metadata), while higher values risk losing more coverage data if the process is killed. At 50 BBs, the maximum data loss is the coverage accumulated in the last 49 BB transitions, typically a few milliseconds of execution.

Conditional compilation and weak-symbol linkage. The two behaviourally distinct binaries: one that records telemetry (instrumented) and one that is telemetry-free (baseline) are produced with a single preprocessor flag, `ENABLE_INSTRUMENTATION`, set by the builder when `trace_mode` $\neq$ `off`.

When defined, the macros in `instrumentation.h` expand to real function calls (`__artifact_checkpoint`, `__artifact_success`, `__artifact_failure`), and `minimal_runtime.h` is included to declare the direct-syscall exit function `__runtime_exit`. When undefined, each macro expands to `((void)0)`, a no-op, and `minimal_runtime.h` is not included, ensuring no runtime symbols are referenced.

```
1 // minimal_runtime.c - weak-symbol conditional
  ↪ flush
2 __attribute__((weak)) extern void
  ↪ __coverage_flush(void);
3 __attribute__((weak)) extern void
  ↪ __trace_flush(void);
4
5 void __runtime_exit(int exit_code) {
6     if (__coverage_flush != NULL)
7         ↪ __coverage_flush();
8     if (__trace_flush != NULL)
9         ↪ __trace_flush();
10    // ... direct syscall exit
11 }
```

At the linking stage, the builder enforces a strict separation. Instrumented builds link two additional object files: `minimal_runtime.o` (providing `__runtime_exit`) and `instrumentation_runtime.o` (providing BB coverage callbacks, line-trace I/O, and checkpoint recording). Baseline builds link *neither* of those two. The baseline exits via CRT `exit()`.

Within `minimal_runtime.c`, the three telemetry flush functions (`__coverage_flush`, `__trace_flush`, `__checkpoint_flush`) are declared with the `__attribute__((weak))` linkage specifier. When `instrumentation_runtime.o` is co-linked, its strong definitions prevail; if it were absent, the weak symbols would resolve to `NULL` and `__runtime_exit` would skip the flush calls. In practice, both object files are always co-linked for instrumented builds, so this mechanism serves as a safety net rather than the normal path.

Direct syscall exit. Instrumented artifacts terminate via `__runtime_exit()` which bypasses all usermode hooks by issuing the `syscall` instruction directly.

The implementation consists of three components. First, `GetSyscallNumber()` dynamically resolves the syscall number for a given `ntdll.dll` export by parsing its stub bytes. If parsing fails, a hardcoded fallback is used: `0x2C` for `NtTerminateProcess` and `0x0F` for `NtClose`.

Second, `DirectSyscall2()` is a naked function that

sets up the Windows x64 syscall calling convention: it moves the first argument to `r10`, loads the syscall number into `eax`, shuffles the remaining arguments into `rcx` and `rdx`, and executes the `syscall` instruction.

Third, `__runtime_exit()` orchestrates the sequence: it flushes all available telemetry (checking each weak symbol for `NULL`), waits 50 ms for file writes to complete (`Sleep`(50)), initialises syscall numbers if needed, and invokes `DirectSyscall2` with `NtTerminateProcess` using the pseudo-handle `-1` for the current process. If the direct syscall fails, it falls back to `ExitProcess()`.

Deferred Loop Tracing

The line tracer (part of the instrumentation design in Section [Instrumentation Design: Evidence That Survives Process Death](#)) must handle loops that run many iterations without overwhelming the named pipe with redundant trace events.

Problem. Naive per-iteration tracing of a 100-iteration loop with 10 traced lines produces 1 000 trace events per invocation. With multiple loop pattern variants active, this overwhelms the named pipe and makes telemetry files unusable for analysis.

Solution: flag-set-in-loop, check-after-loop. Instead of emitting a trace event on every iteration, the tracer sets a static flag inside the loop and checks it after the loop exits:

```
1 static int __seen_L42 = 0;
2 for (int i = 0; i < DECON_ROUNDS; i++) {
3     __seen_L42 = 1;           // flag
4     ↪ set, not a trace call
5     /* ... loop body ... */
6 }
7 if (__seen_L42) __trace_line_binary("file",
8     ↪ 42, __func__); // one event
```

Listing 7: Deferred loop tracing: $O(1)$ trace events per unique source line, regardless of iteration count.

This guarantees $O(1)$ trace events per unique source line regardless of the iteration count.

Exceptions. `return` and `goto` statements inside loops retain eager (per-iteration) tracing. These control-flow exits leave the loop before the deferred check executes, so the flag-based approach would silently drop coverage for the exiting iteration. The tracer detects these constructs during AST analysis and falls back to inline tracing for the affected lines.

Named Pipe Trace Transport

Pipe lifecycle and asymmetric creation. The primary real-time telemetry channel between an instrumented artifact and the worker agent is a Windows named pipe

(\\.\pipe\rededr_trace). The worker agent creates the pipe server *before* spawning the artifact process, using 1 MB buffers for both input and output, with up to 5 creation retries at 200 ms intervals to handle stale pipe handles from previous runs. The artifact opens the pipe with `OPEN_EXISTING` semantics; if the pipe is unavailable, the runtime falls back to file output (`trace.log` in the current directory, with Windows and Unix fallback paths). This asymmetric creation model, server before client, guarantees that the pipe is ready to receive data before the artifact emits its first trace event, eliminating a race condition that would otherwise cause early trace events to be lost.

The protocol details, magic-based auto-detection and binary ISTR record format, are described in Appendix [Named Pipe Trace Protocol](#).

Aggressive flush and death-bed capture. The artifact flushes the pipe buffer after *every* trace event (`__trace_flush()` is called at the end of each `__trace_line_binary` and `__trace_line_b64` invocation) rather than batching writes. This per-event flush calls both `WriteFile` and `FlushFileBuffers` on the pipe handle, forcing the OS to deliver the data to the reader immediately. The trade-off is measurable overhead on tight loops: a loop body that executes thousands of iterations generates one pipe write per iteration. However, combined with the deferred loop tracing optimisation described in Section [Deferred Loop Tracing](#), which collapses N loop iterations into $O(1)$ trace events, the effective write frequency remains bounded. The critical benefit is that when an EDR terminates the process, the last event successfully delivered to the pipe is within one trace call of the actual kill point, the *death-bed telemetry* property. Without aggressive flushing, a buffered approach could lose hundreds of events, making it impossible to localise the detection trigger to a specific source line.

Separate checkpoint pipe. A second named pipe (\\.\pipe\rededr_checkpoints) carries low-frequency structured events: `__artifact_checkpoint(name)`, `__artifact_success(msg)`, and `__artifact_failure(msg, code)`. Each event is a single JSON line with a microsecond timestamp, checkpoint name, and event type:

```
1 {
2   "ts_us":1284507,
3   "checkpoint":"carrier_start",
4   "type":"artifact_checkpoint"
5 }
```

Separating checkpoint events from high-volume line traces serves two purposes. First, it simplifies the worker's parsing pipeline: the trace collector processes a homogeneous stream of either binary ISTR records or Base64 markers, while the checkpoint collector processes newline-

delimited JSON. Second, it avoids interleaving structured JSON with binary data on the same pipe, which would require framing logic to distinguish record boundaries. The checkpoint pipe uses the same asymmetric creation model (worker creates before artifact connects) with identical fallback-to-file behaviour.

Worker Agent Implementation

Resource safety via RAIL guards. The RAIL cleanup contract described in Section [Worker Agent Design: Robust Execution as a Ten-Phase Pipeline](#) is realized through three guards, each addressing a non-trivial resource-management problem.

The ProcessGuard implements a two-stage kill sequence. First, `taskkill /F /T /PID` force-terminates the entire process tree, necessary because the artifact may spawn child processes via `CreateProcess`, and Tokio's `child.kill()` only terminates the immediate child, not its descendants. A 500 ms pause follows to allow the OS to reclaim process resources before the guard proceeds. As a fallback, Tokio's cross-platform `child.kill()` is invoked to handle any remaining orphan.

The MonitorGuard enforces a carefully ordered shutdown sequence to prevent deadlock. The monitor's event consumer task is aborted *first*, and only then is the monitor task itself stopped and awaited. Reversing this order would deadlock: the monitor would attempt to send an event to a full channel while the consumer, blocked waiting to read from the same channel, is never drained.

The ExecutionLockGuard faces a Rust-specific challenge: the `Drop` trait is synchronous, but the execution lock (`tokio::sync::Mutex`) requires an asynchronous `.lock()` call. The guard resolves this by capturing the Tokio runtime handle at construction time; in its `Drop` implementation, it spawns a detached task that performs the async lock release. This ensures the lock is always released even if the execution pipeline panics.

Telemetry deduplication and truncation. Two post-processing steps are applied to trace telemetry before transport, realizing the packaging logic described in Phase 9 of Section [Worker Agent Design: Robust Execution as a Ten-Phase Pipeline](#). Deferred loop tracing (Section [Deferred Loop Tracing](#)) already collapses repetitive loop iterations to $O(1)$ events per source line at emission time; the steps below handle residual redundancy that persists after that reduction.

Even after deferred tracing, execution traces from long-running artifacts retain redundancy across distinct call sites that revisit the same source line, and a single informative record per key suffices. The deduplication algorithm keys on the triple (file, line, function) and retains only the record with the highest sequence number per key. A count field records multi-hit lines for downstream analysis. After deduplication, records are sorted by sequence number to restore chronological order. The choice to retain the highest (most recent) sequence

number is deliberate: if the security product terminates the artifact at a specific line, the most recent execution of that line carries the sequence number closest to the termination point, the information the triage engine needs for detection localization.

Even after deduplication, traces from long-running artifacts may exceed the 4 MB gRPC message size limit. The truncation algorithm uses a binary-search approach: it serializes the current trace slice, checks its size against a 3.5 MB threshold (leaving headroom for Protobuf framing), and if oversized, halves the slice from the beginning, advances to the next newline boundary to avoid splitting a JSON record, and retries. The algorithm truncates from the *head* (earliest events) and preserves the *tail* (most recent events). This is the opposite of naive truncation: detection-relevant behavior clusters near the end of execution, so the sequence of API calls immediately preceding process termination is more informative for triage than the initialization sequence. Truncated payloads are annotated with metadata (compression: "truncated_tail", sent_lines, original_size_bytes) so that downstream processing can account for the lost data.

Idle detection. The ExecutionMonitor implements a two-signal idle detection algorithm that polls at 3-second intervals to distinguish genuine stalls from artifacts performing useful but telemetry-silent work. At each poll, two conditions are evaluated: (1) event staleness, whether the RedEDR event count has changed since the last poll while the process is still alive, and (2) CPU idleness, whether the process's CPU usage is below a 5% threshold.

The conjunction of both signals drives the decision: if events are stale *and* CPU is idle, the idle counter increments, the process is truly inactive. If events are stale but CPU is *not* idle, the counter resets, the process is computing without producing ETW events, and should not be killed. If new events have arrived, the counter also resets regardless of CPU load. When the counter reaches 3 (representing at least 9 seconds of continuous inactivity), the monitor emits an idle-detection signal that triggers process termination via the three-way race described in Section [Worker Agent Design: Robust Execution as a Ten-Phase Pipeline](#). Idle detection is enabled by default but can be disabled; when active, it accelerates the fuzzing loop by terminating quiescent processes well before the full timeout expires, allowing the worker to proceed directly to verdict classification with the corresponding exit code.

Triage and Feedback Loop

Token Extraction and Scoring Pipeline

Three extraction sources. Token extraction draws from three distinct data sources. *In-memory tokens*, module selections and mutation specifications, are extracted synchronously from the RoundSummary struct without

any I/O. Each of the seven module slots produces a `module:<slot>=<impl>` token, and each applied mutation produces a `mutation:<id>:<key>=<value>` token. *ES telemetry tokens* are queried asynchronously from Elasticsearch against the *baseline* run's indexed events, yielding API calls, API arguments, bigrams, image loads, and ETW events. *ES checkpoint tokens* are queried against the *instrumented* run's checkpoint events, producing `checkpoint:<name>` tokens. This split reflects an intentional design choice: behavioural telemetry is drawn from the baseline run, which carries no instrumentation overhead, to ensure the tokens represent unperturbed EDR observations, while checkpoint tokens are drawn from the instrumented run, the only variant that records them. In principle, telemetry tokens could be extracted from both runs and merged; the current single-run approach halves the ES query cost per round without loss of information, and the extraction pipeline is trivially extensible to dual-run extraction if future analysis requires it.

Deterministic mutation token identity. Mutation tokens must be deterministic so that the same mutation applied in different rounds produces the same token string, enabling cross-round correlation. When a MutationSpec includes parameters (e.g. {count: 50, method: "fixed"}), the extractor sorts parameter keys alphabetically before concatenation:

```
mutation:ast.decon_rounds:count=50:method=fixed
```

Values are serialised as: strings verbatim, numbers and booleans via `to_string()`, and complex values (arrays, objects) as compact JSON. An empty or absent parameter object produces a bare `mutation:<id>` token with no parameter suffix. This sorted-key normalisation ensures that parameter insertion order in the JSON object, which could be non-deterministic in many serialisation libraries, does not affect token identity.

Bigram construction with stateful tracking. API-call bigrams `seq2:<prev>+<func>` are constructed by maintaining a `prev_func` variable across the sorted telemetry document stream. Importantly, ETW events (which have `payload_func = null`) do *not* reset this chain. If the event stream contains [NtAllocateVirtualMemory, an ETW event, NtProtectVirtualMemory], the ETW event (which has `payload_func = null`) is skipped for bigram purposes, yielding a single bigram `seq2:NtAllocateVirtualMemory+NtProtectVirtualMemory`, the ETW event is skipped for bigram purposes because it represents a provider-level observation, not a usermode API call in the artifact's execution flow.

This design captures the artifact's actual API-call ordering. All tokens within a single run are deduplicated via a HashSet in insertion order, so repeated calls to the same API (e.g. multiple `NtAllocateVirtualMemory` invocations) produce only one `api:` token but may contribute

multiple distinct bigrams. A frequency count per token could be added if needed, though the planned temporal tokens (e.g. `dt:write+protect_ms<20`) would already, in a sense, capture positional and timing information that subsumes raw repetition counts.

Trustworthiness filter. Not all differential categories produce reliable training signal. The `extract_and_score` function filters the accumulated round history before building the token-round matrix: only rounds classified as `RealDetection`, `Evasion`, or `StaticDetection` are included. Rounds classified as `InstrumentationArtifact` (where instrumentation itself caused detection), `Flaky` (non-reproducible results), `MutationFailed`, or `PayloadFailed` are excluded because their detection outcomes cannot be reliably attributed to the artifact's behavioural features. This filter is applied via the `DifferentialCategory::is_trustworthy()` predicate, ensuring that the scoring model trains only on rounds where the causal link between tokens and detection is credible.

Degenerate lift guard. The scorer (`scorer.rs`) computes lift as $\text{lift}(T) = P(\text{detected} \mid T) / P(\text{detected})$. When the base detection rate $P(\text{detected})$ is 0 (all rounds evaded) or 1 (all rounds detected), lift is either undefined (division by zero) or trivially 1.0 for all tokens. In either case, the scorer returns an empty score vector and produces empty guidance (no avoid, no seek tokens). This guard prevents the selector from receiving degenerate guidance that would dominate early rounds (when only 1–2 rounds have completed and all share the same outcome). As the campaign accumulates rounds with mixed outcomes, the base rate moves away from the boundary, and meaningful lift values emerge naturally. The guidance thresholds (avoid: $\text{lift} > 1.5$, seek: $\text{lift} < 0.667$, both requiring confidence > 0.3 , i.e. at least 2 observations) provide additional protection against premature generalisation from sparse data.

Token Set Differencing

Zeller's delta debugging[37] minimises a failure-inducing input by systematically removing subsets and re-testing until a minimal failure-inducing difference is isolated. Token set differencing applies the same logical structure to round data: the "input" is a round's complete token set (module, mutation, API, ETW, and checkpoint tokens); the "test" is the EDR verdict; the "changed subsets" are the symmetric differences produced by `compare_token_sets()`. Crucially, this is *not* runtime delta debugging, there is no iterative minimisation loop. It is a post-hoc analytical lens applied to accumulated round data using set-differencing primitives, enabling a researcher to ask: "What observables changed between two rounds whose outcomes differed?"

Sequential round comparison. When round N was detected and round $N+1$ evaded (or vice versa), the func-

tion `compare_token_sets` partitions the observable space into three disjoint subsets:

- **Only in A** (round N only) tokens that disappeared when the outcome changed. These are candidate detection triggers: their presence correlated with detection, and their absence correlated with evasion.
- **Only in B** (round $N+1$ only) tokens that appeared when the outcome changed. These are candidate evasion enablers.
- **Common** tokens present in both rounds. These are invariant across the outcome change and therefore not explanatory for the specific transition.

A scalar summary accompanies the partition: the *Jaccard distance*

$$1 - |A \cap B| / |A \cup B|$$

where values near 0.0 indicate near-identical token sets and values near 1.0 indicate near-complete divergence.

Worked example. Consider two consecutive rounds from a campaign:

	Round 5 (Detected)	Round 6 (Evasion)
Mutation	<code>mutation:ast.ru:count=2</code>	<code>mutation:ast.ru:count=200</code>
Module	<code>module:decond=none</code>	<code>module:decond=alloc_loop</code>
API arg	<code>api_arg:VirtProt:RWX</code>	<code>api_arg:VirtProt:R-X</code>
API	<code>api:VirtualAlloc</code>	<code>api:VirtualAlloc</code>

The partition yields:

$$A \setminus B = \{ \text{mutation:ast.ru:count=20,} \\ \text{module:decond=none,} \\ \text{api_arg:VirtProt:RWX} \}$$

$$B \setminus A = \{ \text{mutation:ast.ru:count=200,} \\ \text{module:decond=alloc_loop,} \\ \text{api_arg:VirtProt:R-X} \}$$

$$A \cap B = \{ \text{api:VirtualAlloc} \}$$

$$|A \cup B| = 7, \quad |A \cap B| = 1, \quad d_J = 1 - \frac{1}{7} \approx 0.857$$

The high Jaccard distance (0.857) reflects substantial divergence. The `only_in_a` tokens, `module:deconditioner=none` and the RWX protection argument, are immediate candidates for investigation as detection contributors.

Cross-outcome comparison. Beyond sequential round pairs, a researcher can select the highest-evasion-score round and the highest-detection-score round from an entire campaign and diff their token sets. This isolates tokens associated with the widest outcome gap.

Cross-outcome comparison is complementary to lift scoring (Section [Token-Driven Feedback Loop](#)): lift aggregates statistical signal across *all* rounds, while the cross-outcome diff compares two *specific* rounds at instance level, surfacing concrete examples of detection-relevant differences.

Per-mutation parameter drill-down. When both rounds contain the same mutation identifier, `compare_token_sets` delegates to the `MutationParamSpace` registry for per-parameter normalised distance (Section [Mutation Parameter Distance](#)). For example, if both rounds used `ast.decon_rounds` but with `count=20` versus `count=200`, the parameter distance is:

$$d_{\text{count}} = \frac{|20 - 200|}{500 - 5} = \frac{180}{495} \approx 0.364$$

Mutations present in only one set receive `overall_distance = 1.0` (maximal difference).

Relationship to lift scoring. `compare_token_sets` is complementary to the lift computation in the token-driven feedback loop (Section [Token-Driven Feedback Loop](#)). Lift answers a frequentist question: “What correlates with detection across all accumulated rounds?” Token set differencing answers an instance-level question: “What changed between these two specific rounds?” The two analyses serve different purposes:

Dimension	Lift scoring	Token set differencing
Scope	All trustworthy rounds	Two specific rounds
Output	Per-token importance ranking	Symmetric difference + Jaccard
Question	“What generally matters?”	“What changed here?”
Strength	Statistical robustness	Causal specificity

Currently, `compare_token_sets` is exposed only via the `CompareTokens` gRPC API and is **not** consumed by any selector or feedback path. It serves as a query-only analytical tool for post-hoc investigation (Appendix [Token Comparison Pipeline](#)).

Mutation Parameter Distance

Parameter space registry. The function `default_registry()` defines parameter spaces for all implemented mutations (Section [Evolutionary Mutation Selection](#)). Each entry specifies the mutation identifier and an ordered list of typed parameter definitions. Three parameter types are supported:

- **IntRange:** discrete integer bounds, e.g., `ast.decon_rounds.count` $\in [5, 500]$.
- **FloatRange:** continuous real bounds, e.g., `llvm.nop_insert.density` $\in [0.0, 1.0]$.

- **Categorical:** enumerated choices, e.g., `ast.fill_pattern.pattern` $\in \{\text{xor}, \text{nop_sled}, \text{random}, \text{zero}\}$.

Table 33 in Appendix [Mutation Catalog](#) lists the explored AST mutations and their parameter definitions.

Distance normalisation. `ParamDef::distance()` produces a normalised $[0, 1]$ scalar for any two values of the same parameter:

$$d_{\text{numeric}}(a, b) = \text{clamp}\left(\frac{|a - b|}{\max - \min}, 0, 1\right)$$

$$d_{\text{categorical}}(a, b) = \begin{cases} 0 & \text{if } a = b \\ 1 & \text{otherwise} \end{cases}$$

The function `compare_params()` computes the arithmetic mean of per-parameter distances for a given mutation. Missing keys in either parameter map fall back to the parameter’s declared default value. This yields a single dimensionless distance in $[0, 1]$ for any two configurations of the same mutation, enabling comparison across heterogeneous parameter types.

Current binary novelty. The `TokenSelector` (Section [Selector Strategies as Ablation Framework](#)) applies a binary novelty bonus of +0.4 for mutations never previously tried in trustworthy rounds. This is all-or-nothing: once a mutation has been tried in any configuration, its novelty bonus drops to zero regardless of how different the new configuration might be. No parameter distance is consulted.

Design contribution: graduated distance-based novelty. The binary novelty scheme discards information: a mutation tried once with `count=5` is treated identically to one exhaustively tested across its full parameter range. A graduated alternative would compute the mean parameter distance from a candidate configuration to all prior configurations of the same mutation.

Let $\delta(c, h)$ denote the `overall_distance` field returned by `compare_params(c.params, h.params)`:

$$\text{novelty}(c, H) = \frac{1}{|H_m|} \sum_{h \in H_m} \delta(c, h)$$

where H_m is the set of history entries using mutation m .

This yields a continuous value in $[0, 1]$: 0 when the candidate duplicates a prior configuration exactly, 1 when it is maximally distant from all prior configurations. The selector could then scale its novelty bonus continuously rather than applying a fixed +0.4 step.

Implementation status

This graduated novelty scheme is a *design contribution* it is not implemented in the current codebase. The infrastructure required (parameter distance computation via `compare_params`, history tracking in selectors) already exists; only the integration into the selector scoring function is absent.

Minimum-step analysis. Between a detected and evading round that share the same mutation identifier, the parameter distance quantifies the smallest behavioural change that flipped the detection outcome. This enables minimum-step analysis: identifying which parameters have the most leverage over detection.

Consider two rounds both using `ast.timing_pattern`:

	Round A (Detected)	Round B (Evasion)
<code>min_ms</code>	10	200
<code>max_ms</code>	100	2000

Applying the normalised distance from the preceding paragraph yields $d_{\text{min_ms}} = 0.38$ and $d_{\text{max_ms}} \approx 0.955$, so $d_{\text{overall}} = 0.668$. The large gap reveals that `max_ms` carries more leverage than `min_ms`. If instead only `min_ms` changed ($10 \rightarrow 50$) while `max_ms` stayed at 100, the overall distance drops to 0.04, confirming that `max_ms` dominates the behavioural shift. By ranking per-parameter contributions this way, the system could guide future exploration toward high-leverage parameters.

Relationship to FuzzerSelector. The `FuzzerSelector` uses `perturb_params()` to introduce random variation during genetic mutation, but does not compute distance from prior configurations. Perturbation controls the *step size* (how far to move from the current point), while distance-based novelty would control the *step direction* (whether to move toward explored or unexplored regions). The graduated novelty metric would complement the genetic algorithm: perturbation provides local search around promising configurations, while novelty-based guidance provides global coverage of the parameter space.

Evolutionary Mutation Selection

The `FuzzerSelector` implements the evolutionary strategy introduced in Section [Selector Strategies as Ablation Framework](#) (Table 20) by treating each mutation recipe as a chromosome.

Genome encoding. Each gene corresponds to a `MutationSpec` entry: the recipe carries *fixed genes* inherited from the template and *pool genes* sampled from the search space. Alleles are mutation parameters drawn from a typed `ParamDef` registry that supports three value kinds, `Categorical`, `IntRange`, and `FloatRange`, each providing `sample`, `perturb`, and `distance` operations. The default registry contains 22 mutations

spanning the three layers (Section [Three-Layer Mutation Architecture](#)).

Seeding and evolution phases. Evolution proceeds in two phases. During seeding (rounds 2 through `population_size+1`), each recipe is generated randomly: pool genes are drawn via a Fisher–Yates partial shuffle over the search space, and parameters are sampled uniformly from their definitions. This ensures a diverse initial population before selection pressure is applied. In the evolution phase (subsequent rounds), two parents are chosen by tournament selection ($k=3$). Fixed genes are inherited from a randomly selected parent; pool genes are formed by the union of both parents' pools. Parameter values undergo perturbation (intensity = 0.2), and a structural mutation, adding or removing a gene, occurs with probability 0.2.

Deterministic reproduction. All randomness flows through `SeededRng::new(job_id, round_number)`, which derives an xorshift64 PRNG seed via FNV-1a hashing. Identical job configuration and round history therefore produce identical offspring, satisfying the reproducibility requirement of the build pipeline (Section [Deterministic Compilation Pipeline](#)).

Table 21: Genetic algorithm configuration parameters.

Parameter	Value
Population size	10
Elitism count	2
Parameter mutation rate	0.3
Structural mutation rate	0.2
Tournament size (k)	3

Evaluation

This section evaluates AutoMutate++ along three dimensions: (a) end-to-end evasion capability: can the closed loop turn a universally detected artifact into an evasive one? (b) infrastructure reliability: does the differential protocol produce trustworthy verdicts? and (c) guidance benefit: does feedback-driven mutation selection outperform random mutation? These dimensions map onto the canonical fuzzer evaluation axes: Input, Oracle, and Guidance, adapted to a system where each round takes tens of seconds rather than microseconds. Detailed per-selector diagnostics and infrastructure validation figures are provided in Appendix [Component-Level Evaluation Diagnostics](#) and [Infrastructure Evaluation Diagnostics](#).

Benchmark Structure and Methodology

Campaign structure. Four campaigns were executed, each using a different selector strategy: Random, Coverage, Genetic, and Token, totalling approximately 50 rounds each. All campaigns share the same seed artifact, payload, and EDR configuration (Microsoft Defender for Endpoint on Windows 10), varying only the selector strategy (details in §[Setup](#)). Each round follows an identical procedure: apply the selected mutations, compile a new PE, and execute the three-run differential protocol (baseline, instrumented, dry-run) to classify the outcome.

Theoretical framing. The evaluation is structured around three orthogonal axes, each mapped to concrete metrics:

- **Input** mutation expressiveness (AST coverage), validity (execution rate), and diversity (token diversity across rounds).
- **Oracle** differential protocol reliability (three-run verdict agreement), the EDR as a black-box oracle (blind spots and instrumentation artifacts), and result stability.
- **Guidance** feedback quality (triage tokens informing selector decisions), search efficiency (evasions per round), baseline comparison (Random vs. guided strategies), and scoring convergence.

This evaluation is automated in a dedicated evaluation module; all round data is persisted in Elasticsearch with a known schema, enabling future selectors to be tested and compared without re-running campaigns. Selector-specific metadata can be appended to the schema for richer analysis.

Why not a classical fuzzer. Traditional coverage-guided fuzzers execute thousands to millions of inputs per second, enabling large-sample statistical evaluation. Here, AutoMutate++ operates at roughly 30–90 seconds per round because each round must compile a new PE, transfer it to a virtual machine, and execute the three-run differential protocol under a real EDR. With the 25-second

execution timeout used in the evaluated campaigns, the three runs alone can consume up to 75 seconds in the worst case; adding compilation and transfer overhead yields the observed 30–90 second range. When a payload terminates in milliseconds rather than hitting the timeout, compilation becomes the bottleneck and rounds complete faster. With only ~180 total rounds, classical throughput benchmarks are inapplicable; these round counts are relatively low and represent a proof-of-concept evaluation, longer multi-day campaigns across multiple payloads would strengthen statistical conclusions. The evaluation therefore prioritises *information density per round* and whether the closed loop produces a qualitatively different outcome, actual evasion, compared to unguided approaches.

EDR platform selection. Both Microsoft Defender for Endpoint (MDE) and Cortex XDR were targeted for comparison. Cortex XDR consistently terminated the evaluation harness (**worker-agent.exe**) after a few artifact spawns, rendering sustained campaigns impractical on that platform. MDE on Windows 10 proved stable across all campaign lengths and was therefore selected as the primary evaluation target.

Setup

Hardware and software. Four remote VMware virtual machines serve as targets: two dry-run VMs (Windows 10, no EDR), one Windows 10 VM with MDE, and one Windows 10 VM with Cortex XDR. All target VMs share the same hardware profile: Intel Xeon Gold 6326 @ 2.90 GHz, 4 GB RAM, and a 50 GB SSD (VMware Virtual Disk). The controller runs on a 13th Gen Intel® Core™ i7-1355U @ 1.70 GHz with 32 GB RAM, executing under WSL2. It orchestrates campaigns via gRPC; Elasticsearch 8.11 and Kibana run in Docker containers on WSL2. Compilation used clang 19.1.7. This architecture scales horizontally: additional target VMs can be provisioned on demand, limited only by the available infrastructure resources.

Seed artifact and payload. The seed artifact is a minimal C loader compiled from the AutoMutate++ template, paired with `payload_x64.bin`, a Cobalt Strike shellcode beacon. Unlike a trivial test binary, the beacon performs actual C2 communication after execution, making it representative of real-world post-exploitation payloads and harder to evaluate than a simple `return 0` stub. Due to time constraints, `payload_x64.bin` was the only payload evaluated; it was selected because it could complete campaigns without triggering static detection issues. The 9 binary-layer mutations are activated from round 2 onward, ensuring that every artifact bypasses static signature detection. This bootstrapping lets the evaluation focus exclusively on *behavioural* detection differences produced by the AST mutations. The AST mutation search space is restricted to the *deconditioner*, the module that rehearses the loader's

alloc→write→protect→free pattern with benign data before real payload execution. The selector explores deconditioner-scoped mutations per round; IR and binary mutations are always applied but their arguments are re-randomised each round. In its unmodified form, MDE detects and quarantines the seed artifact within seconds. The mutation catalogue comprises 22 mutations: 10 at the AST layer (explored by the selector within the deconditioner), 3 fixed at the IR layer, and 9 fixed at the binary layer. Each campaign applies the same seed, payload, and VM configuration, varying only the selector strategy.

Question

The evaluation addresses four research questions, structured around the three evaluation axes plus an end-to-end showcase:

RQ1 (Showcase) Can AutoMutate++ transform a universally detected artifact into one that evades Microsoft Defender for Endpoint on Windows 10?

RQ2 (Oracle) Is the three-run differential protocol reliable enough to distinguish genuine EDR detection from instrumentation artifacts?

RQ3 (Guidance) Does feedback-driven mutation selection outperform random mutation in evasion rate, score trajectory, and search space coverage?

RQ4 (Input) Do the 10 AST mutations, scoped to the deconditioner module (listing 9), produce behaviourally diverse artifacts across selector strategies?

Observation

End-to-end evasion showcase. The Genetic selector achieved 6 evasions in 51 rounds (11.8% evasion rate). The strongest evasion round reached a score of 0.899. Token-differential analysis between the best evasion round and a typical detection round reveals which behavioural tokens disappeared (candidate detection triggers) and which appeared (candidate evasion enablers), providing actionable feedback for future mutation design.

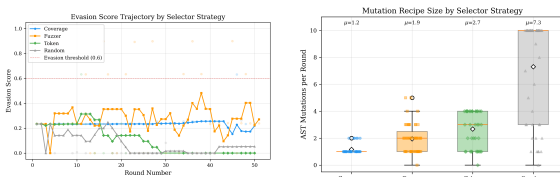


Figure 18: Left: evasion score trajectory for all four selectors over ~50 rounds. Right: AST mutation recipe size per selector (number of mutations applied per round).

Evasion performance comparison. Table 22 compares the four selector strategies; Figure 18 visualises their trajectories side by side. The *Coverage* selector (blue) maintains a stable trajectory around 0.2–0.25 with occasional near-evasion spikes, paired with a conservative recipe size, ($\mu=1.2$ mutations/round) cautious exploration that yields reliable telemetry data but rarely breaks through the evasion threshold. The *Genetic* selector (orange) exhibits high variability with multiple evasion-threshold crossings and a moderate recipe size ($\mu=1.9$); crossover recombines previously successful mutations into novel paths that periodically achieve full evasion. The *Token* selector (green) starts at a comparable level but drops toward zero despite a moderate recipe size ($\mu=2.7$): because ETW tokens cover only a subset of providers (a PoC limitation), the selector cannot reliably determine which mutations influence token presence, so its feedback signal is too noisy to guide the search effectively. *Random* (grey) stays low throughout and applies the largest recipes ($\mu=7.3$, often all ten mutations at once); blindly stacking mutations does not produce evasion, confirming that guidance is necessary.

Table 22: Selector strategy comparison across $n \approx 50$ rounds per campaign. The Genetic selector is the only strategy to achieve evasion. Bold values indicate best-in-column.

Metric	Random	Coverage	Genetic	Token
Evasion performance				
Evasion count	0	0	6	0
Evasion rate	0.0 %	0.0 %	11.8 %	0.0 %
Mean evasion score	0.070	0.226	0.282	0.121
Max evasion score	0.269	0.631	0.899	0.632
Experimental quality				
Real detections	16	34	22	18
Mutation failure rate	68.0 %	10.3 %	33.3 %	52.5 %
Trustworthy ratio	32.0 %	87.2 %	54.9 %	45.0 %
Search space coverage				
Unique AST mutations explored	10	10	10	7
Unique behavioural tokens	202	206	210	206

Infrastructure reliability. Per-run verdicts agree with the final differential category at rates ranging from 32 % (Random) to 87 % (Coverage). The low baseline agreement confirms that a single execution would misclassify the majority of rounds, the three-run differential protocol (baseline, instrumented, dry-run) is essential for reliable outcome labelling regardless of selector. Coverage detection correlation exceeds $r = 0.88$ for three of four selectors, indicating that the telemetry pipeline faithfully captures the behavioural signals that trigger detection. The Genetic selector’s lower correlation ($r = 0.59$) reflects its more aggressive mutation combinations, which introduce execution paths less cleanly separable by coverage alone. All selectors activate eight of nine token categories, confirming broad extraction regardless of strategy.

Table 23: Component-level diagnostic metrics across selector strategies. Random serves as the unguided baseline. Coloured indicators: ▲ strong, ▲ moderate, ● null.

Metric	Random ($n=50$)	Coverage ($n=39$)	Genetic ($n=51$)	Token ($n=40$)
Verdict ↔ differential agreement	0.32 ▲	0.87 ▲	0.55 ▲	0.45 ▲
Coverage ↔ detection correlation	0.92 ▲	0.88 ▲	0.59 ▲	0.94 ▲
Trustworthy round fraction	0.32 ▲	0.87 ▲	0.55 ▲	0.45 ▲
Active token categories (of 9)	8 ▲	8 ▲	8 ▲	8 ▲
Mean tokens per round	107.3	149.2	129.6	124.5
Significant single-mutation effect	0.00 ●	0.00 ●	1.00 ▲	0.00 ●

Experimental quality. Coverage achieves the lowest mutation failure rate (10.3%) and the highest trustworthy ratio (87.2%), meaning nearly nine in ten rounds produce usable differential data. Random fails at the opposite extreme: 68% of its rounds crash before reaching payload logic, yielding only 32% trustworthy data. The Genetic selector’s aggressive recombination produces more evasions but also more failures (33.3%) than the conservative Coverage strategy, a fundamental exploration/exploitation trade-off. A further source of data loss is static detection before dispatch: chained mutation variants are flagged by MDE’s signature cache, discarding rounds before behavioural execution begins (see Section [Deduction](#)). The trustworthy round fraction mirrors verdict agreement: guided selectors that produce more evasive artifacts naturally reduce instrumentation-induced contamination, since the instrumented run is less likely to diverge from baseline when the artifact already evades detection.

Search space coverage. Three of four selectors explore all ten available AST mutations (scoped to the deconditioner module); the Token selector covers only seven, suggesting its bias mechanism prematurely narrows the search. Behavioural token counts are comparable across all strategies (~200–210 unique tokens), indicating that the observable feature space is largely mutation-independent, differences in evasion outcome stem from *how* tokens co-occur, not from which tokens appear. Detailed token presence heatmaps and category proportions are provided in Appendix [Component-Level Evaluation Diagnostics](#).

Infrastructure diagnostics summary. The following infrastructure experiments validate individual pipeline components; full figures and analysis are provided in Appendix [Infrastructure Evaluation Diagnostics](#):

- **I1 (Encoding):** XOR encoding maximises Shannon entropy with minimal size expansion; English encoding shows low entropy and high expansion.
- **I2 (AST mutation):** All ten AST mutations maintain >90% validity with negligible compilation latency overhead.

- **I3 (IR mutation):** -O2 eliminates many IR insertions, confirming that post-optimisation evaluation is necessary.
- **I7 (Token extraction):** All nine token categories are exercised with no token loss across the pipeline.
- **I8 (Scoring):** The lift×confidence formula correctly ranks high-lift tokens on synthetic validation data.
- **I9 (Input diversity):** Guided selectors produce more diverse mutation configurations than Random.
- **I10 (Oracle stability):** Scoring is deterministic and robust to run-order permutation.
- **I11 (Cross-selector):** Comparative analysis of coverage, diversity, mutation frequency, and score trajectories across selectors.
- **I12 (Guidance utilisation):** Token selector shows highest avoid/seek compliance; Genetic partially ignores guidance signals.
- **I13 (Convergence):** Guided selectors approach theoretical convergence rates; Random shows no trend.
- **I14 (Line tracing):** Line tracer provides reliable throughput, validity, and instrumentation density for the coverage correction mechanism.

Convergence. Lift variance stabilises within approximately 10–15 rounds for guided selectors. The Genetic selector converges while still discovering evasions, avoiding premature convergence, a desirable property for an exploration-oriented system. Random shows no convergence trend, consistent with its lack of feedback (see Appendix [Infrastructure Evaluation Diagnostics](#)).

Deduction

The closed loop works. The Genetic selector’s 11.8% evasion rate, starting from a universally detected seed artifact, validates the core design hypothesis of AutoMutate++. The cross-selector trajectory (fig. 18) shows that, within the limited number of rounds executed, only

the Genetic selector achieved full evasion. The Coverage selector came close, reaching near-evasion scores, but its artifacts were ultimately killed during shellcode execution. With additional rounds, Coverage might have crossed the threshold; nevertheless, the Genetic selector's ability to recombine successful mutations gives it a clear advantage in steering the search toward evasive combinations.

Infrastructure validation (RQ2). The three-run differential protocol is essential: single-execution verdicts would misclassify the majority of rounds across all selectors. Within the rounds executed, only the Genetic selector produces a statistically significant single-mutation effect, suggesting it is the strategy most likely to isolate individual mutations with measurable causal impact on detection. The other selectors, particularly Coverage, which approached the evasion threshold, may well contain impactful mutations whose effects are masked by the limited sample size. Across all selectors, evasion appears to be largely an emergent property of mutation *combinations* rather than attributable to any single transform, a meaningful observation about the nature of EDR evasion, though one that warrants confirmation with longer campaigns.

Guidance outperforms random, with caveats (RQ3). The Genetic selector is the clear evasion winner: it is the only strategy to cross the evasion threshold while maintaining full mutation coverage (10/10 deconditioner-scoped AST mutations explored). However, Coverage produces the cleanest experimental data (87.2% trustworthy rounds), making it the preferred choice when the goal is triage quality rather than evasion discovery. This reveals a tension: the best evasion finder is not the best infrastructure validator. The Token selector's coverage of only 7/10 AST mutations reflects both its bias mechanism, which constrains the search prematurely, and the incomplete token-provider mapping noted earlier: because tokens cover only a subset of ETW providers, the selector lacks the feedback signal needed to explore the full mutation space.

Static detection and data scarcity. A significant fraction of rounds were flagged by Defender's static analysis before VM dispatch, reducing the amount of usable behavioural data. Because mutations are applied incrementally in a chain, each new variant structurally resembles the previously submitted one; once AV/EDR records a static signature for a detected variant, subsequent similar artifacts are caught before execution begins. This chaining effect, also observed by Cailleret [17], is the primary reason for the limited round counts across all selectors. The bootstrapping mechanism described in Section [Setup](#), activating binary layer mutations from round 2 onward—was introduced specifically to mitigate this: by altering the binary structure early, it ensures artifacts pass static signature checks and reach behavioural execution, where the AST-level mutations

can be meaningfully evaluated.

Limitations. Round counts (~50 per campaign) are low; results should be validated with longer campaigns spanning multiple days. Only one payload (`payload_x64.bin`) was tested; generalisation to other shellcode types (available in `data/shellcode/`) remains future work. Applying the AST mutations beyond the deconditioner to the full loader or carrier code could reveal additional evasion vectors. Additionally, RedEDR was compiled at its latest available version, yet this version does not emit **VirtualProtect** ETW events, a significant gap, since memory-protection changes are a key indicator in shellcode detection. Downgrading is not feasible because the pipeline depends on the recent API to retrieve telemetry from RedEDR.

Scoring function maturity. The composite evasion score was developed incrementally alongside the system and only critically examined during the final evaluation phase; it should be treated as a first approximation rather than a validated metric. The current formula assigns 70% weight to source-level coverage of the template loader, yet most execution logic resides in the injected shellcode, which the compiler never sees. Injected checkpoints are the only proxy for payload progress, and the score should weight them more heavily—rewarding artifacts that reach deeper into the payload execution path. Several refinements deserve investigation: replacing binary token presence with occurrence counts to distinguish a single **VirtualAlloc** from a burst of allocations; counting injected checkpoints reached, where more reached indicates deeper payload execution; and incorporating survival time more meaningfully. Each extension, however, introduces its own pitfalls: a checkpoint inside a loop would inflate the count without reflecting genuine progress, and a high survival time achieved by a benign **Sleep** call does not indicate evasion quality. Designing a scoring function that generalises across arbitrary shellcode payloads therefore requires careful study and is left as future work.

Mutation-catalogue size. The current catalogue contains only 22 mutations, each of which required non-trivial engineering effort: every transform must be implemented, verified in isolation to confirm it produces the intended code change, and tested in combination to ensure it does not break other mutations or the compilation pipeline. This per-mutation cost is one of the primary reasons why the feedback-loop selectors were not explored more extensively building a sufficiently diverse mutation catalogue was a prerequisite that consumed a significant share of the project timeline, leaving fewer cycles for long-running, multi-day campaigns. Twenty-two mutations may not be enough to consistently evade a production-grade EDR; nevertheless, the evaluation demonstrates that evasion is reachable even under these constrained conditions, suggesting that a broader catalogue combined with longer campaigns would further improve results.

Extensibility as evaluation criterion. Beyond numeric results, the evaluation validates AutoMutate++ as a *platform* for systematic EDR evaluation. Four selector strategies share a common interface; 22 mutations span three compilation layers; token extraction is modular across nine categories. Future selectors, mutations, or token sources can be integrated without redesigning the core loop. The evaluation demonstrates that this extensibility is not merely architectural: each new selector produces meaningfully different campaign dynamics, confirming that the framework supports genuine research variation.

Summary

The evaluation answers the four research questions as follows.

RQ1: Yes, AutoMutate++ produced six evasive variants of a universally detected artifact against Microsoft Defender for Endpoint, with external validation confirming evasion on a clean Windows 11 installation.

RQ2: Yes, the three-run differential protocol is essential; single-execution classification would misclassify the majority of rounds, and guided selectors achieve up to 87 % verdict agreement.

RQ3: Yes, the Genetic selector outperforms all others in evasion rate (11.8 %) and peak score (0.899), while Coverage provides the highest data quality (87.2 % trustworthy); Random never approaches evasion.

RQ4: The deconditioner-scoped mutation space is well explored by guided selectors (10/10 AST mutations for three of four strategies), with ~200–210 unique behavioural tokens observed across all campaigns; differences in outcome stem from token co-occurrence patterns rather than token presence.

AutoMutate++ is a viable platform for systematic, closed-loop EDR evaluation. The Genetic selector is recommended for evasion-oriented research; the Coverage selector for validation campaigns requiring high data fidelity. These results are preliminary; longer campaigns on multiple payloads are needed to confirm generalisability. Detailed per-selector diagnostics and infrastructure validation are provided in Appendix [Component-Level Evaluation Diagnostics](#) and [Infrastructure Evaluation Diagnostics](#).

Future Work

EDR-Side Token Extraction

AutoMutate++ currently derives triage tokens exclusively from the artifact's observable behavior: ETW events emitted by the artifact's process (via RedEDR), execution checkpoints, and module/mutation metadata. The EDR's own decision process, which analysis stages it activates, when it scans memory, how it correlates events, remains a black box. Cailleret [17] demonstrated that this black box can be partially opened: by consuming the `Microsoft-Antimalware-Engine` and `Microsoft-Windows-Kernel-Audit-API-Calls` ETW traces, the EDRi framework observes MDE's internal actions (cache lookups, emulation invocations, memory scans, verdict emissions) in real time, without requiring kernel-mode exploits.

Integrating a similar capability into AutoMutate++ would add a second, complementary token source to the feedback loop. Concretely, each run would produce two parallel token streams:

- **Artifact tokens** (existing): what the artifact *did*, API calls, memory transitions, event sequences.
- **EDR-response tokens** (proposed): what the EDR *did in response*, which analysis stage was active, whether emulation was triggered, whether a memory scan occurred, and the temporal ordering of these actions relative to the artifact's execution timeline.

This dual-stream approach would enable several new analyses:

Causal attribution. Current tokens can identify *what* the artifact did that correlated with detection, but not *which EDR mechanism* caused the detection. EDR-response tokens would distinguish static-scan detections from emulation-based detections from behavioral detections, even when the final verdict is identical. This finer-grained attribution would allow the mutation engine to target the correct layer: if the EDR's emulation engine is the trigger, AST-level anti-emulation mutations are relevant; if a behavioral correlation fires, deconditioner variants should be explored.

Analysis-stage awareness. Cailleret observed that MDE's analysis follows a staged pipeline: static lookup → emulation → behavioral tracking → memory scan. EDR-response tokens would let the feedback loop detect *which stage* an artifact reaches before detection. An artifact that survives static analysis and emulation but is caught by behavioral tracking is fundamentally different from one caught at the static stage, yet both currently receive the same "RealDetection" category. Encoding the EDR's active stage as a token would provide a more informative fitness signal.

Deconditioning feedback. The deconditioner hypothesis (Section [Deconditioner Design](#)) posits that flooding behavioral counters with benign patterns reduces detection. EDR-response tokens could directly test this: if the number of memory scans or behavioral correlations decreases after deconditioning, the hypothesis gains quantitative support. Without EDR-side observability, deconditioning effectiveness can only be inferred indirectly from binary outcomes.

Limitations. This approach is inherently vendor-specific: the Antimalware-Engine trace is a Microsoft Defender artifact and does not exist on Cortex, CrowdStrike, or other EDR products. Generalising to other vendors would require identifying equivalent telemetry sources per product, as Cailleret noted when Cortex's processes crashed under monitoring and emitted no Antimalware events. Nevertheless, since AutoMutate++ already targets one EDR per experimental campaign, vendor-specific EDR-response tokens remain useful within that scope.

Alternative telemetry collectors.

The current pipeline relies on RedEDR as its sole ETW consumer. Kelleet [20] (Section [Event Tracing for Windows \(ETW\)](#)) offers a kernel-level alternative designed for lossless, low-overhead event capture. Substituting or running it alongside RedEDR could reveal differences in event completeness that affect token quality and, consequently, mutation guidance.

API Indirection via Callback Mechanisms

Lepori [48] demonstrated DLL proxy-loading, where `LoadLibrary`/`GetProcAddress` calls are replaced with indirect invocations through `RtlRegisterWait` callbacks, causing the call stack to show DLL loading originating from a legitimate timer callback rather than from the loader itself. While this technique was effective against some EDR products (though not Elastic Defend), AutoMutate++ does not currently implement it. The `@MUTATE:getprocaddress_indirection` marker exists in the carrier source but remains a stub. For carrier configurations that call `LoadLibrary`/`GetProcAddress` directly, the substitution would be straightforward; however, when the PEB-walk mutation is active, the carrier already bypasses `LoadLibrary` entirely, making the indirection redundant in that configuration.

Advanced Carrier Strategies

Once the feedback loop's utility is established with simple carriers and the token scores stabilize, i.e. the system can reliably distinguish which behavioral features drive detection, -a natural extension is to add structurally more complex carrier variants as new gene options. The techniques table (Table 42) identifies several candidates:

- **Module stomping:** Map a legitimate DLL, overwrite

its entry point with shellcode, execute from backed memory. Tests whether EDR detection depends on *unbacked-memory heuristics*, if the token scorer already flags `mem_region:private` as high-lift, module stomping would confirm or refute that finding.

- **Reflective DLL loading:** Load a DLL entirely from memory without disk I/O. Tests *disk-monitoring dependence*, relevant if the static-scan short-circuit (Section [Three-Run Differential Protocol](#)) catches most variants before VM dispatch.
- **Process hollowing:** Spawn a suspended legitimate process, replace its image, resume. Would require extending the worker agent to a *multi-process observation model* (new ETW filters, cross-process token extraction).

Module stomping and reflective loading are natural candidates for future `@MODULE:carrier` implementations, as both techniques execute within the loader's own process and thus preserve the single-process telemetry model. However, reflective loading would require the payload to be structured as a reflective DLL rather than raw shellcode, while module stomping would involve mapping a sacrificial system DLL and overwriting its code section, both representing a departure from the current raw-shellcode-into-allocated-memory pattern shared by all existing carriers. Process hollowing requires cross-process observation and is architecturally more invasive. For all three, the token extractor would need new token types (e.g. `mem_region:image_backed`, `cross_process:NtUnmapViewOfSection`) to capture their distinctive behavioral signals.

Adding these carriers would test whether the token-driven feedback loop generalizes beyond simple injection patterns. If the scorer correctly identifies the behavioral features that make module stomping succeed (backed-memory execution, legitimate DLL context) and those that make it fail (entry-point modification telemetry), it would validate that the closed-loop methodology applies to the full spectrum of injection techniques, not just the canonical pattern used in the initial evaluation.

Additional tokens

Temporal tokens.

The current token taxonomy captures *which* events occurred and their local ordering (bigrams), but not *how quickly* they followed each other. Several EDR detection rules are known to be timing-sensitive: for example, a `VirtualAlloc` followed by `VirtualProtect(RWX)` within a few milliseconds is far more suspicious than the same pair separated by seconds of benign activity. Temporal tokens would encode these inter-event intervals as coarse-grained classes, e.g. `dt:VirtualAlloc→VirtualProtect_ms<20` (under 20 ms apart) versus `dt:VirtualAlloc→VirtualProtect_ms>1000`.

The infrastructure to support this already exists: every telemetry event stored in ElasticSearch carries a `timestamp` field and a monotonic `payload_id` sequence number, so the extractor could compute pairwise deltas between consecutive API calls without additional data collection.

The expected benefit is twofold. First, temporal tokens would let the scorer identify timing-dependent detection rules that are invisible to presence-only and ordering-only tokens: if a particular inter-event delay consistently correlates with detection, it would surface as a high-lift avoid token. Second, the `ast.timing_pattern` mutation, which inserts inter-operation `Sleep()` calls, could then be guided by temporal token feedback, inserting delays specifically between the event pairs that the scorer flags as timing-sensitive, rather than applying uniform delays everywhere.

Localization tokens.

The instrumented run already captures approximate kill-point data: the AST line tracer records the last executed source line before termination, and the check-point system records the last coarse progress marker (e.g. `decode_complete`, `launching`). However, this information is not yet extracted into triage tokens. Localization tokens such as `trunc:carrier.c:47` or `trunc:between:decode_complete:launching` would encode the kill-point so that the scorer can identify whether detections cluster around a specific module or code region, for example, consistently triggering between memory allocation and protection change. This would let the selector target mutations at or near the vulnerable code section rather than exploring the mutation space uniformly from token information.

MSVC Toolchain Adoption

The current compilation pipeline uses Clang and LLD, which provide access to LLVM IR and enable IR-level mutations. However, Clang-produced PE binaries lack several metadata structures that legitimate Windows software carries: native Rich headers encoding the Microsoft linker version matrix, standard Microsoft section names, and authentic debug directory entries. The binary mutation layer (Section [Binary Mutations at the PE Byte Level](#)) can approximate these artefacts via `bin.rich_header`, `bin.section_rename`, and `bin.debug_dir`, but the current implementations have known limitations: synthesised Rich headers may not survive statistical consistency checks that compare tool counts against actual section content, section renaming can produce duplicate names, and the underlying PE still carries inherent Clang/LLD signatures, that a sufficiently detailed static analyser could fingerprint independently of the mutated fields.

Switching to the MSVC toolchain (`cl.exe` + `link.exe`) would produce these structures natively, eliminating an

entire class of static detection signals without requiring binary-level patching. The tradeoff is significant: losing the LLVM IR layer removes not only IR mutations but also the AFL-style basic-block edge coverage instrumentation. In practice, this limitation is mitigated by the fact that the system primarily relies on line-level tracing rather than basic-block coverage for execution path analysis, so the loss of -fsanitize-coverage in an MSVC-only pipeline would not critically impair the feedback loop.

Mutation Token Profiling

The token selector (§ [Selector Strategies as Ablation Framework](#)) currently guides mutation selection using a binary novelty bonus and avoid/seek lists derived from token scoring, but it operates without knowledge of *which* tokens each mutation can produce or remove. Selection is therefore exploratory: the selector must discover mutation–token associations empirically across rounds, which is sample-inefficient when the mutation space is large.

Isolated extraction. A more targeted approach would pre-compute a mapping $\text{mutation_id} \times \text{params} \rightarrow \{\text{tokens_added}, \text{tokens_removed}\}$ by compiling minimal C programs with each mutation applied in isolation, extracting their token sets, and differencing against a clean baseline. This profiling step needs to run only once per mutation catalogue version and would produce a static database that the selector can query at runtime.

Targeted selection. With this database, the token selector could directly match avoid-tokens to mutations known to remove them, and seek-tokens to mutations known to produce them, rather than relying on the novelty bonus to stumble upon useful mutations. For example, if `evt:VirtualProtect` appears as a high-lift avoid token, the selector could immediately prioritise `ast.syscall_replace` (which would replace the API call with a direct syscall, removing the event) over mutations that do not affect API-call tokens. This would complement the existing distance and novelty heuristics in the selector (§ [Selector Strategies as Ablation Framework](#)) with a causal, pre-computed signal.

Machine Learning on Accumulated Data

The six Elasticsearch index families: jobs, rounds, runs, telemetry, tokens, and artifacts (§ [Elasticsearch Index Schema](#)) accumulate structured data across campaigns: per-round mutation recipes, evasion scores, token sets, ETW event sequences, coverage maps, and detection outcomes. As the number of completed campaigns grows, this dataset becomes amenable to machine-learning approaches that could augment or replace the hand-tuned scoring heuristics.

Detection prediction. A classifier trained on $(\text{mutation_recipe}, \text{token_set}) \rightarrow \text{detection_outcome}$ could predict whether a candidate artifact will be detected

before it is executed in the VM, allowing the selector to prune unpromising individuals and reduce the number of expensive instrumented runs per round.

Non-linear token interactions. The current token scorer uses linear lift $(P(\text{detect} \mid \text{token})/P(\text{detect}))$, which treats each token independently. In practice, detection rules often depend on *combinations* of tokens, e.g. `evt:VirtualAlloc` alone is benign, but its co-occurrence with `evt:WriteProcessMemory` and `arg:PAGE_EXECUTE_READWRITE` triggers a behavioural signature. A gradient-boosted tree or small neural model could capture these interactions from the accumulated data, producing a more accurate fitness signal than the linear scorer provides.

Learned weights. The blending weights in the evasion scoring formula (§ [Composite Evasion Scoring](#)) and the selector bonus thresholds are currently hand-tuned. Given sufficient campaign data, these could be replaced with learned parameters optimised to maximise evasion improvement per round, adapting the system to each EDR product’s detection characteristics without manual experimentation.

Completing the Token Feedback Loop

The token-driven feedback loop currently operates as a simplified pipeline: tokens are extracted, scored with linear lift, and fed to the selector as flat avoid/seek lists. Several design simplifications limit the loop’s precision and could be addressed in future iterations.

Token combination scoring. The linear lift scorer treats tokens independently, missing interaction effects where two individually benign tokens become suspicious in combination. Extending the scorer to evaluate token *pairs* or small subsets would require combinatorial lift computation, which is tractable for the current token vocabulary size but would need pruning heuristics to avoid an explosion of spurious associations.

Mutation–token mapping. The selector currently treats mutations as atomic units without knowing which tokens they affect. As described above, a pre-computed mutation–token database would close this gap. Without it, the selector cannot verify whether a selected mutation *actually* addresses the tokens it was chosen to target, nor can it predict the token-level side effects of combining multiple mutations in a single recipe.

Evidence decay. The avoid and seek lists grow monotonically: once a token is classified as high-lift, it remains on the avoid list indefinitely, even if subsequent rounds show that removing it had no effect on detection. Introducing a recency-weighted confidence score, where token lift estimates decay toward the prior as they age and are reinforced only by fresh observations, would prevent the selector from perseverating on stale token associations and allow it to adapt between campaigns.

Conclusion

Evaluating endpoint detection and response systems has historically been a manual, ad-hoc process: a red-team operator crafts a variant, executes it, inspects the outcome, and iterates by intuition. This workflow is neither reproducible nor scalable, and its results are difficult to generalise beyond the specific artifact and moment in time. AutoMutate++ was designed to close this gap by providing the first fully automated, closed-loop infrastructure for systematic EDR evaluation.

Contributions. The system makes six concrete contributions, each validated by the experimental campaign.

First, the *closed-loop architecture* integrates build, execution, telemetry collection, token extraction, scoring, and mutation selection into a single automated pipeline that operates without manual intervention. The modular five-layer decomposition (controller, build system, worker agent, telemetry plane, infrastructure automation) ensures that each concern can evolve independently while preserving end-to-end reproducibility through seeded randomness and pinned toolchains.

Second, the *multi-layer mutation engine* applies 22 mutations across three compilation stages, 10 at the AST level, 3 at the LLVM IR level, and 9 at the post-link binary level, in a deterministic, composable pipeline. This layered design targets fundamentally different detection surfaces (source-level behavioural patterns, intermediate-representation structure, and binary-level static features) within a single build.

Third, the *three-run differential protocol* emerged as an essential innovation. Each round executes the same artifact under three conditions: baseline, instrumented, and dryrun; and classifies the outcome into one of seven mutually exclusive categories. The evaluation confirmed that single-execution classification would misclassify the majority of rounds: a dryrun on a clean VM may complete in hundreds of milliseconds, while the same artifact under EDR instrumentation may hit the 25-second timeout, or conversely, the antivirus may terminate the process in milliseconds while the dryrun runs to completion. These fundamentally non-equivalent timing profiles make naïve comparison meaningless. The differential protocol handles this gracefully by requiring equivalent *verdict classification*, not equivalent execution duration, enabling the Coverage selector to achieve 87.2% verdict agreement between per-run and differential categories. This is a non-trivial systems engineering achievement that makes the fuzzer-like feedback loop possible.

Fourth, the *token-based attribution system* decomposes EDR telemetry into nine categories of triage tokens (API calls, API arguments, call-sequence n -grams, ETW events, loaded images, execution checkpoints, module choices, mutation identifiers, and mutation parameters) and scores each by lift $\times$ confidence to produce

ranked avoid/seek lists. Across all four campaigns, selectors activated 8 of 9 token categories and observed approximately 200–210 unique behavioural tokens, demonstrating that the token vocabulary is rich enough to distinguish detection-relevant behavioural patterns.

Fifth, four *interchangeable minimal selector strategies*: Random, Coverage, Genetic, and Token; share a common interface and can be assigned per job for controlled ablation. Even with minimal tuning and short campaign lengths of approximately 50 rounds each, the Genetic selector achieved an 11.8% evasion rate (6 evasive variants from a universally detected seed) with a peak evasion score of 0.899, while the Random baseline never approached evasion. This result confirms that feedback-driven selection outperforms unguided search, and that the infrastructure provides a meaningful fitness signal.

Sixth, the *modular template architecture* with seven gene slots (carrier, decoder, anti-emulation, guardrail, virtualprotect, decoy, deconditioner) enables combinatorial exploration of evasion-relevant design choices while maintaining structural compilability guarantees.

An infrastructure-first contribution. The primary contribution of this work is the *platform and instrumentation*, not the selectors themselves. The time constraints of a master's thesis necessarily limited selector development to basic strategies: the Genetic selector uses a standard tournament-selection evolutionary algorithm, the Coverage selector applies ϵ -greedy exploration, and the Token selector implements a straightforward avoid/seek heuristic. Yet even with these minimal selectors, the system produced genuine evasions from a seed artifact that was universally detected. This outcome validates the infrastructure's core premise: the differential protocol, the scoring function, the token extraction pipeline, and the storage layer together provide all the tools needed to operate in a fuzzer-like manner. Future work can now focus entirely on selector intelligence, longer campaigns, more sophisticated evolutionary strategies, reinforcement learning, or neural-guided mutation scheduling, without modifying the core loop or instrumentation layer.

Future directions. The infrastructure is immediately ready for longer, multi-day campaigns that would allow selectors to explore the mutation space more thoroughly and produce statistically stronger results. New mutations, new modules, new payloads, and new EDR targets can be integrated without architectural changes: each is a configuration-level extension rather than a structural modification. Integrating EDR-side token extraction, as demonstrated by Cailleret's EDRI framework, would add a complementary telemetry stream enabling causal attribution of detections to specific EDR analysis stages. As campaigns accumulate data, machine-learning approaches over the token, outcome corpus become viable, potentially replacing the current statistical scoring with learned models that capture higher-order interaction effects. The

scoring function itself is a first approximation, the infrastructure to refine it is already in place.

AutoMutate++ demonstrates that automated, evidence-driven EDR evaluation is not only feasible but productive. Starting from a universally detected artifact and operating with minimal selector tuning over short campaigns, the system produced genuine evasions, generated trustworthy behavioural data, and provided ranked token-level hypotheses about detection triggers. The platform transforms EDR evasion research from craft to engineering: where manual workflows rely on operator intuition, AutoMutate++ substitutes systematic, reproducible, and extensible experimentation.

Acknowledgements

I would like to sincerely thank my academic supervisor, Professor Sanidhya Kashyap, for his guidance throughout this project. His course on operating systems sparked my passion for low-level systems programming and shaped the way I approach the problems tackled in this thesis.

I am equally grateful to my industry supervisor at the Cyber-Defence Campus, Daniel Hulliger, whose support was invaluable. From weekly meetings to hands-on debugging, including fixing the RedEDR integration, his willingness to share his experience, answer every question, and offer thoughtful advice made this project possible.¹

¹ChatGPT was used to assist with phrasing during the writing of this report. GitHub Copilot was used for code autocompletion suggestion during development. Claude Code was used at the end of the project to generate code documentation, comments, and repository README files. All tools were used in accordance with EPFL guidelines; all content was reviewed to ensure accuracy and academic integrity.

References

- [1] Rakesh Agrawal, Tomasz Imieliński, and Arun Swami. “Mining Association Rules between Sets of Items in Large Databases.” In: *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*. ACM, 1993, pp. 207–216.
- [2] Kotaiba Alachkar et al. “EvilEDR: Repurposing EDR as an Offensive Tool.” In: *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2025. url: <https://www.usenix.org/conference/usenixsecurity25/presentation/alachkar>.
- [3] Joma Alrzini and Diane Rasmussen Pennington. “A review of polymorphic malware detection techniques.” In: 2020. url: <https://api.semanticscholar.org/CorpusID:222282316>.
- [4] Eslam Amer and Ivan Zelinka. “A dynamic Windows malware detection and prediction method based on contextual understanding of API call sequence.” In: *Computers & Security* 92 (2020), p. 101760. issn: 0167-4048. doi: [10.1016/j.cose.2020.101760](https://doi.org/10.1016/j.cose.2020.101760).
- [5] Dennis Andriesse. *Practical Binary Analysis: Build Your Own Linux Tools for Binary Instrumentation, Analysis, and Disassembly*. No Starch Press, 2019. isbn: 978-1-59327-912-7.
- [6] Asad Arfeen et al. “Endpoint Detection & Response: A Malware Identification Solution.” In: *2021 International Conference on Cyber Warfare and Security (ICWS)*. 2021, pp. 1–8. doi: [10.1109/ICWS53234.2021.9703010](https://doi.org/10.1109/ICWS53234.2021.9703010).
- [7] Cornelius Aschermann et al. “REDQUEEN: Fuzzing with Input-to-State Correspondence.” In: *26th Annual Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2019. doi: [10.14722/ndss.2019.23371](https://doi.org/10.14722/ndss.2019.23371).
- [8] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. “RESTler: Stateful REST API Fuzzing.” In: *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 2019, pp. 748–758. doi: [10.1109/ICSE.2019.00083](https://doi.org/10.1109/ICSE.2019.00083).
- [9] Weiheng Bai et al. *AFLGopher: Accelerating Directed Fuzzing via Feasibility-Aware Guidance*. 2025. arXiv: [2511.10828 \[cs.SE\]](https://arxiv.org/abs/2511.10828).
- [10] Bitdefender. *How Bitdefender Uses Machine Learning for Threat Detection*. Accessed: 2025-10-05. 2024. url: <https://www.bitdefender.co.th/resources/Business/Bitdefender-2017-TechnicalBrief-MachineLearning-crea2103-A4-en-EN-2-GenericUse.pdf>.
- [11] Bitdefender. *What is Advanced Threat Control (ATC)?* Accessed: 2025-10-05. 2024. url: <https://www.bitdefender.com/business/support/en/77209-376309-advanced-threat-control.html>.
- [12] Tim Blazytko et al. “GRIMOIRE: Synthesizing Structure while Fuzzing.” In: *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, 2019, pp. 1985–2002.

- [13] Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. "Coverage-based Greybox Fuzzing as Markov Chain." In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2016, pp. 1032–1043. doi: [10.1145/2976749.2978428](https://doi.org/10.1145/2976749.2978428).
- [14] Jean-Marie Borello and Ludovic Mé. "Code Obfuscation Techniques for Metamorphic Viruses." In: *Journal in Computer Virology* 4 (Aug. 2008), pp. 211–220. doi: [10.1007/s11416-008-0084-2](https://doi.org/10.1007/s11416-008-0084-2).
- [15] Alexei Bulazel. "Windows Offender: Reverse Engineering Windows Defender's Antivirus Emulator." In: *Black Hat USA 2018*. Accessed: 2026-01-20. Las Vegas, NV, Aug. 2018. url: <https://i.blackhat.com/us-18/Thu-August-9/us-18-Bulazel-Windows-Offender-Reverse-Engineering-Windows-Defenders-Antivirus-Emulator.pdf>.
- [16] Levi Cailleret. *Decondition Everything*. Accessed: 2026-03-09. 2025. url: <https://blog.levi.wiki/post/2025-12-05-decondition-everything>.
- [17] Levi Cailleret. "EDR Introspection: Enabling Deep Insights into EDR Detection Approaches." Mentor: Dr. P. Heinrich (ZHAW); Expert: D. Hulliger (CYD/arma-suisse). Master's Thesis. ZHAW School of Engineering, 2025.
- [18] R. L. Castro et al. "AIMED: Evolving Malware with Genetic Programming to Evade Detection." In: *Proceedings of the 2019 IEEE Conference on Communications and Network Security (CNS)*. Source 46 in provided list. 2019. url: <https://www.rapha.ai/files/AIMED.pdf>.
- [19] Aurélien Chalot. *From Windows Drivers to a Almost Fully Working EDR*. Accessed: 2025-10-02. Jan. 2024. url: <https://blog.whiteflag.io/blog/from-windows-drivers-to-a-almost-fully-working-edr/>.
- [20] Tieming Chen et al. "Kollect: A Kernel-Based Efficient and Lossless Event Log Collector for Windows Security." In: *Computers & Security* 150 (2025), p. 104203.
- [21] Fred Cohen. "Computer Viruses: Theory and Experiments." In: *Computers & Security* 6.1 (1987), pp. 22–35. doi: [10.1016/0167-4048\(87\)90122-2](https://doi.org/10.1016/0167-4048(87)90122-2).
- [22] Mila Dalla Preda et al. "A semantics-based approach to malware detection." In: vol. 42. Jan. 2007, pp. 377–388. doi: [10.1145/1190216.1190270](https://doi.org/10.1145/1190216.1190270).
- [23] deeb. *Avred background: Advances in Reversing Defender Signature Format*. accessed: 2025-10-02. 2024. url: <https://blog.deeb.ch/posts/avred-update/>.
- [24] deeb. *How EDR Works*. Accessed: 2025-10-02. 2023. url: <https://blog.deeb.ch/posts/how-edr-works/>.
- [25] Dobin. *RedEDR: Red Team EDR Evasion Research Tool*. MIT License; ETW-based telemetry collection. 2024. url: <https://github.com/dobin/RedEDR>.
- [26] dob. *SuperMega: A Shellcode Loader Framework*. Accessed: 2026-03-08. 2024. url: <https://github.com/dobin/SuperMega>.
- [27] M. Duratny. "Modern obfuscation techniques." Source 10 in provided list. MA thesis. Masaryk University, 2018. url: https://is.muni.cz/th/v1f9y/Modern_obfuscation_techniques.pdf.
- [28] ESET. *ESET LiveGrid and LiveGuard Advanced Technologies*. Accessed: 2025-10-05. 2024. url: https://help.eset.com/ees_mac/8/en-US/cloud-based_protection.html.
- [29] Jannatul Ferdous et al. "A Survey on ML Techniques for Multi-Platform Malware Detection: Securing PC, Mobile Devices, IoT, and Cloud Environments." In: *Sensors* 25.4 (2025), p. 1153. doi: [10.3390/s25041153](https://doi.org/10.3390/s25041153).
- [30] Shuitao Gan et al. "GREYONE: Data Flow Sensitive Fuzzing." In: *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 2020, pp. 2577–2594.
- [31] Jiaxuan Geng et al. "A Survey of Strategy-Driven Evasion Methods for PE Malware: Transformation, Concealment, and Attack." In: *Computers & Security* 137 (2024), p. 103595. doi: [10.1016/j.cose.2023.103595](https://doi.org/10.1016/j.cose.2023.103595).
- [32] gRPC Authors. *gRPC Documentation*. Accessed: 2025-10-02. 2024. url: <https://grpc.io/docs/>.
- [33] Matt Hand. *Evading EDR: sensors, telemetry, and how to bypass them*. San Francisco, CA: No Starch Press, 2024. isbn: 978-1-7185-0335-9. url: <https://nostarch.com/evading-edr>.
- [34] Weiwei Hu and Ying Tan. "Generating Adversarial Malware Examples for Black-Box Attacks Based on GAN." In: *arXiv preprint arXiv:1702.05983* (2017).
- [35] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual*. Volume 3A: System Programming Guide, Part 1, Section 6.3 – Breakpoint Exception (#BP). Intel Corporation. 2024. url: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>.
- [36] Suman Jana and Vitaly Shmatikov. "Abusing File Processing in Malware Detectors for Fun and Profit." In: *2012 IEEE Symposium on Security and Privacy*. 2012, pp. 80–94. doi: [10.1109/SP.2012.15](https://doi.org/10.1109/SP.2012.15).
- [37] Somesh Jha et al. "Testing Malware Detectors." In: *Proceedings of the 2007 International Symposium on Software Testing and Analysis (ISSTA)*. Source 19 in provided list; Applies Delta Debugging to malware. 2007. url: <https://research.cs.wisc.edu/wisa/papers/issta04/issta.pdf>.
- [38] Jinho Jung et al. "WINNIE: Fuzzing Windows Applications with Harness Synthesis and Fast Cloning." In: *28th Annual Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2021. doi: [10.14722/ndss.2021.24334](https://doi.org/10.14722/ndss.2021.24334).
- [39] George Karantzas and Constantinos Patsakis. "An Empirical Assessment of Endpoint Detection and Response Systems against Advanced Persistent Threats Attack Vectors." In: *Journal of Cybersecurity and Privacy* 1.3 (July 2021), pp. 387–421. issn: 2624-800X. doi: [10.3390/jcp1030021](https://doi.org/10.3390/jcp1030021). url: <http://dx.doi.org/10.3390/jcp1030021>.
- [40] Kaspersky. *System Watcher: Behavioral Detection Technology*. Accessed: 2025-10-05. 2024. url: <https://support.kaspersky.com/ksv5la/148848>.

- [41] Kaspersky Lab. *Machine Learning for Malware Detection*. Tech. rep. Accessed: 2025-02-26. Kaspersky Lab, 2017. url: <https://media.kaspersky.com/en/enterprise-security/Kaspersky-Lab-Whitepaper-Machine-Learning.pdf>.
- [42] J. Kastler et al. "Source code obfuscation with genetic algorithms using LLVM code optimizations." In: *Logic Journal of the IGPL* 33.5 (2024). Source 22 in provided list; Discusses optimization trade-offs. url: <https://academic.oup.com/jigpal/article/33/5/jzae069/7670720>.
- [43] Harpreet Kaur et al. "Evolution of Endpoint Detection and Response (EDR) in Cyber Security: A Comprehensive Review." In: *E3S Web of Conferences* 556 (Aug. 2024), p. 01006. doi: [10.1051/e3sconf/202455601006](https://doi.org/10.1051/e3sconf/202455601006).
- [44] Youngjoon Ki, Eunjin Kim, and Huy Kang Kim. "A Novel Approach to Detect Malware Based on API Call Sequence Analysis." In: *International Journal of Distributed Sensor Networks* 2015 (2015), pp. 1–9. doi: [10.1155/2015/659101](https://doi.org/10.1155/2015/659101).
- [45] Samuel Kim. "PE Header Analysis for Malware Detection." Master's Project. San Jose State University, 2018. doi: [10.31979/etd.q3dd-gp9u](https://doi.org/10.31979/etd.q3dd-gp9u). url: https://scholarworks.sjsu.edu/etd_projects/624.
- [46] Raphael Labaca Castro, Corinna Schmitt, and Gabi Dreo Rodosek. "ARMED: How Automatic Malware Modifications Can Evade Static Detection?" In: *2019 5th International Conference on Information Management (ICIM)*. IEEE, 2019, pp. 20–27. doi: [10.1109/INFOMAN.2019.8714698](https://doi.org/10.1109/INFOMAN.2019.8714698).
- [47] Caroline Lemieux and Koushik Sen. "FairFuzz: A Targeted Mutation Strategy for Increasing Greybox Fuzz Testing Coverage." In: *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE)*. ACM, 2018, pp. 475–485. doi: [10.1145/3238147.3238176](https://doi.org/10.1145/3238147.3238176).
- [48] Andrea Lepori. "Automated Code Transformations to Bypass and Understand Endpoint Detection and Response (EDR) Systems." Accessed: 2025-12-18. MA thesis. ETH Zurich, May 2025. doi: [10.3929/ethz-b-000737933](https://doi.org/10.3929/ethz-b-000737933). url: <https://www.research-collection.ethz.ch/handle/20.500.11850/737933>.
- [49] Chenyang Lyu et al. "MOpt: Optimized Mutation Scheduling for Fuzzers." In: *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, 2019, pp. 1949–1966.
- [50] Oladipo A. Madamidola, Felix Ngobigha, and Adnane Ez-zizi. "Detecting new obfuscated malware variants: A lightweight and interpretable machine learning approach." In: *Intelligent Systems with Applications* 25 (Mar. 2025), p. 200472. issn: 2667-3053. doi: [10.1016/j.iswa.2024.200472](https://doi.org/10.1016/j.iswa.2024.200472). url: <http://dx.doi.org/10.1016/j.iswa.2024.200472>.
- [51] Björn Mathis, Rahul Gopinath, and Andreas Zeller. "Learning Input Tokens for Effective Fuzzing." In: *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2020, pp. 27–37. doi: [10.1145/3395363.3397348](https://doi.org/10.1145/3395363.3397348).
- [52] William M. McKeeman. "Differential Testing for Software." In: *Digital Technical Journal* (1998). Introduces oracle-free differential testing.
- [53] Guozhu Meng et al. "MYSTIQUE: Evolving Android Malware for Auditing Anti-Malware Tools." In: *Proceedings of the 11th ACM on Asia Conference on Computer and Communications Security (ASIA CCS '16)*. ACM, 2016, pp. 365–376. doi: [10.1145/2897845.2897856](https://doi.org/10.1145/2897845.2897856).
- [54] Microsoft. *Event Tracing for Windows (ETW)*. Accessed: 2025-10-05. 2024. url: <https://learn.microsoft.com/en-us/windows/win32/etw/event-tracing-portal>.
- [55] Microsoft. *Overview of Windows Components*. Accessed: 2025-10-02. 2025. url: <https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/overview-of-windows-components>.
- [56] Microsoft. *PE Format – Win32 Apps*. Accessed: 2025-10-05. 2025. url: <https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>.
- [57] MITRE. *Process Injection, Technique T1055 – ATT&CK Enterprise*. Accessed: 2025-10-05. 2024. url: <https://attack.mitre.org/techniques/T1055/>.
- [58] Philip O'Kane, Sakir Sezer, and Kieran McLaughlin. "Obfuscation: The Hidden Malware." In: *IEEE Security & Privacy* 9.5 (2011), pp. 41–47. doi: [10.1109/MSP.2011.98](https://doi.org/10.1109/MSP.2011.98).
- [59] Matt Pietrek. "Peering Inside the PE: A Tour of the Win32 Portable Executable File Format." In: *Microsoft Systems Journal* 9.3 (1994). url: [https://learn.microsoft.com/en-us/previous-versions/ms809762\(v=msdn.10\)](https://learn.microsoft.com/en-us/previous-versions/ms809762(v=msdn.10)).
- [60] Sanjay Rawat et al. "VUzzer: Application-aware Evolutionary Fuzzing." In: *24th Annual Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2017. doi: [10.14722/ndss.2017.23404](https://doi.org/10.14722/ndss.2017.23404).
- [61] Retooling.io. *An Unexpected Journey into Microsoft Defender's Signature World*. Accessed: 2025-10-02. 2023. url: <https://retooling.io/blog/an-unexpected-journey-into-microsoft-defenders-signature-world>.
- [62] Dobin Rutishauser. *MalDev Myths*. Accessed: 2025-12-15. 2025. url: <https://blog.deeb.ch/posts/maldev-myths/>.
- [63] Dobin Rutishauser. "My First and Last Shellcode Loader." In: *HITBSecConf 2024 Bangkok, CommSec Track*. Accessed: 2025-12-15. 2024. url: <https://conference.hitb.org/hitbsecconf2024bkk/speaker/dobin-rutishauser/>.
- [64] Dobin Rutishauser. *The (Anti-)EDR Compendium*. Accessed: 2025-12-15. 2024. url: <https://blog.deeb.ch/posts/how-edr-works/>.
- [65] Christopher Salls et al. "Token-Level Fuzzing." In: *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2021, pp. 2795–2809.
- [66] Dongdong She et al. "NEUZZ: Efficient Fuzzing with Neural Program Smoothing." In: *2019 IEEE Symposium on Security and Privacy (SP)*. 2019, pp. 803–817. doi: [10.1109/SP.2019.00052](https://doi.org/10.1109/SP.2019.00052).

- [67] Avast Software. *Behavior Shield in Avast Antivirus*. Accessed: 2025-10-05. 2024. url: https://businesshelp.avast.com/Content/Products/AfB_Management_Consoles/ConfiguringSettingsandPolicies/BehaviorShield.htm.
- [68] Avast Software. *What is CyberCapture in Avast Antivirus?* Accessed: 2025-10-05. 2024. url: <https://www.avast.com/en-us/business/resources/data-sheet/cloud-sandbox-faq-data-sheet>.
- [69] Diomidis Spinellis. "Reliable identification of bounded-length viruses is NP-complete." In: *Information Theory, IEEE Transactions on* 49 (Feb. 2003), pp. 280–284. doi: [10.1109/TIT.2002.806137](https://doi.org/10.1109/TIT.2002.806137).
- [70] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. Adaptive Computation and Machine Learning. Cambridge, MA: MIT Press, 2018. isbn: 9780262039246. url: <http://incompleteideas.net/book/the-book-2nd.html>.
- [71] Pang-Ning Tan, Vipin Kumar, and Jaideep Srivastava. "Selecting the Right Interestingness Measure for Association Patterns." In: *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York, NY, USA: ACM, 2002, pp. 32–41. doi: [10.1145/775047.775053](https://doi.org/10.1145/775047.775053).
- [72] Kostas Tsale. *EDR Telemetry Project: Comparing Endpoint Detection Sources*. GitHub: <https://github.com/tsale/EDR-Telemetry>. Accessed: 2025-12-15. 2024. url: <https://www.edr-telemetry.com/>.
- [73] John Uhlmann. *Kernel ETW is the best ETW*. Elastic Security Labs. Accessed: 2025-09-21. Sept. 2024. url: <https://www.elastic.co/security-labs/kernel-etw-best-etw>.
- [74] Apurva Virkud et al. "How does Endpoint Detection use the MITRE ATT&CK Framework?" In: *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 3891–3908. isbn: 978-1-939133-44-1. url: <https://www.usenix.org/conference/usenixsecurity24/presentation/virkud>.
- [75] VirusTotal. *YARA: The pattern matching swiss knife*. Accessed: 2026-03-08. 2024. url: <https://github.com/virustotal/yara>.
- [76] Junjie Wang et al. "Superion: Grammar-Aware Greybox Fuzzing." In: *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 2019, pp. 724–735. doi: [10.1109/ICSE.2019.00081](https://doi.org/10.1109/ICSE.2019.00081).
- [77] George D. Webster et al. "Finding the Needle: A Study of the PE32 Rich Header and Respective Malware Triage." In: *Detection of Intrusions and Malware, and Vulnerability Assessment*. Ed. by Michalis Polychronakis and Michael Meier. Cham: Springer International Publishing, 2017, pp. 119–138. isbn: 978-3-319-60876-1.
- [78] Pim van de Wouw. *API Unhooking and the Transitive Hooking Problem*. Unverifiable – no public record found. Verify source manually. 2024.
- [79] Pavel Yosifovich et al. *Windows Internals, Part 1*. User/kernel separation, processes, memory management. Microsoft Press, 2017. isbn: 9780735684188.
- [80] Michał Zalewski. *American Fuzzy Lop*. Coverage-guided fuzzing framework. 2014. url: <https://lcamtuf.coredump.cx/afl/>.
- [81] Shuofei Zhu et al. "Measuring and Modeling the Label Dynamics of Online Anti-Malware Engines." In: *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 2361–2378. isbn: 978-1-939133-17-5. url: <https://www.usenix.org/conference/usenixsecurity20/presentation/zhu>.

Appendix

Build and Execution Instructions

The runtime and all associated tooling for this work are available at:

<https://github.com/2Tricky4u/Automated-Analysis-and-Mutation-of-Software-Artifacts-against-AV-EDR/>

Environment Requirements

The framework requires a **Windows** host with the following features enabled:

- **Hyper-V** and **VirtualMachinePlatform**
- **WSL2** hosts the controller, Elasticsearch 8.11, and Kibana 8.11 in Docker containers.
- **IsolationSwitch** a Hyper-V internal switch on 10.200.200.0/24 connecting VMs to the host.

A master configuration file (`automation/config.yaml`) defines network topology, worker VM specifications, and service ports. Template-based config generation (`generate-configs.ps1`) expands `{{TOKEN}}` placeholders in TOML templates to produce per-component configuration files.

Dev Container and Toolchain

The WSL2 environment is bootstrapped by `02-wsl-bootstrap.sh`, which installs:

- Rust (stable + nightly) with `llvm-tools` component,
- Clang/LLVM 17 and LLD (cross-compilation to Windows PE via `xwin` SDK),
- Protocol Buffers compiler (`protoc` 25.1) and `grpcurl` 1.8.6,
- Docker Engine + Compose (Elasticsearch 8.11.0 and Kibana 8.11.0).

Worker VMs require Rust (stable), Visual Studio 2022 Build Tools (C++ workload + Windows SDK), `protoc`, and the VC++ redistributable, all installed automatically by `04-vm-init.ps1`.

Automated Setup

The entire infrastructure is provisioned by running `setup-all.ps1` as Administrator, which executes five numbered scripts in sequence:

1. **Host setup** (`01-host-setup.ps1`) enables Hyper-V and WSL2, creates the internal switch with NAT, configures IP forwarding and firewall rules.
2. **WSL bootstrap** (`02-wsl-bootstrap.sh`) installs the Rust toolchain, Docker, and starts the Elasticsearch/Kibana stack as a systemd service.
3. **VM creation** (`03-create-worker-vm.ps1`) creates Hyper-V Gen2 VMs with TPM 2.0 and dynamic VHDX disks. Windows 10/11 ISOs are downloaded and hash-verified automatically.
4. **VM initialization** (`04-vm-init.ps1`) a 10-step script run inside each VM: static IP assignment, development tool installation, RedEDR extraction and driver registration, audit policy configuration, and boot-time settings (`testsigning`, kernel debug).
5. **Baseline checkpoint** (`05-create-baseline.ps1`) creates a clean Hyper-V snapshot per VM, enabling fast revert between experimental rounds.

After setup, day-to-day operation uses `start-environment.ps1` and `stop-environment.ps1` to bring all services and VMs up or down. A 12-point health check (`validate-environment.ps1`) verifies Windows features, WSL state, network connectivity, Elasticsearch health, and VM status.

Security Controls

Three host-enforced scripts manage VM network access:

- `toggle-vm-internet.ps1` enables or disables NAT (air-gap mode; VMs retain host access).
- `manage-egress-filter.ps1` default-deny outbound firewall with a whitelist (DNS, HTTP/S, NTP).
- `toggle-vm-isolation.ps1` blocks VM-to-VM traffic, permitting only VM-to-host communication.

Remote Deployment

For workers not hosted locally, `deploy-remote-worker.ps1` deploys the agent to any reachable Windows machine over SSH. It builds the worker binary, generates configuration, transfers files via SCP, and executes initialization remotely. Workers self-register with the controller at startup via gRPC.

Loader Template

The following listings show the modular template files that serve as the starting point for artifact assembly (Section [Template Assembly](#)). Listing 8 shows the core loader logic including the carrier, decoder, and guardrail integration points. Listing 9 shows a basic deconditioner module, the one that was used in evaluation.

All templates can be looked at in the Github repo at [build/templates](#)

Listing 8: Template of loader implementation in C

```

1  #include <windows.h>
2  #include "instrumentation.h"
3
4  @MODULE:payload
5
6  @MODULE:definitions
7
8  @MODULE:decoder
9
10 @MODULE:virtualprotect
11
12 @MODULE:antiemulation
13
14 @MODULE:guardrail
15
16 @MODULE:decoy
17
18 @MODULE:carrier
19
20 // =====
21 // MAIN ENTRY POINT
22 // =====
23 int main(void) {
24     @MUTATE:benign_preamble
25     @MUTATE:timing_jitter
26
27     @MUTATE:opaque_predicate
28     int gr = guardrail();
29     if (gr != 0) {
30         @MUTATE:dead_code_insertion
31         return gr;
32     }
33     ARTIFACT_CHECKPOINT("guardrails_passed");
34
35     @MUTATE:timing_jitter
36     antiemulation();
37     ARTIFACT_CHECKPOINT("antiemulation_passed");
38     @MUTATE:api_sequence_obfuscation
39
40     deconditioner();
41     ARTIFACT_CHECKPOINT("deconditioner_passed");
42
43     @MUTATE:timing_jitter
44     decoy();
45     ARTIFACT_CHECKPOINT("decoy_passed");
46     @MUTATE:api_sequence_obfuscation
47
48     @MUTATE:timing_jitter
49     return carrier();
50 }
```

Listing 9: Template of basic deconditioner module implementation in C

```

1  /* MODULE: DECONDITIONER
2  * TYPE: basic
3  * DESC: Research-informed deconditioning template (Decondition Everything).
4  *
5  *      Mirrors the carrier's alloc->write->protect->[exec]->free pattern with
6  *      benign data for N rounds. Theory: EDR behavioral rules have per-process
7  *      cutoff counters. After N benign triggers, the counter overflows and
8  *      subsequent triggers (including malicious) bypass the rule.
9  *
10 *      Key parameters (varied by AST mutations):
11 *      - decon_rounds:      iteration count (must exceed EDR cutoff)
12 *      - fill_pattern:      benign data type (affects entropy-based scans)
13 *      - exec_decoy:        execute from alloc'd memory (normalizes #1 signal)
14 *      - timing_pattern:    delays between API calls (affects temporal tokens)
15 *      - protection_transition: memory protection changes (must match carrier rule)
16 *
17 *      Default: 20 rounds, XOR fill, no execution, no delay, RW->RX.
18 */
19 #include "../header/definitions.h"
20
21 #ifndef DECON_ROUNDS
22 #define DECON_ROUNDS 20
23 #endif
24
25 void deconditioner() {
26     DWORD old_prot;
27
28     @MUTATE:decon_rounds
29     for (int i = 0; i < DECON_ROUNDS; i++) {
30
31         char *buf = (char*)VirtualAlloc(NULL, PAYLOAD_LEN, 0x3000, p_RW);
32         if (!buf) continue;
33
34         @MUTATE:fill_pattern
35         for (int k = 0; k < PAYLOAD_LEN; k++) {
36             buf[k] = (char)(k ^ (i + 0x41));
37         }
38
39         @MUTATE:timing_pattern
40         @MUTATE:protection_transition
41         VirtualProtect(buf, PAYLOAD_LEN, p_RX, &old_prot);
42
43         @MUTATE:exec_decoy
44
45         @MUTATE:timing_pattern
46         VirtualFree(buf, 0, 0x8000);
47     }
48 }

```

Pattern Signature Internals

Signature-based defense mechanisms, as employed by modern antivirus (AV) and endpoint detection solutions, operate by comparing binary content against predefined patterns, or *signatures*. Each signature is typically decomposed into multiple *sub-rules*. A sub-rule encodes a specific byte sequence or structural feature of the target binary, and each is assigned a numerical *weight*. When a binary is scanned, the AV engine accumulates the weights of all matching sub-rules.

A signature further defines a *score threshold* T , which represents the minimum cumulative weight required for detection. If the total weight of the matched sub-rules satisfies

$$\sum_{i=1}^n w_i \geq T,$$

the signature is triggered and the binary is flagged as malicious.

This design allows for two types of matches:

- **Non-dominant matches:** Detection is triggered by the aggregation of several lower-weight sub-rules, none of which individually reach the threshold. For instance, if $T = 10$ and each of 8 sub-rules has $w_i = 4$, at least three sub-rules must match for detection to occur.
- **Dominant matches:** A single sub-rule with weight $w_i \geq T$ is sufficient to trigger detection immediately. Such rules usually correspond to highly characteristic or unique byte patterns.

The thresholded sub-rule approach improves resilience of signature-based detection by tolerating partial obfuscation or minor code mutations: detection still occurs as long as sufficient sub-rules survive transformation. Conversely, adversaries may bypass detection by mutating enough code regions such that the cumulative score remains below the threshold, effectively evading non-dominant matches [23].

Wildcard Patterns in Signatures

Beyond the concept of weighted sub-rules, modern signature systems also employ *wildcards*. Wildcards allow a rule to tolerate differences in a file without failing the overall match. This is important because malware samples often differ in minor details, for example by inserting padding bytes or using slightly different compiler output. Instead of demanding a byte-for-byte perfect match, the wildcard acts as a placeholder: “there may be some bytes here, but their exact value does not matter.”

From reverse engineering it has become clear that these wildcards follow a regular encoding pattern. For instance, some encodings indicate that “any one byte is acceptable here,” while others allow for multiple arbitrary bytes. When several such positions are defined inside a sub-rule, the signature can still match malware families even if certain offsets vary between samples.

Of course, the exact patterns and their encodings are regularly updated as signatures evolve. For presentation and understanding, however, these patterns can often be summarized in a YARA-like notation. For example, a signature rule for PlugX.A uses wildcards to ignore one byte at certain positions. In YARA syntax this would simply be expressed with the ?? symbol:

```
rule Pattern_example {
  strings:
    $a = { 45 78 69 74 C7 85 ?? FF FF FF
          54 68 72 65 66 C7 85 ?? 04 FF FF FF 61 64 }
  condition:
    $a
}
```

This shows how, while the underlying engine encodes the rule in its own proprietary format, it can be conceptually represented as a familiar YARA pattern with wildcards. In this sense, wildcards complement the weighted sub-rule mechanism: they provide resilience against minor variation without weakening the overall detection logic [61].

PE File Format Details

This appendix provides the field-level detail of the Portable Executable (PE) structures introduced in Section [The Portable Executable \(PE\) Format](#), based on the Microsoft PE/COFF specification [56]. All offsets are relative to the start of the respective header unless noted otherwise.

DOS Header

The DOS header occupies the first 64 bytes of every PE file. The two fields relevant to PE parsing are:

- `e_magic` (offset 0x00, 2 bytes): Magic number `0x5A4D` (ASCII "MZ"), identifying the file as a valid DOS executable.
- `e_lfanew` (offset 0x3C, 4 bytes): File offset to the PE signature. The loader reads this value to locate the start of the PE headers, skipping the DOS stub.

COFF Header

Immediately after the four-byte PE signature (`0x00004550`), the 20-byte COFF header contains:

- `Machine` (2 bytes): Target architecture (e.g., `0x8664` for x86-64, `0x014C` for i386).
- `NumberOfSections` (2 bytes): Count of entries in the section table.
- `TimeStamp` (4 bytes): Unix timestamp of the link step. This field is often used as a heuristic feature in static analysis.
- `SizeOfOptionalHeader` (2 bytes): Size of the Optional header that follows; determines which Data Directory entries are present.
- `Characteristics` (2 bytes): Bit flags indicating whether the file is an executable, a DLL, supports large addresses, etc.

Optional Header (PE32+)

For 64-bit executables the Optional header is 240 bytes and includes:

- `AddressOfEntryPoint` (4 bytes): RVA of the first instruction the loader executes.
- `ImageBase` (8 bytes): Preferred base address; if unavailable, the loader applies relocations.
- `SectionAlignment` (4 bytes): Alignment of sections when loaded into memory (typically `0x1000`).
- `FileAlignment` (4 bytes): Alignment of raw section data on disk (typically `0x200`).
- `SizeOfImage` (4 bytes): Total virtual size of the loaded image, rounded up to `SectionAlignment`.
- `NumberOfRvaAndSizes` (4 bytes): Number of Data Directory entries (usually 16).
- **Data Directory array** (16×8 bytes): Each entry is an RVA/size pair pointing to a well-known structure. Key indices include: 0 (Export Table), 1 (Import Table), 2 (Resource Table), 5 (Base Relocation Table), 6 (Debug Directory), and 14 (CLR Runtime Header).

Section Table

Each 40-byte section table entry describes one section:

- `Name` (8 bytes): Null-padded ASCII name (e.g., `.text`, `.rdata`).
- `VirtualSize` (4 bytes): Actual size of section content in memory.
- `VirtualAddress` (4 bytes): RVA where the section is mapped.
- `SizeOfRawData` (4 bytes): Size of initialized data on disk, rounded to `FileAlignment`.
- `PointerToRawData` (4 bytes): File offset to the section's raw data.
- `Characteristics` (4 bytes): Bit flags encoding permissions (`IMAGE_SCN_MEM_EXECUTE`, `IMAGE_SCN_MEM_READ`, `IMAGE_SCN_MEM_WRITE`) and content type (`IMAGE_SCN_CNT_CODE`, `IMAGE_SCN_CNT_INITIALIZED_DATA`).

Rich Header Encoding

The Rich header is an undocumented block inserted by the MSVC linker between the DOS stub and the PE signature. Its structure is:

1. A variable-length padding region filled with the XOR key.
2. The marker Rich (4 bytes, ASCII), followed by the 4-byte **checksum key**.
3. Preceding the marker, a sequence of 8-byte records, each XOR-encoded with the checksum key. After decoding, each record yields a `comp.id` (product ID in the upper 16 bits, build number in the lower 16 bits) and a count (number of object files produced by that toolchain component).
4. The block begins (after decoding) with the signature DanS followed by three zero-padded DWORDs.

The block is XOR-obfuscated using a 32-bit key stored after the Rich marker. Published reverse-engineering describes a checksum-style computation over DOS header bytes and the decoded records to derive and validate this key [77]; exact details vary slightly across descriptions. Security tools use the decoded product IDs to identify MSVC compiler, assembler, and linker versions, flagging inconsistencies as indicators of manual tampering or alternative toolchains.

Import Directory and IAT Resolution

The Import Directory (Data Directory index 1) consists of an array of `IMAGE_IMPORT_DESCRIPTOR` entries, one per imported DLL, terminated by a null entry. Each descriptor contains:

- **OriginalFirstThunk**: RVA of the *Import Lookup Table* (ILT), an array of pointers to `IMAGE_IMPORT_BY_NAME` entries (each holding a 2-byte hint and a null-terminated function name). If this field is zero, the loader uses **FirstThunk** for name/ordinal lookup as well.
- **Name**: RVA of the null-terminated DLL name string.
- **FirstThunk**: RVA of the *Import Address Table* (IAT), initially a mirror of the ILT.

At load time the Windows loader processes each descriptor: it loads (or locates) the named DLL, iterates the ILT, resolves each function by name or ordinal via the DLL's export table, and overwrites the corresponding IAT entry with the resolved virtual address. After loading, compiled code calls imported functions through the IAT pointers (typically via `call [rip+offset]`), making the IAT a natural interception point for IAT-patching hooks, while many EDRs additionally rely on inline hooks in user-mode libraries (e.g., `ntdll.dll`) and kernel-level telemetry.

Vectored Exception Handling and INT3 Checkpoints

This appendix describes the low-level mechanisms behind the INT3-based checkpoint system referenced in Section [Windows Primitives for Code Execution](#) and used by the runtime implementation for lightweight execution tracing.

Vectored Exception Handling (VEH)

Windows provides two exception-handling mechanisms that operate *before* frame-based Structured Exception Handling (SEH): **Vectored Exception Handlers** (VEH) and Vectored Continue Handlers. A process registers a VEH callback via

`AddVectoredExceptionHandler`:

```
PVOID AddVectoredExceptionHandler(ULONG First, PVECTORED_EXCEPTION_HANDLER Handler);
```

When an exception occurs, the kernel dispatches an `EXCEPTION_RECORD` to user mode. The VEH dispatch chain is walked *before* any SEH frame: if `First` is non-zero, the handler is inserted at the head of the chain. The handler receives a pointer to an `EXCEPTION_POINTERS` structure containing both the `EXCEPTION_RECORD` (exception code, faulting address) and the thread's `CONTEXT` (register state at the point of the exception). Returning `EXCEPTION_CONTINUE_EXECUTION` resumes execution at the instruction pointer stored in the context; returning `EXCEPTION_CONTINUE_SEARCH` passes the exception to the next handler in the chain.

INT3 Breakpoint Instruction

The x86 `INT3` instruction (opcode `0xCC`) is a single-byte software breakpoint. When the CPU executes `0xCC`, it raises a `#BP` exception (vector 3), which the OS translates into an `EXCEPTION_BREAKPOINT` exception record (code `0x80000003`). This mechanism mirrors the classic implementation of software breakpoints used by debuggers: the debugger replaces the first byte of a target instruction with the `INT3` (`0xCC`) opcode, which triggers a breakpoint exception when executed; the debugger then restores the original opcode and resumes execution [35, 79, Vol. 3A, Sec. 6.3].

Checkpoint Application

The runtime combines VEH and INT3 into a lightweight checkpoint mechanism:

1. **Registration:** At startup, a VEH handler is installed at the head of the chain.
2. **Patching:** For each checkpoint site (identified by address), the runtime saves the original byte and overwrites it with `0xCC`. Each address is associated with a checkpoint ID.
3. **Execution:** When the patched instruction is reached, the CPU raises `EXCEPTION_BREAKPOINT`. The VEH handler:
 - (a) Identifies the checkpoint by the faulting address (`ExceptionRecord->ExceptionAddress`).
 - (b) Logs the checkpoint ID (or increments a coverage counter).
 - (c) Restores the original byte at the faulting address.
 - (d) Sets the instruction pointer in the thread context to `ExceptionRecord->ExceptionAddress` (so execution resumes at the now-restored original instruction).
 - (e) Returns `EXCEPTION_CONTINUE_EXECUTION`.
4. **Result:** Execution continues transparently from the original instruction, and the checkpoint has been recorded with minimal overhead.

Because the original byte is restored and `0xCC` is not re-inserted, each checkpoint is *one-shot*: it fires exactly once and then disappears, leaving the original instruction stream intact for all subsequent executions of that address. This approach provides instruction-granularity tracing without requiring recompilation or binary rewriting of the target. The trade-off is the per-checkpoint exception overhead, which is acceptable for the sparse checkpoint densities used in the experimental pipeline. The recursive-descent patching algorithm that selects checkpoint sites is described in the Implementation chapter.

Kernel Patch Protection and the End of Third-Party Kernel Hooking

Before Kernel Patch Protection, third-party security drivers routinely modified kernel code at runtime to intercept system calls, a technique known as **inline kernel hooking**.

The most common approach involved patching the System Service Descriptor Table (**SSDT**), which maps system call numbers to their kernel handler addresses: a driver would replace an SSDT entry with a pointer to its own filter function, inspect the call arguments, and then forward the request to the original handler.

An alternative was to overwrite the first bytes of a kernel function's prologue with an unconditional jump (**JMP**) to the hook routine, which provided interception at an even finer granularity. Both methods gave security products *synchronous, real-time* visibility into every system call, enabling them to block malicious operations before they completed.

Kernel Patch Protection (KPP/PatchGuard). Starting with Windows Vista x64, Microsoft introduced **Kernel Patch Protection** (commonly called **PatchGuard**), a mechanism that periodically verifies the integrity of critical kernel structures, including the SSDT, the Interrupt Descriptor Table (IDT), the Global Descriptor Table (GDT), and key kernel code regions [79].

If tampering is detected, PatchGuard triggers a **CRITICAL_STRUCTURE_CORRUPTION** bugcheck, immediately halting the system with a blue screen. Because PatchGuard's checks are obfuscated and run at unpredictable intervals, reliably bypassing it is impractical for commercial software.

Consequences for endpoint security. With inline kernel hooking no longer viable, security vendors were forced to adopt Microsoft-sanctioned kernel APIs for visibility: notification callbacks (**PsSetCreateProcessNotifyRoutineEx**, **PsSetLoadImageNotifyRoutine**, **ObRegisterCallbacks**), minifilter drivers for I/O interception, and, critically, asynchronous **kernel ETW** providers for after-the-fact telemetry.

This architectural shift is the primary reason modern EDRs depend heavily on kernel ETW as described in Section **Event Tracing for Windows (ETW)**: it is the only sanctioned, non-invasive mechanism that provides broad kernel-level visibility without modifying protected structures.

Elasticsearch Index Schema

The controller stores all experimental data in Elasticsearch, organized into six index families with time-based rotation (Table 24).

Table 24: Elasticsearch index families, rotation strategies, and key fields.

Index pattern	Rotation	Doc ID	Key fields
jobs-YYYY.MM	Monthly	job_id	config, selector, status
rounds-YYYY.MM	Monthly	job_id/round_id	category, evasion_score, mutations
runs-YYYY.MM	Monthly	run_id	outcome, elapsed_ms, trace_mode
telemetry-YYYY.MM.DD	Daily	auto	ETW events, payload_* fields
tokens-YYYY.MM	Monthly	auto	token, round, detected
artifacts-YYYY.MM	Monthly	auto	sha256, path, size

Monthly rotation keeps index sizes manageable for jobs, rounds, runs, tokens, and artifacts. Telemetry indices rotate daily because ETW event volume is substantially higher.

Dynamic templates. Telemetry documents contain arbitrary fields from different ETW providers. Dynamic templates map `payload_*` string fields to the `keyword` type and numeric fields to `long`, eliminating manual schema changes when new providers are added. The `mutation_recipe.params` field is stored with `enabled:false` (opaque JSON blob, not indexed), and `assembled_source` is stored with `index:false` (retrievable but not searchable), avoiding unnecessary index overhead for large or opaque data.

Write consistency. Index operations use `Refresh::WaitFor` to guarantee read-after-write consistency, which the asynchronous coverage-correction pipeline (Section [Asynchronous Coverage Correction](#)) depends on when querying trace data immediately after round finalization. Document updates resolve the target monthly index via a `find_index()` search and retry up to three times on HTTP 409 version conflicts.

Token Comparison Pipeline

The conceptual framework for token set differencing and parameter distance is described in Sections [Token Set Differencing](#) and [Mutation Parameter Distance](#). This section documents the implementation of the `CompareTokens` gRPC endpoint that exposes these capabilities.

gRPC entry point. The `CompareTokens` RPC accepts a `CompareTokensRequest` containing two run identifiers (`run_id_a`, `run_id_b`). The handler in `api/job.rs` implements a four-stage pipeline:

1. **Run lookup.** Query Elasticsearch for both run documents via `query_runs_by_ids()`, extracting `job_id` and `round_id` for each run.
2. **Token retrieval.** Query the `tokens-YYYY.MM` index for both rounds in parallel using `tokio::join!`, extracting token string arrays from the stored JSON documents.
3. **Set comparison.** Call `token_diff::compare_token_sets(&tokens_a, &tokens_b, &default_registry())` to compute set differences and per-mutation parameter distances.
4. **Proto conversion.** Map the Rust `TokenSetComparison` struct to `TokenSetComparisonProto` for gRPC transport.

The parallel token retrieval is significant: since the two rounds may reside in different monthly index shards, the `tokio::join!` ensures both Elasticsearch queries execute concurrently rather than sequentially.

Set comparison algorithm. `compare_token_sets` in `token_diff.rs` operates in three phases:

1. **Set operations.** Build `HashSet<str>` from both input slices. Compute symmetric difference, intersection, and union. Jaccard distance: $1 - |A \cap B| / |A \cup B|$, with the convention that two empty sets have distance 0.0.
2. **Mutation token parsing.** Iterate both input arrays, calling `parse_mutation_token()` on each. Tokens starting with "mutation:" are parsed into a `ParsedMutationToken` struct: the mutation identifier (first colon-separated segment) and a `HashMap<String, String>` of key-value parameters (subsequent `k=v` segments). Parsed tokens are grouped by mutation identifier.
3. **Per-mutation comparison.** For each mutation identifier present in either set: if present in both, look up the mutation in the `MutationParamSpace` registry via `find_param_space()`; if found, delegate to `compare_params()` for per-parameter distance; otherwise, compare raw token strings (0.0 if identical, 1.0 if different). Mutations present in only one set receive `overall_distance=1.0` (maximal difference).

Results are sorted by `mutation_id` for deterministic output.

Distance computation detail. For `IntRange` parameters, `ParamDef::distance()` in `param_space.rs` computes:

```
1 let range = (*max - *min) as f64;
2 let va: f64 = a.parse().unwrap_or(*min as f64);
3 let vb: f64 = b.parse().unwrap_or(*min as f64);
4 let norm = if range <= 0.0 { 0.0 }
5           else { ((va - vb).abs() / range).clamp(0.0, 1.0) };
```

Missing keys in either parameter map fall back to the parameter's default value:

```
1 let default = p.default_value();
2 let va = a.get(p.name()).unwrap_or(&default);
3 let vb = b.get(p.name()).unwrap_or(&default);
```

The `overall_distance` for a mutation is the arithmetic mean of its per-parameter normalised distances:

```
1 distances.iter().map(|d| d.distance.normalized).sum::<f64>()
2 / distances.len() as f64
```

Controller Communication and Synchronization

All inter-component communication in the controller uses typed mpsc channels; no mutable state is shared except through the DashMap-backed RunPool and TargetManager, each of which localizes synchronization to individual entries. This appendix documents the concrete channel topology and synchronization primitives that underpin the scheduling design (Section [Controller Design: Jobs, Rounds, and Fair Scheduling](#)) and the dispatch mechanisms (Section [Dryrun Dispatch and Grace Period](#)).

Channel Topology

Nine typed channels connect the system's components (Table 25). Each channel carries exactly one message type, enabling Rust's type system to prevent protocol mismatches at compile time.

Table 25: Typed channel topology: producers, consumers, message types, and capacities.

#	Producer	Consumer	Message Type	Cap.
1	API	Orchestrator	JobSession	128
2	API	Orchestrator	JobControlCommand	64
3	TargetManager	Orchestrator	TargetEvent	4096
4	JobWorker	Orchestrator	JobWorkerEvent	256
5	RunPool	JobWorker	JobRunResult	64/job
6	VMExecutor	StreamHandler	ControllerMessage	128
7	StreamHandler	VMExecutor	RemoteRunResult	128
8	Orchestrator	JobWorker	CoverageCorrection	per-job
9	Triage spawn	JobWorker	TriageGuidance	per-job

Channel capacities encode backpressure semantics. The telemetry channel (#3, capacity 4096) absorbs bursts from concurrent VM streams without blocking the stream handlers. The job-control channel (#2, capacity 64) is bounded tightly because stop commands must never be dropped or delayed behind a deep queue. Per-job channels (#5, #8, #9) are scoped to a single JobWorker and are created and destroyed with the job lifecycle.

RunPool Synchronization Stack

The RunPool uses a three-layer synchronization strategy (Table 26), choosing each primitive based on its access pattern rather than relying on a single global lock.

Table 26: RunPool synchronization primitives and their roles.

Layer	Primitive	Purpose
Storage	DashMap<RunId, RunEnvelope>	Lock-free concurrent reads and writes of run envelopes
Ordering	Mutex<VecDeque<RunId> per OS	FIFO ordering within each OS-sharded partition
Wake	tokio::sync::Notify	Signal-driven wake of idle VMExecutors when work arrives

DashMap provides $O(1)$ lock-free reads for the common case (envelope lookup by RunId). The per-OS Mutex is held only for the microseconds required to push or pop a RunId from the queue, ensuring that OS-heterogeneous VM pools never contend across partitions. Notify replaces polling: idle executors park on a single notification and are woken when add_runs() inserts new work. The TargetManager uses a similar DashMap<TargetId, Target> pattern, with per-entry writes for reserve and release operations and no global mutex anywhere in the system.

Capability Discovery via Deferred Executor Spawn

The VMExecutor is not started when a gRPC stream is established. Instead, a deferred task waits up to 15 seconds for the worker's Registration message, which carries the OS version and capability set (e.g., whether the VM is designated for dryrun execution). Only when this message arrives does the executor start and begin pulling from the RunPool with accurate capability information. On timeout or stream closure, the executor is never created. This *capability-before-activation* pattern prevents a race condition in which an executor could take runs from the pool before knowing its OS or capabilities, leading to mismatched dispatches, the same invariant enforced by the bidirectional dryrun guard (Section [Dryrun Dispatch and Grace Period](#)).

Worker Exit Code Classification

Table 27 documents the complete mapping from process exit codes to per-run detection verdicts used by the Worker Agent's classifier (Section [Worker Agent Design: Robust Execution as a Ten-Phase Pipeline](#)).

Table 27: Worker-agent exit code classification (5 provisional verdicts). The controller may later promote Ambiguous to MutationFailed, PayloadFailed, Detected, or Anomaly using dryrun context. The synthetic $-100\,00x$ codes live in a range unreachable by `ExitProcess(UINT)`, so they never collide with real process exit codes or `NTSTATUS` values.

Code(s)	Source	Meaning	Verdict
$-100\,004$ (EXIT_INFRA)	Engine	Process spawn or infrastructure failure	InfraError
$-100\,001$ (EXIT_WAIT_FAILED)	Engine	OS wait on process failed	InfraError
10–19	Artifact	Guardrail rejection (environment check)	InfraError
0	Process	Clean exit	Evasion
$-100\,003$ (EXIT_TIMEOUT)	Engine	Timeout, payload launched	Evasion
$-100\,003$ (EXIT_TIMEOUT)	Engine	Timeout, payload <i>not</i> launched	Stalled
$-100\,002$ (EXIT_NO_CODE)	Engine	Process vanished without exit code	Detected
$0xC0000906$	Windows	STATUS_VIRUS_INFECTED	Detected
$0xC0000907$	Windows	STATUS_VIRUS_DELETED	Detected
$0xFFFFFFFF$	Windows	EDR forced kill (e.g. Cortex XDR)	Detected
$0xC0000005$	Windows	STATUS_ACCESS_VIOLATION	Ambiguous
$0xC0000409$	Windows	STATUS_STACK_BUFFER_OVERRUN	Ambiguous
$0xC00000FD$	Windows	STATUS_STACK_OVERFLOW	Ambiguous
$0xC0000374$	Windows	STATUS_HEAP_CORRUPTION	Ambiguous
$0xC0000094$	Windows	STATUS_INTEGER_DIVIDE_BY_ZERO	Ambiguous
30–39	Artifact	Carrier-stage failure	Ambiguous
Other nonzero	—	Unrecognized exit code	Ambiguous

Controller Verdict Resolution

Table 28 documents the controller's dryrun-based override logic. When a dryrun result is available, `override_with_dryrun()` replaces the worker's provisional verdict using the decision tree below. This tree never produces Ambiguous, the dryrun always resolves ambiguity. When no dryrun is available, the worker's provisional verdict is used directly (Section [Worker Agent Design: Robust Execution as a Ten-Phase Pipeline](#)). Static detection is resolved before VM dispatch and short-circuits the entire protocol.

Table 28: Controller-side verdict resolution. When a dryrun is available, `override_with_dryrun()` replaces the worker's provisional verdict. *Launched* refers to whether the instrumented run reached the Launching checkpoint. Static detection is resolved before VM dispatch and short-circuits the entire protocol.

#	Dryrun	Baseline	Verdict	Rationale
0	(static scan hit)		StaticDetection	Defender flagged artifact on disk before execution
1	nonzero, !timeout	—	MutationFailed or PayloadFailed	Artifact broken on clean VM; PayloadFailed if no mutations and launched
2	timeout, !launched	—	MutationFailed	Stalled identically without AV : loader bug
3	clean	clean	Evasion	Runs and survives with and without AV
4	clean	nonzero	Detected	Works on clean VM, killed under AV
5	timeout, launched	timeout, launched	Evasion	Payload runs forever with or without AV
6	timeout, !launched	timeout	MutationFailed	Stalled identically : artifact bug
7a	timeout, launched	clean	Anomaly	Finishes with AV but stalls without : contradictory
7b	timeout, launched	nonzero, !timeout	Detected	AV killed what would have kept running
8	same nonzero exit code		InfraError	Identical failure mode : not AV-related
9	different nonzero exit codes		Detected	AV changed the failure mode

When no dryrun is available, the worker's provisional verdict is used directly. This tree never produces Ambiguous the dryrun always resolves ambiguity.

*Instrumentation: Evolution from Weak Symbols to Conditional Compilation**Initial Design: Structural Identity via Weak Linkage*

The initial instrumentation design aimed for **structural identity** between baseline and instrumented binaries. The approach was:

1. **Always link `minimal_runtime.o`** into every artifact, regardless of trace mode.

2. **Declare telemetry functions as weak symbols** in `minimal_runtime.c`:

```
__attribute__((weak)) extern void __coverage_flush(void);
__attribute__((weak)) extern void __trace_flush(void);
__attribute__((weak)) extern void __checkpoint_flush(void);
```

3. **Conditionally link `instrumentation_runtime.o`** only when `trace_mode` $\neq$ "off".

4. At runtime, `__runtime_exit()` checks each weak symbol against NULL before calling:

```
if (__coverage_flush != NULL) {
    __coverage_flush();    // Resolves to real impl OR NULL
}
```

Motivation: If both variants linked the same `minimal_runtime.o`, they would share identical PE structure, same imports, same sections, same entry point behavior. Any detection difference could then be attributed solely to the additional telemetry calls in the instrumented variant, simplifying the differential analysis.

Why This Was Abandoned

Testing revealed that linking `minimal_runtime.o` in baseline builds introduced several EDR-visible artifacts that contaminated the ground truth:

Artifact	Source in <code>minimal_runtime.c</code>	EDR Impact
Direct <code>syscall</code> instruction	<code>DirectSyscall12()</code> (inline x64 <code>syscall</code> opcode)	Triggers direct-syscall heuristics; many EDRs flag inline <code>syscall</code> instructions as suspicious
<code>NtTerminateProcess</code> string	<code>GetSyscallNumber("NtTerminateProcess")</code>	String reference to NT native API; scanned by static and runtime heuristics
"NtClose" string	<code>GetSyscallNumber("NtClose")</code>	Native API string reference
<code>ntdll.dll</code> string	<code>GetModuleHandleA("ntdll.dll")</code>	Module name reference for runtime resolution
<code>Sleep</code> import	<code>Sleep(50)</code> delay before exit	Additional IAT entry visible in static analysis
<code>OutputDebugStringA</code> import	Fallback error reporting	Additional IAT entry; sometimes flagged as anti-debug API
<code>GetModuleHandleA</code> + <code>GetProcAddress</code>	Syscall number resolution	Dynamic API resolution pattern, a common shellcode indicator

These artifacts shifted the baseline's detection profile: EDR products could detect the direct-syscall pattern or the `ntdll` resolution code even in the *baseline* run, producing false `RealDetection` classifications when the detection was actually caused by instrumentation infrastructure.

Replacement Design: Conditional Compilation + Conditional Linking

The current design eliminates all instrumentation traces from the baseline binary through two mechanisms:

```
#ifdef ENABLE_INSTRUMENTATION
#include "minimal_runtime.h"
#define ARTIFACT_CHECKPOINT(name) __artifact_checkpoint(name)
#define ARTIFACT_SUCCESS(msg) __artifact_success(msg)
#define ARTIFACT_FAILURE(msg, code) __artifact_failure(msg, code)
#else
#define ARTIFACT_CHECKPOINT(name) ((void)0)
#define ARTIFACT_SUCCESS(msg) ((void)0)
```

```
#define ARTIFACT_FAILURE(msg, code) ((void)0)
#endif
```

The builder passes `-DENABLE_INSTRUMENTATION` only when `trace_mode ≠ "off"` (builder.rs:519,592). In baseline builds, all instrumentation macros compile to no-ops and are eliminated by the optimizer.

```
// Compile minimal_runtime.o only when instrumentation is enabled.
let minimal_runtime_str = if link_runtime {
    let obj = self.ensure_minimal_runtime().await?;
    Some(obj.to_string_lossy().into_owned())
} else {
    None // Baseline: no minimal_runtime.o
};

// If linking with instrumentation, compile instrumentation_runtime too
let runtime_obj_str = if link_runtime {
    // ... compile and link instrumentation_runtime.o
} else {
    None // Baseline: no instrumentation_runtime.o
};
```

The baseline binary links only the artifact's own object file against standard Windows libraries (`kernel32`, `advapi32`, `libcmnt`). It contains no instrumentation symbols, no direct-syscall code, and no telemetry infrastructure.

Variant	Exit Path	Hooks Bypassed?
Baseline	<code>return carrier() → CRT → ExitProcess</code>	No
Instrumented	<code>__runtime_exit() → flush telemetry → syscall NtTerminateProcess</code>	Yes

Exit mechanism divergence. The baseline uses the standard CRT exit path, which passes through any EDR hooks on `ExitProcess/NtTerminateProcess`. The instrumented variant bypasses hooks via direct syscall to ensure telemetry flush completes even under aggressive EDR interception.

Handling the Divergence

The structural differences between baseline and instrumented binaries mean that detection outcomes may differ for reasons unrelated to the mutation being tested. The three-run differential protocol handles this through **outcome categorization** (Table 14):

Baseline	Instrumented	Dryrun	Category	Feed to Scorer?
Detected	Detected	Clean	RealDetection	Yes
Clean	Detected	Clean	InstrumentationArtifact	No
Detected	Clean	Clean	Flaky	No
Clean	Clean	Clean	Evasion	Yes

The `InstrumentationArtifact` category specifically filters cases where the instrumentation infrastructure, rather than the mutation, caused detection. This filtering mechanism replaces the structural-identity guarantee of the original weak-symbol design.

Retained Role of Weak Symbols

The weak symbol declarations remain in `minimal_runtime.c` (lines 32–34) as a **defensive safety measure**. If a future build configuration links `minimal_runtime.o` without `instrumentation_runtime.o`, the NULL checks in `__runtime_exit()` (lines 140–148) prevent crashes from unresolved symbol references. In the current architecture this path is never exercised, both objects are linked together or neither is linked.

Named Pipe Trace Protocol

Dual-protocol auto-detection. The worker's `TraceCollector` peeks at the first 4 bytes received on the pipe. If they match the binary protocol magic value `0x49535452` (ASCII ISTR), the binary parser is activated; otherwise, the Base64 text parser is selected. No protocol negotiation occurs, the format is determined entirely by the artifact's compile-time instrumentation mode. IR-level instrumentation emits binary ISTR records; AST-level line tracing can emit Base64-encoded markers (prefixed with `YjY0`, the Base64 encoding of "b64"). This design avoids per-event format headers and allows the collector to commit to a single, optimised parsing path for the entire session.

```
1 // worker/agent/src/telemetry/collectors/trace.rs - protocol detection
2 let mut peek_buf = [0u8; 4];
3 stream.read_exact(&mut peek_buf).await?;
4 let magic = u32::from_le_bytes(peek_buf);
5 if magic == 0x49535452 { // MAGIC_BINARY
6     self.read_binary_stream(&mut server, peek_buf).await?;
7 } else {
8     self.read_text_stream(&mut server, peek_buf).await?;
9 }
```

Binary ISTR protocol. Each binary record consists of a 30-byte packed header (`#pragma pack(push, 1)`) followed by a variable-length UTF-8 payload. The header fields are: a 4-byte magic (`0x49535452`), a 2-byte version (currently 1), a 2-byte event type (1 = line trace), a 4-byte Windows thread ID from `GetCurrentThreadId()`, an 8-byte monotonic sequence number from `InterlockedIncrement64`, an 8-byte microsecond-precision timestamp from `QueryPerformanceCounter`, and a 4-byte payload length. The payload for line-trace events is a "file:line:func" UTF-8 string. Because the struct uses `#pragma pack(1)`, it has no padding bytes, and the worker parses header fields with `std::ptr::read_unaligned` to avoid undefined behaviour on architectures that require aligned memory access. The binary format is approximately 5× more compact than an equivalent JSON representation for high-frequency line-trace events, reducing pipe bandwidth consumption during tight loops.

Mutation Catalog

This appendix catalogs all mutations in the AutoMutate++ system. Table A lists the 22 implemented mutations verified against source code; Table B lists 14 proposed mutations documented via @MUTATE markers that have no implementation in the AST mutator dispatch table.

Scope *Positional* mutations apply at // @MUTATE:<name> marker locations in template code. *Global* mutations transform the entire source, IR, or PE without requiring markers.

Fixed vs Explored Fixed mutations are always applied to every artifact (PE normalization). Explored mutations are selected per round by the mutation selector using triage feedback. Binary mutations (9) and IR mutations (3) are Fixed; AST mutations (10) are Explored.

Parameters Shown as name=default in the description. Overridden via MutationSpec CLI syntax.

Table A: Implemented Mutations (22)

*

AST Layer Positional (7 mutations)

Applied at // @MUTATE:<name> marker locations. Each is dispatched through AstMutator::apply_at_marker().

Table 29: Implemented AST positional mutations.

ID	F/E	Description	EDR Effect
ast.decon_rounds	E	Replaces loop bound literal at marker. method=fixed inserts a constant; method=runtime inserts GetTickCount()-based dynamic count. Params: count=20, method=fixed.	Shifts benign-to-suspicious API call ratio
ast.fill_pattern	E	Swaps fill loop body at marker. Options: nop, random, zero, xor. Param: pattern=xor.	Alters memory content heuristics
ast.exec_decoy	E	Inserts alternative execution call as decoy (direct_call or CreateThread). Param: method=none.	Dilutes execution N-gram signature
ast.timing_pattern	E	Inserts Sleep () with randomized duration between operations. Params: min_ms=10, max_ms=100.	Breaks temporal sequence detection
ast.protection_transition	E	Selects memory permission change sequence: rw_rx, rw_rwx, or rw_r_rx (intermediate R step). Param: pattern=rw_rx.	Shifts VirtualProtect argument tokens
ast.benign_preamble	E	Inserts 1–3 benign SystemQuery-group calls before critical code using seeded selection. Params: count=2, seed=0xBE41.	Dilutes API N-grams before suspicious sequences
ast.api_sequence_obfuscation	E	Interleaves 1–2 benign calls between suspicious operations from all behavior groups. Params: count=2, seed=0xBE41.	Disrupts seq2: tokens directly

*

AST Layer Global (3 mutations)

Transform the entire source file without requiring @MUTATE markers. Applied after marker-based mutations.

Table 30: Implemented AST global mutations.

ID	F/E	Description	EDR Effect
ast.string_xor	E	Tree-sitter walks all <code>string_literal</code> nodes. Each string is replaced with a XOR-encoded static <code>unsigned char[]</code> and a lazy runtime decode via GCC statement expression. Runs last to catch strings added by other mutations. Param: <code>xor_key=0xAA</code> .	Removes cleartext strings from the binary
ast.const_obfuscation	E	Tree-sitter walks all <code>number_literal</code> nodes above <code>min_value</code> . Each constant N is decomposed into a volatile sum pair: $X + Y = N$ with PRNG-derived X . Params: <code>min_value=2</code> , <code>seed=0xDEAD</code> .	Prevents pattern matching on suspicious constants
ast.benign_syscall_insert	E	Uses BehaviorGraph for dependency-ordered insertion across 3 behavior groups. Distributes calls at density-controlled intervals throughout the target function body. Params: <code>count=8</code> , <code>density=0.5</code> , <code>seed=0xBE41</code> .	Floods function with benign <code>api:</code> and <code>seq2:</code> tokens

*

IR Layer Global (3 mutations)

Text-based LLVM IR transforms using a deterministic LCG for density-based insertion. All operate on .ll text without LLVM C-API dependency.

Table 31: Implemented IR global mutations.

ID	F/E	Description	EDR Effect
llvm.nop_insert	F	Inserts <code>call void asm sideeffect "nop" after basic block labels at configurable density. Opaque to LLVM, survives -O2. Param: <code>density=0.3</code>.</code>	Shifts code offsets within functions
llvm.opaque_predicate	F	Converts unconditional <code>br label %target</code> to always-true conditional branches. <i>Robust mode</i> : uses inline asm opaque to LLVM. <i>Trivial mode</i> : uses <code>icmp eq i32 0, 0</code> (folds at -O2). Params: <code>density=0.3</code> , <code>mode=robust</code> .	Changes CFG structure, defeats linear control flow analysis
llvm.junk_block	F	Inserts unreachable blocks with arithmetic and unreachable terminator before function closing braces. Param: <code>count=2</code> .	Increases code section size, adds disassembly noise

*

Binary Layer Global (9 mutations)

Post-link PE transforms operating on compiled PE bytes using direct byte manipulation. `string_inject`, `entropy_normalize`, `size_pad`, and `debug_dir` are consolidated into a single `.rdata` section to avoid adding multiple non-standard sections.

Table 32: Implemented binary global mutations.

ID	F/E	Description	EDR Effect
binary.rich_header	F	Injects a Rich header from legitimate Windows binaries (notepad, calc, explorer) with correct XOR-based checksum. Param: donor=notepad.	Bypasses “unusual compiler” heuristics
binary.import_pad	F	Adds benign imports from a pool of 14 DLLs (~45 functions). Creates new IAT entries with proper ILT/HNT structures. Param: count=50.	Normalizes suspicious import ratio
binary.resource_inject	F	Creates a .rsrc section containing VS_VERSIONINFO and an application manifest. Params: product_name, company, original_filename.	Resource presence is an ML legitimacy signal
binary.section_rename	F	Renames sections based on characteristics flags to MSVC defaults (.text, .data, .rdata).	Removes non-standard section names
binary.timestamp	F	Sets COFF TimeDateStamp with FNV1a-based jitter derived from binary content. Param: age_days=365.	Avoids “just compiled” heuristic
binary.debug_dir	F	Creates IMAGE_DEBUG_DIRECTORY with CodeView RSDS record and plausible PDB path. Param: pdb_path=auto.	Suggests legitimate development process
binary.string_inject	F	Embeds Windows-style error/status messages in .rdata. Param: count=20.	Shifts string pool ML features toward legitimate profile
binary.entropy_normalize	F	Adds low-entropy padding (mixed strings, zero blocks, integer sequences) to reach target bits-per-byte. Param: target=6.0.	Reduces entropy from packed range to legitimate range
binary.size_pad	F	Pads binary to target size using low-entropy data. Param: target_kb=256.	Normalizes file size to typical application range

Table 33: AST mutation parameter space.

Mutation ID	Param	Type	Range
ast.decon_rounds	count	IntRange	[5, 500]
	method	Categorical	{fixed, runtime}
ast.fill_pattern	pattern	Categorical	{xor, nop_sled, random, zero}
ast.exec_decoy	method	Categorical	{none, direct, thread}
ast.timing_pattern	min_ms	IntRange	[0, 500]
	max_ms	IntRange	[10, 2000]
ast.protection_transition	pattern	Categorical	{rw_rx, rw_rwx, rw_r_rx}
ast.string_xor	xor_key	IntRange	[1, 255]
ast.const_obfuscation	min_value	IntRange	[2, 2]
ast.benign_syscall_insert	groups	Categorical	{system_query, ...}
	count	IntRange	[1, 9]
	density	FloatRange	[0.1, 1.0]
	target_fn	Categorical	{carrier, decon, ...}
ast.benign_preamble	count	IntRange	[1, 3]
ast.api_sequence_obfuscation	count	IntRange	[1, 3]

Table B: Proposed / Not-Yet-Implemented Mutations (14)

These mutations are documented via @MUTATE markers in the architecture document. **None have handlers in the dispatch table**, unrecognized marker names log at debug level and return the source unchanged.

Three proposed markers are **excluded** because they are functionally subsumed by existing implementations:

Table 34: Excluded proposed markers (subsumed by existing implementations).

Excluded Marker	Subsumed By	Reason
literal_encoding	ast.const_obfuscation	Same goal and EDR effect. The global implementation already covers every constant the positional marker would target.
opaque_predicate (AST)	llvm.opaque_predicate	AST-level always-true conditions are eliminated by -O2; the IR version uses inline asm that survives optimization.
dead_code_insertion	llvm.junk_block	AST-level if(0) blocks are eliminated by constant folding; IR-level unreachable blocks are preserved.

Additionally, syntactic_substitution has been merged into loop_restructuring as its loop transforms duplicate that entry and its non-loop transforms are a natural extension.

*

Import / API Resolution (3)

Table 35: Proposed import/API resolution mutations.

ID	Layer	Description	EDR Effect
api_wrapper_injection	AST	At each marker, generate a static wrapper function and replace the direct call with a call to the wrapper.	Changes call graph and stack trace origin
getprocaddress_indirection	AST	Replace static API call with <u>GetProcAddress</u> ((<u>GetModuleHanc</u> runtime resolution.	Removes function from IAT; adds GetProcAddress tokens
api_swap	AST	Substitute API call with semantically equivalent alternative (e.g., <u>VirtualAlloc</u> → <u>HeapAlloc</u>).	Changes which api: tokens appear

*

Code Structure (2)

Table 36: Proposed code structure mutations.

ID	Layer	Description	EDR Effect
string_splitting	AST	Convert string literals to character arrays (e.g., "kernel32.dll" → {'k','e',...,0}). Complements ast.string_xor.	Defeats static string matching
loop_restructuring	AST	Transform control structures: for↔while conversion, partial unrolling, switch-dispatch conversion, if→switch. Subsumes syntactic_substitution.	Changes CFG structure and instruction patterns

*

Behavioral / Execution (4)

*

Cryptographic (3)

*

Advanced (2)

Table 37: Proposed behavioral/execution mutations.

ID	Layer	Description	EDR Effect
execution_method	Behavioral	Replace direct function pointer call with alternative: callback abuse (EnumWindows), fiber conversion, or thread pool (QueueUserWorkItem).	Completely changes execution ETW event patterns
staged_rx	Behavioral	Replace direct RW→RX with RW→R→(exception-triggered)→RX using SEH/VEH handler. Overlaps: ast.protection_transition.	Splits protection transition; RX set in exception handler context
loop_mutation	AST	Replace fixed loop counts with GetTickCount() % N or similar runtime-derived values. Overlaps: ast.decon_rounds:method=runtime.	Defeats emulators that fast-forward known loop counts
logic_mutation	AST	Replace memory write patterns with CPU-based math equivalents using register operations and a final single write.	Reduces write operations visible to kernel callbacks

Notes

Fixed vs Explored Classification. The 22 implemented mutations split into two operational categories: The mutation selector (CoverageSelector, FuzzerSelector, TokenSelector, or RandomSelector) chooses which AST mutations to apply each round. Fixed mutations provide a consistent baseline so that detection differences between rounds can be attributed to the explored AST mutations.

Overlap Analysis. Two of the 14 proposed mutations partially overlap with existing implementations: Additionally, string_splitting complements (but does not duplicate) ast.string_xor char arrays vs XOR-encoded byte arrays serve different tradeoffs. Three further proposed markers (literal_encoding, opaque_predicate AST, dead_code_insertion) were excluded entirely as subsumed by existing implementations (Table 34).

Highest-Impact Proposed Mutations. Based on expected telemetry impact:

1. execution_method Changes the entire execution primitive, producing fundamentally different ETW event patterns.
2. control_flow_flattening Destroys structural CFG patterns that detection rules may match.
3. key_upgrade (AES variant) Adds entirely new API tokens (BCrypt*) and DLL loads.
4. api_swap Directly changes which api: tokens appear, though functional equivalence constraints limit applicability.

Table 38: Proposed cryptographic mutations.

ID	Layer	Description	EDR Effect
key_upgrade	AST	Replace simple XOR decoder with RC4, AES (via BCrypt/CNG), rolling XOR, or multi-stage decode.	Different crypto primitives produce different instruction patterns and API calls
key_derivation	AST	Derive decryption key from environment (hostname hash, timestamp) instead of hardcoding.	Adds environment-querying APIs; makes static decryption impossible
control_flow_flattening	IR	Convert structured control flow into a state-machine switch-dispatch loop. Each basic block becomes a switch case.	Destroys structural CFG patterns

Table 39: Proposed advanced mutations.

ID	Layer	Description	EDR Effect
inline_assembly	AST	Replace C code with hand-crafted inline assembly at marker locations (primarily PEB-walk carrier).	Changes instruction patterns; defeats source-level pattern matching
exception_based_flow	AST	Use SEH/VEH for control transfer instead of normal branches. Trigger exceptions to invoke handlers that perform the intended operations.	Creates unusual execution patterns; moves operations to handler context

Table 40: Fixed vs Explored mutation classification.

Category	Count	Mutations	Selector-Varied?	Purpose
Fixed	12	9 binary + 3 IR	No	PE normalization and code layout noise
Explored	10	10 AST	Yes	Behavioral changes that shift detection signals

Table 41: Overlap between proposed and implemented mutations.

Proposed	Overlaps With	Key Difference
staged_rx	ast.protection_transition	Both add intermediate R step; staged_rx uses exception-based transition rather than a direct second VirtualProtect call.
loop_mutation	ast.decon_rounds	General (any loop) vs specific (deconditioner only).

*List of techniques***Table 42:** EDR bypass and evasion techniques with their conceptual mechanisms, defeated mechanisms, and telemetry signatures or drawbacks.

Technique	Conceptual Mechanism	Defeated Mechanism	Telemetry/Drawback
Direct Syscalls / Indirect Syscalls	Code calls Windows kernel functions directly, bypassing the standard Windows execution flow that typically goes through userland API functions exported by ntdll.dll. Indirect syscalls reuse parts of ntdll.dll functions after the hook invocation point.	Userland API Hooking: These hooks are set by EDRs to intercept and monitor API calls in user space.	This technique can be reliably detected by ETW and call stack analysis. If direct syscalls are used, the call stack looks "wrong" by default.
Unhooking	The attacker loads a fresh, unhooked version of ntdll.dll (or other target DLLs) into the process memory after the EDR agent has already loaded and hooked the initial version. May attempt to "re-hook" the EDR at the end of the operation to conceal their tracks.	Userland Hooks.	The EDR is rendered blind to monitored code. EDRs that rely solely on user-mode hooks are considered obsolete. Restoring the patched DLL itself can generate telemetry.
AMSI (Patching)	Bypass Methods like memory patching, reflection, or COM server hijacking are used to subvert the Anti-Malware Scan Interface (AMSI) framework. This is commonly achieved by changing memory permissions with VirtualProtect and overwriting the function prologue with a return instruction.	AMSI Visibility/Scanning of scripts, .NET, PowerShell, and WMI components.	Successfully bypassing AMSI is often trivial. Patching may require changing the memory permissions of the target region, which can generate telemetry.
ETW (Patching)	Bypass The attacker manually patches specific ETW functions, such as EtwEventWrite or EtwTiLogReadWriteVm, by inserting a return instruction (RET/0xC3) at the beginning of the function.	Event Tracing for Windows (ETW) logging/telemetry, particularly kernel instrumentation used by MDE (EtwTi).	Patching an EtwTi function is generally not KPP-safe but may avoid a Blue Screen of Death (BSOD) if the attacker quickly restores the original state. Requires admin/high-IL context to blind the EtwTi sensor telemetry.
Tampering with Kernel Callbacks	Requires achieving high integrity level code execution to attack EDRs at the kernel level. Techniques include zeroing out the address of the callback routine from the kernel callback array, unregistering the routine, or patching the routine (e.g., with a RET instruction).	Kernel-level telemetry providers (e.g., PsSetCreateProcessNotifyRoutine, PsSetLoadImageNotifyRoutine, ObRegisterCallbacks).	This requires admin or kernel access. Vulnerable signed drivers (BYOVD) are often used to gain kernel access and perform these actions.
DLL Sideloading	An attacker uses a legitimate, signed application (e.g., MS Teams) and places a malicious DLL clone in the application's folder, ensuring the encrypted shellcode is triggered when the benign process is executed.	Detection of malicious execution, since the code is run under the context of a signed and benign binary process.	The malicious DLL is often unsigned, which can sometimes trigger defense mechanisms. This technique is highly successful.
Reflective Loading	DLL A code injection technique where a DLL is loaded directly from memory into an existing process, instead of being written to and loaded from the disk.	Disk monitoring and detection systems that rely on monitoring DLLs as they are loaded from disk.	EDRs address this by monitoring the allocation of local memory areas and preventing Portable Executable (PE) loading attempts from those regions.

Continued on next page. . .

Continued — List of EDR Bypass and Evasion Techniques

Technique	Conceptual Mechanism	Defeated Mechanism	Telemetry/Drawback
Process Hollowing / Injection	Creating a process in suspended state, unmapping its original image via NtUnmapViewOfSection , writing the malicious PE with relocated base addresses, updating the thread context to the new entry point, and resuming execution.	Process-image attribution: the malicious code executes under the identity and token of a legitimate, signed executable.	Modifying another process's memory and unmapping its image section are inherently suspicious. NtUnmapViewOfSection , cross-process writes, and suspended-process-then-resume patterns are strong detection signals for modern EDRs.
Obfuscating Constants and Code	Making constants inside the source code a result of a simple binary operation (like addition or XOR) or inserting simple NOP instructions into short assembly snippets. Obfuscating PE headers in memory by erasing optional headers after reflective loading.	Static signatures and signatures based on known constants or short assembly snippets inside the malicious code. Memory scanners looking for PE magic bytes/metadata.	This is generally effective and fast for evading static AVs. However, EDRs, particularly those using advanced heuristics/ML, are more sensitive and require more complex encoding techniques, such as a sub-byte map.
EDR Deconditioning (Siriallocalot)	Repeatedly performing benign operations, such as allocating memory, copying fake data, and changing memory permissions to RX, to overwhelm the EDR's performance budget before executing the real shellcode.	Performance-intensive memory scanning, which is only performed when triggered by suspicious activity (like memory protection changes).	The technique aims to make the EDR "tired of scanning" the memory.
Command Spoofing	Line Modifying command line arguments stored in the user-modifiable Process Environment Block (PEB). Attackers can create a suspended process with fake arguments, overwrite the PEB with real arguments, and then overwrite the PEB back to fake arguments for cleanup before the EDR queries the process.	Tricking the EDR into recording innocent command line arguments, hiding indicators like "privilege::debug".	Overwrite/restore PEB sequence; requires careful timing to avoid leaving inconsistent process metadata.
PPID Spoofing	Instructing the CreateProcessW() function to provide a fake parent PID (PPID) using attributes in the STARTUPINFOEX structure. This sets the fake PPID in the EPROCESS kernel structure.	Creating a suspicious process (e.g., a beacon) that appears to be spawned by a trusted process (e.g., svchost.exe).	Alters kernel parent-child attribution; can be detected by correlating kernel-level process create callbacks with on-disk parent process activity.
Module Stomping	Injecting a benign DLL into a remote process, then overwriting its AddressOfEntryPoint with shellcode and creating a remote thread at that address.	Unbacked-memory heuristics: EDR rules that flag code execution from private, non-file-backed memory regions.	Executes from backed, on-disk module ranges to reduce anomaly flags; modifying module code regions is nonetheless suspicious if integrity checks or image load events are monitored.
Callstack Spoofing	Manipulating the function call hierarchy (callstack) to make it appear genuine. This is a supporting technique used to cover tracks when primary techniques (like direct/indirect syscalls) create a "wrong" callstack.	Evading call stack analysis rules used by EDRs to identify suspicious origins (e.g., non-backed memory).	Aims to make stack-shape heuristics fail; defenders can counter by correlating stacks with known image ranges and kernel syscall telemetry.
Memory Encryption	Encrypting suspicious memory regions before the process enters a sleep state and decrypting them upon wake.	Bypassing Memory Scanning performed by the EDR, which is often triggered by suspicious activity or performed at regular intervals.	Shifts detection surface to pre/post-decryption phases; requires reliable key/material management and may be detectable by monitoring memory-protection/change events.

Continued on next page. . .

Continued — List of EDR Bypass and Evasion Techniques

Technique	Conceptual Mechanism	Defeated Mechanism	Telemetry/Drawback
Variants of Image Spoofing / Process Injection	Several advanced variants treated as distinct evasion methods: • Process Doppelgänger — overwrite a file using Transactional NTFS (TxF), start the process, then roll back the transaction; • Process Herpaderping — write malicious code to an executable, create a process, then quickly replace the malicious content with a non-malicious one before scanning; • Process Ghosting — create an empty file, semi-delete it, write malicious data, and create a process from it.	Evading standard on-disk scanning and process-image attribution by abusing file-transactional and file-state edge-cases.	Variants exploit filesystem/loader edge cases; defenders must correlate file-system transactions, image creation events, and process start timeline to detect.
Applnit_DLLs / AppCertDLLs	Registry / loader-based DLL injection: adversary registers DLL names under Applnit_DLLs or AppCertDLLs (or other legacy loader hooks) so the OS loader injects the DLL into many processes at load time.	Per-process injection monitors: the OS loader itself performs the injection at load time, bypassing CreateRemoteThread / WriteProcessMemory-style API hooks.	Registry writes and image-load events are noisy: ETW/Sysmon image-loads, registry modification events, and unexpected DLLs in common processes are strong signals. Modern Windows editions and secure policies limit Applnit usage (requires admin/compat settings), so success rates vary.
Image File Execution Options (IFEO) Debugger Hijack	Set a process' IFEO "Debugger" key to point to a malicious binary; when a target executable is launched, the OS starts the attacker binary instead of (or before) the target. Also used to force arbitrary code to run in privileged contexts.	On-launch execution attribution and pre-load tampering; useful to gain early execution without CreateProcess-based hooking.	IFEO registry changes and the subsequent child process lineage are visible to ETW/Windows Event Log. Defender/EDR rules often watch IFEO modifications; requires admin or write access to HKLM.
Living-off-the-Land Binaries (LOLBins)	Abuse of signed, benign system utilities (e.g., msbuild.exe, regsvr32.exe, rundll32.exe, certutil.exe, werfault.exe) to execute payloads or load malicious code (fileless patterns) so behavior executes under trusted image names.	Evasion of signature/filename-based detections and reliance on benign process context for execution.	Telemetry: process image names, unusual command line arguments, parent/child lineage anomalies and file writes performed by typically non-scripting binaries. Detection is possible by correlating behaviors (e.g., certutil writing exe data) rather than filename alone.
WMI Event Subscription / Permanent Event Consumers (Fileless)	Register WMI permanent event consumers or scheduled WMI providers that trigger payload execution (CommandLineEventConsumer, ActiveScriptEventConsumer), execution occurs without dropping a persistent EXE on disk.	Evasion of on-disk scanning and many simple AV heuristics that only scan files; allows truly fileless persistence/execution.	Telemetry: WMI repository changes are logged, and ETW/sysmon may surface the resulting process launches. Detection requires monitoring WMI repository writes, COM registrations, and long-lived event consumer entries; many EDRs add heuristics for suspicious WMI consumers.

Continued on next page. . .

Continued — List of EDR Bypass and Evasion Techniques

Technique	Conceptual Mechanism	Defeated Mechanism	Telemetry/Drawback
Hotpatching / Legitimate In-Memory Patch Mechanisms	Use documented/undocumented OS hotpatch or patching APIs (or legitimate update paths) to modify in-memory code pages of a running process (e.g., using sanctioned update loaders or atomic rename + replace flows) to execute attacker code while avoiding typical RWX allocations.	Evasion of RWX/memory-protection heuristics and userland memory-allocation monitors that flag VirtualAlloc/Write/Protect sequences.	Telemetry: page protection changes, image-load/unload events, and unusual use of service/update APIs are signals. Hotpatching can be subtle but often leaves traces in image relocation events and integrity checks. Requires careful timing and may need elevated privileges.
Bring-Your-Own-Vulnerable-Driver (BYOVD) / Signed Driver Abuse	Load a signed but vulnerable kernel driver to obtain kernel privileges or to disable kernel callbacks (e.g., patch kernel data structures), attacker leverages legitimate signing to bypass driver signing checks.	Defeats kernel-level protections and allows disabling or tampering with EDR kernel callbacks (PsSetCreateProcessNotifyRoutine, Image load hooks) from ring-0.	Telemetry/Drawback: driver load events, new kernel module signatures, and changes to kernel callback lists can be captured by kernel ETW and a kernel-debug adapter. Using real vulnerable drivers creates a strong forensic footprint (driver name, load path) and often requires admin rights to install.
SetWindowHookEx Code Injection	Install a Windows hook (e.g., WH_KEYBOARD, WH_CALLWNDPROC) either as a global hook DLL or thread-specific hook so that the target process executes attacker-controlled code in the context of its thread when a matching Windows message is delivered. Global hooks load an attacker DLL into many GUI processes automatically.	Userland injection that abuses the window/message dispatch mechanism to execute code inside another process (bypasses CreateRemoteThread-style primitives).	Telemetry / Drawback: Hook registrations and unexpected DLL loads across many GUI processes are noisy, ETW image-loads, Windows message patterns, and hook-related API calls (SetWindowsHookEx/UnhookWindowsHookEx) are observable. Requires an interactive desktop session or active windowing context, and many modern endpoints restrict global hooks or require admin/desktop session, reducing reliability.
Finding Kernel32 Base and Function Addresses in Shellcode	Position-independent shellcode resolves API entry points at runtime by walking OS structures (PEB/TEB), parsing export tables, or scanning memory to locate the base of kernel32/ntdll and compute function addresses (GetProcAddress-like resolution in pure shellcode).	Defeats static import-time detection and signature rules that rely on Imports/Export tables being present in a PE; allows payloads to be fully position-independent and avoid static IAT-based indicators.	Telemetry / Drawback: The characteristic memory reads/walks of PEB/TEB and parsing of in-memory PE headers produce distinctive telemetry (suspicious memory reads, repeated scanning across module ranges). Heuristic engines flag these behaviors; using such logic in many samples has led to signature rules. Also slightly increases shellcode size/complexity and can fail under certain process hardening measures.

Continued on next page. . .

Continued — List of EDR Bypass and Evasion Techniques			
Technique	Conceptual Mechanism	Defeated Mechanism	Telemetry/Drawback
DLL Injection via a Custom .NET Garbage Collector	Abuse the COMPLUS_GCName environment variable to specify a native DLL exporting GC_VersionInfo; when any .NET application starts, the CLR loads this DLL as a custom garbage collector, executing attacker code. The path is unsanitized, allowing directory traversal.	Bypasses classic injection monitors (no CreateRemoteThread or WriteProcessMemory) by leveraging the CLR's internal loader to pull attacker code into the process address space.	DLL- (no or by Telemetry / Drawback: Changes to environment variables, unexpected CLR/managed assembly loads, or unusual AppDomain/CLR hosting calls are observable. Detection signals include redeployments of .NET modules, msbuild/regsvr32-like chains that spawn the CLR, and dyndebug / JIT-related ETW events. This technique is constrained to hosts running the .NET runtime and may require specific CLR versions or startup parameters.
Injecting .NET Assembly to an Unmanaged Process	Host the CLR inside an unmanaged/native process using CLR hosting APIs (e.g., CLRCreateInstance → ICLRRuntimeHost, or legacy CorBindToRuntimeEx) or use remote-thread techniques to call into the runtime loader so that a managed assembly runs inside a native process context. Also achievable by staged reflective loading of the CLR.	Avoids on-disk native payloads and leverages the managed runtime to execute code that would otherwise be flagged if present as a native PE; defeats detectors that focus only on native executable signatures.	Telemetry / Drawback: Loading the CLR into processes that historically never host it produces clear telemetry: image loads for mscorEE.dll/cclr.dll, JIT compilation events, CLR initialization traces, and unexpected managed assemblies in process memory. EDRs can detect anomalous CLR host events and flag processes that load the runtime unexpectedly. Requires the target process to be compatible with CLR hosting and may fail on heavily sandboxed / restricted hosts.

Component-Level Evaluation Diagnostics

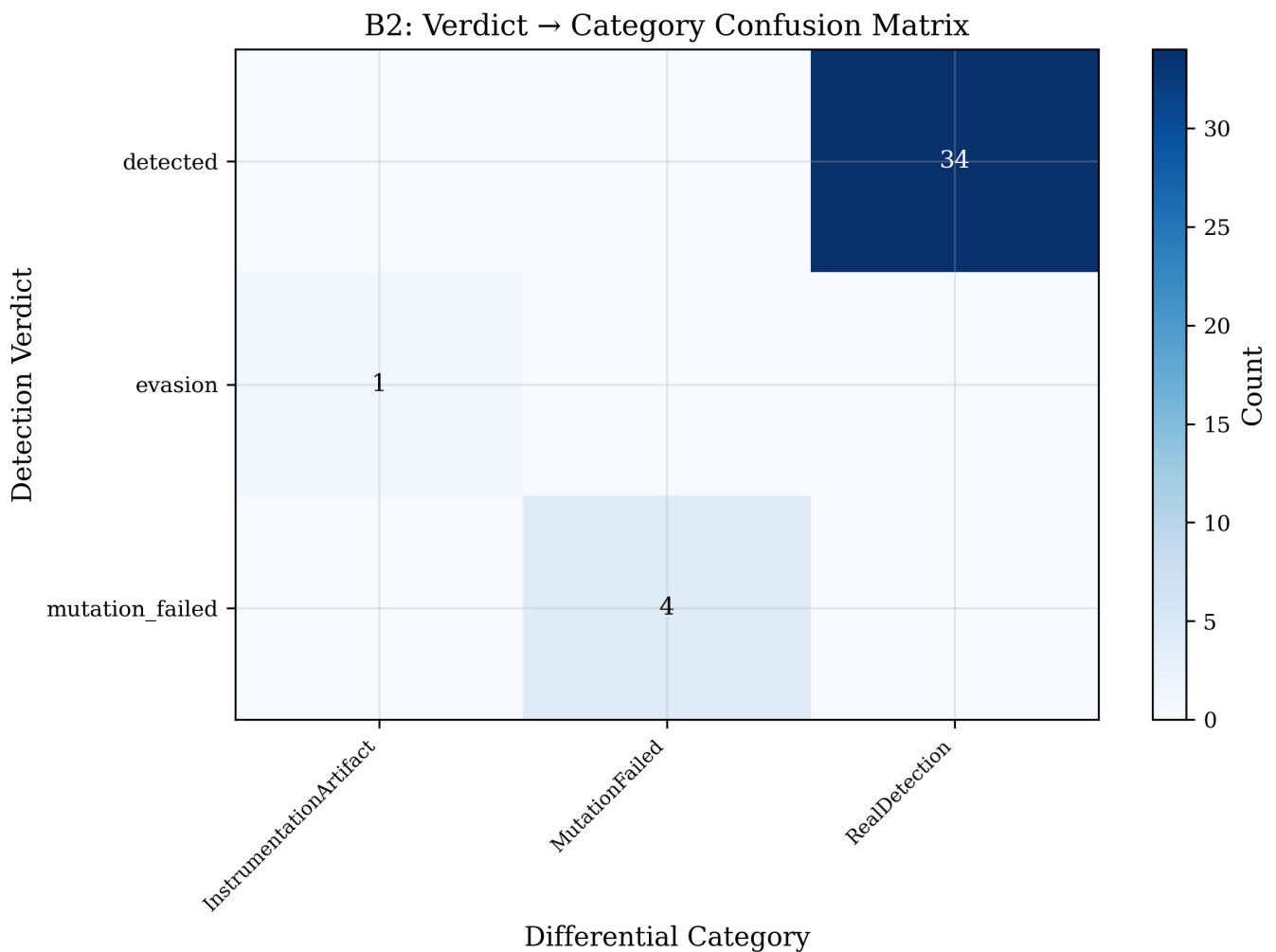
This appendix presents per-selector diagnostic figures from the evaluation pipeline. Each subsection corresponds to one of the four selector campaigns and contains the full set of classifier analysis, telemetry completeness, token coverage, scoring convergence, and counterfactual validation plots. These figures complement the condensed observations in the main evaluation section (§Observation) and support the deductions drawn in §Deduction.

Coverage Selector (n=39)

The Coverage selector achieves the highest verdict-differential agreement (87%) and trustworthy ratio (87.2%), producing the cleanest experimental data of all four campaigns. Its conservative mutation strategy results in only 4 mutation failures across 39 rounds.

Table 43: Coverage Selector Evaluation Metrics

Metric ID	Label	Value
b2.classifier_analysis.confusion_matrix	Confusion matrix (verdict → differential category)	3.0000
b2.classifier_analysis.sankey_flows	Verdict → category Sankey diagram flows	3.0000
b2.classifier_analysis.score_by_category	Evasion score distribution by differential category	3.0000
b2.classifier_analysis.verdict_agreement	Verdict differential category agreement rate	0.8718
b3.telemetry_completeness.coverage_by_verdict	Coverage distribution by differential category	78.4333
b3.telemetry_completeness.coverage_detection_corr	Coverage detection Pearson correlation	0.8836
b3.telemetry_completeness.coverage_histogram	Coverage percent histogram	39.0000
b3.telemetry_completeness.token_counts	Token counts by differential category	39.0000
b3.telemetry_completeness.trustworthy_trajectory	Trustworthy round fraction (1 - contamination rate)	0.8718
c3.token_coverage.category_proportions	Per-round token category proportions (for stacked bar)	8.0000
c3.token_coverage.category_table	Token category coverage (active categories / 9 total)	0.8889
c3.token_coverage.presence_heatmap	Token presence heatmap (bottom-20 rarest tokens × rounds)	399.0000
c3.token_coverage.tokens_per_round	Mean tokens per round	149.2051
c4.scoring_convergence.actionable_stability	Round where actionable token count stabilizes	5.0000
c4.scoring_convergence.lift_variance	Lift variance convergence (final variance)	0.0000
c4.scoring_convergence.top5_overlap	Top-5 token overlap convergence (final overlap fraction)	0.0000
c5.counterfactual.effect_sizes	Largest absolute detection rate delta	0.0000
c5.counterfactual.forest_plot	Counterfactual validation: significant tokens (Bonferroni α	0.0000

**Figure 19:** Coverage: verdict vs. differential category confusion matrix. High diagonal concentration confirms that per-run verdicts align well with the three-run differential outcome.

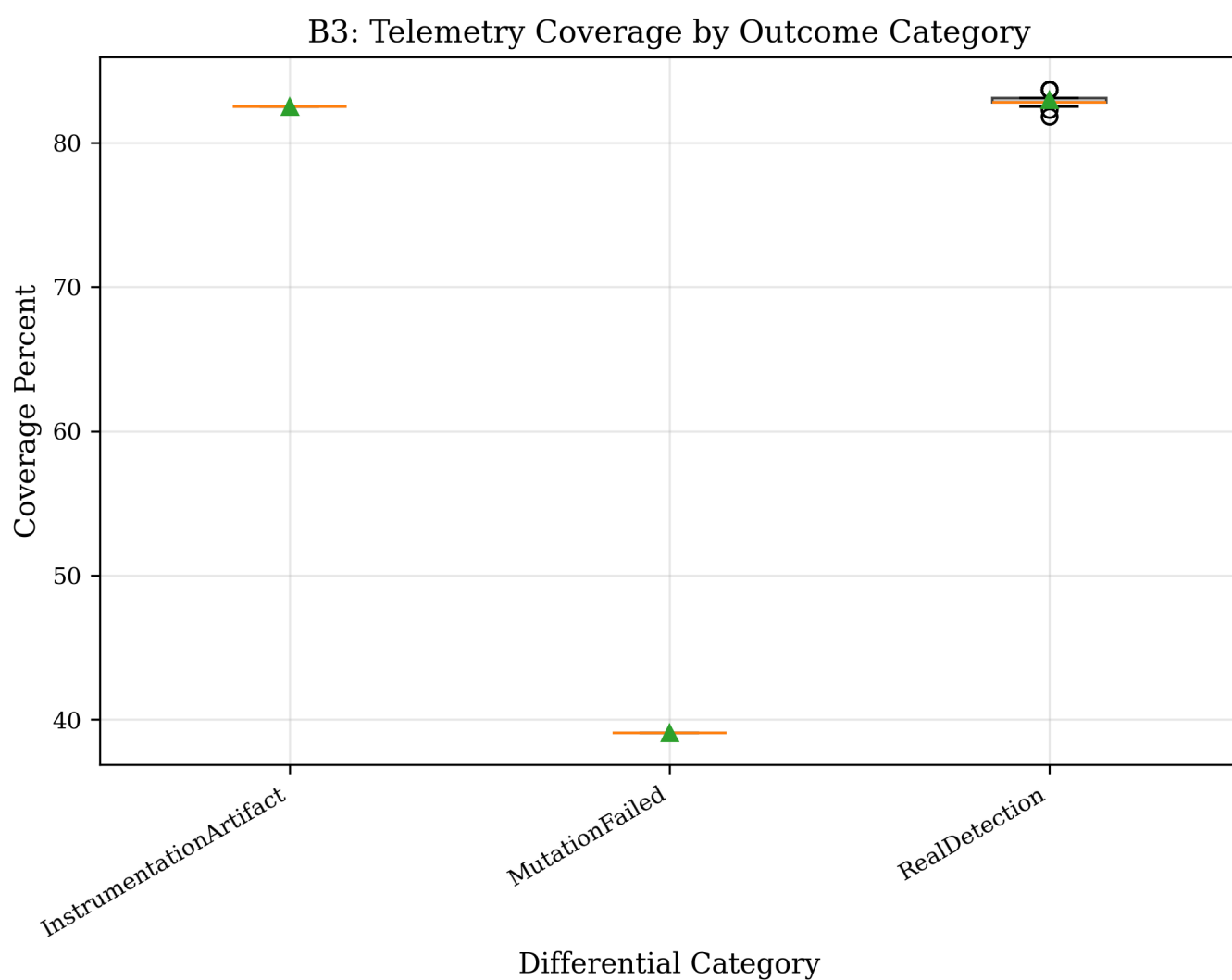


Figure 20: Coverage: code coverage distribution by differential outcome. Median coverage is high across all outcome categories, reflecting the selector's preference for valid, high-coverage mutation configurations.

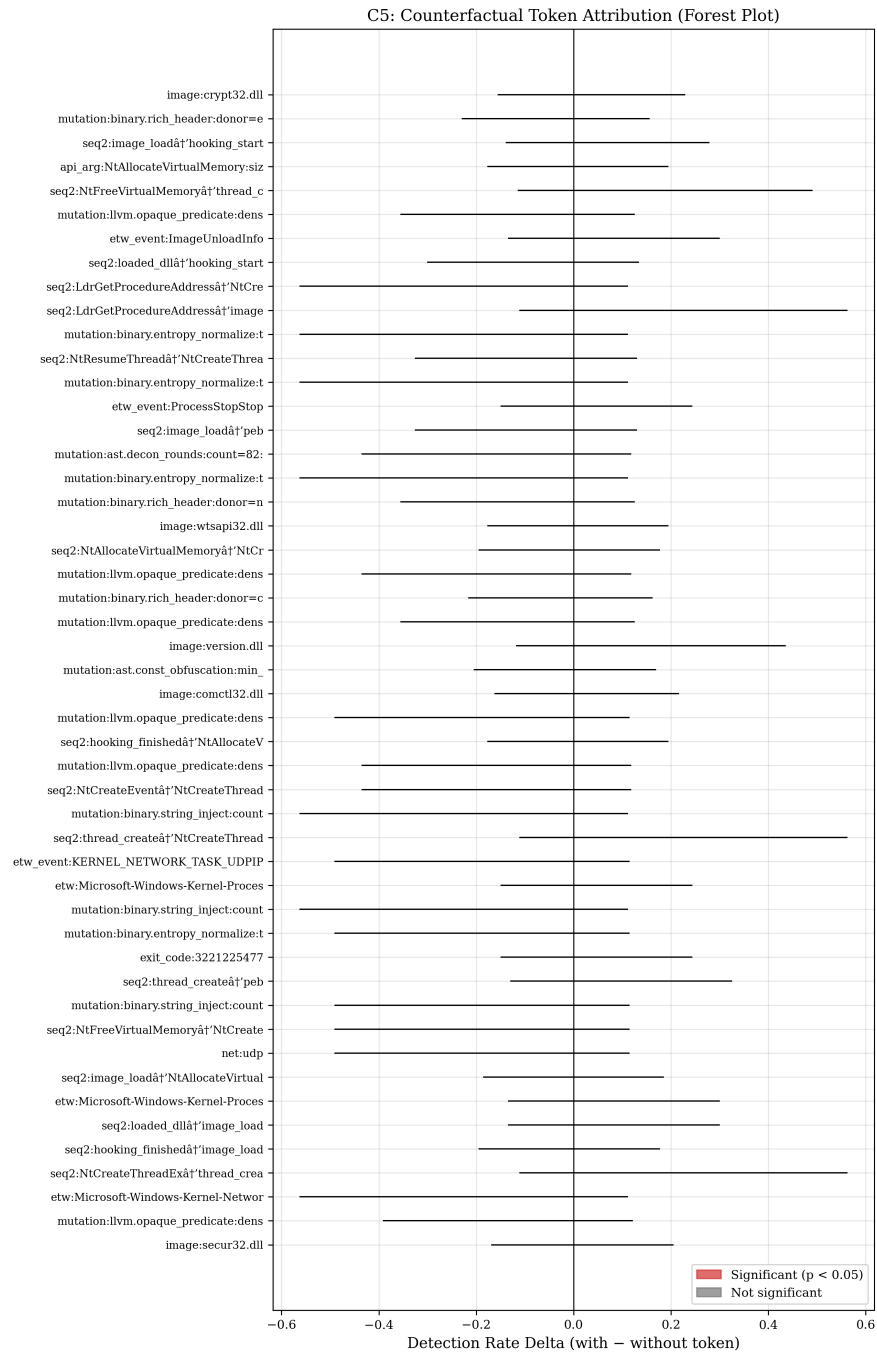


Figure 23: Coverage: forest plot of token effect sizes with Bonferroni correction. No individual token reaches significance, consistent with the absence of evasion in this campaign.

*Genetic Selector (n=51)***Table 44:** Genetic Selector Evaluation Metrics

Metric ID	Label	Value	<i>n</i>
b2.classifier_analysis.confusion_matrix	Confusion matrix (verdict → differential category)	3.0000	51
b2.classifier_analysis.sankey_flows	Verdict → category Sankey diagram flows	4.0000	51
b2.classifier_analysis.score_by_category	Evasion score distribution by differential category	4.0000	51
b2.classifier_analysis.verdict_agreement	Verdict differential category agreement rate	0.5490	51
b3.telemetry_completeness.coverage_by_verdict	Coverage distribution by differential category	68.6320	51
b3.telemetry_completeness.coverage_detection_corr	Coverage detection Pearson correlation	0.5887	51
b3.telemetry_completeness.coverage_histogram	Coverage percent histogram	50.0000	51
b3.telemetry_completeness.token_counts	Token counts by differential category	51.0000	51
b3.telemetry_completeness.trustworthy_trajectory	Trustworthy round fraction (1 - contamination rate)	0.5490	51
c1.token_sensitivity.default_operating_point	Default operating point (lift=1.5, confidence=0.3)	0.0630	51
c1.token_sensitivity.heatmap	Token sensitivity heatmap (5×5 lift × confidence grid)	0.2741	51
c1.token_sensitivity.lift_distribution	Token lift distribution statistics	0.3232	51
c3.token_coverage.category_proportions	Per-round token category proportions (for stacked bar)	8.0000	51
c3.token_coverage.category_table	Token category coverage (active categories / 9 total)	0.8889	51
c3.token_coverage.presence_heatmap	Token presence heatmap (bottom-20 rarest tokens × rounds)	306.0000	51
c3.token_coverage.tokens_per_round	Mean tokens per round	129.5686	51
c4.scoring_convergence.actionable_stability	Round where actionable token count stabilizes	0.0000	51
c4.scoring_convergence.lift_variance	Lift variance convergence (final variance)	0.1221	51
c4.scoring_convergence.top5_overlap	Top-5 token overlap convergence (final overlap fraction)	0.0000	51
c5.counterfactual.effect_sizes	Largest absolute detection rate delta	1.0000	51
c5.counterfactual.forest_plot	Counterfactual validation: significant tokens (Bonferroni	0.3020	51

The Genetic selector is the only strategy to achieve evasion (6/51 rounds, 11.8%). Its crossover and mutation operators produce more aggressive artifact variants, resulting in a higher failure rate (33.3%) but also the richest evasion data.

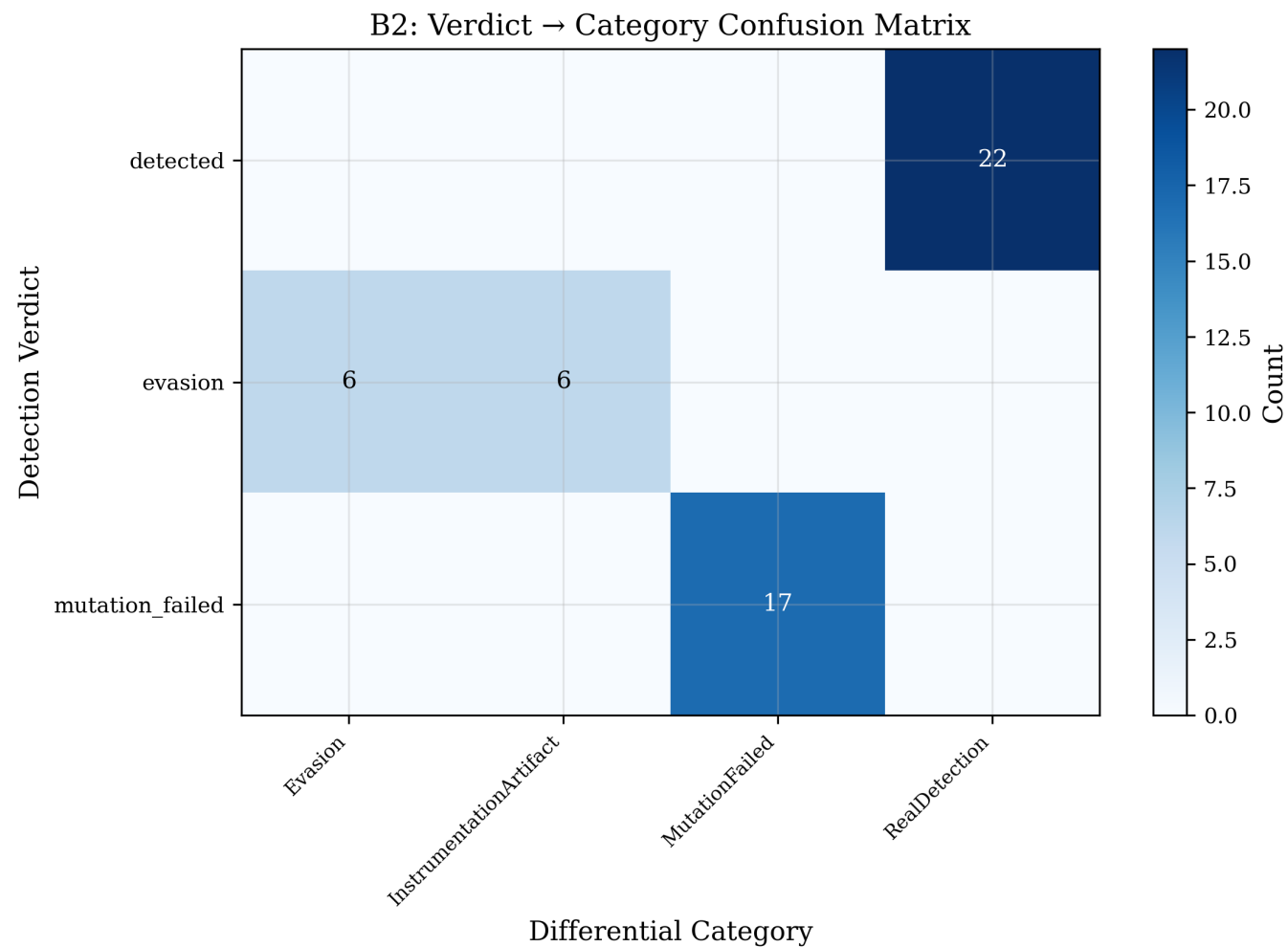


Figure 24: Genetic: verdict vs. differential category confusion matrix. The presence of evasion rounds introduces off-diagonal entries not seen in other selectors.

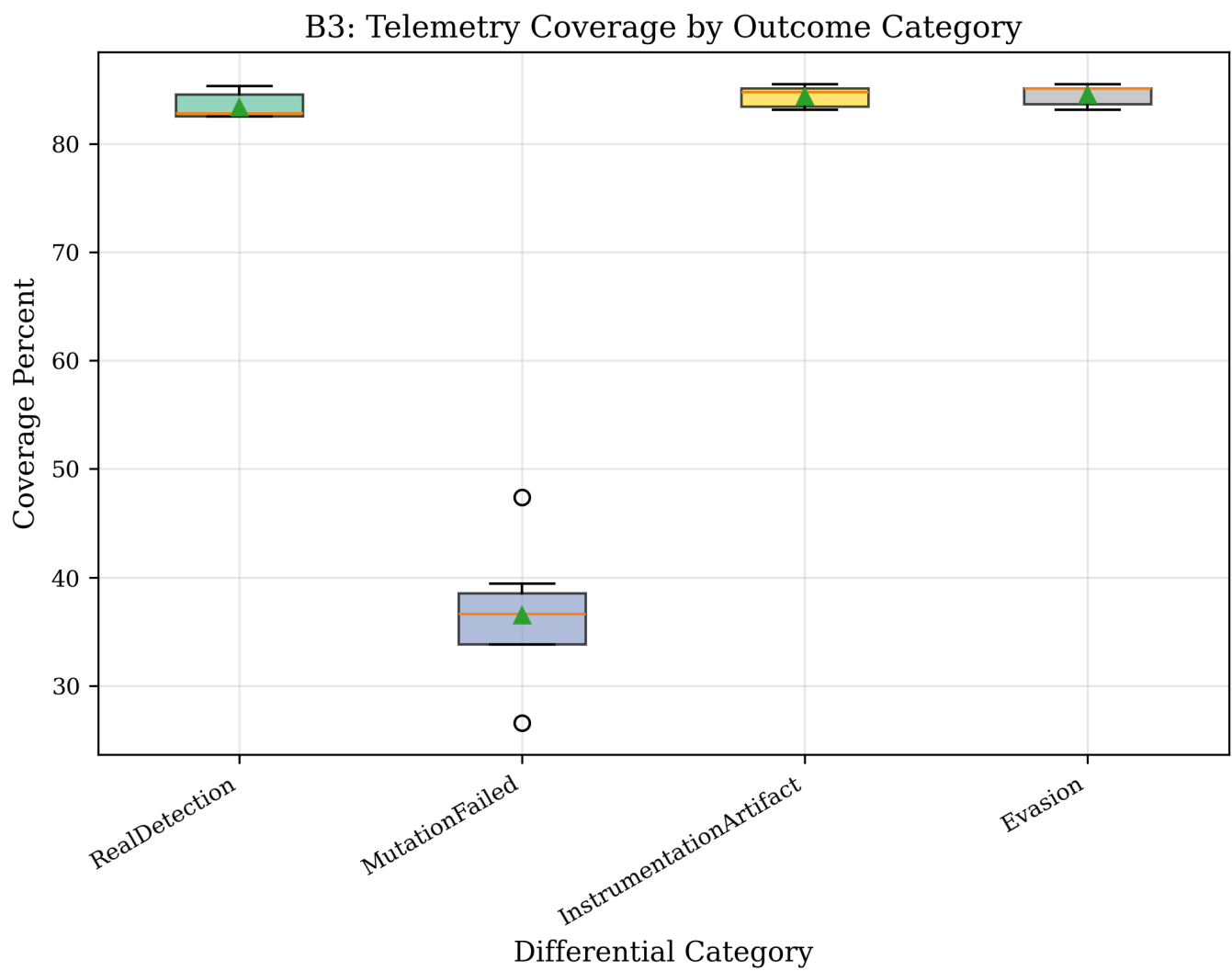


Figure 25: Genetic: code coverage distribution by differential outcome. Evasion rounds show distinct coverage profiles, with wider spread reflecting the aggressive mutation strategy.

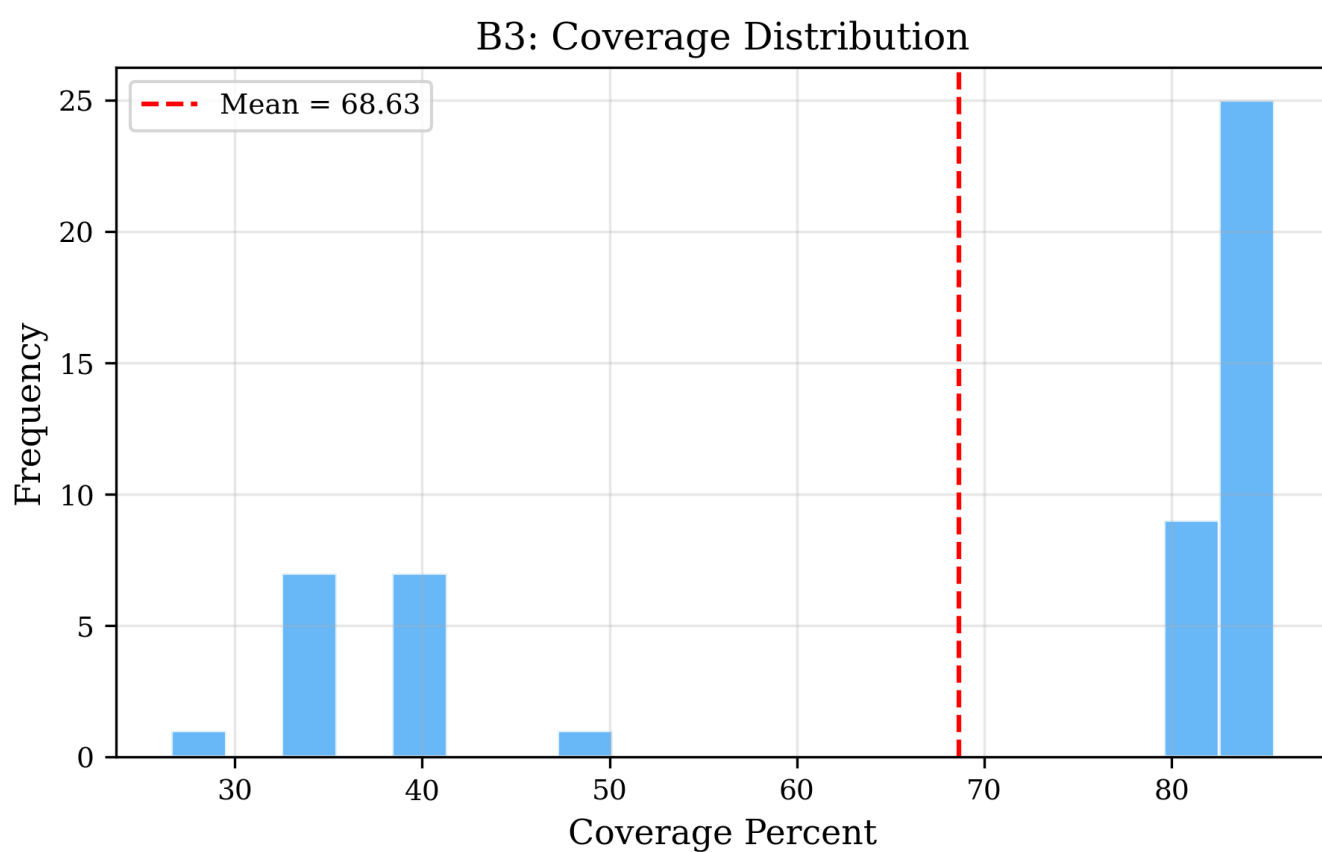


Figure 26: Genetic: full code coverage histogram. The bimodal distribution reflects the split between successful executions and mutation failures.

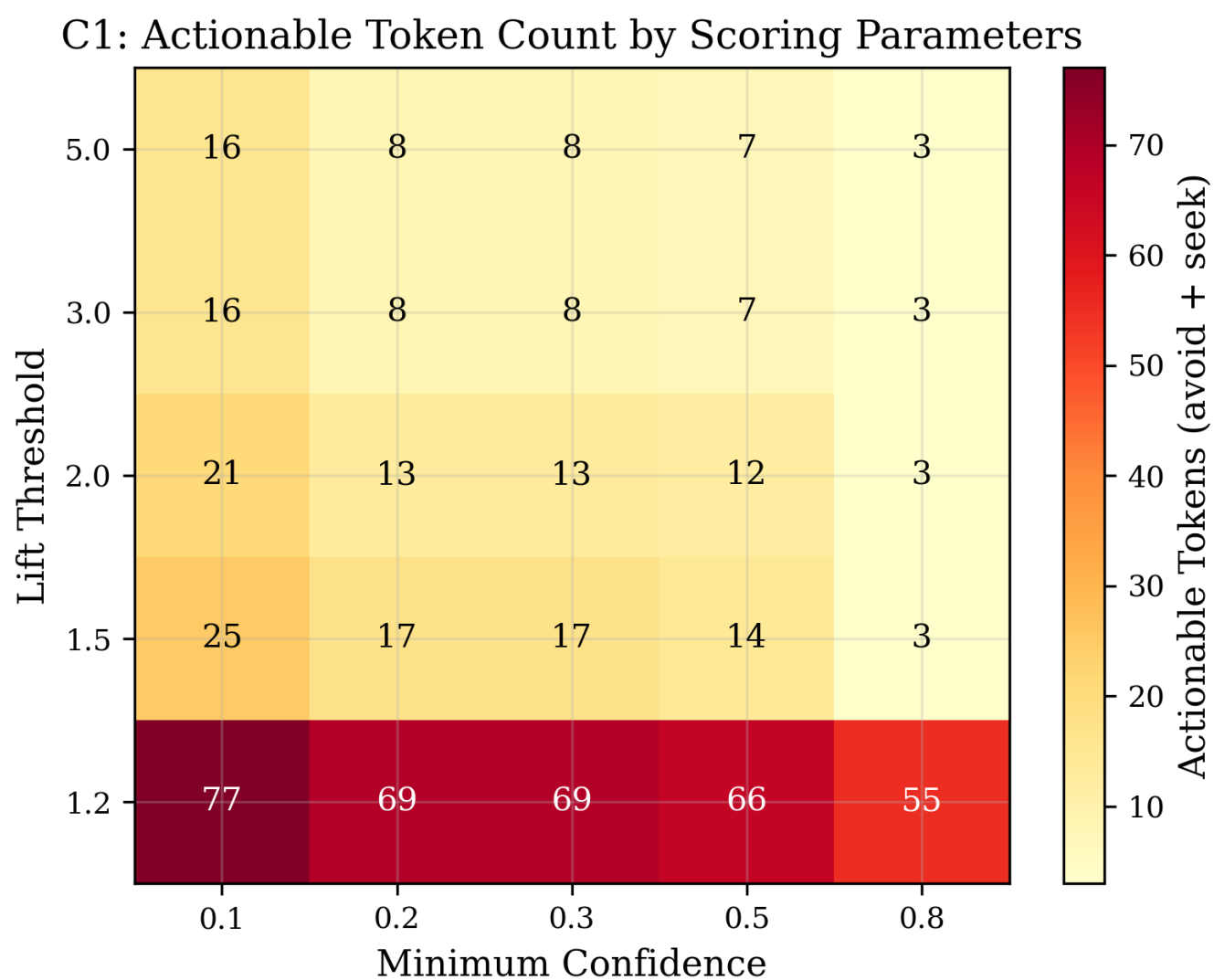
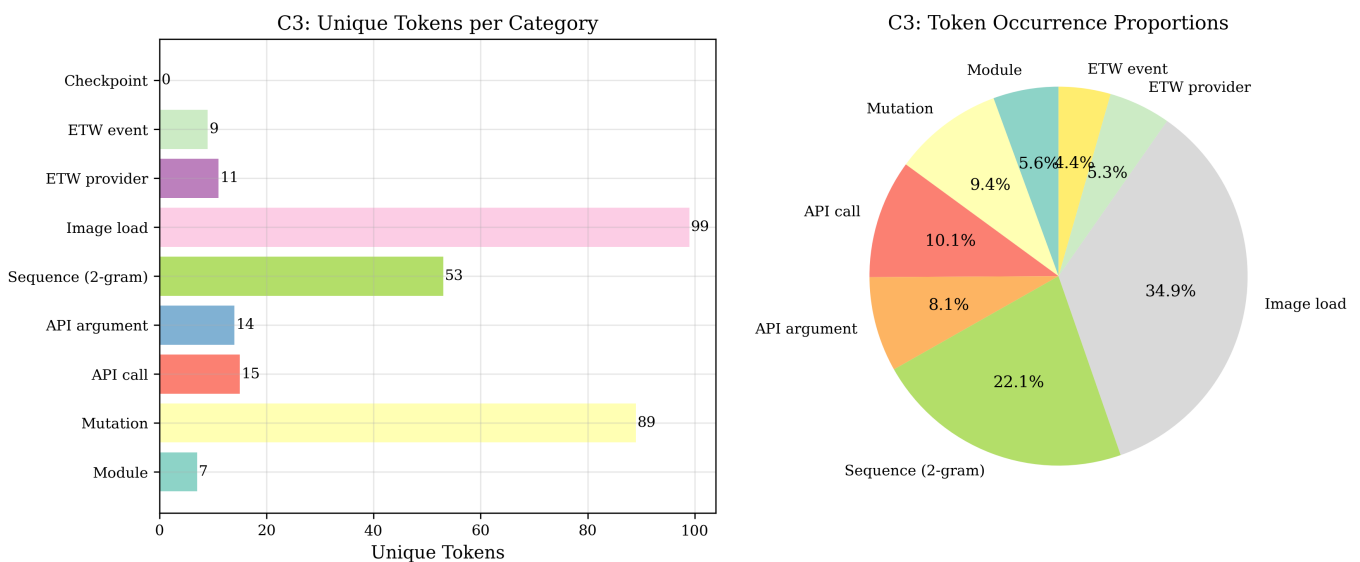
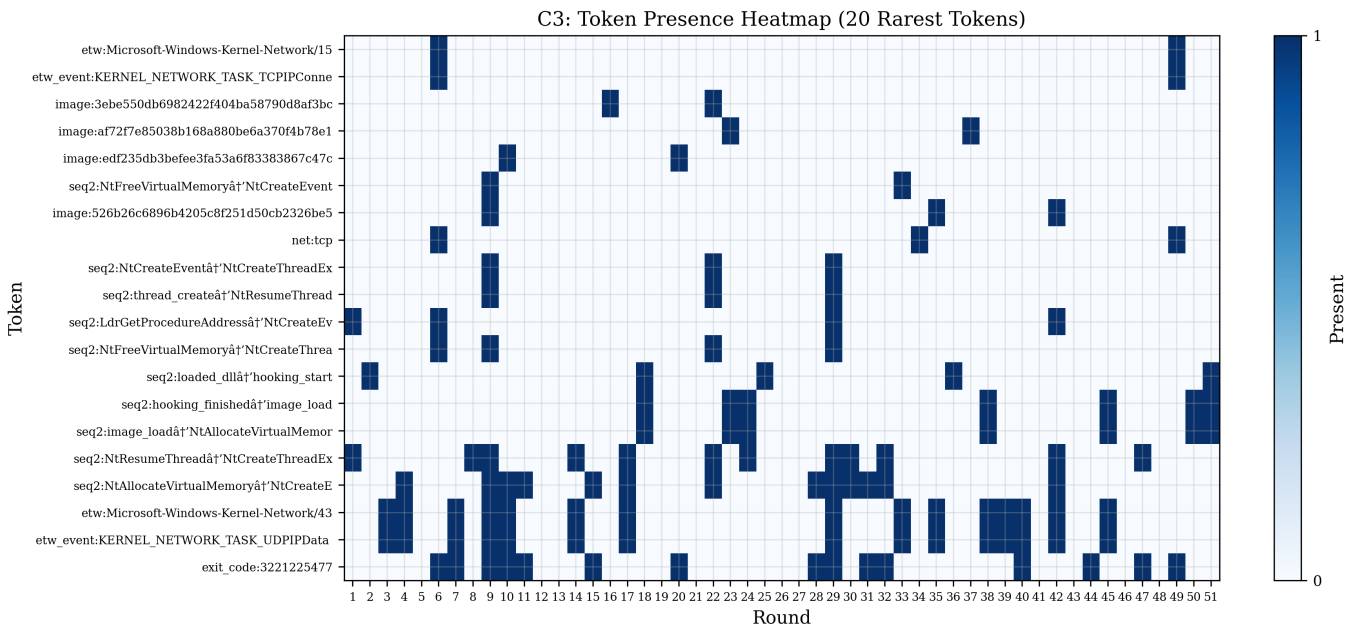


Figure 27: Genetic: 5×5 lift–confidence sensitivity heatmap. This GA-only figure shows how the discriminative token set varies with threshold parameters; other selectors lack sufficient evasion variation to produce this analysis.



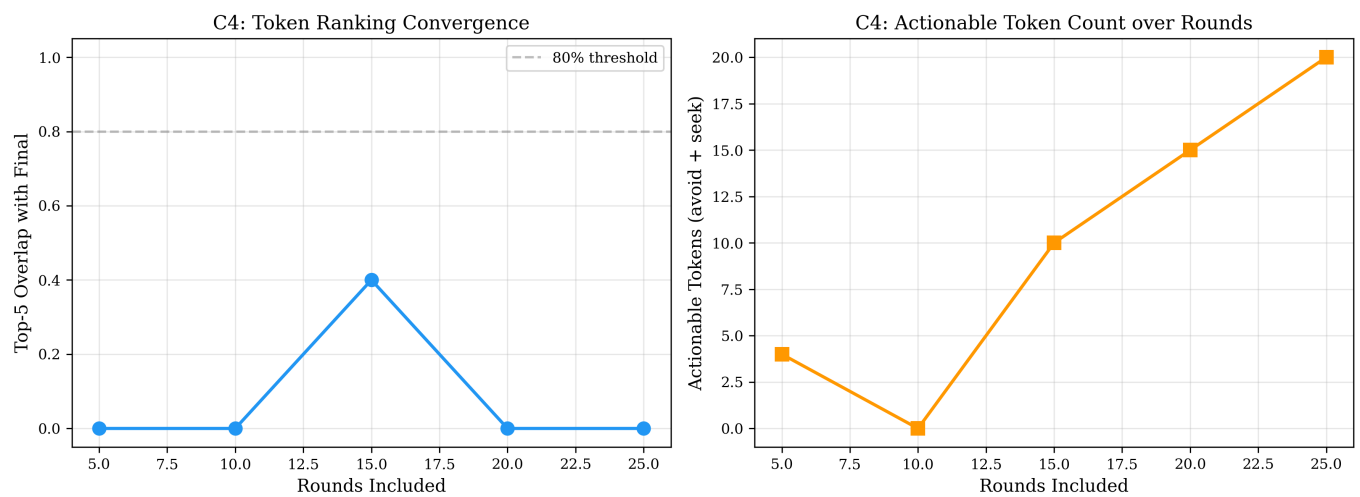


Figure 30: Genetic: lift variance convergence over rounds. The selector converges while still discovering evasions, avoiding premature convergence a desirable property.

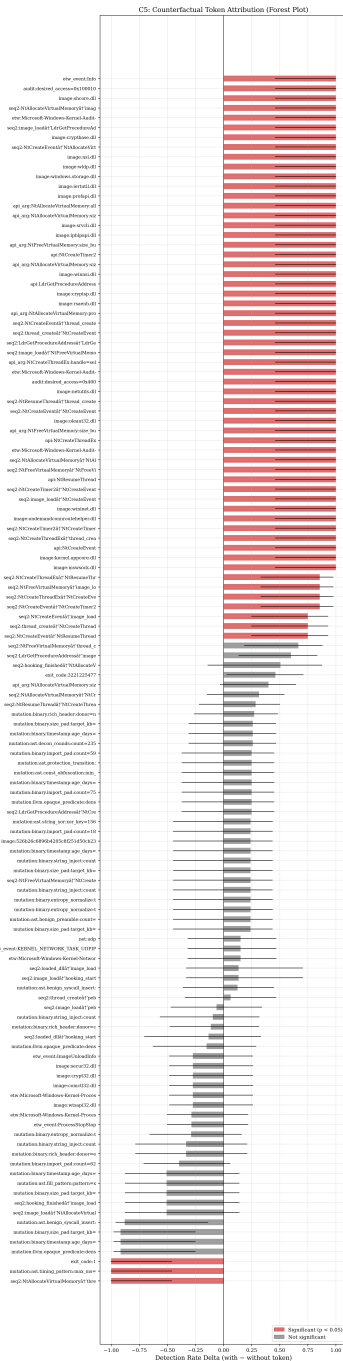


Figure 31: Genetic: forest plot of token effect sizes with Bonferroni correction. The Genetic selector is the only one to produce statistically significant single-mutation effects.

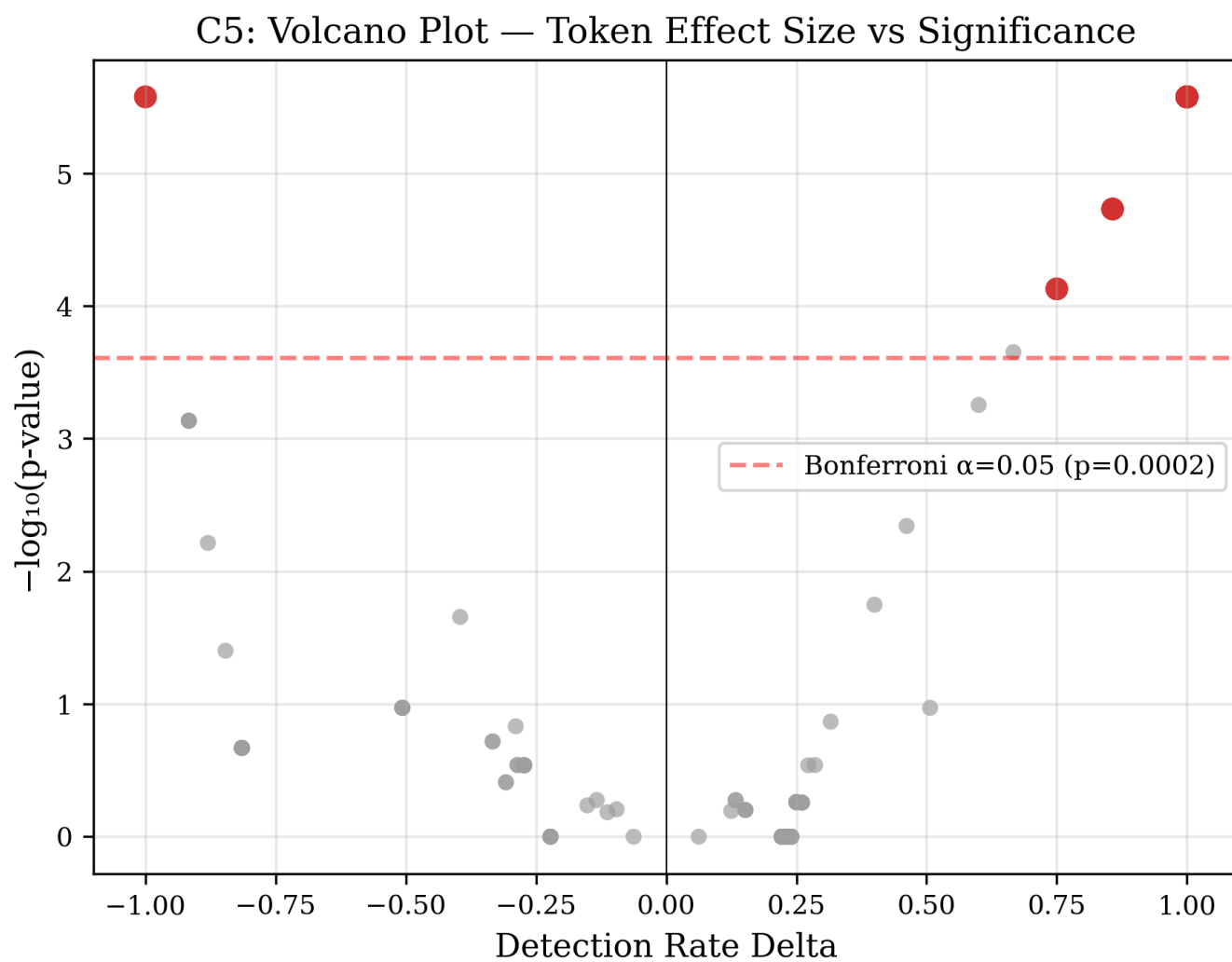


Figure 32: Genetic: volcano plot (log-odds vs. effect size). Tokens in the upper-right quadrant are both statistically significant and strongly associated with detection outcome.

*Random Selector (n=50)***Table 45:** Random Selector Evaluation Metrics

Metric ID	Label	Value	<i>n</i>
b2.classifier_analysis.confusion_matrix	Confusion matrix (verdict → differential category)	2.0000	50
b2.classifier_analysis.sankey_flows	Verdict → category Sankey diagram flows	2.0000	50
b2.classifier_analysis.score_by_category	Evasion score distribution by differential category	2.0000	50
b2.classifier_analysis.verdict_agreement	Verdict differential category agreement rate	0.3200	50
b3.telemetry_completeness.coverage_by_verdict	Coverage distribution by differential category	47.1360	50
b3.telemetry_completeness.coverage_detection_corr	Coverage detection Pearson correlation	0.9191	50
b3.telemetry_completeness.coverage_histogram	Coverage percent histogram	50.0000	50
b3.telemetry_completeness.token_counts	Token counts by differential category	50.0000	50
b3.telemetry_completeness.trustworthy_trajectory	Trustworthy round fraction (1 - contamination rate)	0.3200	50
c3.token_coverage.category_proportions	Per-round token category proportions (for stacked bar)	8.0000	50
c3.token_coverage.category_table	Token category coverage (active categories / 9 total)	0.8889	50
c3.token_coverage.presence_heatmap	Token presence heatmap (bottom-20 rarest tokens × rounds)	633.0000	50
c3.token_coverage.tokens_per_round	Mean tokens per round	107.3400	50
c4.scoring_convergence.actionable_stability	Round where actionable token count stabilizes	5.0000	50
c4.scoring_convergence.lift_variance	Lift variance convergence (final variance)	0.0000	50
c4.scoring_convergence.top5_overlap	Top-5 token overlap convergence (final overlap fraction)	0.0000	50
c5.counterfactual.effect_sizes	Largest absolute detection rate delta	0.0000	50
c5.counterfactual.forest_plot	Counterfactual validation: significant tokens (Bonferroni	0.0000	50

The Random selector serves as the unguided baseline. Its 68 % mutation failure rate and 32 % trustworthy ratio demonstrate the difficulty of generating valid, evasive artifacts without feedback.

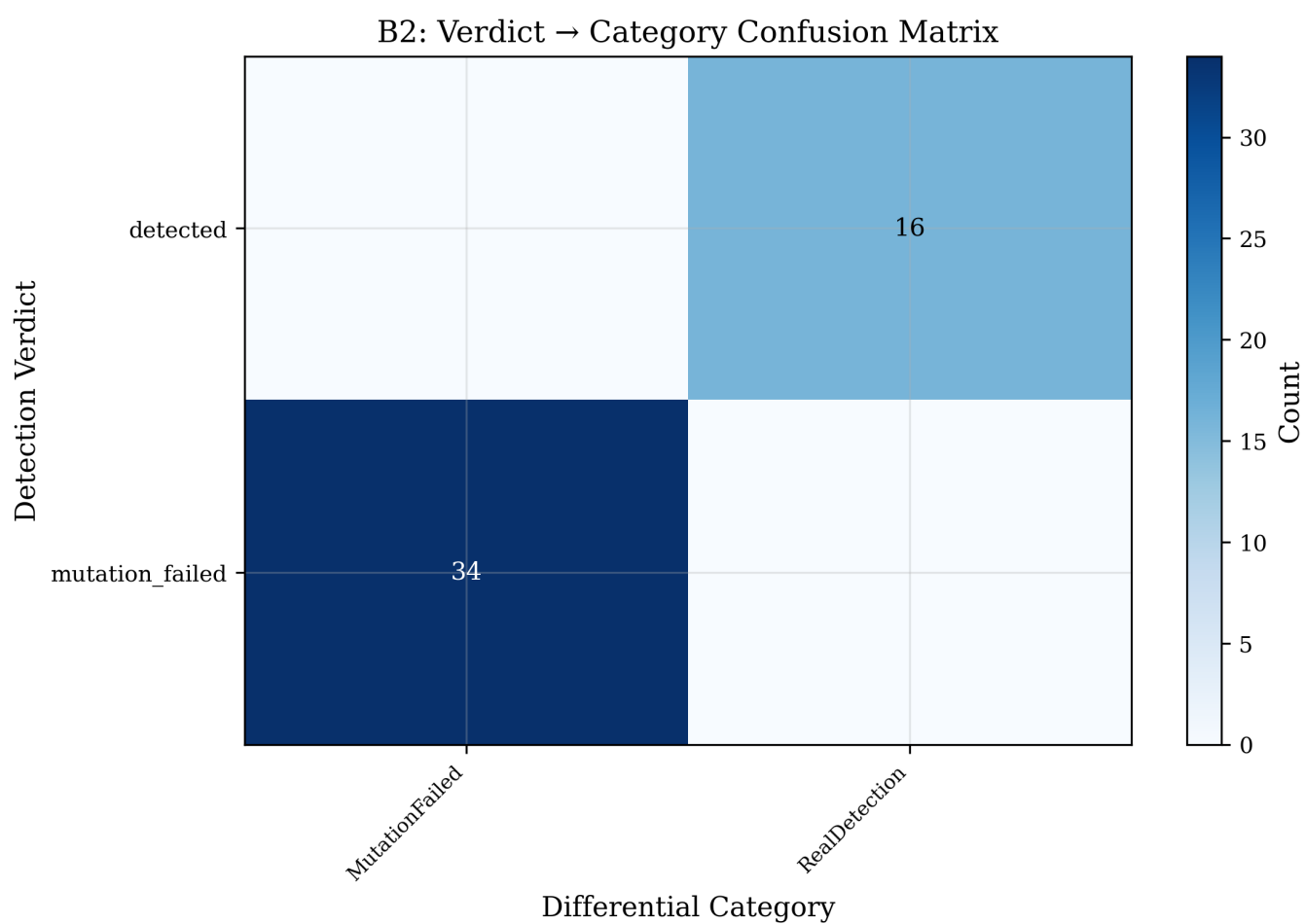


Figure 33: Random: verdict vs. differential category confusion matrix. Low diagonal concentration reflects the high rate of mutation failures and instrumentation artifacts in unguided campaigns.

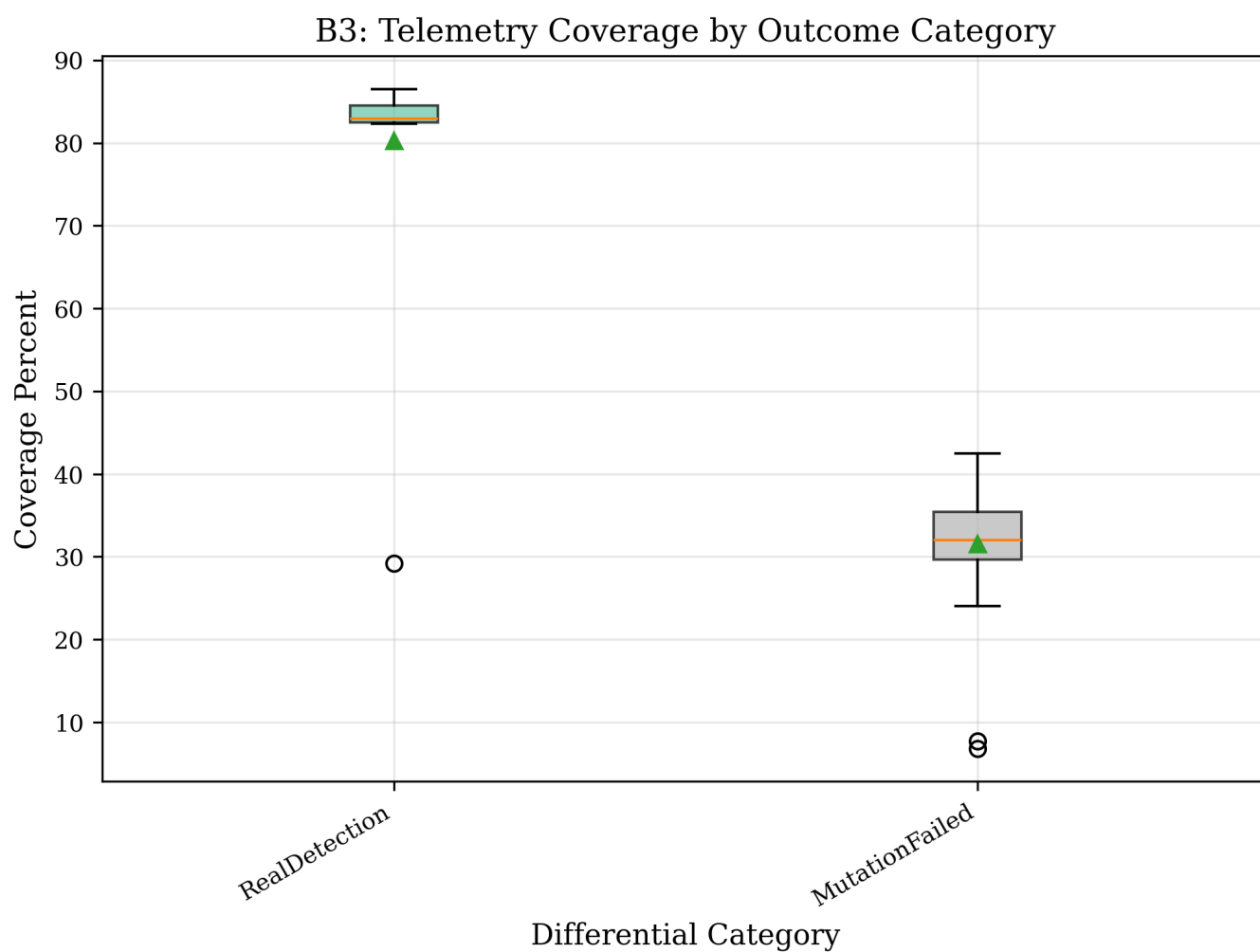


Figure 34: Random: code coverage distribution by differential outcome. Coverage values are generally lower and more variable than in guided selectors.

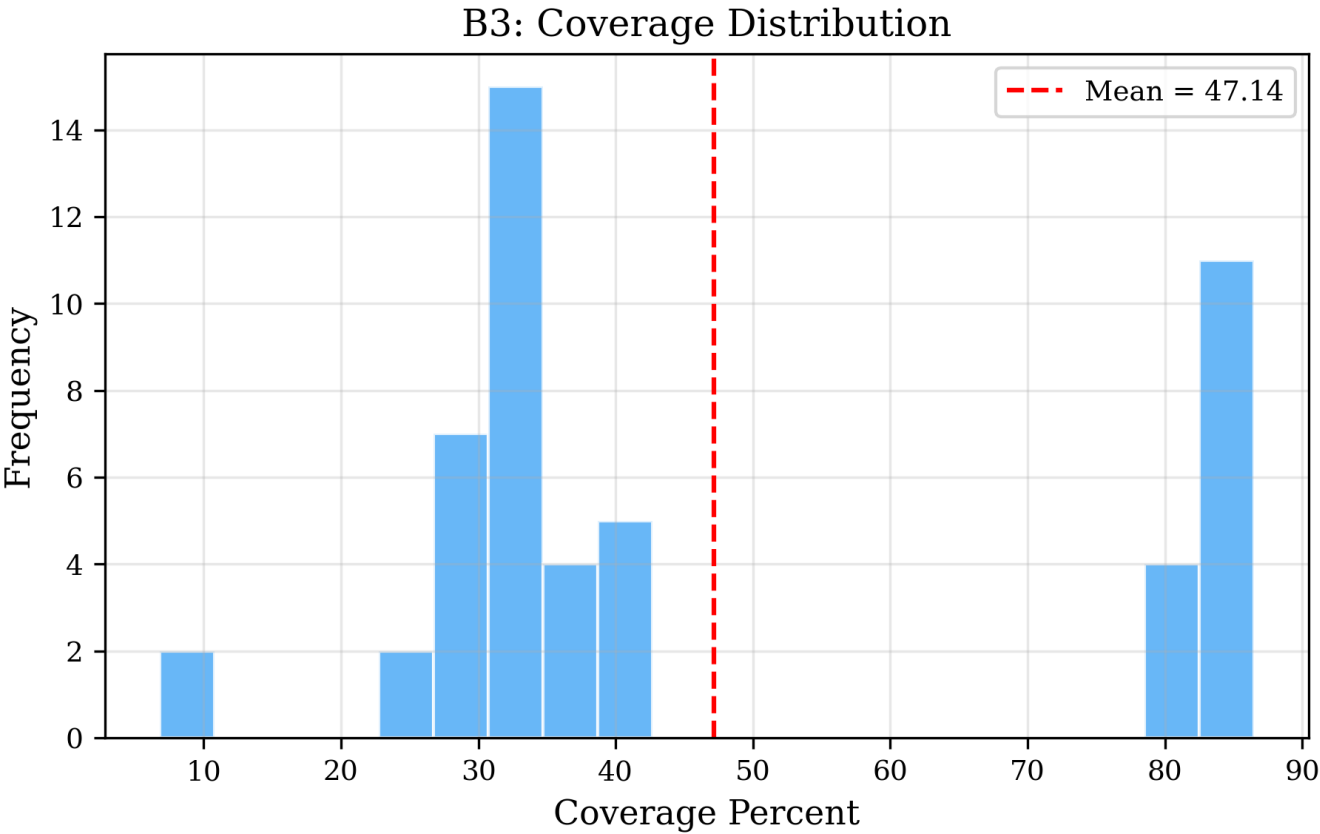


Figure 35: Random: full code coverage histogram. The distribution skews left, reflecting frequent early-termination failures.

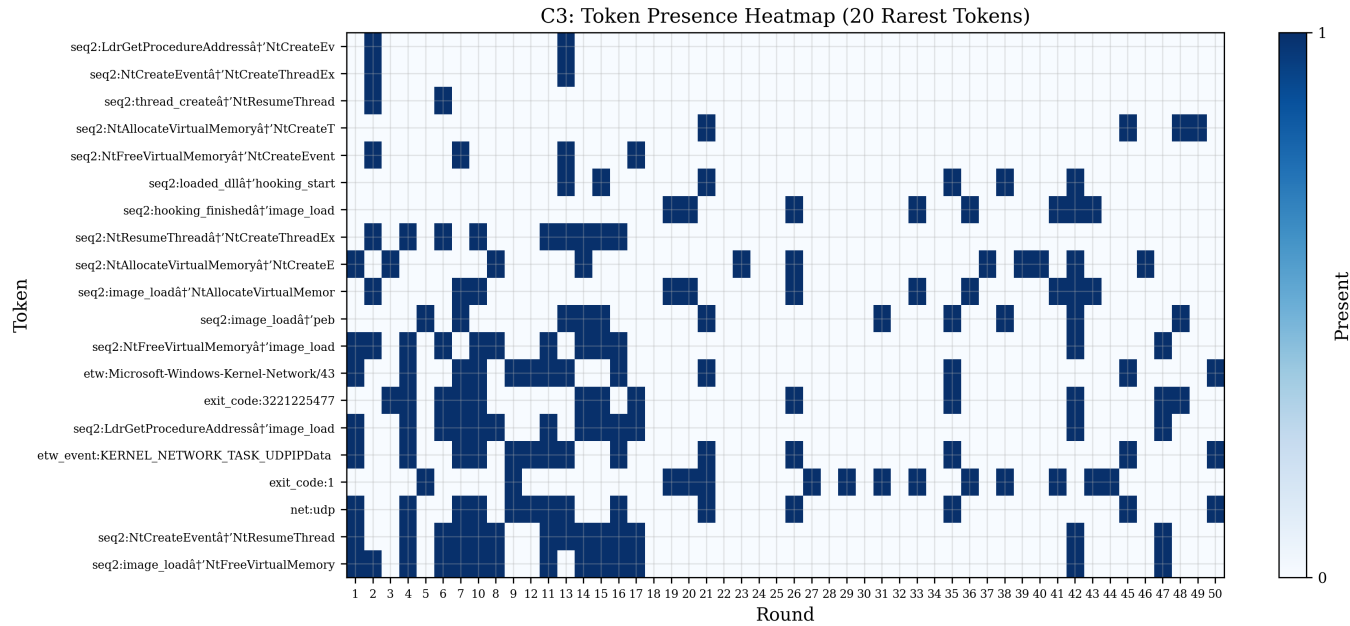


Figure 36: Random: rarest-20 behavioural tokens across rounds. Token presence is sparser than in guided campaigns, consistent with the high failure rate reducing observable behaviour.

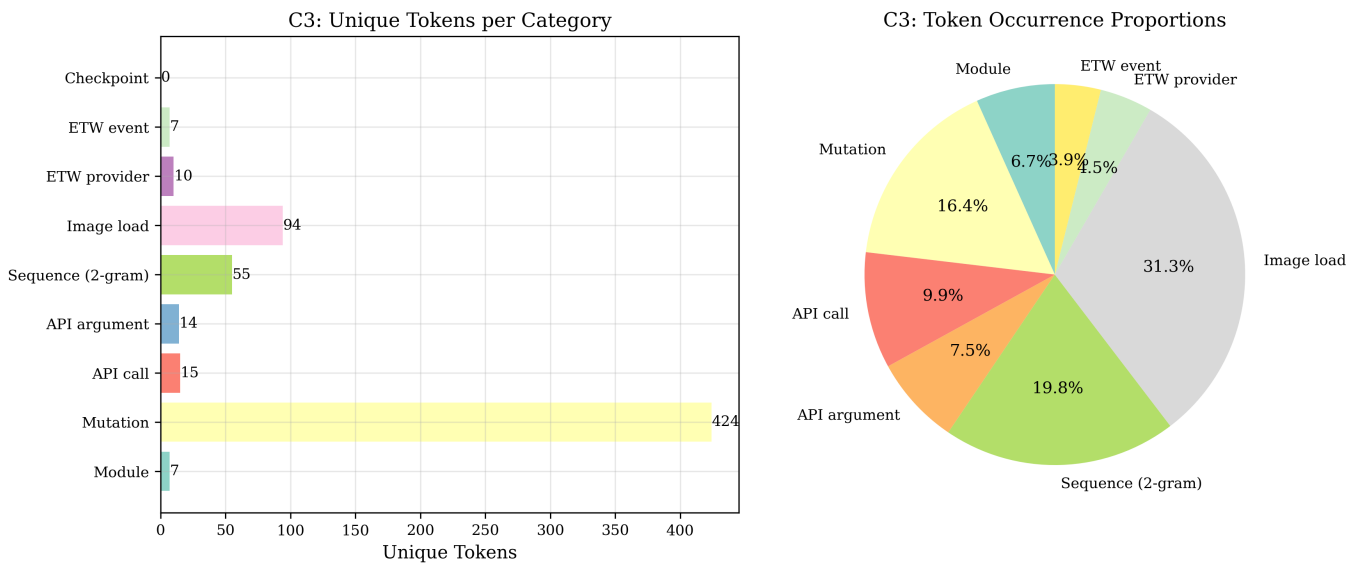


Figure 37: Random: token category proportions per round. Despite the high failure rate, successful rounds still activate all eight token categories.

Token Selector (n=40)

Table 46: Token Selector Evaluation Metrics

Metric ID	Label	Value	<i>n</i>
b2.classifier_analysis.confusion_matrix	Confusion matrix (verdict → differential category)	3.0000	40
b2.classifier_analysis.sankey_flows	Verdict → category Sankey diagram flows	3.0000	40
b2.classifier_analysis.score_by_category	Evasion score distribution by differential category	3.0000	40
b2.classifier_analysis.verdict_agreement	Verdict differential category agreement rate	0.4500	40
b3.telemetry_completeness.coverage_by_verdict	Coverage distribution by differential category	55.5150	40
b3.telemetry_completeness.coverage_detection_corr	Coverage detection Pearson correlation	0.9447	40
b3.telemetry_completeness.coverage_histogram	Coverage percent histogram	40.0000	40
b3.telemetry_completeness.token_counts	Token counts by differential category	40.0000	40
b3.telemetry_completeness.trustworthy_trajectory	Trustworthy round fraction (1 - contamination rate)	0.4500	40
c3.token_coverage.category_proportions	Per-round token category proportions (for stacked bar)	8.0000	40
c3.token_coverage.category_table	Token category coverage (active categories / 9 total)	0.8889	40
c3.token_coverage.presence_heatmap	Token presence heatmap (bottom-20 rarest tokens × rounds)	445.0000	40
c3.token_coverage.tokens_per_round	Mean tokens per round	124.4750	40
c4.scoring_convergence.actionable_stability	Round where actionable token count stabilizes	5.0000	40
c4.scoring_convergence.lift_variance	Lift variance convergence (final variance)	0.0000	40
c4.scoring_convergence.top5_overlap	Top-5 token overlap convergence (final overlap fraction)	0.0000	40
c5.counterfactual.effect_sizes	Largest absolute detection rate delta	0.0000	40
c5.counterfactual.forest_plot	Counterfactual validation: significant tokens (Bonferroni	0.0000	40

The Token selector uses triage token scores to bias mutation selection toward configurations that maximise token-based feedback. Despite guided selection, it achieves 0% evasion and explores only 7/10 AST mutations, suggesting premature search narrowing.

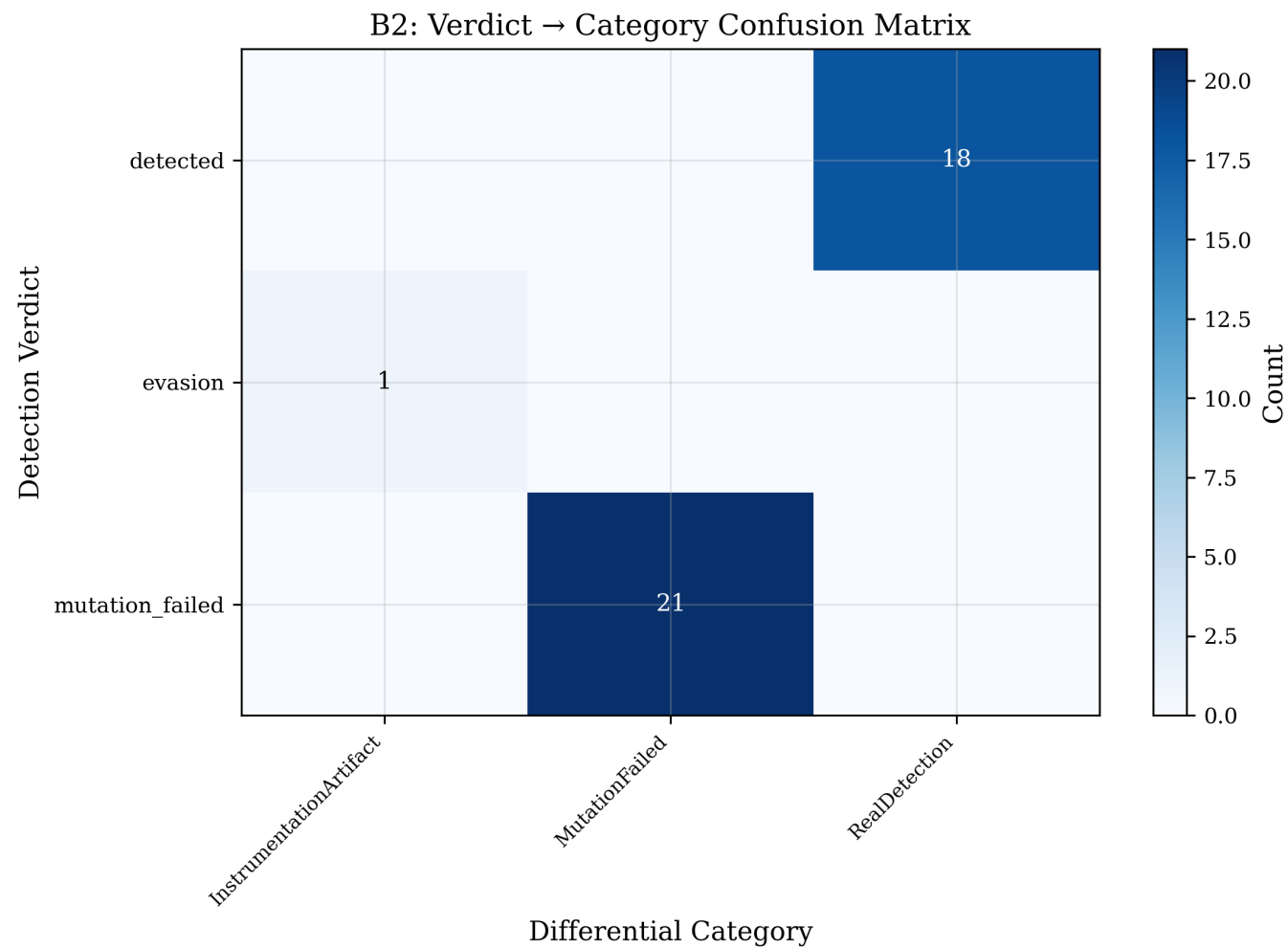


Figure 38: Token: verdict vs. differential category confusion matrix. Moderate diagonal agreement (45 %) places this selector between Coverage and Random.

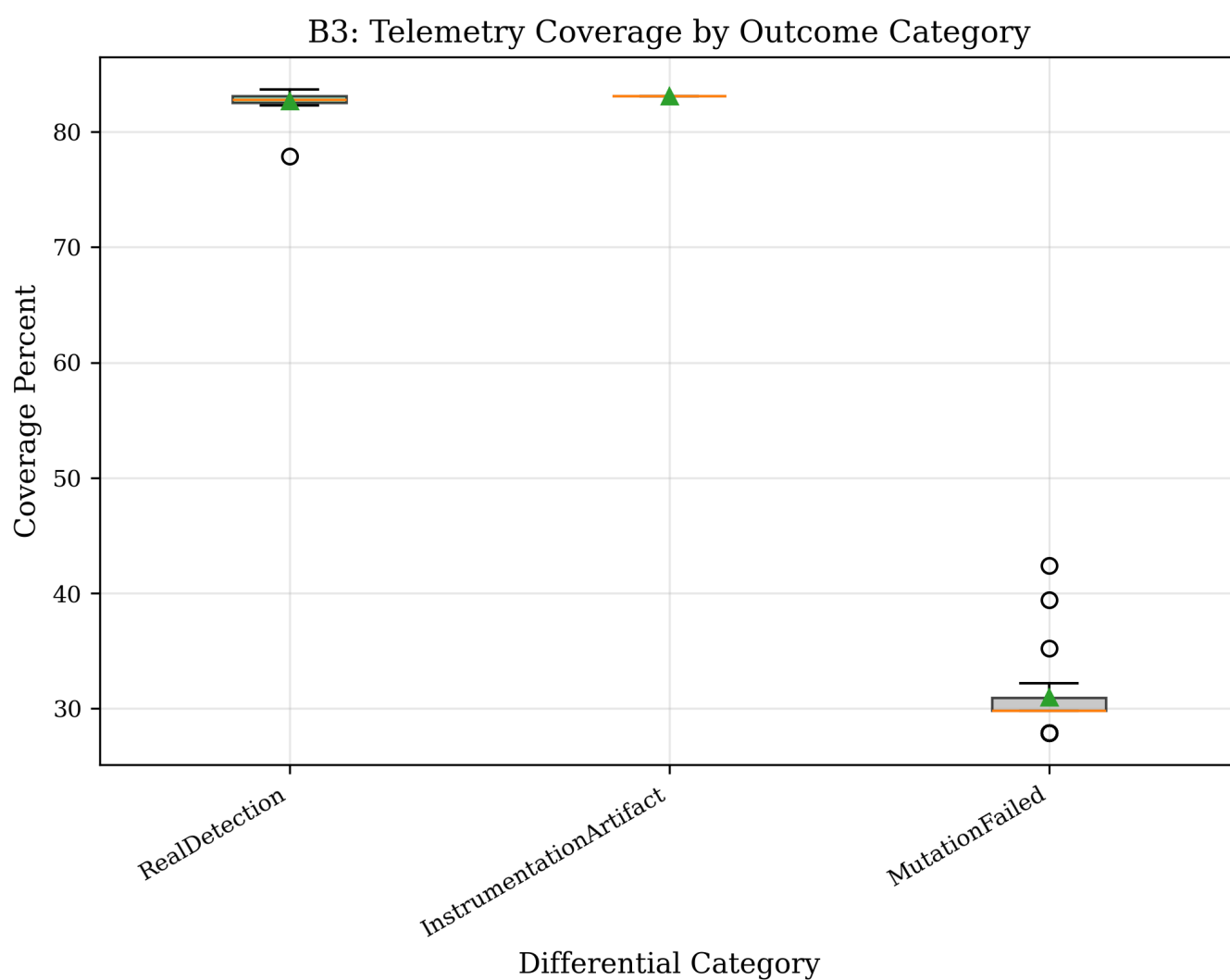


Figure 39: Token: code coverage distribution by differential outcome. Coverage values are moderate, reflecting the selector's preference for token-rich configurations over high-coverage ones.

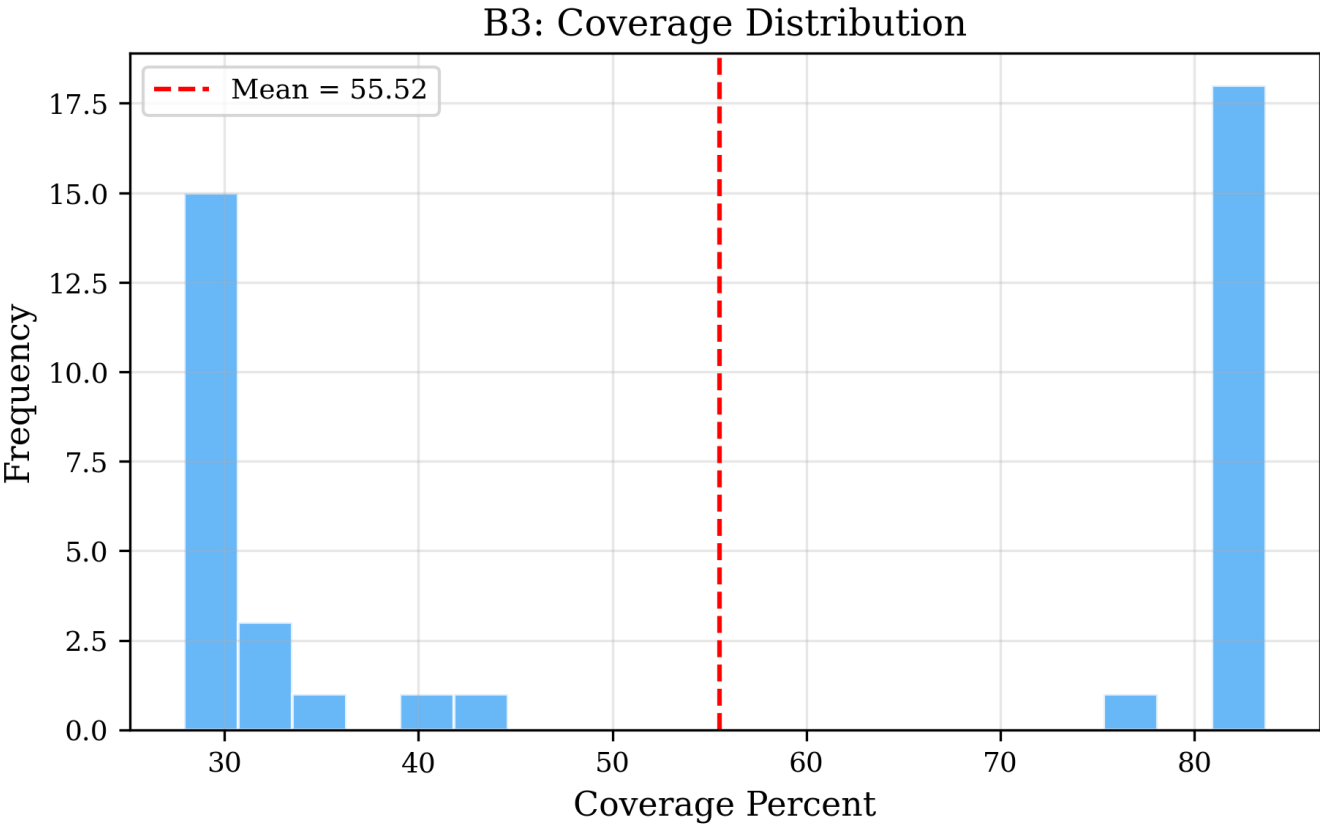


Figure 40: Token: full code coverage histogram. A notable fraction of rounds cluster at low coverage, corresponding to the 52.5 % mutation failure rate.

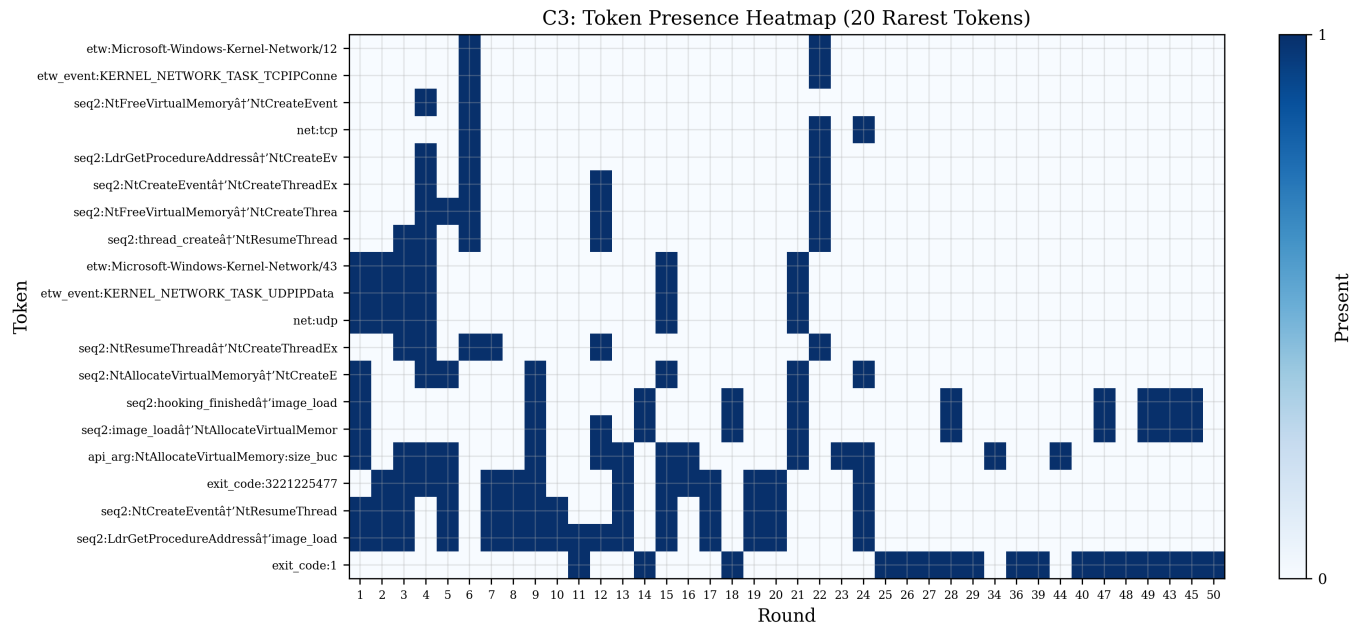


Figure 41: Token: rarest-20 behavioural tokens across rounds. The heatmap reveals narrower token diversity compared to Coverage and Genetic selectors, consistent with the reduced AST mutation coverage (7/10).

Infrastructure Evaluation Diagnostics

This appendix validates individual pipeline components independently of the selector strategy. Each subsection corresponds to one infrastructure experiment. These experiments verify that the mutation engine, token extraction, scoring formula, oracle protocol, and telemetry pipeline operate correctly before campaign-level results are interpreted.

I1: Encoding Validation

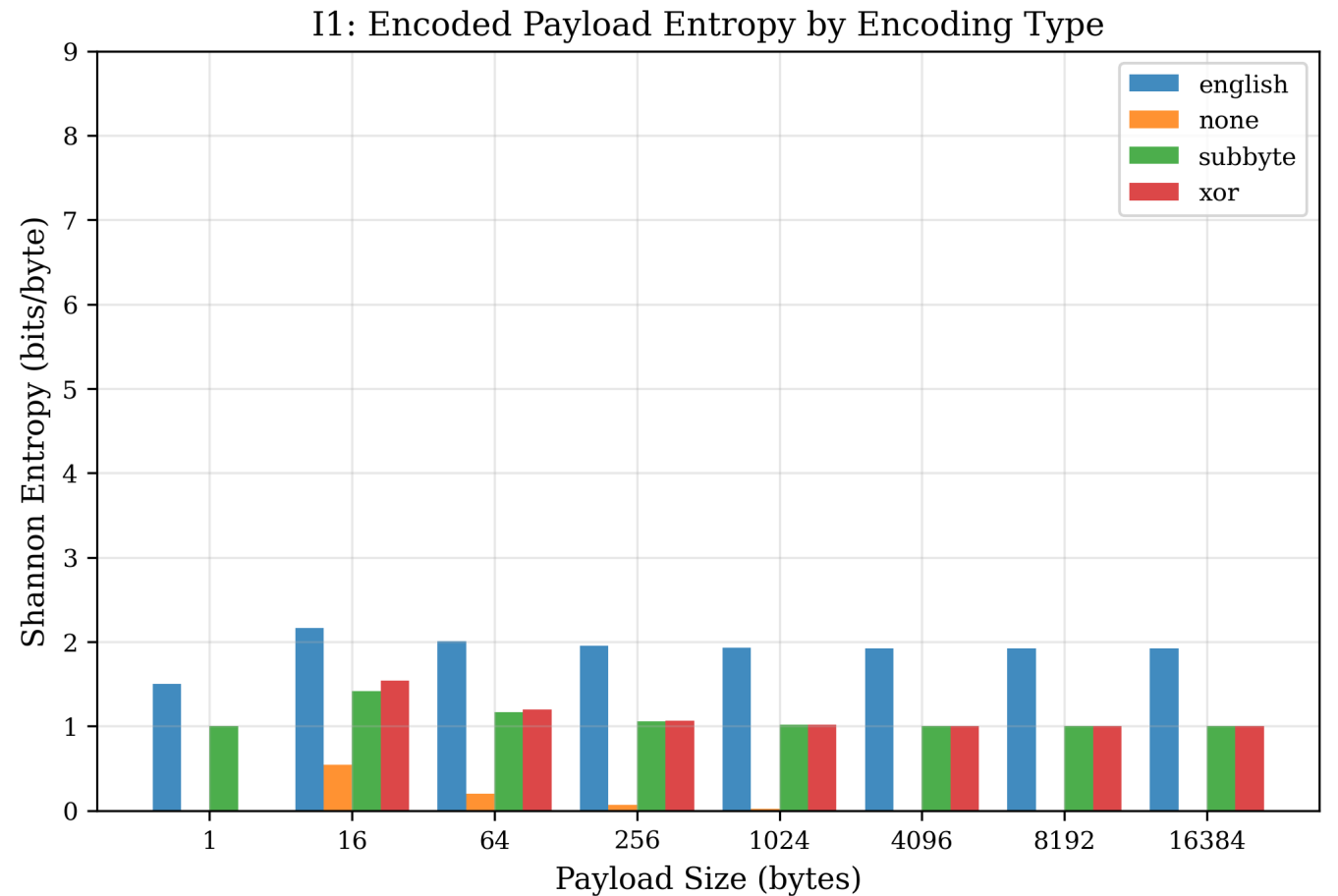


Figure 42: Shannon entropy comparison across four encoding types (XOR, English, SubByte, None). XOR encoding achieves maximum entropy, consistent with its role as a pseudo-random byte transformation.

XOR encoding maximises Shannon entropy with near-zero size overhead, making it the preferred encoding for payload obfuscation. English encoding produces low-entropy output with substantial size expansion, trading stealth for surface plausibility.

I10: Oracle Stability

Repeated evaluation of the same artifact produces identical scores, confirming determinism. Permuting the order of the three differential runs does not alter the final verdict, validating the protocol's robustness against execution-order effects.

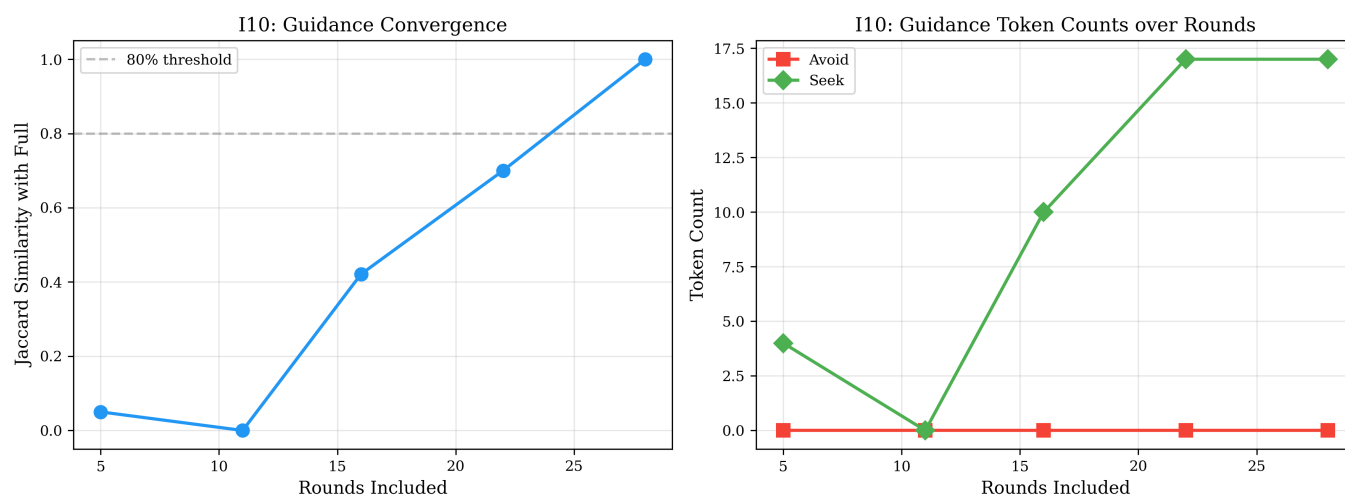


Figure 43: Scoring determinism and permutation robustness. Scores are stable across repeated evaluations and robust to run-order permutation.

I11: Cross-Selector Comparison

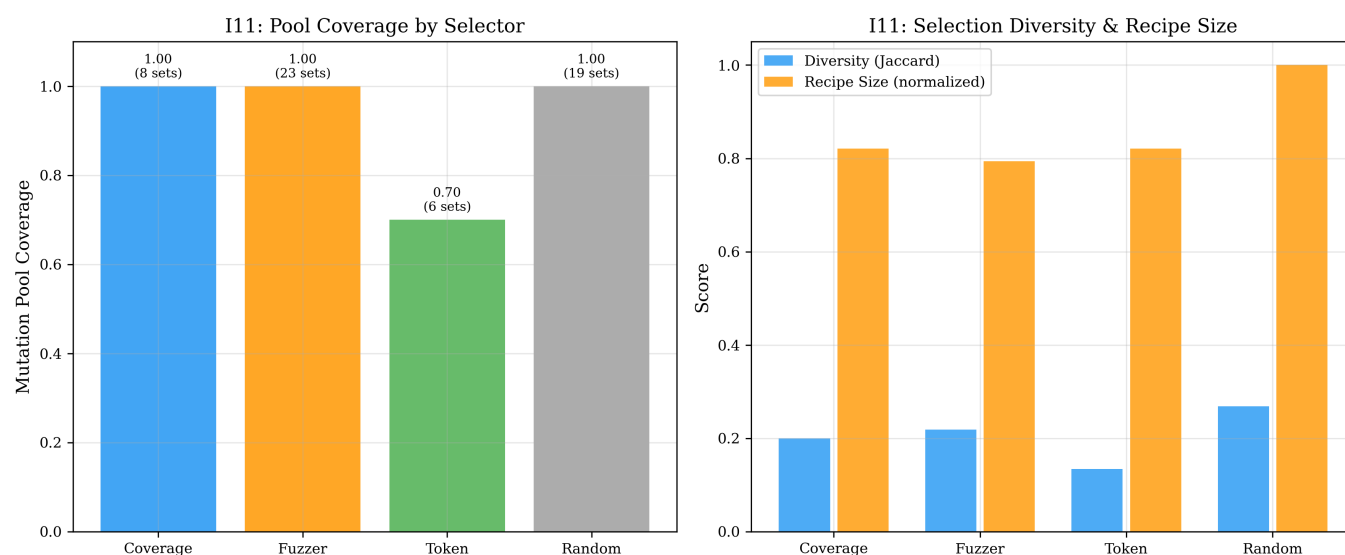


Figure 44: Cross-selector coverage and diversity bar chart. The Genetic selector achieves the widest coverage while maintaining high diversity.

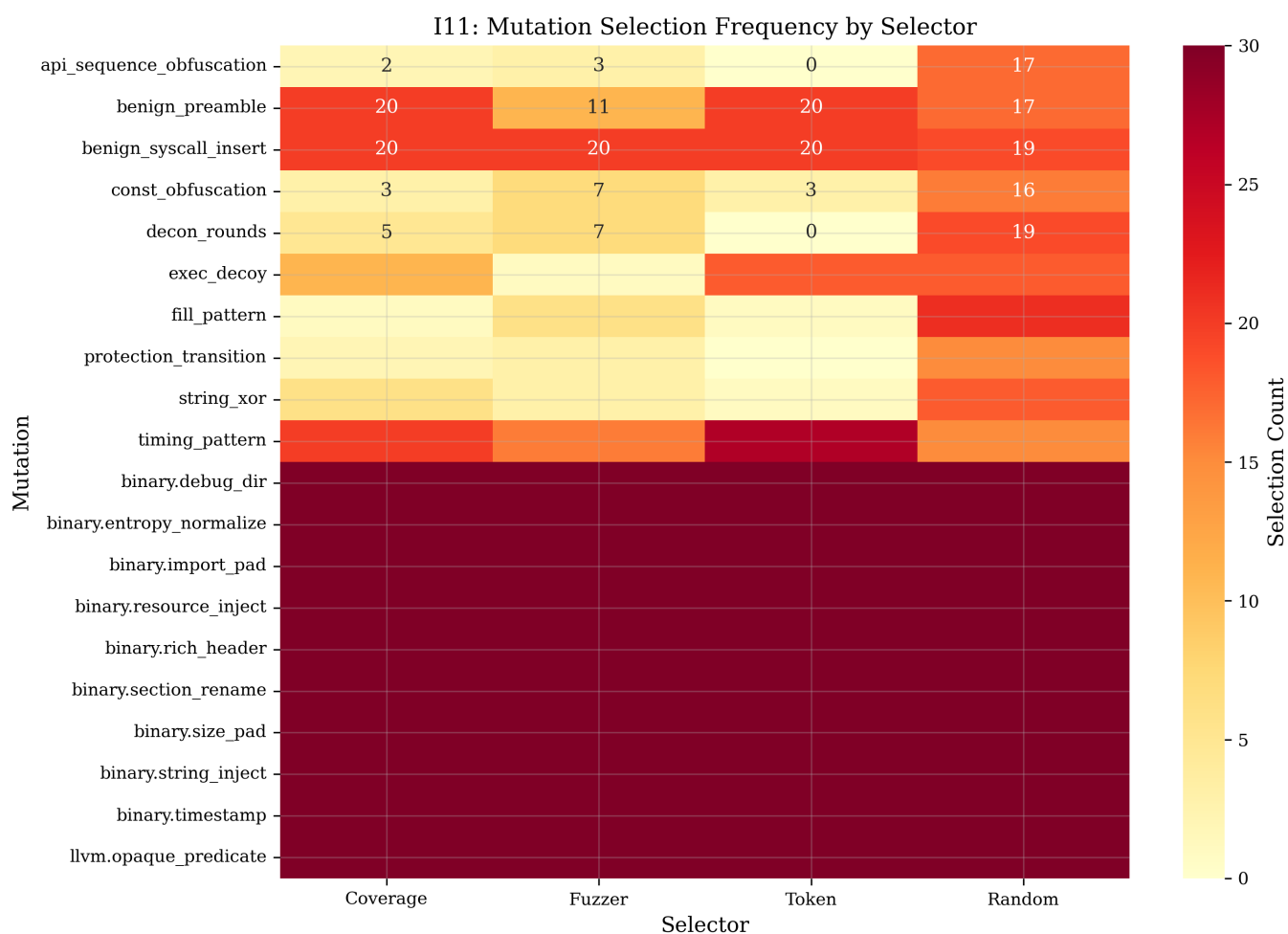


Figure 45: Mutation selection frequency matrix (mutation $\times$ selector). The Token selector concentrates on fewer mutations, consistent with its 7/10 AST coverage.

I12: Guidance Utilisation

I12: Mutation Frequency With/Without Guidance by Selector

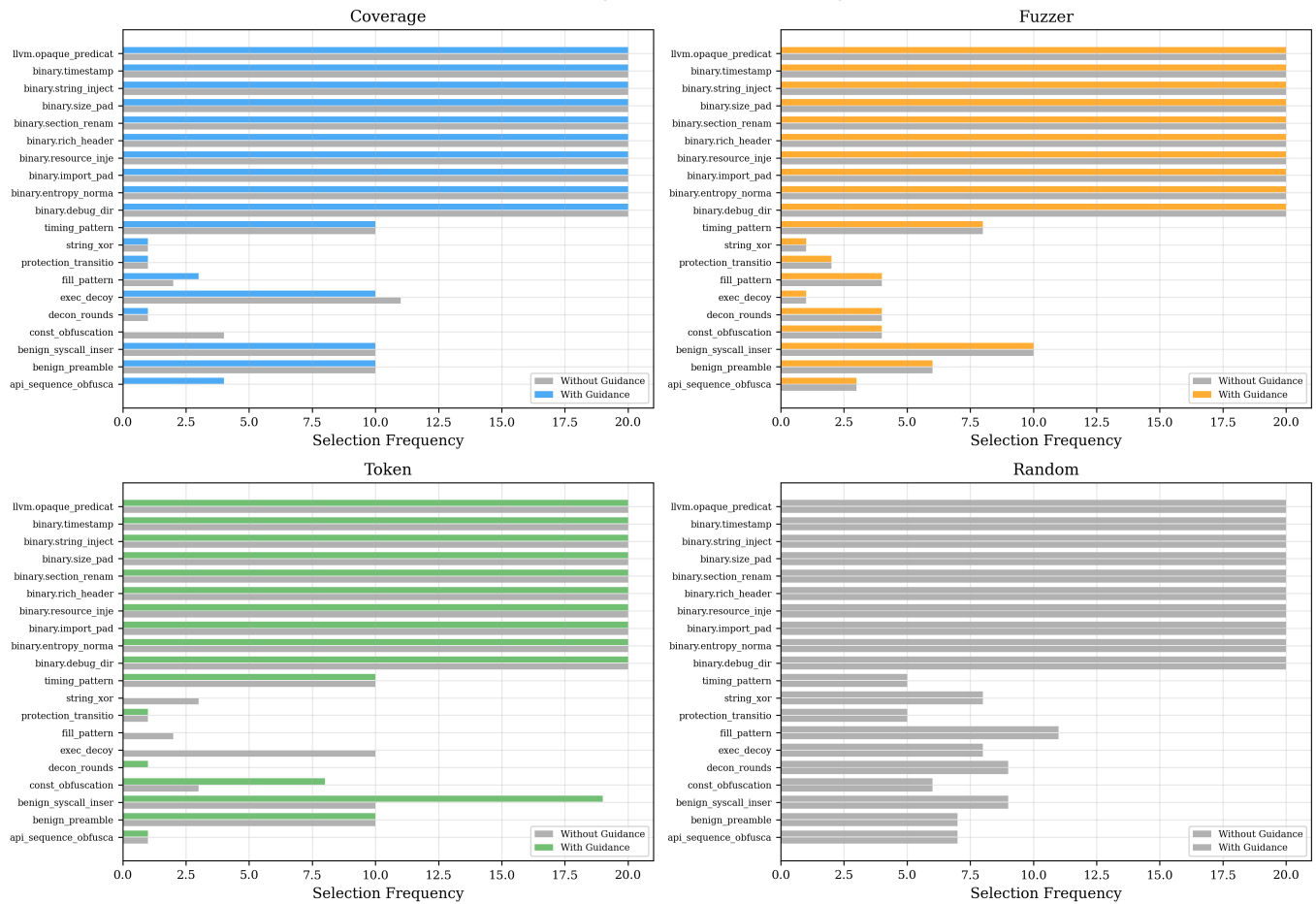


Figure 46: Avoid/seek compliance rates per selector. The Token selector shows highest compliance with triage guidance; the Genetic selector partially ignores avoid signals in favour of exploration.

Compliance rates measure how faithfully each selector follows the triage system's avoid/seek recommendations. The Token selector is most compliant, which paradoxically contributes to its narrower search. The Genetic selector's partial non-compliance enables broader exploration and, ultimately, evasion.

I13: Convergence Analysis

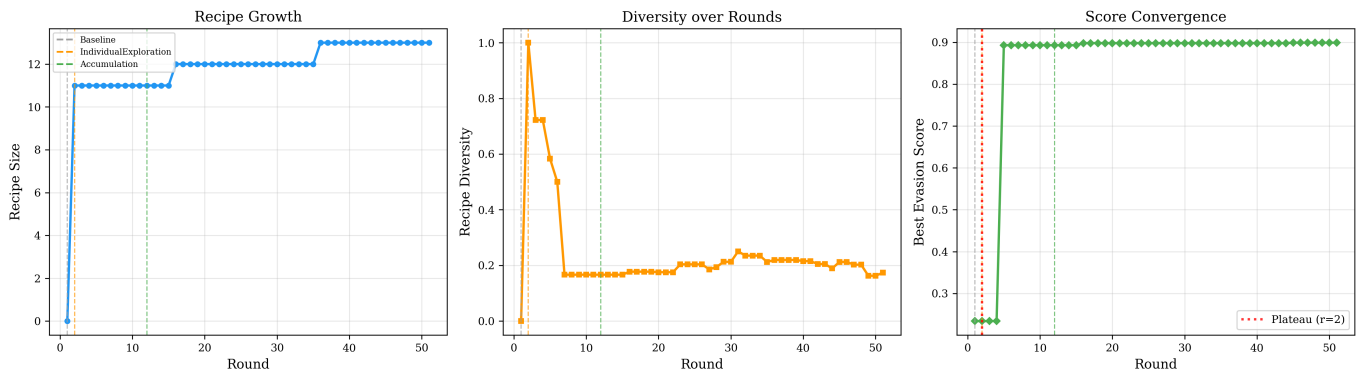


Figure 47: Simulated convergence on synthetic data. Theoretical convergence rates serve as the upper bound against which actual selector convergence is compared.

I14: Line Tracing

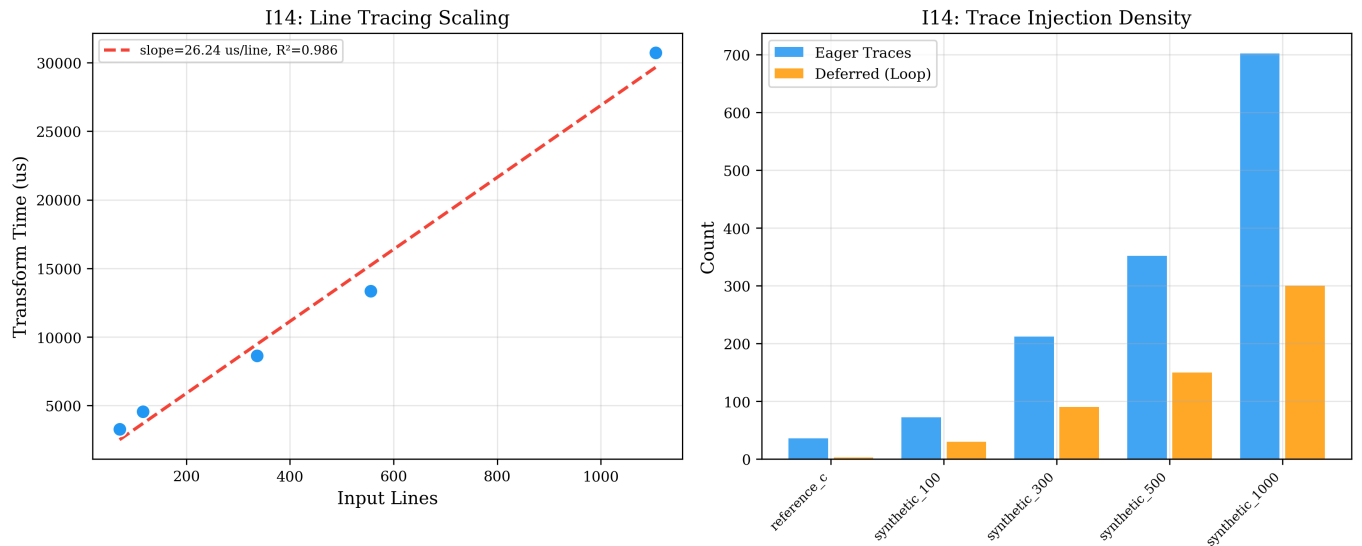


Figure 48: Line tracer throughput, validity, and instrumentation density. The tracer provides reliable coverage data at acceptable performance cost.

The line tracing infrastructure instruments compiled artifacts to measure code coverage during execution. Throughput, validity, and instrumentation density are all within acceptable ranges, confirming that the coverage correction mechanism operates on reliable data.

Condensed Metrics Tables

Table 47: Infrastructure-Level Evaluation Metrics - Automated

Metric ID	Label	Value	<i>n</i>
i1.payload_encoding.entropy_comparison	Encoding entropy comparison (max range across types)	1.8099	32
i1.payload_encoding.latency	Mean encode latency (μ s)	556.9250	32
i1.payload_encoding.roundtrip_correctness	Roundtrip correctness (fraction correct)	1.0000	32
i1.payload_encoding.size_expansion	Size expansion ratio (encoded/original)	2.2161	32
i10.oracle_stability.determinism	Scoring determinism (1.0 if all repeated runs identical)	0.0000	1
i10.oracle_stability.incremental_convergence	Fraction of incremental snapshots with Jaccard > 0.8	0.2000	1
i10.oracle_stability.lift_variance	Lift stability score: $1/(1+\text{mean_variance})$	1.0000	1
i10.oracle_stability.permutation_robustness	Mean top-5 Jaccard similarity across round permutations	0.0222	1
i11.selector_comparison.coverage_by_selector	Mutation pool coverage by selector strategy	0.9250	4
i11.selector_comparison.coverage_saturation	Round at which each selector hits 80% pool coverage	8.2500	4
i11.selector_comparison.coverage_trajectory	Cumulative mutation coverage trajectory per selector	0.9625	4
i11.selector_comparison.diversity_by_selector	Mean pairwise Jaccard distance of round selections	0.2054	4
i11.selector_comparison.diversity_trajectory	Rolling pairwise Jaccard diversity trajectory (window=5)	0.2054	4
i11.selector_comparison.exploration_rate	Per-selector exploration fraction	0.2833	4
i11.selector_comparison.guided_vs_random_delta	Coverage delta: guided selectors vs Random baseline	-0.1000	4
i11.selector_comparison.mutation_frequency_heatmap	Mutation selection frequency matrix (mutation $\times$ selector)	0.0000	4
i12.guidance_utilization.avoidance_rate	Fraction of guided rounds avoiding avoid-correlated mutation	0.3000	4
i12.guidance_utilization.guidance_trajectory	Per-round avoidance and seek compliance per selector	0.3000	4
i12.guidance_utilization.recipe_delta	Mean recipe difference with vs without guidance	0.0508	4
i12.guidance_utilization.seek_adoption_rate	Fraction of guided rounds adopting seek-correlated mutations	0.0000	4
i13.convergence_simulation.diversity_preservation	Minimum recipe diversity during accumulation phase	0.1628	40
i13.convergence_simulation.phase_transition_round	Round of first Accumulation phase entry (as fraction of total)	0.2353	1
i13.convergence_simulation.recipe_growth_rate	Recipe size growth rate during accumulation (mutations/round)	0.0495	40
i13.convergence_simulation.score_plateau_round	Round where best score plateaus (as fraction of total)	0.0392	51
i13.convergence_simulation.selector_convergence	Per-selector score, diversity, and recipe size trajectories	0.0000	4
i14.line_tracing.injection_density	Mean trace calls per source line	0.6053	5
i14.line_tracing.scaling	Latency scaling coefficient (μ s/line)	26.2415	5
i14.line_tracing.throughput	Mean source throughput (chars/ μ s)	0.6122	5
i14.line_tracing.validity	Fraction of outputs that parse as valid C	1.0000	5
i2.ast_mutation.line_impact	Per-mutation line delta table	3.0000	10
i2.ast_mutation.parse_validity	Fraction of mutations producing valid C	1.0000	10
i2.ast_mutation.transform_latency	Mean AST transform latency (μ s)	4354.9300	10
i3.ir_mutation.determinism	Fraction producing deterministic output from same seed	1.0000	4
i3.ir_mutation.insertion_effectiveness	Insertions per input line (by mutation)	0.1889	4
i3.ir_mutation.line_bloat	Mean output/input line ratio	1.1889	4
i3.ir_mutation.o2_survival	Fraction surviving -O2 optimization	0.0000	4
i7.token_extraction.category_coverage	Active token categories / total (9)	0.2222	5
i7.token_extraction.determinism	Fraction of runs producing deterministic output	1.0000	5
i7.token_extraction.latency_distribution	Extraction latency distribution (μ s)	235.2600	5
i7.token_extraction.tokens_per_doc	Mean tokens per input document	0.7192	5
i8.token_scoring.guidance_correctness	Avoid/seek classification accuracy	0.6667	3
i8.token_scoring.lift_accuracy	Lift computation accuracy ($1 - \text{max_error}$)	1.0000	3
i9.input_diversity.encoding_entropy_spread	Entropy range across encoding types (bits)	2.1631	32
i9.input_diversity.output_uniqueness	Fraction of mutation pairs producing distinct output	0.8667	45
i9.input_diversity.pairwise_distance	Mean pairwise normalized distance between mutation outputs	0.6213	45
i9.input_diversity.param_sensitivity	Fraction of mutations where different params produce differe	0.0000	0